

Package *javax.telephony*

The Java Telephony API (JTAPI) is a portable, object-oriented application programming interface for Java-based computer-telephony applications.

See:

Description

Interface Summary	
<u>Address</u>	An Address object represents what we commonly think of as a "telephone number".
<u>AddressEvent</u>	The AddressEvent interface is the base interface for all Address-related events.
<u>AddressListener</u>	The AddressListener interface reports all changes which happen to the Address object.
<u>AddressObserver</u>	The AddressObserver interface reports all changes which happen to the Address object.
<u>Call</u>	A Call object models a telephone call.
<u>CallEvent</u>	The CallEvent interface is the base interface for all Call-related events.
<u>CallListener</u>	The CallListener interface reports all changes to the Call object.
<u>CallObserver</u>	The CallObserver interface reports all changes which happen to the Call object and all of the Connection and TerminalConnection objects which are part of the Call.
<u>Connection</u>	A Connection represents a link (i.e. an association) between a Call object and an Address object.
<u>ConnectionEvent</u>	The ConnectionEvent interface is the base event interface for all Connection-related events.
<u>ConnectionListener</u>	This interface is an extension of the CallListener interface, and reports state changes both of the Call and its Connections.
<u>Event</u>	The Event interface is the parent of all JTAPI event interfaces.
<u>JtapiPeer</u>	The JtapiPeer interface represents a vendor's particular implementation of the Java Telephony API.
<u>MetaEvent</u>	The MetaEvent interface is the base interface for all JTAPI Meta events.
<u>MultiCallMetaEvent</u>	The MultiCallMetaEvent interface is the base interface for all multiple-call Call Meta events (multi-call MetaEvent).

<u>Provider</u>	A Provider represents the telephony software-entity that interfaces with a telephony subsystem.
<u>ProviderEvent</u>	The ProviderEvent interface is the base interface for all Provider related events.
<u>ProviderListener</u>	The ProviderListener interface reports all changes which happen to the Provider object.
<u>ProviderObserver</u>	The ProviderObserver interface reports all changes which happen to the Provider object.
<u>SingleCallMetaEvent</u>	The SingleCallMetaEvent interface is the base interface for all single-call Call Meta events.
<u>Terminal</u>	A Terminal represents a physical hardware endpoint connected to the telephony domain.
<u>TerminalConnection</u>	The TerminalConnection represents the relationship between a Connection and a Terminal .
<u>TerminalConnectionEvent</u>	The TerminalConnectionEvent interface is the base event interface for all TerminalConnection -related events.
<u>TerminalConnectionListener</u>	This interface is an extension of the ConnectionListener interface, and reports state changes of the Call , its Connections and its TerminalConnections .
<u>TerminalEvent</u>	The TerminalEvent interface is the base interface for all Terminal -related events.
<u>TerminalListener</u>	The TerminalListener interface reports all changes which happen to the Terminal object.
<u>TerminalObserver</u>	The TerminalObserver interface reports all changes which happen to the Terminal object.

Class Summary

<u>JtapiPeerFactory</u>	The JtapiPeerFactory class is a class by which applications obtain a Provider object.
--------------------------------	---

Exception Summary	
<u>InvalidArgumentException</u>	An <code>InvalidArgumentException</code> indicates an argument passed to the method is invalid.
<u>InvalidPartyException</u>	An <code>InvalidPartyException</code> indicates that a party given as an argument to the method call was invalid.
<u>InvalidStateException</u>	An <code>InvalidStateException</code> indicates the current state of an object involved in the method invocation does not meet the acceptable pre-conditions for the method.
<u>JtapiPeerUnavailableException</u>	The <code>JtapiPeerUnavailableException</code> indicates that the <code>JtapiPeer</code> (i.e.
<u>MethodNotSupportedException</u>	The <code>MethodNotSupportedException</code> indicates that the method which was invoked is not supported by the implementation.
<u>PlatformException</u>	A <code>PlatformException</code> indicates an implementation-specific exception.
<u>PrivilegeViolationException</u>	A <code>PrivilegeViolationException</code> indicates that an action pertaining to a certain object failed because the application did not have the proper security permissions to execute that command.
<u>ProviderUnavailableException</u>	The <code>ProviderUnavailableException</code> indicates that the Provider is currently not available to the application.
<u>ResourceUnavailableException</u>	The <code>ResourceUnavailableException</code> indicates that a resource inside the system is not available to complete an operation.

Package javax.telephony Description

The Java Telephony API (JTAPI) is a portable, object-oriented application programming interface for Java-based computer-telephony applications.

The Java™ Telephony API

Introduction

JTAPI serves a broad audience, from call center application developers to web page designers. JTAPI supports both first- and third-party telephony application domains. The API is designed to make programming simple applications easy, while providing those features necessary for advanced telephony applications.

The Java Telephony API is, in fact, a set of API's. The "core" API provides the basic *call model* and rudimentary telephony features, such as placing telephone calls and answering telephone calls. The core API is surrounded by standard extension APIs providing functionality for specific telephony domains, such as call centers and media stream access. The JTAPI core and extension package architectures are described later in this document.

Applications written using the Java Telephony API are portable across the various computer platforms and telephone systems. Implementations of JTAPI will be available for existing computer-telephony integration platforms such as Sun Microsystem's SunXTL™, Microsoft and Intel's TAPI, Novell and Lucent's TSAPI, and IBM's CallPath. Additionally, independent hardware vendors may choose to provide implementations of the Java Telephony API on top of their own proprietary hardware.

Overview Document Organization

This document is organized into the following sections:

<u>Java Telephony API Features</u>	Describes the features of JTAPI and the principles on which it was designed.
<u>Supported Configurations</u>	Summarizes the environments in which JTAPI may be used and the computer and software configurations for which it was designed.
<u>Java Telephony Package Architecture</u>	Summarizes how the Java Telephony API is organized into various Java language packages. A brief description accompanies each package along with links to more detailed documentation.
<u>The Java Telephony Call Model</u>	Describes how telephone calls and different objects that make up telephone calls are represented in this API.
<u>Core Package Methods</u>	Provides a brief summary of the key methods available in the core package which perform the most basic telephony operations, such as placing a telephone call, answering a telephone call, and disconnecting a connection to a telephone call.
<u>Connection Object States</u>	Describes the states in which the Connection object can exist. It provides a description of the allowable transitions from each state.
<u>TerminalConnection Object States</u>	Describes the states in which the TerminalConnection object can exist. It provides a description of the allowable transitions from each state.
<u>Placing a Telephone Call</u>	One of the most common features used in any telephony API is placing a telephone call. This section describes the JTAPI method invocations required to place a telephone call, and examines state changes in the call model. This analysis will explain how calls are placed, answered, and terminated.
<u>The Java Telephony Observer Model</u>	Describes the JTAPI observer model. Applications use observers for asynchronous notification of changes in the state of the JTAPI call model.
<u>Application Code Examples</u>	Provides two real-life code examples using the Java Telephony API. One example places a telephone call to a specified telephone number. The other example shows a designated Terminal answering an incoming telephone call.
<u>Locating and Obtaining Providers</u>	Describes the manner in which applications create and obtain JTAPI Provider objects.
<u>Security</u>	Summarizes the JTAPI security strategy.

History of the Java Telephony API

The Java Telephony API specification represents the combined efforts of design teams from Sun Microsystems, Lucent, Nortel, Novell, Intel, and IBM, operating under the direction of JavaSoft on versions 1.0 and 1.1, and teams from Sun Microsystems, Lucent, Nortel, IBM, Siemens, and Dialogic, working within the Enterprise Computer Telephony Forum on version 1.2 and 1.3.

The Java Telephony API version 1.0 specification was released to the public on November 1, 1996. Version 1.1 was released to the public on February 1, 1997; version 1.2 was released to the public in December 1997; version 1.3 was released for early access to the public on April 7, 1999.

Java Telephony Features

The features and guiding design principles for the Java Telephony API are:

- Brings simplicity to the most basic telephony applications
- Provides a scalable framework that spans desktop applications to distributed call center telephony applications
- Interfaces applications directly to service providers or acts as a Java interface to existing telephony APIs, such as SunXTL™, TSAPI, and TAPI
- Based on a simple core that is augmented with standard extension packages
- Runs on a wide range of hardware configurations, wherever Java run-time can be used

Supported Configurations

JTAPI runs on a variety of system configurations, including centralized servers with direct access to telephony resources, and remote network computers with access to telephony resources over a network. In the first configuration, a network computer is running the JTAPI application and is accessing telephony resources over a network, as illustrated in [Figure 1](#). In the second configuration, the application is running on a computer with its own telephony resources, as illustrated in [Figure 2](#).

Network Computer (NC) Configuration

In a network configuration, the JTAPI application or Java applet runs on a remote workstation. This workstation can be a network computer with only a display, keyboard, processor, and some memory. It accesses network resources, making use of a centralized server that manages telephony resources. JTAPI communicates with this server via a remote communication mechanism, such as Java's Remote Method Invocation (RMI) or a telephony protocol. The following diagram shows this configuration.

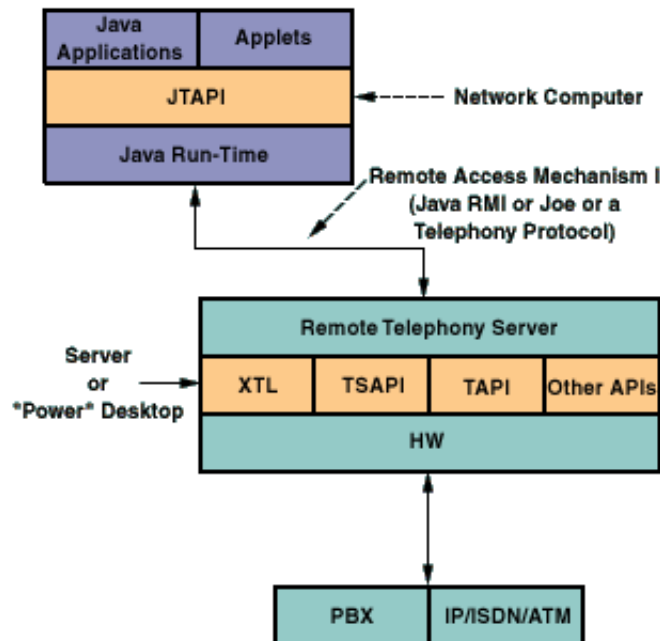


Figure 1: Network Configuration

Desktop Computer Configuration

In a desktop configuration, the JTAPI application or Java applet runs on the same workstation that houses the telephony resources. The following diagram shows the desktop configuration.

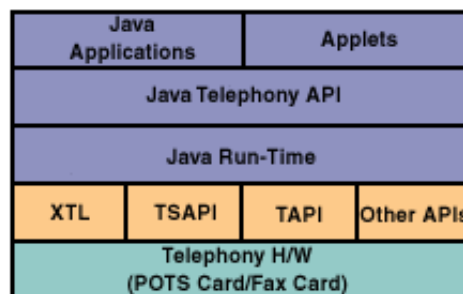


Figure 2: Desktop Configuration

Java Telephony Package Architecture

The Java Telephony API is composed of a set of Java language *packages*. Each package provides a specific piece of functionality for a certain aspect of computer-telephony applications. Implementations of telephony servers choose the packages they support, depending upon the capabilities of their underlying

platform and hardware. Applications may query for the packages supported by the implementation they are currently using. Additionally, application developers may concern themselves with only the supported packages applications need to accomplish a task. The diagram below depicts the architecture of the JTAPI packages.

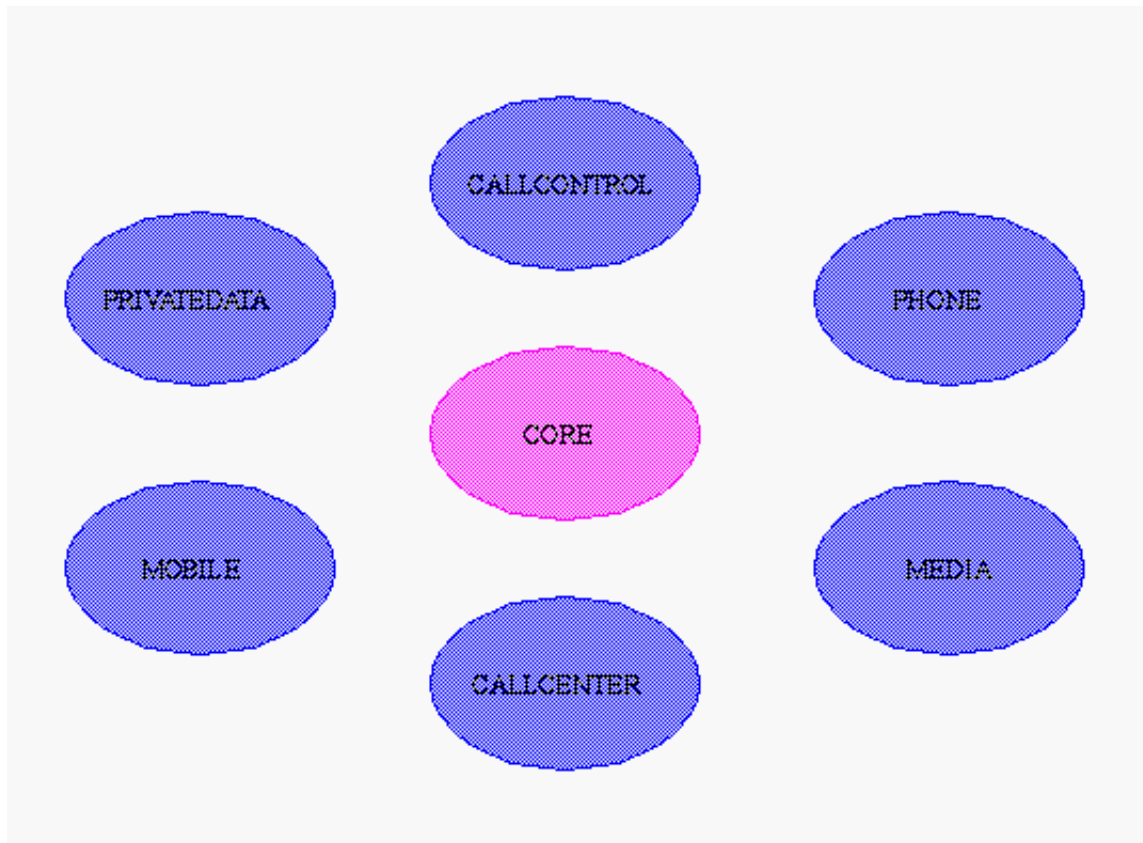


Figure 3: Core/Extension Package Relationship

At the center of the Java Telephony API is the "core" package. The core package provides the basic framework to model telephone calls and rudimentary telephony features. These features include placing a telephone call, answering a telephone call, and disconnecting a connection to a telephone call. Simple telephony applications will only need to use the core to accomplish their tasks, and do not need to concern themselves with the details of other packages. For example, the core package permits applet designers to add telephone capabilities to a Web page with ease.

A number of "standard extension" packages extend the JTAPI core package. These extension packages each bring additional telephony functionality to the API. Currently, the following extension packages exist for this API: call control, call center, media, phone, private data, and capabilities packages. Each package is summarized below in terms of the features it brings to JTAPI, and is linked to a separate overview document and specifications.

The JTAPI package architecture is a two-way street for both implementations and applications. In other words, telephony server implementations choose which extension packages (in addition to the core package) they implement, based upon the capabilities of the underlying hardware. Applications choose the extension packages (in addition to the core package) they need to use to accomplish the desired tasks of the application. Applications may query the implementation for the extension packages the implementation supports, and the application developer does not need to concern himself/herself with the details of any packages not needed for the application.

Java Telephony Standard Extension Packages

Each JTAPI extension package has its own specification describing its extensions to the core API, and in most cases has its own separate overview document describing it. The chart below lists each extension package available, with a link to the individual overview document, if it exists.

<u>Call Control Package</u>	The <i>javax.telephony.callcontrol</i> package extends the core package by providing more advanced call-control features such as placing calls on hold, transferring telephone calls, and conferencing telephone calls. This package also provides a more detailed state model of telephone calls.
<u>Call Center Package</u>	The <i>javax.telephony.callcenter</i> package provides applications the ability to perform advanced features necessary for managing large call centers. Examples of these advanced features include: Routing, Automated Call Distribution (ACD), Predictive Calling, and associating application data with telephony objects.
<u>Media Package</u>	The <i>javax.telephony.media</i> Package provides applications access to the media streams associated with a telephone call. They are able to read and write data from these media streams. DTMF (touch-tone) and non-DTMF tone detection and generations is also provided in the <code>java.telephony.media</code> package
<u>Phone Package</u>	The <i>javax.telephony.phone</i> package permits applications to control the physical features of telephone hardware phone sets. Implementations may describe Terminals as collections of components, where each of these component-types has interfaces in this package.
<u>Capabilities Package</u>	The <i>javax.telephony.capabilities</i> package allows applications to query whether certain actions may be performed. Capabilities take two forms: <i>static</i> capabilities indicate whether an implementation supports a feature; <i>dynamic</i> capabilities indicate whether a certain action is allowable given the current state of the call model.
<u>Private Data Package</u>	The <i>javax.telephony.privatedata</i> package enables applications to communicate data directly with the underlying hardware switch. This data may be used to instruct the switch to perform a switch-specific action. Applications may also use the package to "piggy-back" a piece of data with a Java Telephony API object.

The Java Telephony Call Model

The JTAPI *call model* consists of a half-dozen Java objects. These objects are defined using Java interfaces in the core package. Each call model object represents either a physical or logical entity in the telephone world. The primary purpose of these call model objects is to describe telephone calls and the endpoints involved in a telephone call. These call model objects are related to one another in specific ways, which is summarized below and described in more detail in the core package specification.

The following diagram shows the JTAPI call model and the objects that compose the call model. A description of each object follows the diagram.

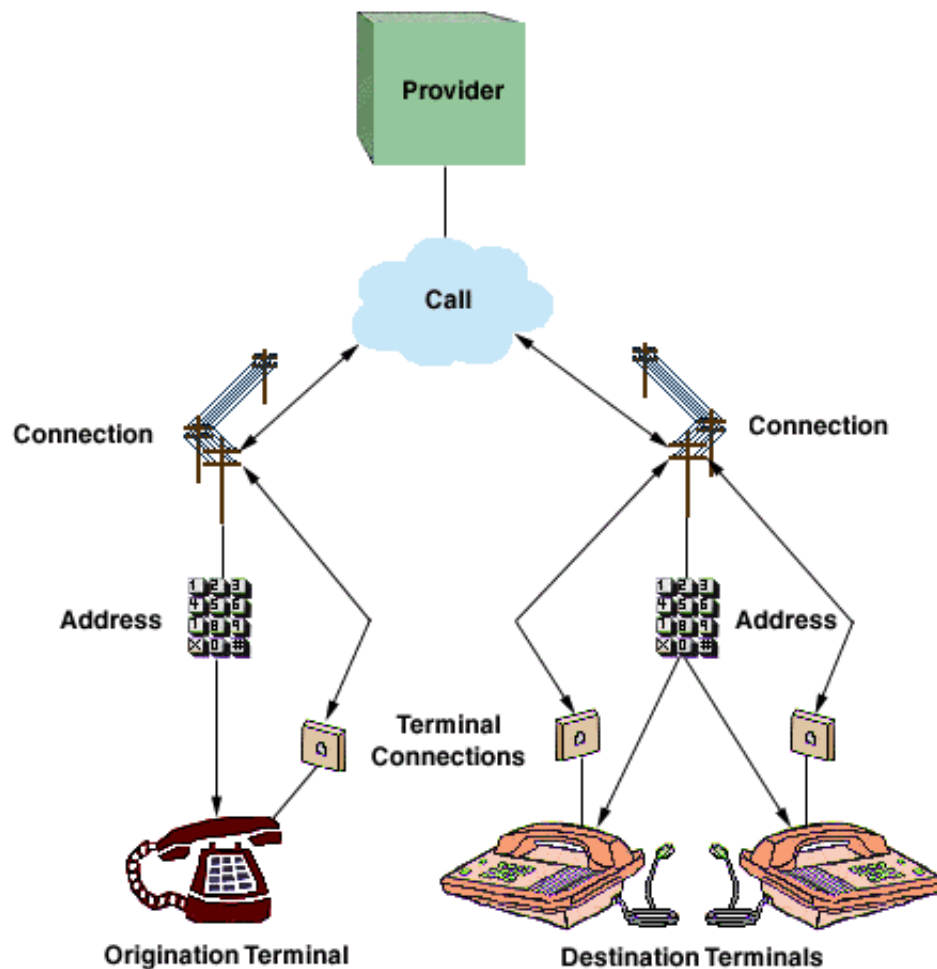


Figure 4: JTAPI Call Model

Provider Object

The Provider object is an abstraction of telephony service-provider software. The provider might manage a PBX connected to a server, a telephony/fax card in a desktop machine, or a computer networking technology, such as IP. A Provider hides the service-specific aspects of the telephony subsystem and enables Java applications and applets to interact with the telephony subsystem in a device-independent manner.

Call Object

The Call object represents a telephone call, the information flowing between the service provider and the call participants. A telephone call comprises a Call object and zero or more connections. In a two-party call scenario, a telephone call has one Call object and two connections. A conference call is three or more connections associated with one Call object.

Address Object

The Address object represents a telephone number. It is an abstraction for the logical endpoint of a telephone call. Note that this is quite distinct from a physical endpoint. In fact, one address may correspond to several physical endpoints (i.e. Terminals)

Connection Object

A Connection object models the communication link between a Call object and an Address object. This relationship is also referred to as a "logical" view, because it is concerned with the relationship between the Call and the Address (i.e. a logical endpoint). Connection objects may be in one of several states, indicating the current state of the relationship between the Call and the Address. These Connection states are summarized later.

Terminal Object

The Terminal object represents a physical device such as a telephone and its associated properties. Each Terminal object may have one or more Address Objects (telephone numbers) associated with it, as in the case of some office phones capable of managing multiple call appearances. The Terminal is also known as the "physical" endpoint of a call, because it corresponds to a physical piece of hardware.

TerminalConnection Object

TerminalConnection objects model the relationship between a Connection and the physical endpoint of a Call, which is represented by the Terminal object. This relationship is also known as the "physical" view of the Connection (in contrast to the Connection, which models the logical view). The TerminalConnection describes the current state of relationship between the Connection and a particular Terminal. The states associated with the TerminalConnection are described later in this document.

Core Package Methods

The core package defines three methods to support its primary features: placing a telephone call, answering a telephone call, and disconnecting a connection to a telephone call. These methods are **Call.connect()**, **TerminalConnection.answer()**, and **Connection.disconnect()**, respectively.

Call.connect()

Once an application has an idle call object (obtained via **Provider.createCall()**), it may place a telephone call using the **Call.connect()** method. The application must specify the originating Terminal (physical endpoint) and the originating Address (logical endpoint) on that Terminal (in the case that a Terminal has multiple telephone numbers on it). It also provides the destination telephone number string. Two Connection objects are returned from the **Call.connect()** method, representing the originating and destination ends of the telephone call.

TerminalConnection.answer()

Applications monitor with observers (discussed later) on Terminal for when incoming calls are presented. An incoming telephone call to a Terminal is indicated by a TerminalConnection to that Terminal in the RINGING state (see TerminalConnection states below). At that time, applications may invoke the **TerminalConnection.answer()** to answer that incoming telephone call.

Connection.disconnect()

The **Connection.disconnect()** method is used to remove an Address from the telephone call. The Connection object represents the relationship of that Address to the telephone call. Applications typically invoke this method when the Connection is in the CONNECTED state, resulting in the Connection moving to the DISCONNECTED state. In the core package, application may only remove entire Addresses from the Call, and all of the Terminals associated with that Address which are part of the call are removed as well. The call control extension package provides the ability for application to remove individual Terminals only from the Call.

Connection Object States

A Connection object is always in a *state* that reflects the relationship between a Call and an Address. The state in which a Connection exists is not only important to the application for information purposes, it is always an indication of which methods and actions can be invoked on the Connection object.

The state changes which Connection objects undergo are governed by rules shown below in a state transition diagram. This diagram guarantees to application developers the possible states in which the Connection object can transition given some current state. These state transition rules are invaluable to application developers. The diagram below shows the possible state transitions for the Connection object. Following this diagram is a brief summary of the meaning of each state.

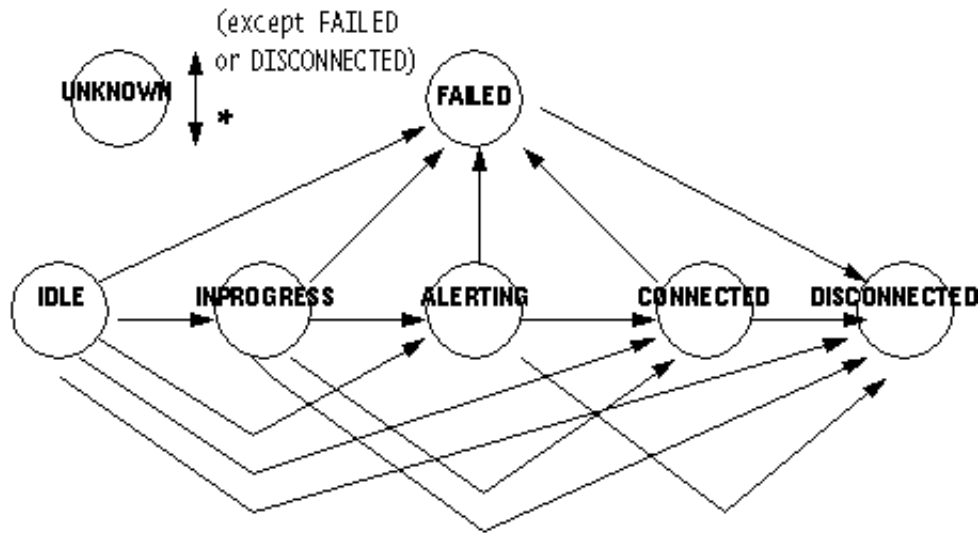


Figure 5: Connection State Transitions

IDLE state

The IDLE state is the initial state for all new Connection objects. Connections typically transition quickly out of the IDLE state into another state. A Connection in the IDLE state indicates that the party has just joined the telephone call in some form. No Core methods are valid on Connections in the IDLE state.

INPROGRESS state

The INPROGRESS state indicates that a telephone call is currently being placed to this destination endpoint.

ALERTING state

The ALERTING state indicates that the destination party of a telephone call is being alerted to an incoming telephone call.

CONNECTED state

The CONNECTED state indicates that a party is actively part of a telephone call. A Connection in the CONNECTED state implies that the associated party is talking to the other parties on the call or is connected to tone.

DISCONNECTED state

The DISCONNECTED state indicates that a party is no longer a part of a telephone call. No methods are valid for Connections in the DISCONNECTED state.

FAILED state

The FAILED state indicates that a telephone call placed to the endpoint has failed. For example, if an application uses Call.connect() to place a telephone call to a party who is busy, the Connection associated with the called party transitions into the FAILED state.

UNKNOWN state

The UNKNOWN state indicates that the Provider cannot determine the state of the Connection at the present time. A Connection may transition in and out of the UNKNOWN state at any time, unless it is in either the DISCONNECTED or FAILED state. The effects of the invocation of any method on a Connection in this state are unpredictable.

TerminalConnection Object States

The TerminalConnection object represents the relationship between a Terminal and a Connection. As mentioned previously, these objects represent a physical view of the Call, describing which physical Terminal endpoints are part of the telephone call. Similar to Connection objects, TerminalConnection objects have their own set of states and state transition diagram. This state transition diagram, with a brief description of each state follows.

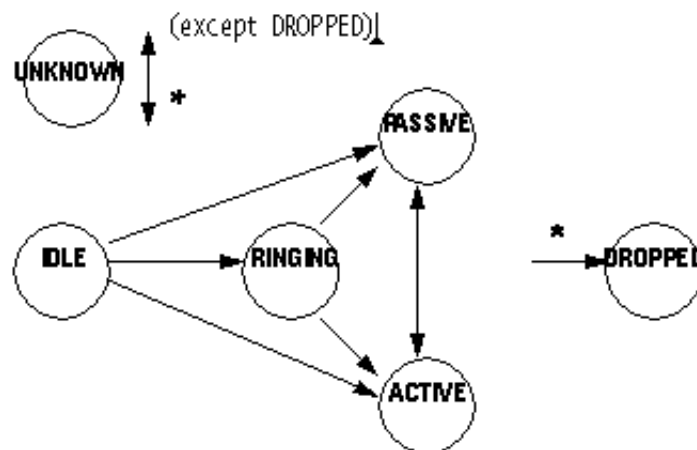


Figure 6: TerminalConnection state transitions

IDLE state

The IDLE state is the initial state for all TerminalConnection objects. It has the same connotation for the Connection object's IDLE state.

ACTIVE state

The ACTIVE state indicates a Terminal is actively part of a telephone call. This often implies that the Terminal handset is off-hook.

RINGING state

The RINGING state indicates that a Terminal is signaling to a user that an incoming telephone call is present at the Terminal.

DROPPED state

The DROPPED state indicates that a Terminal was once part of a telephone call, but has since dropped off of that telephone call. The DROPPED state is the final state for all TerminalConnections.

PASSIVE state

The PASSIVE state indicates a Terminal is part of a telephone call, but not actively so. A TerminalConnection in the PASSIVE state indicates that a resource on the Terminal is being used by this telephone call. Packages providing advanced features permit Terminals to join calls from the PASSIVE state.

UNKNOWN state

The UNKNOWN state indicates that the Provider is unable to determine the current state of a TerminalConnection. It has a similar connotation to that of the Connection object's UNKNOWN state.

Placing a Telephone Call

The past several sections have outlined the JTAPI call model, the essential methods in the core package, and the Connection and TerminalConnection states. This section ties all of this information together, presenting a common scenario found in most telephony applications. This section describes the state changes the entire call model typically undergoes when an application places a simple telephone call. Readers will come away with a coherent understanding of the call model changes for this simple example.

The vehicle used to describe the state changes undergone by the call model is the diagram below. This diagram is a *call model timing diagram*, where changes in the various objects are depicted as times increases down the vertical axis. Such a diagram is given below describing the typical state changes after an application invokes the **Call.connect()** method.

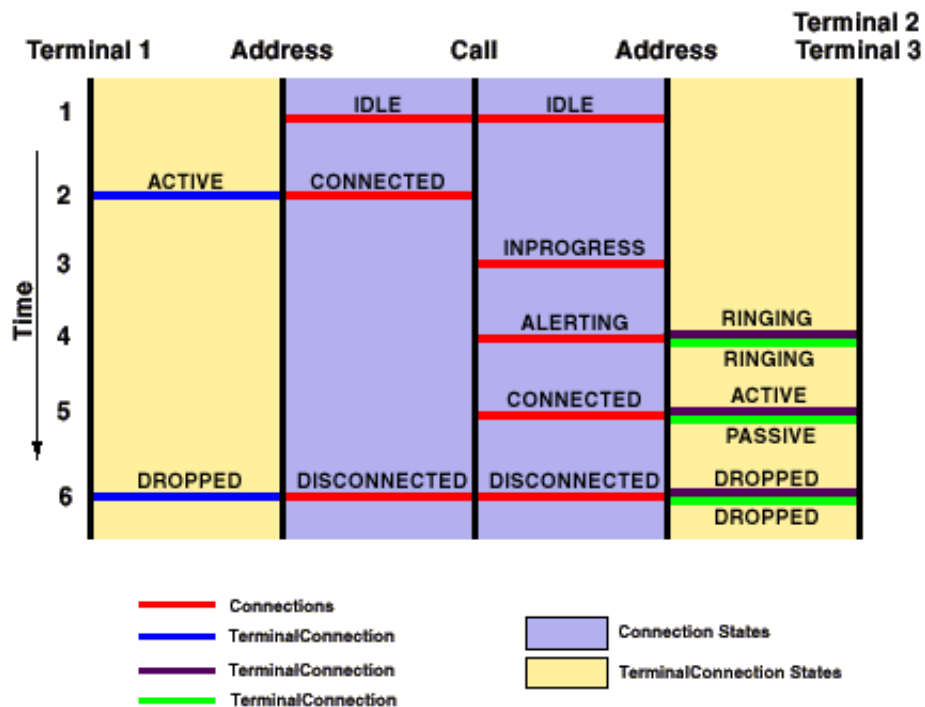


Figure 7: Call Model timing diagram

In the diagram above, discrete time steps are denoted by integers down the vertical axis. Time increases down this axis, but the integers are not meant to indicate real (clock) time.

This diagram, as a whole, represents a single telephone Call. In this case, the diagram represents a two-party telephone call (The **Call.connect()** method always results in a two-party call). The diagram may be broken into two parts: the left half and the right half. The left half represents the originating-end of the telephone call and the right half represents the destination-end of the telephone call.

On the left-hand (originating) side of the diagram, the two vertical lines represent the originating Terminal and Address (which are arguments to the **Call.connect()** method) objects, as indicated on the diagram. The horizontal lines represent either Connection objects or TerminalConnection objects as marked. Note that the Connection objects are drawn in the inner- most regions, whereas the TerminalConnection objects are drawn in the outer- most regions.

Similarly, on the right-hand (destination) side of the diagram, the two vertical lines represent the destination Address and Terminals. In this example, there are two destination Terminals associated with the destination Address. This configuration has been depicted previously in Figure 4. Note that since there

are two Terminals, there are two TerminalConnection objects on the destination side.

This diagram can be read as follows: as time passes the Connection and TerminalConnection objects change states. The appearance of a new Connection or TerminalConnection horizontal line corresponds to a new object of that type being created.

In the example of placing a telephone call, we see that after the two Connections are created in the IDLE state, the originating Connection transitions to the CONNECTED state, while the destination Connection transitions to the INPROGRESS state. At that time, a TerminalConnection to the originating Terminal is created and transitions to the ACTIVE state. When the destination Connection transitions to the ALERTING state, two TerminalConnections are created in the RINGING state.

At this point, a person at one of the destination Terminals answers the call. When this happens, that TerminalConnection moves to the ACTIVE state, and the other TerminalConnection moves to the PASSIVE state. At the same time, the destination Connection concurrently moves to the CONNECTED state. When the telephone call ends, all Connections move to the DISCONNECTED state, and all TerminalConnections move to the DROPPED state.

As a final point, this document has used the terms "logical" and "physical" view of a telephone call. This diagram makes these concepts clear. An application can monitor the state changes of the Connection object (i.e. the logical view). By looking at the diagram, the reader can understand that these states provide a higher-level view of the progress of the telephone call. The TerminalConnection state changes represent the physical view. By monitoring the TerminalConnection state changes, applications can find out what is happening at each physical endpoint.

The Java Telephony Observer Model

The Java Telephony API uses the Java observer/observable model to asynchronously notify the application of various changes in the JTAPI call model. These changes may include the state change of an object or the creation of an object.

The Provider, Call, Terminal, and Address objects have Observers. The interfaces corresponding to these observers are ProviderObserver, CallObserver, TerminalObserver, and AddressObserver, respectively.

The ProviderObserver reports all state changes to the Provider object. For the core package, state changes are reported when the Provider changes state from OUT_OF_SERVICE, to IN_SERVICE, to SHUTDOWN.

The Call observer reports state change information for all Connections and TerminalConnections that are part of the telephone call as well as state changes to the Call itself. These state changes are reported on neither the Address nor the Terminal observers.

At times, the application may want to monitor Address or Terminal objects for incoming telephone calls. In these instances, the application uses the **Address.addCallObserver()** or the **Terminal.addCallObserver()** methods. These methods instruct the implementation to automatically add a CallObserver to any calls that come to an Address or Terminal. These CallObservers are removed once the call leaves the Address or Terminal.

The Address and Terminal observers report any state changes in these objects. In the core package there are no events for these objects. The AddressObserver and TerminalObserver interfaces still exist, however, so other packages may extend these interfaces.

Application Code Examples

This section presents two application code examples. The first places a telephone call, and the second answers an incoming telephone call to a Terminal.

Note that the outgoing application code example does not make any blocking JTAPI calls in observer code - it only examines the evlist it receives as a parameter - while the incoming call application code uses "inner classes", defined in JDK™ 1.1, to avoid making blocking JTAPI calls in Observer code.

Outgoing Telephone Call Example

The following code example places a telephone call using the core Call.connect() method. It, however, looks for the states provided by the Call Control package.

```
import javax.telephony.*;
import javax.telephony.events.*;

/*
 * The MyOutCallObserver class implements the CallObserver
 * interface and receives all events associated with the Call.
 */

public class MyOutCallObserver implements CallObserver {

    public void callChangedEvent(CallEv[] evlist) {

        for (int i = 0; i < evlist.length; i++) {

            if (evlist[i] instanceof ConnEv) {

                String name = null;
                try {
                    Connection connection = ((ConnEv)evlist[i]).getConnection();
                    Address addr = connection.getAddress();
                    name = addr.getName();
                } catch (Exception excp) {
                    // Handle Exceptions
                }
                String msg = "Connection to Address: " + name + " is ";

                if (evlist[i].getID() == ConnAlertingEv.ID) {
                    System.out.println(msg + "ALERTING");
                }
                else if (evlist[i].getID() == ConnInProgressEv.ID) {
                    System.out.println(msg + "INPROGRESS");
                }
                else if (evlist[i].getID() == ConnConnectedEv.ID) {
                    System.out.println(msg + "CONNECTED");
                }
            }
        }
    }
}
```

```

    }
    else if (evlist[i].getID() == ConnDisconnectedEv.ID) {
        System.out.println(msg + "DISCONNECTED");
    }
}
}
}
}
}

```

```

import javax.telephony.*;
import javax.telephony.events.*;
import MyOutCallObserver;

```

```

/*
 * Places a telephone call from 476111 to 5551212
 */
public class Outcall {

    public static final void main(String args[]) {

        /*
         * Create a provider by first obtaining the default implementation of
         * JTAPI and then the default provider of that implementation.
         */
        Provider myprovider = null;
        try {
            JtapiPeer peer = JtapiPeerFactory.getJtapiPeer(null);
            myprovider = peer.getProvider(null);
        } catch (Exception excp) {
            System.out.println("Can't get Provider: " + excp.toString());
            System.exit(0);
        }

        /*
         * We need to get the appropriate objects associated with the
         * originating side of the telephone call. We ask the Address for a list
         * of Terminals on it and arbitrarily choose one.
         */
        Address origaddr = null;
        Terminal origterm = null;
        try {
            origaddr = myprovider.getAddress("4761111");

            /* Just get some Terminal on this Address */
            Terminal[] terminals = origaddr.getTerminals();
            if (terminals == null) {
                System.out.println("No Terminals on Address.");
                System.exit(0);
            }
            origterm = terminals[0];
        } catch (Exception excp) {
            // Handle exceptions;
        }
    }
}

```

```

    }

    /*
     * Create the telephone call object and add an observer.
     */
    Call mycall = null;
    try {
        mycall = myprovider.createCall();
        mycall.addObserver(new MyOutCallObserver());
    } catch (Exception excp) {
        // Handle exceptions
    }

    /*
     * Place the telephone call.
     */
    try {
        Connection c[] = mycall.connect(origterm, origaddr, "5551212");
    } catch (Exception excp) {
        // Handle all Exceptions
    }
}
}

```

Incoming Telephone Call Example

The following code example illustrates how an application answers a Call at a particular Terminal. It shows how applications accept calls when (and if) offered. This code example greatly resembles the core InCall code example.

```

import javax.telephony.*;
import javax.telephony.events.*;

import javax.telephony.*;
import javax.telephony.events.*;

/*
 * The MyInCallObserver class implements the CallObserver and
 * receives all Call-related events.
 */

public class MyInCallObserver implements CallObserver {

    public void callChangedEvent(CallEv[] evlist) {
        TerminalConnection termconn;
        String name;
        for (int i = 0; i < evlist.length; i++) {

            if (evlist[i] instanceof TermConnEv) {
                termconn = null;
                name = null;

                try {
                    TermConnEv tcev = (TermConnEv)evlist[i];

```

```

        Terminal term = termconn.getTerminal();
        termconn = tcev.getTerminalConnection();
        name = term.getName();
    } catch (Exception excp) {
        // Handle exceptions.
    }

    String msg = "TerminalConnection to Terminal: " + name + " is ";

    if (evlist[i].getID() == TermConnActiveEv.ID) {
        System.out.println(msg + "ACTIVE");
    }
    else if (evlist[i].getID() == TermConnRingingEv.ID) {
        System.out.println(msg + "RINGING");

        /* Answer the telephone Call using "inner class" thread */
        try {
            final TerminalConnection _tc = termconn;
            Runnable r = new Runnable() {
                public void run(){
                    try{
                        _tc.answer();
                    } catch (Exception excp){
                        // handle answer exceptions
                    }
                }
            };

            Thread T = new Thread(r);
            T.start();
        } catch (Exception excp) {
            // Handle Exceptions;
        }
    } else if (evlist[i].getID() == TermConnDroppedEv.ID) {
        System.out.println(msg + "DROPPED");
    }
}
}
}
}

-----

import javax.telephony.*;
import javax.telephony.events.*;
import MyInCallObserver;

/*
 * Create a provider and monitor a particular terminal for an incoming call.
 */
public class Incall {

    public static final void main(String args[]) {

        /*
         * Create a provider by first obtaining the default implementation of
         * JTAPI and then the default provider of that implementation.

```

```

    */
    Provider myprovider = null;
    try {
        JtapiPeer peer = JtapiPeerFactory.getJtapiPeer(null);
        myprovider = peer.getProvider(null);
    } catch (Exception excp) {
        System.out.println("Can't get Provider: " + excp.toString());
        System.exit(0);
    }

    /*
    * Get the terminal we wish to monitor and add a call observer to that
    * Terminal. This will place a call observer on all call which come to
    * that terminal. We are assuming that Terminals are named after some
    * primary telephone number on them.
    */
    try {
        Terminal terminal = myprovider.getTerminal("4761111");
        terminal.addCallObserver(new MyInCallObserver());
    } catch (Exception excp) {
        System.out.println("Can't get Terminal: " + excp.toString());
        System.exit(0);
    }
}
}
}

```

Locating and Obtaining Providers

The Java Telephony API defines a convention by which telephony server implementations of JTAPI make their services available to applications.

The two elements that link an application to a server are:

JtapiPeerFactory

The JtapiPeerFactory class is the first point of contact for an application that needs telephony services. It has the ability to return a named JtapiPeer object or a default JtapiPeer object. It is defined as a static class.

JtapiPeer

The JtapiPeer interface is the basis for a vendor's particular implementation of the Java Telephony API. Each vendor that provides an implementation of JTAPI must implement this interface in a class that can be loaded by the JtapiPeerFactory.

It is through a class that implements the JtapiPeer object that an application gets a Provider object.

JtapiPeerFactory: Getting Started

The `JtapiPeerFactory` is a static class defined in JTAPI. Its sole public method, `getJtapiPeer()` gets the `JtapiPeer` implementation requested or it returns a default implementation.

`getJtapiPeer()` takes the name of the desired JTAPI server implementation class as a parameter to return an object instance of that class. If no name is provided, `getJtapiPeer()` returns the default JTAPI server implementation object.

JtapiPeer: Getting a Provider Object

`JtapiPeer` is an interface. It is used by the JTAPI server implementors. It defines the methods that applications use to get Provider objects, to query services available on those providers, and to get the name of the `JtapiPeer` object instance. By creating a class that implements the `JtapiPeer` interface, JTAPI implementations make the following methods available to applications.

Applications use the `JtapiPeer.getProvider()` method to obtain new Provider objects. Each implementation may support one or more different "services" (e.g. for different types of underlying network substrate). A list of available services can be obtained via the `JtapiPeer.getServices()` method.

Applications may also supply optional arguments to the Provider. These arguments are appended to the string argument passed to the `JtapiPeer.getProvider()` method. The string argument has the following format:

< service name > ; arg1 = val1; arg2 = val2; ...

Where < service name > is not optional, and each optional argument pair which follows is separated by a semi-colon. The keys for these arguments are implementation specific, except for two standard-defined keys:

1. login: provides the login user name to the Provider.
2. passwd: provides a password to the Provider.

Applications use the `JtapiPeer.getName()` method to get the name of this `JtapiPeer` object instance. It has a *name* parameter, which is the same name used as an argument to the `JtapiPeerFactory.getJtapiPeer()` method.

Security in the Java Telephony API

JTAPI peer implementations use the Java "sandbox" model for controlling access to sensitive operations. Callers of JTAPI methods are categorized as "trusted" or "untrusted", using criteria determined by the runtime system. Trusted callers are allowed full access to JTAPI functionality. Untrusted callers are limited to operations that cannot compromise the system's integrity.

JTAPI may be used to access telephony servers or implementations that provide their own security mechanisms. These mechanisms remain in place; parameters such as user name and password are provided through parameters on the `JtapiPeer.getProvider()` method.

Hierarchy For Package javax.telephony

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
 - class javax.telephony.**JtapiPeerFactory**
 - class java.lang.Throwable (implements java.io.Serializable)
 - class java.lang.Exception
 - class javax.telephony.**InvalidArgumentException**
 - class javax.telephony.**InvalidPartyException**
 - class javax.telephony.**InvalidStateException**
 - class javax.telephony.**JtapiPeerUnavailableException**
 - class javax.telephony.**MethodNotSupportedException**
 - class javax.telephony.**PrivilegeViolationException**
 - class javax.telephony.**ResourceUnavailableException**
 - class java.lang.RuntimeException
 - class javax.telephony.**PlatformException**
 - class javax.telephony.**ProviderUnavailableException**

Interface Hierarchy

- interface javax.telephony.**Address**
- interface javax.telephony.**AddressObserver**
- interface javax.telephony.**Call**
- interface javax.telephony.**CallObserver**
- interface javax.telephony.**Connection**
- interface javax.telephony.**Event**
 - interface javax.telephony.**AddressEvent**
 - interface javax.telephony.**CallEvent**
 - interface javax.telephony.**ConnectionEvent**
 - interface javax.telephony.**TerminalConnectionEvent**
 - interface javax.telephony.**MetaEvent**
 - interface javax.telephony.**MultiCallMetaEvent**
 - interface javax.telephony.**SingleCallMetaEvent**
 - interface javax.telephony.**ProviderEvent**
 - interface javax.telephony.**TerminalEvent**
- interface java.util.EventListener
 - interface javax.telephony.**AddressListener**

- interface javax.telephony.**CallListener**
 - interface javax.telephony.**ConnectionListener**
 - interface javax.telephony.**TerminalConnectionListener**
 - interface javax.telephony.**ProviderListener**
 - interface javax.telephony.**TerminalListener**
 - interface javax.telephony.**JtapiPeer**
 - interface javax.telephony.**Provider**
 - interface javax.telephony.**ProviderObserver**
 - interface javax.telephony.**Terminal**
 - interface javax.telephony.**TerminalConnection**
 - interface javax.telephony.**TerminalObserver**
-
-

javax.telephony **Interface Address**

All Known Subinterfaces:

CallControlAddress, MobileAddress

public interface **Address**

An `Address` object represents what we commonly think of as a "telephone number".

Introduction

In implementations where the underlying network is not a telephone network, this address may represent something else. For example, if the underlying network is IP, this address might represent an IP address (e.g. 18.203.0.49). An `Address` object has a string *name* which corresponds to this telephone address. The `Address` object does not attempt to interpret this string in any way. This name is first assigned when the `Address` object is created and does not change throughout the lifetime of the object. The method `Address.getName()` returns the name of the `Address` object.

`Address` objects may be classified into two categories: *local* and *remote*. Local `Address` objects are those addresses which are part of the Provider's local domain. These `Address` objects are created by the implementation of the `Provider` object when it is first instantiated. All of the Provider's local addresses are reported via the `Provider.getAddresses()` method. Remote `Address` objects are those outside of the Provider's domain which the Provider learns about during its lifetime through various happenings (e.g. an incoming call from a currently unknown address). Remote `Addresses` are **not** reported via the `Provider.getAddresses()` method. Note that applications never explicitly create new `Address` objects.

Address and Terminal Objects

`Address` and `Terminal` objects exist in a many-to-many relationship. An `Address` object may have zero or more `Terminals` associated with it. Each `Terminal` associated with an `Address` must reflect its association with the `Address`. Since the implementation creates `Address` (and `Terminal`) objects, it is responsible for insuring the correctness of these relationships. The `Terminals` associated with an `Address` is given by the `Address.getTerminals()` method.

An association between an `Address` and `Terminal` indicates that the `Terminal` is addressable via that `Address`. In many instances, a telephone set (represented by a `Terminal` object) has only one telephone number (represented by an `Address` object) associated with it. In more complex configurations, telephone sets may have several telephone numbers associated with them. Likewise, a telephone number may appear on more than one telephone set. For example, feature phones in PBX environments may exhibit this configuration.

Address and Call Objects

Address objects represent the *logical* endpoints of a telephone call. A logical view of a telephone call views the call as originating from one Address endpoint and terminates at another Address endpoint.

Address objects are related to Call objects via the Connection object. The Connection object has a *state* which describes the current relationship between the Call and the Address. Each Address object may be part of more than one telephone call, and in each case, is represented by a separate Connection object. The `Address.getConnections()` method returns all Connection objects currently associated with the Call.

An Address is associated with a Call until the Connection moves into the `Connection.DISCONNECTED` state. At that time, the Connection is no longer reported via the `Address.getConnections()` method. Therefore, the `Address.getConnections()` method will never report a Connection in the `Connection.DISCONNECTED` state.

Existing Telephone Calls

The Java Telephony API specification states that the implementation is responsible for reporting all existing telephone calls when a Provider is first created. This implies that an Address object must report information regarding existing telephone calls to that Address. In other words, Address objects must reports all Connection objects which represent existing telephone calls.

Address Listeners and Events

All changes in an Address object are reported via the AddressListener interface. Applications instantiate an object which implements this interface and begins this delivery of events to this object using the `Address.addAddressListener()` method. All Address-related events extend the `AddressEvent` interface provided in the core package. Applications receive events on an listener until the listener is removed via the `Address.removeAddressListener()` method or until the Address is no longer observable. In these instances, each AddressListener receives a `ADDRESS_EVENT_TRANSMISSION_ENDED` as its final event.

Currently in the core package, the only Address-related event is `ADDRESS_EVENT_TRANSMISSION_ENDED`.

Call Listeners

At times, applications may want to monitor a particular Address for all Calls which come to that Address. For example, a customer service agent application is only interested in telephone calls associated with a particular agent address. To achieve this sort of Address-based Call monitoring applications may *add* CallListeners to an Address via the `Address.addCallListener()` method.

When a CallListener is added to an Address, this listener instance is immediately added to all Calls at this Address and is added to all Calls which come to this Address in the future. These listeners remain on the telephone call as long as the Address is associated with the telephone call.

The specification of `Address.addCallListener()` contains more precise semantics.

See Also:

[AddressListener](#), [CallListener](#)

Method Summary	
<code>void</code>	<u>addAddressListener</u> (<u>AddressListener</u> listener) Adds an listener to the Address.
<code>void</code>	<u>addCallListener</u> (<u>CallListener</u> listener) Adds an listener to a Call object when this Address object first becomes part of that Call.
<code>void</code>	<u>addCallObserver</u> (<u>CallObserver</u> observer) Adds an observer to a Call object when this Address object first becomes part of that Call.
<code>void</code>	<u>addObserver</u> (<u>AddressObserver</u> observer) Adds an observer to the Address.
<u>AddressCapabilities</u>	<u>getAddressCapabilities</u> (<u>Terminal</u> terminal) Deprecated. Since JTAPI v1.2. This method has been replaced by the <code>Address.getCapabilities()</code> method.
<u>AddressListener</u> []	<u>getAddressListeners</u> () Returns a list of all JAddressListeners associated with this Address object.
<u>CallListener</u> []	<u>getCallListeners</u> () Returns a list of all CallListeners associated with this Address object.
<u>CallObserver</u> []	<u>getCallObservers</u> () Returns a list of all CallObservers associated with this Address object.
<u>AddressCapabilities</u>	<u>getCapabilities</u> () Returns the dynamic capabilities for this instance of the Address object.
<u>Connection</u> []	<u>getConnections</u> () Returns an array of Connection objects currently associated with this Address object at the instant the <code>getConnections()</code> method is called.
<code>java.lang.String</code>	<u>getName</u> () Returns the name of the Address.
<u>AddressObserver</u> []	<u>getObservers</u> () Returns a list of all AddressObservers associated with this Address object.
<u>Provider</u>	<u>getProvider</u> () Returns the Provider associated with this Address.

<u>Terminal</u> []	getTerminals() Returns an array of Terminals associated with this Address object.
void	removeAddressListener (<u>AddressListener</u> listener) Removes the given listener from the Address.
void	removeCallListener (<u>CallListener</u> listener) Removes the given CallListener from the Address.
void	removeCallObserver (<u>CallObserver</u> observer) Removes the given CallObserver from the Address.
void	removeObserver (<u>AddressObserver</u> observer) Removes the given observer from the Address.

Method Detail

getName

public java.lang.String **getName()**

Returns the name of the Address. Each Address possesses a unique name. This name is a way of referencing an endpoint in a telephone call.

Returns:

The name of this Address.

getProvider

public Provider **getProvider()**

Returns the Provider associated with this Address. This Provider object is valid throughout the lifetime of the Address and does not change once the Address is created.

Returns:

The Provider associated with this Address.

getTerminals

public Terminal[] **getTerminals()**

Returns an array of Terminals associated with this Address object. If no Terminals are associated with this Address, this method returns null. This list does not change throughout the lifetime of the Address object.

Returns:

An array of Terminal objects associated with this Address.

getConnections

```
public Connection[] getConnections()
```

Returns an array of Connection objects currently associated with this Address object at the instant the `getConnections()` method is called. When a Connection moves into the `Connection.DISCONNECTED` state, the Address object loses the reference to the Connection and the Connection no longer returned by this method. Therefore, all Connections returned by this method will never be in the `Connection.DISCONNECTED` state. If the Address has no Connections associated with it, this method returns null.

Post-conditions:

1. Let `Connection c[] = this.getConnections()`
2. `c == null` or `c.length >= 1`
3. For all `i`, `c[i].getState() != Connection.DISCONNECTED`

Returns:

An array of Connection objects associated with this Address.

addObserver

```
public void addObserver(AddressObserver observer)  
    throws ResourceUnavailableException,  
           MethodNotSupportedException
```

Adds an observer to the Address. The AddressObserver reports all Address- related state changes as events. The Address object will report events to this AddressObserver object for the lifetime of the Address object or until the observer is removed with the `Address.removeObserver()` method or until the Address is no longer observable. In these instances, a `AddrObservationEndedEv` is delivered to the observer as its final event. The observer will receive no events after `AddrObservationEndedEv` unless the observer is explicitly re-added via the `Address.addObserver()` method. Also, once an observer receives an `AddrObservationEndedEv`, it will no longer be reported via the `Address.getObservers()`.

If an application attempts to add an instance of an observer already present on this Address, this attempt will silently fail, i.e. multiple instances of an observer are not added and no exception will be thrown.

Currently, only the `AddrObservationEndedEv` event is defined by the core package and delivered to the AddressObserver.

Post-conditions:

1. `observer` is an element of `this.getObservers()`

Parameters:

`observer` - The observer being added.

Throws:

MethodNotSupportedException - The observer cannot be added at this time

ResourceUnavailableException - The resource limit for the number of observers has been exceeded.

getObservers

```
public AddressObserver[] getObservers()
```

Returns a list of all AddressObservers associated with this Address object. If there are no observers associated with this Address object, this method returns null.

Post-conditions:

1. Let AddressObserver[] obs = this.getObservers()
2. obs == null or obs.length >= 1

Returns:

An array of AddressObserver objects associated with this Address.

removeObserver

```
public void removeObserver(AddressObserver observer)
```

Removes the given observer from the Address. If successful, the observer will no longer receive events generated by this Address object. As its final event, the AddressObserver receives is an AddrObservationEndedEv event.

If an observer is not part of the Address, then this method fails silently, i.e. no observer is removed and no exception is thrown.

Post-conditions:

1. An AddrObservationEndedEv event is reported to the observer as its final event.
2. observer is not an element of this.getObservers()

Parameters:

`observer` - The observer which is being removed.

addCallObserver

```
public void addCallObserver(CallObserver observer)
    throws ResourceUnavailableException,
           MethodNotSupportedException
```


Adds an observer to a Call object when this Address object first becomes part of that Call. This method permits applications to select an Address object in which they are interested and automatically have the implementation attach an observer to all present and future Calls which come to this Address.

JTAPI v1.0 Semantics

In version 1.0 of the Java Telephony API specification, the application monitored the Address object for the AddrCallAtAddrEv event. This event indicated that a Call has come to this Address. Then, applications would manually add an observer to the Call. With this architecture, potentially dangerous race conditions existed while an application was adding an observer to the Call. As a result, this mechanism has been replaced in version 1.1.

JTAPI v1.1 Semantics

In version 1.1 of the specification, the AddrCallAtAddrEv event does not exist and this method replaces the functionality described above. Instead of monitoring for a AddrCallAtAddrEv event, this application simply uses the `Address.addCallObserver()` method, and observer will be added to new telephone calls at this Address automatically.

If an application attempts to add an instance of a call observer already present on this Address, these repeated attempts will silently fail, i.e. multiple instances of a call observer are not added and no exception will be thrown.

When a call observer is added to an Address with this method, the following behavior is exhibited by the implementation.

1. It is immediately associated with any existing calls at the Address and a snapshot of those calls are reported to the call observer. For each of these calls, the observer is reported via `Call.getObservers()`.
2. It is associated with all future calls which comes to this Address. For each new call, the observer is reported via `Call.getObservers()`.

Note that the definition of the term ".. when a call is at an Address" means that the Call contains one Connection which has this Address as its Address.

Call Observer Lifetime

For all call observers which are present on Calls because of this method, the following behavior is exhibited with respect to the lifetime of the call.

1. The call observer receives events until the Call is no longer at this Address. In this case, the call observer will be re-applied to the Call if the Call returns to this Address at some point in the future.
2. The call observer is removed with `Call.removeObserver()`. Note that this only affects the instance of the call observer for that particular call. If the Call subsequently leaves and returns to the Address, the observer will be re-applied.
3. The Call can no longer be monitored by the implementation.
4. The Call moves into the `Call.INVALID` state.

When the CallObserver leaves the Call because of one of the reasons above, it receives a CallObservationEndedEv as its final event.

Call Observer on Multiple Addresses and Terminals

It is possible for an application to add CallObservers at more than one Address and Terminal (using Address.addCallObserver() and Terminal.addCallObserver(), respectively). The rules outlined above still apply, with the following additions:

1. A CallObserver is not added to a Call more than once, even if it has been added to more than one Address/Terminal which are present on the Call.
2. The CallObserver leaves the call only if *all* of the Addresses and Terminals on which the Call Observer was added leave the Call. If one of those Addresses/Terminals becomes part of the Call again, the call observer is re-applied to the Call.

Post-Conditions:

1. observer is an element of this.getCallObservers()
2. observer is an element of Call.getObservers() for each Call associated with the Connections from this.getConnections().
3. An array of snapshot events are reported to the observer for existing calls associated with this Address.

Parameters:

observer - The observer being added.

Throws:

MethodNotSupportedException - The observer cannot be added at this time.

ResourceUnavailableException - The resource limit for the numbers of observers has been exceeded.

See Also:

Call

getCallObservers

```
public CallObserver[] getCallObservers()
```

Returns a list of all CallObservers associated with this Address object. That is, it returns a list of CallObserver object which have been added via the Address.addCallObserver() method. If there are no CallObservers associated with this Address object, this method returns null.

Post-conditions:

1. Let CallObserver[] obs = this.getCallObservers()
2. obs == null or obs.length >= 1

Returns:

An array of CallObserver objects associated with this Address.

removeCallObserver

```
public void removeCallObserver(CallObserver observer)
```

Removes the given CallObserver from the Address. In other words, it removes a CallObserver which was added via the `Address.addCallObserver()` method. If successful, the observer will no longer be added to new Calls which are presented to this Address, however it does not affect CallObservers which have already been added at a Call.

Also, if the CallObserver is not part of the Address, then this method fails silently, i.e. no observer is removed and no exception is thrown.

Post-conditions:

1. observer is not an element of `this.getCallObservers()`

Parameters:

observer - The CallObserver which is being removed.

getCapabilities

```
public AddressCapabilities getCapabilities()
```

Returns the dynamic capabilities for this instance of the Address object. Dynamic capabilities tell the application which actions are possible at the time this method is invoked based upon the implementations knowledge of its ability to successfully perform the action. This determination may be based upon argument passed to this method, the current state of the call model, or some implementation-specific knowledge. These indications do not guarantee that a particular method will succeed when invoked, however.

The dynamic Address capabilities require no additional arguments.

Returns:

The dynamic Address capabilities.

getAddressCapabilities

```
public AddressCapabilities getAddressCapabilities(Terminal terminal)  
throws InvalidArgumentException,  
       PlatformException
```

Deprecated. Since JTAPI v1.2. This method has been replaced by the `Address.getCapabilities()` method.

Gets the AddressCapabilities object with respect to a Terminal. If null is passed as a Terminal parameter, the general/provider-wide Address capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Address.getCapabilities()` method returns the dynamic Address capabilities. This method now should simply invoke the `Address.getCapabilities()` method.

Parameters:

`terminal` - This argument is ignored in JTAPI v1.2 and later.

Throws:

`InvalidArgumentException` - This exception is never thrown in JTAPI v1.2 and later.

`PlatformException` - A platform-specific exception occurred.

addAddressListener

```
public void addAddressListener(AddressListener listener)
                               throws ResourceUnavailableException,
                                       MethodNotSupportedException
```

Adds an listener to the Address. The AddressListener reports all Address- related state changes as events. The Address object will report events to this AddressListener object for the lifetime of the Address object or until the listener is removed with the `Address.removeAddressListener()` method or until the Address is no longer observable. In these instances, a `ADDRESS_EVENT_TRANSMISSION_ENDED` is delivered to the listener as its final event. The listener will receive no events after `ADDRESS_EVENT_TRANSMISSION_ENDED` unless the listener is explicitly re-added via the `Address.addAddressListener()` method. Also, once an listener receives an `ADDRESS_EVENT_TRANSMISSION_ENDED`, it will no longer be reported via the `Address.getAddressListeners()`.

If an application attempts to add an instance of an listener already present on this Address, this attempt will silently fail, i.e. multiple instances of an listener are not added and no exception will be thrown.

Currently, only the `ADDRESS_EVENT_TRANSMISSION_ENDED` event is defined by the core package and delivered to the AddressListener.

Post-conditions:

1. listener is an element of this.getAddressListeners()

Parameters:

`listener` - The listener being added.

Throws:

`MethodNotSupportedException` - The listener cannot be added at this time

`ResourceUnavailableException` - The resource limit for the number of listeners has been exceeded.

getAddressListeners

```
public AddressListener[] getAddressListeners()
```

Returns a list of all JAddressListeners associated with this Address object. If there are no listeners associated with this Address object, this method returns null.

Post-conditions:

1. Let AddressListener[] obs = this.getAddressListeners()
2. obs == null or obs.length >= 1

Returns:

An array of AddressListener objects associated with this Address.

removeAddressListener

```
public void removeAddressListener(AddressListener listener)
```

Removes the given listener from the Address. If successful, the listener will no longer receive events generated by this Address object. As its final event, the AddressListener receives is an ADDRESS_EVENT_TRANSMISSION_ENDED event.

If an listener is not part of the Address, then this method fails silently, i.e. no listener is removed and no exception is thrown.

Post-conditions:

1. An ADDRESS_EVENT_TRANSMISSION_ENDED event is reported to the listener as its final event.
2. listener is not an element of this.getAddressListeners()

Parameters:

listener - The listener which is being removed.

addCallListener

```
public void addCallListener(CallListener listener)  
    throws ResourceUnavailableException,  
           MethodNotSupportedException
```

Adds an listener to a Call object when this Address object first becomes part of that Call. This method permits applications to select an Address object in which they are interested and automatically have the implementation attach an listener to all present and future Calls which come to this Address.

JTAPI v1.0 Semantics

In version 1.0 of the Java Telephony API specification, the application monitored the Address object for the AddrCallAtAddrEv event. This event indicated that a Call has come to this Address. Then, applications would manually add an listener to the Call. With this architecture, potentially dangerous race conditions existed while an application was adding an listener to the Call. As a result, this mechanism has been replaced in version 1.1.

JTAPI v1.1 Semantics

In version 1.1 of the specification, the AddrCallAtAddrEv event does not exist and this method replaces the functionality described above. Instead of monitoring for a AddrCallAtAddrEv event, this application simply uses the `Address.addCallListener()` method, and listener will be added to new telephone calls at this Address automatically.

If an application attempts to add an instance of a call listener already present on this Address, these repeated attempts will silently fail, i.e. multiple instances of a call listener are not added and no exception will be thrown.

When a call listener is added to an Address with this method, the following behavior is exhibited by the implementation.

1. It is immediately associated with any existing calls at the Address and a snapshot of those calls are reported to the call listener. For each of these calls, the listener is reported via `Call.getAddressListeners()`.
2. It is associated with all future calls which comes to this Address. For each new call, the listener is reported via `Call.getAddressListeners()`.

Note that the definition of the term ".. when a call is at an Address" means that the Call contains one Connection which has this Address as its Address.

Call Listener Lifetime

For all call listeners which are present on Calls because of this method, the following behavior is exhibited with respect to the lifetime of the call.

1. The call listener receives events until the Call is no longer at this Address. In this case, the call listener will be re-applied to the Call if the Call returns to this Address at some point in the future.
2. The call listener is removed with `Call.removeCallListener()`. Note that this only affects the instance of the call listener for that particular call. If the Call subsequently leaves and returns to the Address, the listener will be re-applied.
3. The Call can no longer be monitored by the implementation.
4. The Call moves into the `Call.INVALID` state.

When the CallListener leaves the Call because of one of the reasons above, it receives a `CALL_EVENT_TRANSMISSION_ENDED` as its final event.

Call Listener on Multiple Addresses and Terminals

It is possible for an application to add CallListeners at more than one Address and Terminal (using `Address.addCallListener()` and `Terminal.addCallListener()`, respectively). The rules outlined above still apply, with the following additions:

1. A CallListener is not added to a Call more than once, even if it has been added to more than one Address/Terminal which are present on the Call.
2. The CallListener leaves the call only if *all* of the Addresses and Terminals on which the Call Listener was added leave the Call. If one of those Addresses/Terminals becomes part of the Call again, the call listener is re-applied to the Call.

Call Listener Event level of Granularity

The application shall receive the event reports associated with the methods which are defined in the CallListener interface that the application implements - either the CallListener, ConnectionListener or TerminalConnectionListener interface. Note that the CallListener, ConnectionListener and TerminalConnectionListener form a hierarchy of interfaces, and so provide successively increasing amounts of call information and successively finer-grained event granularity.

Post-Conditions:

1. listener is an element of `this.getCallListeners()`
2. listener is an element of `Call.getCallListeners()` for each Call associated with the Connections from `this.getConnections()`.
3. An array of snapshot events are reported to the listener for existing calls associated with this Address.

Parameters:

listener - The listener being added.

Throws:

MethodNotSupportedException - The listener cannot be added at this time.

ResourceUnavailableException - The resource limit for the numbers of listeners has been exceeded.

See Also:

Call

getCallListeners

```
public CallListener[] getCallListeners()
```

Returns a list of all CallListeners associated with this Address object. That is, it returns a list of CallListener object which have been added via the `Address.addCallListener()` method. If there are no CallListeners associated with this Address object, this method returns null.

Post-conditions:

1. Let `CallListener[] obs = this.getCallListeners()`
2. `obs == null` or `obs.length >= 1`

Returns:

An array of CallListener objects associated with this Address.

removeCallListener

```
public void removeCallListener(CallListener listener)
```

Removes the given CallListener from the Address. In other words, it removes a CallListener which was added via the `Address.addCallListener()` method. If successful, the listener will no longer be added to new Calls which are presented to this Address, however it does not affect CallListeners which have already been added at a Call.

Also, if the CallListener is not part of the Address, then this method fails silently, i.e. no listener is removed and no exception is thrown.

Post-conditions:

1. listener is not an element of `this.getCallListeners()`

Parameters:

`listener` - The CallListener which is being removed.

javax.telephony **Interface AddressEvent**

public interface **AddressEvent**
extends [Event](#)

The `AddressEvent` interface is the base interface for all Address-related events. All Address-related events must extend this interface. Events which extend this interface are reported via the `AddressListener` interface.

An individual `AddressEvent` conveys one of a series of different possible Address state changes; the specific Address state change is indicated by the `Event.getID()` value returned by the event.

The only event ID defined in the core package for the Address is the `ADDRESS_EVENT_TRANSMISSION_ENDED` event.

This event ID is reported via a method in the `AddressListener` interface (see `CallListener.callInvalidEvent()`)

The `AddressEvent.getAddress()` method on this interface returns the Address associated with the Address event.

See Also:

[Event](#), [AddressListener](#), [Address](#)

Field Summary

static int	<code>ADDRESS_EVENT_TRANSMISSION_ENDED</code> The <code>ADDRESS_EVENT_TRANSMISSION_ENDED</code> event indicates that the application will no longer receive Address events on the instance of the <code>AddressListener</code> .
------------	--

Fields inherited from interface `javax.telephony.Event`

[CAUSE_CALL_CANCELLED](#), [CAUSE_DEST_NOT_OBTAINABLE](#),
[CAUSE_INCOMPATIBLE_DESTINATION](#), [CAUSE_LOCKOUT](#),
[CAUSE_NETWORK_CONGESTION](#), [CAUSE_NETWORK_NOT_OBTAINABLE](#),
[CAUSE_NEW_CALL](#), [CAUSE_NORMAL](#), [CAUSE_RESOURCES_NOT_AVAILABLE](#),
[CAUSE_SNAPSHOT](#), [CAUSE_UNKNOWN](#)

Method Summary

<u>Address</u>	<u>getAddress()</u> Returns the Address associated with this Address event.
----------------	---

Methods inherited from interface javax.telephony.Event

getCause, getID, getMetaEvent, getSource

Field Detail

ADDRESS_EVENT_TRANSMISSION_ENDED

```
public static final int ADDRESS_EVENT_TRANSMISSION_ENDED
```

The ADDRESS_EVENT_TRANSMISSION_ENDED event indicates that the application will no longer receive Address events on the instance of the AddressListener.

This constant indicates a specific event passed via a AddressEvent event, and is reported on the AddressListenerEnded method of the AddressListener interface.

Method Detail

getAddress

```
public Address getAddress()
```

Returns the Address associated with this Address event.

Returns:

The Address associated with this event.

javax.telephony **Interface AddressListener**

public interface **AddressListener**
extends java.util.EventListener

The `AddressListener` interface reports all changes which happen to the `Address` object.

Introduction

Such a state change is reported by a call to the method of this interface which corresponds to the type of state change that occurred. An event (an `AddressEvent`) which has an event ID which corresponds to the state change is passed as a parameter.

To receive all future events associated with an `Address` object, applications instantiate an object which implements this interface and then register their object as an `Address` Listener using the `Address.addListener()` method.

The `AddressListener` methods each receive one event at a time.

Address Observation Ending

At various times, the underlying implementation may not be able to observe the `Address`. In these instances, the `AddressListener` is sent an event with event ID `AddressEvent.ADDRESS_EVENT_TRANSMISSION_ENDED`. This indicates that the application will not receive further events associated with the `Address` object. The listener is no longer reported via the `Address.getAddressListeners()` method.

See Also:

[AddressEvent](#), [Address](#)

Method Summary

void	<code>addressListenerEnded</code> (<u>AddressEvent</u> event) The <code>ADDRESS_EVENT_TRANSMISSION_ENDED</code> event indicates that the application will no longer receive <code>Address</code> events on the instance of the <code>AddressListener</code> .
------	--

Method Detail

addressListenerEnded

public void **addressListenerEnded**(AddressEvent event)

The ADDRESS_EVENT_TRANSMISSION_ENDED event indicates that the application will no longer receive Address events on the instance of the AddressListener.

This constant indicates a specific event passed via a AddressEvent event, and is reported on the AddressListener interface.

See Also:

AddressEvent, Address

javax.telephony **Interface AddressObserver**

All Known Subinterfaces:

CallControlAddressObserver

public interface **AddressObserver**

The AddressObserver interface reports all changes which happen to the Address object.

Introduction

These changes are reported as events to the AddressObserver .addressChangedEvent () method. Applications must instantiate an object which implements this interface and then use the Address.addObserver () method to register the object to receive all future events associated with the Address object.

The AddressObserver .addressChangedEvent () method receives an array of events which all must extend the AddrEv interface. Since several changes may happen to a single JTAPI object at once, a list of events is needed to convey those changes which happen at the same time. Applications iterate through the array of events provided.

Address Observation Ending

At various times, the underlying implementation may not be able to observe the Address. In these instances, the AddressObserver is sent an AddrObservationEndedEv event. This indicates that the application will not receive further events associated with the Address object. The observer is no longer reported via the Address.getObservers () method.

See Also:

AddrEv, AddrObservationEndedEv

Method Summary

void	<u>addressChangedEvent</u> (AddrEv[] eventList) Reports all events associated with the Address object.
------	--

Method Detail

addressChangedEvent

public void **addressChangedEvent**(AddrEv[] eventList)

Reports all events associated with the Address object. This method passes an array of AddrEv objects as its arguments which correspond to the list of events representing the changes to the Address object.

Parameters:

eventList - The list of Address events.

javax.telephony **Interface Call**

All Known Subinterfaces:

CallControlCall

public interface **Call**

A `Call` object models a telephone call.

Introduction

A `Call` can have zero or more Connections. A two-party call has two `Connections`, and a conference call has three or more `Connections`. Each `Connection` models the relationship between a `Call` and an `Address`, where an `Address` identifies a particular party or set of parties on a `Call`.

Creating Call Objects

Applications create instances of a `Call` object with the `Provider.createCall()` method, which returns a `Call` object that has zero `Connections` and is in the `Call.IDLE` state. The `Call` maintains a reference to its `Provider` for the life of that `Call` object. This `Provider` object instance does not change throughout the lifetime of the `Call` object. The `Provider` associated with a `Call` is obtained via the `Call.getProvider()` method.

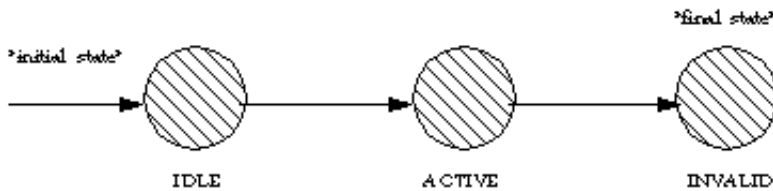
Call States

A `Call` has a *state* which is obtained via the `Call.getState()` method. This state describes the current progress of a telephone call, where it is in its life cycle, and how many `Connections` exist on the `Call`. The `Call` state may be one of three values: `Call.IDLE`, `Call.ACTIVE`, or `Call.INVALID`. The following is a description of each state:

<code>Call.IDLE</code>	This is the initial state for all <code>Calls</code> . In this state, the <code>Call</code> has zero <code>Connections</code> , that is <code>Call.getConnections()</code> <i>must</i> return null.
<code>Call.ACTIVE</code>	A <code>Call</code> with some current ongoing activity is in this state. <code>Calls</code> with one or more associated <code>Connections</code> must be in this state. If a <code>Call</code> is in this state, the <code>Call.getConnections()</code> method must return an array of size at least one.
<code>Call.INVALID</code>	This is the final state for all <code>Calls</code> . <code>Call</code> objects which lose all of their <code>Connections</code> objects (via a transition of the <code>Connection</code> object into the <code>Connection.DISCONNECTED</code> state) moves into this state. <code>Calls</code> in this state have zero <code>Connections</code> and these <code>Call</code> objects may not be used for any future action. In this state, the <code>Call.getConnections()</code> <i>must</i> return null.

Call State Transitions

The possible Call state transitions are given in the diagram below:



Calls and Connections

A Call maintain a list of the Connections on that Call. Applications obtain an array of Connections associated with the Call via the `Call.getConnections()` method. A Call retains a reference to a Connection only if it is not in the `Connection.DISCONNECTED` state. Therefore, if a Call has a reference to a Connection, then that Connection must not be in the `Connection.DISCONNECTED` state. When a Connection moves into the `Connection.DISCONNECTED` state (e.g. when a party hangs up), the Call loses its reference to that Connection which is no longer reported via the `Call.getConnections()` method.

The `Call.connect()` method

The primary method on this interface is the `Call.connect()` method. Applications use this method to place a telephone call from an originating endpoint to a destination address string. The result of this method on the call model is to create an originating and destination Connection and move the Call into the `Call.ACTIVE` state. As the new telephone call progresses during its lifetime, the state of various objects associated with the Call may change and new objects may be created and associated with the Call. See the specification of the `Call.connect()` method below for more details.

Listeners and Observers

In JTAPI 1.2 and earlier versions, the way a JTAPI application arranged to receive events about telephony activity was to register an application object as an **Observer** of one or more JTAPI objects.

Starting with JTAPI 1.3, the primary way to arrange for a provider to report events is to use Listeners, following the pattern established in Java release 1.1 (see **Event** for more details). The Observer model is supported but its use is deprecated with this release.

Listeners and Events

The `CallListener` interface reports all events pertaining to the Call object. Events delivered to this interface must extend the `CallEvent` interface. Applications add listeners to a Call object via the `Call.addCallListener()` method.

Connection-related and TerminalConnection-related events are also reported via the `CallListener` interface. These events include the creation of these objects and their state changes. Events which are reported via the `CallListener` interface pertaining to Connections and TerminalConnections extend the `ConnectionEvent` interface and the `TerminalConnectionEvent` interface, respectively.

An event is delivered to the application whenever the state of the Call changes. The event interfaces corresponding to Call state changes are: `CallActiveEventID` and `CallInvalidEventID`.

When Call Event Transmission Ends

At times it may become impossible for the implementation to report events to an application. In this case, a `CALL_EVENT_TRANSMISSION_ENDED` is delivered to an object registered as a `CallListener` (or an extension of that interface).

This is the final event receives by the Listener, and it is no longer reported via the `Call.getCallListeners()` method.

Event Granularity

An application may control the granularity of events which are reported. Registering for the highest level interface (`CallListener`) will direct the implementation to send only Call-related events (including `MetaEvents`). To register as a Listener at a lower level interfaces (`ConnectionListener` or `TerminalConnectionListener`) directs the implementation to provide successively more detailed events.

Registering Call Listeners via Address and Terminal

Applications may receive events about a Call by adding an Listener via the Address or Terminal objects using the `Address.addCallListener()` and `Terminal.addCallListener()` (and related) methods. These methods provide the ability for an application to receive Call-related events when a Call contains a particular Address or Terminal. In particular, methods exist to add a `CallListener`, `ConnectionListener` and `TerminalConnectionListener` via Address and Terminal. See the specifications for Address and Terminal for more details.

See Also:

[CallListener](#), [ConnectionListener](#), [TerminalConnectionListener](#),
[Connection](#), [Address](#), [Terminal](#), [TerminalConnection](#), [CallEvent](#)

Field Summary	
static int	<u>ACTIVE</u> The <code>Call.ACTIVE</code> state indicates the Call has one or more Connections, none of which are in the <code>Connection.DISCONNECTED</code> state.
static int	<u>IDLE</u> The <code>Call.IDLE</code> state indicates the Call has zero Connections.
static int	<u>INVALID</u> The <code>Call.INVALID</code> state indicates the Call has lost all of its connections - that is, all of its associated Connection objects have moved into the <code>Connection.DISCONNECTED</code> state and are no longer associated with the Call.

Method Summary	
void	<u>addCallListener</u> (<u>CallListener</u> listener) Adds an listener to the Call.
void	<u>addObserver</u> (<u>CallObserver</u> observer) Adds an observer to the Call.
<u>Connection</u> []	<u>connect</u> (<u>Terminal</u> origterm, <u>Address</u> origaddr, java.lang.String dialedDigits) Places a telephone call from an originating endpoint to a destination address string.
<u>CallCapabilities</u>	<u>getCallCapabilities</u> (<u>Terminal</u> term, <u>Address</u> addr) Deprecated. Since JTAPI v1.2. This method has been replaced by the <i>Call.getCapabilities()</i> method.
<u>CallListener</u> []	<u>getCallListeners</u> () Returns an array of all <u>CallListeners</u> on this Call.
<u>CallCapabilities</u>	<u>getCapabilities</u> (<u>Terminal</u> terminal, <u>Address</u> address) Returns the dynamic capabilities for the instance of the Call object.
<u>Connection</u> []	<u>getConnections</u> () Returns an array of <u>Connections</u> associated with this call.
<u>CallObserver</u> []	<u>getObservers</u> () Returns an array of all <u>CallObservers</u> on this Call.
<u>Provider</u>	<u>getProvider</u> () Returns the <u>Provider</u> associated with the Call.
int	<u>getState</u> () Returns the current state of the telephone call.
void	<u>removeCallListener</u> (<u>CallListener</u> listener) Removes the given listener from the Call.
void	<u>removeObserver</u> (<u>CallObserver</u> observer) Removes the given observer from the Call.

Field Detail

IDLE

```
public static final int IDLE
```

The `Call.IDLE` state indicates the Call has zero Connections. It is the initial state of all Call objects.

ACTIVE

public static final int **ACTIVE**

The `Call.ACTIVE` state indicates the Call has one or more Connections, none of which are in the `Connection.DISCONNECTED` state.

INVALID

public static final int **INVALID**

The `Call.INVALID` state indicates the Call has lost all of its connections - that is, all of its associated Connection objects have moved into the `Connection.DISCONNECTED` state and are no longer associated with the Call. A Call in this state cannot be used for future actions.

Method Detail

getConnections

public Connection[] **getConnections()**

Returns an array of Connections associated with this call. Note that none of the Connections returned will be in the `Connection.DISCONNECTED` state. Also, if the Call is in the `Call.IDLE` state or the `Call.INVALID` state, this method returns null. Otherwise, it returns one or more Connection objects.

Post-conditions:

1. Let `Connection[] conn = Call.getConnections()`
2. if `this.getState() == Call.IDLE` then `conn = null`
3. if `this.getState() == Call.INVALID` then `conn = null`
4. if `this.getState() == Call.ACTIVE` then `conn.length >= 1`
5. For all `i`, `conn[i].getState() != Connection.DISCONNECTED`

Returns:

An array of the Connections associated with the call.

getProvider

public Provider **getProvider()**

Returns the Provider associated with the Call. This Provider reference remains valid throughout the lifetime of the Call object, despite the state of the Call object. This Provider reference does not change once the Call object has been created.

Returns:

The Provider associated with the call.

getState

```
public int getState()
```

Returns the current state of the telephone call. The state will be either `Call.IDLE`, `Call.ACTIVE`, or `Call.INVALID`.

Returns:

The current state of the call.

connect

```
public Connection[] connect(Terminal origterm,  
                             Address origaddr,  
                             java.lang.String dialedDigits)  
    throws ResourceUnavailableException,  
           PrivilegeViolationException,  
           InvalidPartyException,  
           InvalidArgumentException,  
           InvalidStateException,  
           MethodNotSupportedException
```

Places a telephone call from an originating endpoint to a destination address string.

The Call must be in the `Call.IDLE` state (and therefore have no existing associated Connections and the Provider must be in the `Provider.IN_SERVICE` state. The successful effect of this method is to place the telephone call and create and return two Connections associated with this Call.

Method Arguments

This method has three arguments. The first argument is the originating Terminal for the telephone call. The second argument is the originating Address for the telephone Call. This Terminal/Address pair must reference one another. That is, the originating Address must appear on the Terminal (via `Address.getTerminals()`) and the originating Terminal must appear on the Address (via `Terminal.getAddress()`). If not, an `InvalidArgumentException` is thrown.

The third argument is a destination string whose value represents the address to which the telephone call is placed. This destination address must be valid and complete.

Method Post-conditions

This method returns successfully when the Provider can successfully initiate the placing of the telephone call. As a result, when the `Call.connect()` method returns, the `Call` will be in the `Call.ACTIVE` state and exactly two `Connections` will be created and returned. The `Connection` associated with the originating endpoint is the first `Connection` in the returned array, and the `Connection` associated with the destination endpoint is the second `Connection` in the returned array. These two `Connections` must at least be in the `Connection.IDLE` state. That is, if one of the `Connections` progresses beyond the `Connection.IDLE` state while this method is completing, this `Connection` may be in a state other than the `Connection.IDLE`. This state must be reflected by an event sent to the application.

Call Scenarios or Flows

As a telephone call progresses during its lifetime, the various objects associated with the `Call` may change state and new objects may be created and associated with the `Call`. These changes typically occur after this method has successfully returned.

How the `Call` and its associated objects progress depends upon real-world conditions. There is a large but finite number of ways in which the state of a `Call` may progress. It is difficult to enumerate each possible way in which the state of a `Call` may progress. Instead, several illustrative *scenarios* (also called *flows*) are presented for common real-world conditions. These scenarios obey the valid state transitions as defined by the `Call`, `Connection`, and `TerminalConnection` objects.

Two common scenarios are presented below. Note that there may exist additional scenarios very similar to these which may only differ in a single step or state change. Any implementation which adheres to the rules outlined by the `Call`, `Connection`, and `TerminalConnection` objects is considered a proper implementation with respect to call flows.

Normal `Call.connect()` Scenario

This scenario, a telephone call is placed to a telephone number address. This address is outside the Provider's domain. The destination party answers the telephone call.

1. The `Call.connect()` method is invoked with the given arguments. Two `Connection` objects are created and returned, each in the `Connection.IDLE` state.

Events delivered to the application: a `CallActiveEvent` and two `ConnectionCreatedEventID`, one for each `Connection`.

2. Once the Provider begins to place the telephone call, the originating `Connection` moves from the `Connection.IDLE` state into the `Connection.CONNECTED` state. A `TerminalConnection` is created in the `TerminalConnection.IDLE` state and moves into the to model the `TerminalConnection.ACTIVE` relationship between the originating `Terminal` and the `Connection`.

Events delivered to the application: a `ConnectionEvent.CONNECTION_CONNECTED` for the originating `Connection`, a `TerminalConnectionEvent.TERMINAL_CONNECTION_CREATED` and a

TerminalConnection.TERMINAL_CONNECTION_ACTIVE for the new TerminalConnection.

Note: Depending upon the configuration of the switch, additional TerminalConnection objects associated with the originating Connection may be created. If the originating Address has more than one Terminal, these additional Terminals may be involved in the telephone call. For each TerminalConnection created a TerminalConnectionEvent.TERMINAL_CONNECTION_CREATED is delivered. Typically, these TerminalConnections will be in the TerminalConnection.PASSIVE state and a TerminalConnectionEvent.TERMINAL_CONNECTION_PASSIVE is delivered for each.

3. The destination Connection into the Connection.INPROGRESS state as the Call proceeds.

Events delivered to the application: a ConnectionEvent.CONNECTION_IN_PROGRESS for the destination Connection.

4. The destination Connection moves into the Connection.ALERTING state as the destination is alerted to the telephone call. TerminalConnection object may be created to model the relationship between any known destination Terminals associated with the Call, each in the TerminalConnection.RINGING state. If the destination Terminals are unknown, then no TerminalConnections are created.

Events delivered to the application: a ConnectionEvent.CONNECTION_ALERTING for the destination Connection, a TerminalConnectionEvent.TERMINAL_CONNECTION_CREATED and TerminalConnectionEvent.TERMINAL_CONNECTION_RINGING for any destination TerminalConnections created.

5. The destination Connection moves into the Connection.CONNECTED state when the called party answers the telephone call. If the destination Terminals are known, the answering TerminalConnection moves into the TerminalConnection.ACTIVE state.

Events delivered to the application: a ConnectionEvent.CONNECTION_CONNECTED for the destination Connection and a TerminalConnectionEvent.TERMINAL_CONNECTION_ACTIVE for the answering TerminalConnection, if known.

Note: For all other non-answering destination TerminalConnections known, either one of two state changes will occur depending upon the configuration of the switch. These TerminalConnections will either move into the TerminalConnection.PASSIVE state or the TerminalConnection.DROPPED state, depending upon whether or not these Terminals continue as part of the telephone call. For each, either a TerminalConnectionEvent.TERMINAL_CONNECTION_PASSIVE or TerminalConnectionEvent.TERMINAL_CONNECTION_DROPPED is delivered.

At the conclusion of this scenario, the Call will be in the Call.ACTIVE state, both Connections will be in the Connection.CONNECTED state, and the originating TerminalConnection will be in the TerminalConnection.ACTIVE state.

Failure `Call.connect()` Scenario

In this scenario, a telephone call is placed to a destination address. The destination cannot be reached, perhaps because the destination address is busy.

The first three steps of the first scenario are the same as in this scenario. They are not repeated here for brevity. The fourth and final step of this scenario is:

1. The destination Connection moves into the `Connection.FAILED` because the destination party could not be reached. (e.g. busy)

Events delivered to the application: a `ConnectionEvent.CONNECTION_FAILED` is delivered for the destination Connection.

At the conclusion of this scenario, the Call will be in the `Call.ACTIVE` state, the originating Connection will be in the `Connection.CONNECTED` state, the destination Connection will be in the `Connection.FAILED` state, and the originating TerminalConnection will be in the `TerminalConnection.ACTIVE` state.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.ACTIVE`
3. Let Connection `c[] = this.getConnections()`
4. `c.length == 2`
5. `c[0].getState() == Connection.IDLE` (at least)
6. `c[1].getState() == Connection.IDLE` (at least)

Parameters:

`origterm` - The originating Terminal for this telephone call.
`origaddr` - The originating Address for this telephone call.
`dialedDigits` - The destination address string for this telephone call.

Returns:

array of Connections

Throws:

`ResourceUnavailableException` - An internal resource necessary for placing the phone call is unavailable.

`PrivilegeViolationException` - The application does not have the proper authority to place a telephone call.

`InvalidPartyException` - Either the originator or the destination does not represent a valid party required to place a telephone call.

`InvalidArgumentException` - An argument provided is not valid either by not providing enough information for this method or is inconsistent with another argument.

`InvalidStateException` - Some object required by this method is not in a valid state as designated by the pre-conditions for this method.

`MethodNotSupportedException` - The implementation does not support this method.

See Also:[CallEvent](#), [ConnectionEvent](#), [TerminalConnectionEvent](#)

addObserver

```
public void addObserver(CallObserver observer)
    throws ResourceUnavailableException,
           MethodNotSupportedException
```

Adds an observer to the Call. The `CallObserver` reports all Call-related events. This includes changes in the state of the Call and all Connection-related and TerminalConnection-related events. The observer added with this method will report events on the call for as long as the implementation can observe the Call. In the case that the implementation can no longer observe the Call, the applications receives a `CallObservationEndedEv`. The observer receives no more events after it receives the `CallObservationEndedEv` and is no longer reported via the `Call.getObservers()` method.

Observer Lifetime

The `CallObserver` will receive events until one of the following occurs, whereupon the observer receives a `CallObservationEndedEv` and the observer is no longer reported via the `Call.getObservers()` method.

1. The observer is removed by the application with `Call.removeObserver()`.
2. The implementation can no longer monitor the call.
3. The Call has completed and moved into the `Call.INVALID` state.

Event Snapshots

By default, when an observer is added to a telephone call, the first batch of events may be a "snapshot". That is, if the observer was added after state changes in the Call, the first batch of events will inform the application of the current state of the Call. Note that these snapshot events do NOT provide a history of all events on the Call, rather they provide the minimum necessary information to bring the application up-to-date with the current state of the Call. The meta code for all of these events will be `Ev.META_SNAPSHOT`.

CallObservers from Addresses and Terminals

There may be additional call observers on the call which were not added by this method. These observers may have become part of the call via the `Address.addCallObserver()` and `Terminal.addCallObserver()` methods. See the specifications for these methods for more information.

Multiple Invocations

If an application attempts to add an instance of an observer already present on this Call, there are two possible outcomes:

1. If the observer was added by the application using this method, then a repeated invocation will silently fail, i.e. multiple instances of an observer are not added and no exception will be thrown.
2. If the observer is part of the call because an application invoked `Address.addObserver()` or `Terminal.addObserver()`, either of these methods modifies the behavior of that observer as if it were added via this method instead. That is, the observer is no longer removed when the call leaves the Address or Terminal. The observer now receives events until one of the conditions in "Observer Lifetime" is met.

Post-Conditions:

1. observer is an element of `this.getObservers()`
2. A snapshot of events is delivered to the observer, if appropriate.

Parameters:

`observer` - The observer being added.

Throws:

[MethodNotSupportedException](#) - The observer cannot be added at this time

[ResourceUnavailableException](#) - The resource limit for the numbers of observers has been exceeded.

getObservers

```
public CallObserver\[\] getObservers()
```

Returns an array of all `CallObservers` on this Call. If no observers are on this Call object, then this method returns null. This method returns all observers on this call no matter how they were added to the Call. Call observers may be added to this call in one of three ways:

1. The application added the observer via `Call.addObserver()`.
2. The application added the observer via `Address.addObserver()` and the call came to that Address.
3. The application added the observer via `Terminal.addObserver()` and the call came to that Terminal.

An instance of a `CallObserver` object will only appear once in this list.

Post-Conditions:

1. Let `CallObserver[] obs = this.getObservers()`
2. `obs == null` or `obs.length >= 1`

Returns:

An array of `CallObserver` objects associated with this Call.

removeObserver

public void **removeObserver**(CallObserver observer)

Removes the given observer from the Call. If successful, the observer will receive a `CallObservationEndedEv` as the last event it receives and will no longer be reported via the `Call.getObservers()` method.

This method has different effects depending upon how the observer was added to the Call, as follows:

1. If the observer was added via `Call.addObserver()`, this method removes the observer until it is re-applied by the application.
2. If the observer was added via `Address.addCallObserver()` or `Terminal.addCallObserver()`, this method removes the observer for this call only. It does not affect whether this observer will be added to future calls which come to that Address. See `Address.addCallObserver()` and `Terminal.addCallObserver()` for more details.

If an observer is not part of the Call, then this method fails silently, i.e. no observer is removed and no exception is thrown.

Post-Conditions:

1. observer is not an element of `this.getObservers()`
2. `CallObservationEndedEv` is delivered to the application

Parameters:

observer - The observer which is being removed.

getCapabilities

public CallCapabilities **getCapabilities**(Terminal terminal,
 Address address)
 throws InvalidArgumentException

Returns the dynamic capabilities for the instance of the Call object. Dynamic capabilities tell the application which actions are possible at the time this method is invoked based upon the implementations knowledge of its ability to successfully perform the action. This determination may be based upon argument passed to this method, the current state of the call model, or some implementation-specific knowledge. These indications do not guarantee that a particular method can be successfully invoked, however.

The dynamic call capabilities are based upon a Terminal/Address pair as well as the instance of the Call object. These parameters are used to determine whether certain call actions are possible at the present. For example, the `CallCapabilities.canConnect()` method will indicate whether a telephone call can be placed using the Terminal/Address pair as the originating endpoint.

Parameters:

`terminal` - Dynamic capabilities are with respect to this Terminal.

`address` - Dynamic capabilities are with respect to this Address.

Returns:

The dynamic Call capabilities.

Throws:

InvalidArgumentException - Indicates the Terminal and/or Address argument provided was not valid.

getCallCapabilities

```
public CallCapabilities getCallCapabilities(Terminal term,
                                           Address addr)
    throws InvalidArgumentException,
           PlatformException
```

Deprecated. Since JTAPI v1.2. This method has been replaced by the `Call.getCapabilities()` method.

Gets the `CallCapabilities` object with respect to a Terminal and an Address. If null is passed as a Terminal parameter, the general/provider-wide Call capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Call.getCapabilities()` method returns the dynamic Call capabilities. This method now should simply invoke the `Call.getCapabilities()` method with the given Terminal and Address arguments.

Parameters:

`term` - Dynamic Call capabilities in JTAPI v1.2 are with respect to this Terminal.

`addr` - Dynamic Call capabilities in JTAPI v1.2 are with respect to this Address.

Returns:

dynamic call capabilities

Throws:

InvalidArgumentException - Indicates the Terminal and/or Address argument provided was not valid.

PlatformException - A platform-specific exception occurred.

addCallListener

```
public void addCallListener(CallListener listener)
    throws ResourceUnavailableException,
           MethodNotSupportedException
```

Adds an listener to the Call. The `CallListener` reports all Call-related events. *This includes changes in the state of the Call and all Connection-related and TerminalConnection-related events.* The listener added with this method will report events on the call for as long as the implementation can listener the Call. In the case that the implementation can no longer observe the Call, the applications receives a `CALL_EVENT_TRANSMISSION_ENDED`. The listener receives no more events after it receives the `CALL_EVENT_TRANSMISSION_ENDED` and is no longer reported via

the `Call.getCallListeners()` method.

Listener Lifetime

The `CallListener` will receive events until one of the following occurs, whereupon the listener receives a `CALL_EVENT_TRANSMISSION_ENDED` and the listener is no longer reported via the `Call.getCallListeners()` method.

1. The listener is removed by the application with `Call.removeCallListener()`.
2. The implementation can no longer monitor the call.
3. The `Call` has completed and moved into the `Call.INVALID` state.

Event Snapshots

By default, when an listener is added to a telephone call, the first batch of events may be a "snapshot". That is, if the listener was added after state changes in the `Call`, the first batch of events will inform the application of the current state of the `Call`. Note that these snapshot events do NOT provide a history of all events on the `Call`, rather they provide the minimum necessary information to bring the application up-to-date with the current state of the `Call`. The meta code for all of these events will be `Event.META_SNAPSHOT`.

CallListeners from Addresses and Terminals

There may be additional call listeners on the call which were not added by this method. These listeners may have become part of the call via the `Address.addCallListener()` and `Terminal.addCallListener()` (or related) methods. See the specifications for these methods for more information.

Multiple Invocations

If an application attempts to add an instance of an listener already present on this `Call`, there are two possible outcomes:

1. If the listener was added by the application using this method, then a repeated invocation will silently fail, i.e. multiple instances of an listener are not added and no exception will be thrown.
2. If the listener is part of the call because an application invoked `Address.addCallListener()` or `Terminal.addCallListener()`, either of these methods modifies the behavior of that listener as if it were added via this method instead. That is, the listener is no longer removed when the call leaves the `Address` or `Terminal`. The listener now receives events until one of the conditions in "Listener Lifetime" is met.

Post-Conditions:

1. listener is an element of `this.getCallListeners()`
2. A snapshot of events is delivered to the listener, if appropriate.

Parameters:

`listener` - The listener being added.

Throws:

`MethodNotSupportedException` - The listener cannot be added at this time

`ResourceUnavailableException` - The resource limit for the numbers of listeners has been

exceeded.

getCallListeners

```
public CallListener[] getCallListeners()
```

Returns an array of all `CallListeners` on this `Call`. If no listeners are on this `Call` object, then this method returns null. This method returns all listeners on this call no matter how they were added to the `Call`. `Call` listeners may be added to this call in one of three ways:

1. The application added the listener via `Call.addCallListener()`.
2. The application added the listener via `Address.addCallListener()` and the call came to that `Address`.
3. The application added the listener via `Terminal.addCallListener()` and the call came to that `Terminal`.

An instance of a `CallListener` object will only appear once in this list.

Post-Conditions:

1. Let `CallListener[] obs = this.getCallListeners()`
2. `obs == null` or `obs.length >= 1`

Returns:

An array of `CallListener` objects associated with this `Call`.

removeCallListener

```
public void removeCallListener(CallListener listener)
```

Removes the given listener from the `Call`. If successful, the listener will receive a `CALL_EVENT_TRANSMISSION_ENDED` as the last event it receives and will no longer be reported via the `Call.getCallListeners()` method.

This method has different effects depending upon how the listener was added to the `Call`, as follows:

1. If the listener was added via `Call.addCallListener()`, this method removes the listener until it is re-applied by the application.
2. If the listener was added via `Address.addCallListener()` or `Terminal.addCallListener()`, this method removes the listener for this call only. It does not affect whether this listener will be added to future calls which come to that `Address`. See `Address.addCallListener()` and `Terminal.addCallListener()` for more details.

If an listener is not part of the `Call`, then this method fails silently, i.e. no listener is removed and no exception is thrown.

Post-Conditions:

1. listener is not an element of this.getCallListeners()
2. CALL_EVENT_TRANSMISSION_ENDED is delivered to the application

Parameters:

listener - The listener which is being removed.

*javax.telephony****Interface ConnectionListener*****All Known Subinterfaces:**TerminalConnectionListener

public interface **ConnectionListener**
extends CallListener

This interface is an extension of the `CallListener` interface, and reports state changes both of the `Call` and its `Connections`.

Introduction

As is true for the `CallListener` interface, specific `Connection` object state changes are reported to specific corresponding `ConnectionListener` methods. The `eventID` carried by the event (a `ConnectionEvent`) corresponds to the method called, and indicates which specific state change occurred.

Adding a Listener to a Call

Applications may add a `ConnectionListener` to a `Call` via one of several mechanisms. They may directly add a `ConnectionListener` by implementing the `ConnectionListener` interface, and then invoking the `Call.addCallListener` method. Applications may also add `ConnectionListener` to `Calls` indirectly by implementing the `ConnectionListener` interface, and then invoking the `Address.addCallListener` and `Terminal.addCallListener` methods. These methods add the given listener to the `Call` when the `Call` comes to the `Address` or `Terminal`. See the specifications for `Call`, `Address`, and `Terminal` for more information.

Call State Changes and Connection Events

In the core package, an event is delivered whenever the `Call` changes state. An event is delivered to a method of this interface specifically when a `Connection` object changes state. The method called, and the event ID reported, indicate the new `Call` state. The methods in this interface are: `connectionAlerting`, `connectionConnected`, `connectionDisconnected`, `connectionFailed`, `connectionInProgress`, and `connectionUnknown`.

Connection Events

All events pertaining to the `Connection` object are reported on this interface. Connection events are instances of the `ConnectionEvent` interface, which in turn, extends the `CallEvent` event interface. In the core package, an event is delivered to this interface whenever a `Connection` changes state.

See Also:

[CallEvent](#), [Connection](#), [ConnectionEvent](#), [TerminalConnectionEvent](#),
[MetaEvent](#), [CallListener](#), [TerminalConnectionListener](#)

Method Summary

void	connectionAlerting (ConnectionEvent event) The <code>connectionAlerting</code> method is called to report that the state of the <code>Connection</code> object has changed to <code>Connection.ALERTING</code> .
void	connectionConnected (ConnectionEvent event) The <code>connectionConnected</code> method is called to report that the state of the <code>Connection</code> object has changed to <code>Connection.CONNECTED</code> .
void	connectionCreated (ConnectionEvent event) The <code>connectionCreated</code> method is called to report that a new <code>Connection</code> object has been created.
void	connectionDisconnected (ConnectionEvent event) The <code>connectionDisconnected</code> method is called to report that the state of the <code>Connection</code> object has changed to <code>Connection.DISCONNECTED</code> .
void	connectionFailed (ConnectionEvent event) The <code>connectionFailed</code> method is called to report that the state of the <code>Connection</code> object has changed to <code>Connection.FAILED</code> .
void	connectionInProgress (ConnectionEvent event) The <code>connectionInProgress</code> method is called to report that the state of the <code>Connection</code> object has changed to <code>Connection.IN_PROGRESS</code> .
void	connectionUnknown (ConnectionEvent event) The <code>connectionUnknown</code> method is called to report that the state of the <code>Connection</code> object has changed to <code>Connection.UNKNOWN</code> .

Methods inherited from interface javax.telephony.CallListener

[callActive](#), [callEventTransmissionEnded](#), [callInvalid](#),
[multiCallMetaMergeEnded](#), [multiCallMetaMergeStarted](#),
[multiCallMetaTransferEnded](#), [multiCallMetaTransferStarted](#),
[singleCallMetaProgressEnded](#), [singleCallMetaProgressStarted](#),
[singleCallMetaSnapshotEnded](#), [singleCallMetaSnapshotStarted](#)

Method Detail

connectionAlerting

```
public void connectionAlerting(ConnectionEvent event)
```

The `connectionAlerting` method is called to report that the state of the `Connection` object has changed to `Connection.ALERTING`.

Parameters:

`event` - A `ConnectionEvent` with eventID `Connection.ALERTING`.

connectionConnected

```
public void connectionConnected(ConnectionEvent event)
```

The `connectionConnected` method is called to report that the state of the `Connection` object has changed to `Connection.CONNECTED`.

Parameters:

`event` - A `ConnectionEvent` with eventID `Connection.CONNECTED`.

connectionCreated

```
public void connectionCreated(ConnectionEvent event)
```

The `connectionCreated` method is called to report that a new `Connection` object has been created.

Parameters:

`event` - A `ConnectionEvent` with eventID `ConnectionEvent.IDLE`.

connectionDisconnected

```
public void connectionDisconnected(ConnectionEvent event)
```

The `connectionDisconnected` method is called to report that the state of the `Connection` object has changed to `Connection.DISCONNECTED`.

Parameters:

`event` - A `ConnectionEvent` with eventID `Connection.DISCONNECTED`.

See Also:

[Connection](#), [ConnectionEvent](#)

connectionFailed

```
public void connectionFailed(ConnectionEvent event)
```

The `connectionFailed` method is called to report that the state of the `Connection` object has changed to `Connection.FAILED`.

Parameters:

event - A `ConnectionEvent` with eventID `Connection.FAILED`.

See Also:

[Connection](#), [ConnectionEvent](#)

connectionInProgress

```
public void connectionInProgress(ConnectionEvent event)
```

The `connectionInProgress` method is called to report that the state of the `Connection` object has changed to `Connection.IN_PROGRESS`.

Parameters:

event - A `ConnectionEvent` with eventID `Connection.IN_PROGRESS`.

See Also:

[Connection](#), [Connection](#)

connectionUnknown

```
public void connectionUnknown(ConnectionEvent event)
```

The `connectionUnknown` method is called to report that the state of the `Connection` object has changed to `Connection.UNKNOWN`.

Parameters:

event - A `ConnectionEvent` with eventID `Connection.UNKNOWN`.

See Also:

[Connection](#), [ConnectionEvent](#)

javax.telephony

Interface CallEvent

All Known Subinterfaces:

[ConnectionEvent](#), [TerminalConnectionEvent](#)

public interface **CallEvent**
extends [Event](#)

The `CallEvent` interface is the base interface for all Call-related events. All Call-related events must extend this interface. Events which extend this interface are reported via the `CallListener` interface.

An individual `CallEvent` conveys one of a series of different possible Call state changes; the specific Call state change is indicated by the `Event.getID()` value returned by the event.

The core package defines events which are reported when the Call changes state. The event IDs (as returned by `Event.getID()`) are: `CALL_ACTIVE` and `CALL_INVALID`. Also, the core package defines the `CALL_EVENT_TRANSMISSION_ENDED` event ID which is sent when the the application can no longer listen for Call events.

These event IDs are reported via methods in the `CallListener` interface (see method `CallListener.CallInvalid`)

The `ConnectionEvent` and `TerminalConnectionEvent` interfaces extend this interface. This reflects the fact that all Connection and TerminalConnection events are reported via the `CallListener` interface.

The `CallEvent.getCall()` method on this interface returns the Call associated with the Call event.

See Also:

[Event](#), [ConnectionEvent](#), [TerminalConnectionEvent](#), [CallListener](#), [Call](#)

Field Summary

static int	<u>CALL_ACTIVE</u> The <code>CALL_ACTIVE</code> event indicates that the state of the Call object has changed to <code>Call.ACTIVE</code> .
static int	<u>CALL_EVENT_TRANSMISSION_ENDED</u> The <code>CALL_EVENT_TRANSMISSION_ENDED</code> event indicates that the application will no longer receive Call events on the instance of the <code>CallListener</code> .
static int	<u>CALL_INVALID</u> The <code>CALL_INVALID</code> event indicates that the state of the Call object has changed to <code>Call.INVALID</code> .

Fields inherited from interface javax.telephony.Event

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN

Method Summary

<u>Call</u>	<u>getCall()</u> Returns the Call object associated with this Call event.
-------------	---

Methods inherited from interface javax.telephony.Event

getCause, getID, getMetaEvent, getSource

Field Detail*CALL_ACTIVE*

```
public static final int CALL_ACTIVE
```

The CALL_ACTIVE event indicates that the state of the Call object has changed to `Call.ACTIVE`.

This constant corresponds to a specific call state change, is passed via a `CallEvent` event, and is reported to the `CallListener.callActive` method.

CALL_INVALID

```
public static final int CALL_INVALID
```

The CALL_INVALID event indicates that the state of the Call object has changed to `Call.INVALID`.

This constant corresponds to a specific call state change, is passed via a `CallEvent` event, and is reported to the `CallListener.callInvalid` method.

CALL_EVENT_TRANSMISSION_ENDED

public static final int **CALL_EVENT_TRANSMISSION_ENDED**

The `CALL_EVENT_TRANSMISSION_ENDED` event indicates that the application will no longer receive Call events on the instance of the `CallListener`.

This constant corresponds to this specific circumstance, is passed via a `CallEvent` event, and is reported to the `CallListener.callEventTransmissionEnded` method.

Method Detail

getCall

public Call **getCall()**

Returns the Call object associated with this Call event.

Returns:

The Call associated with this event.

javax.telephony

Interface CallListener

All Known Subinterfaces:

ConnectionListener, TerminalConnectionListener

public interface **CallListener**
extends java.util.EventListener

The `CallListener` interface reports all changes to the `Call` object.

Introduction

Call Events and Call MetaEvents

JTAPI generally provides a common model and standard interface to telephony applications, to enable the application to portably represent and control calls. Again generally there is a telephony platform on the other side of the API, which internally represents the objects and operations differently - often at a different level of granularity.

Since some individual changes to a call as represented by the telephony platform result in many changes in state in the corresponding JTAPI objects, the telephony platform's JTAPI implementation needs a way to mark a group of JTAPI events, calling them related, all originating from a single change in the telephony platform and its calls.

Applications may care because they may draw incorrect conclusions about the state of Calls after having processed only a fraction of a set of events generated in response to a single provider change of state.

In JTAPI 1.2, in order to indicate that a collection of JTAPI events all were in response to a single common telephony platform event, the convention was to deliver JTAPI events in *batches*. JTAPI 1.2 events delivered in a single batch were produced in response to a single telephony platform change of state.

JTAPI 1.3 introduces the Java 1.2 Event Model pattern of *Event Listeners*, which expect to receive events one at a time; hence batches are no longer used. Since the related events do not arrive as a group, JTAPI 1.3 provides new events, called MetaEvents, which show when such a group begins and ends.

In addition to helping the application know how the events are grouped, MetaEvents also convey information about larger granularity events. Some specifically carry information about call merging and transferring (see `MultiCallMetaEvent`).

Registering as a Listener

Therefore the CallListener interface conveys JTAPI Events and MetaEvents which indicate changes to the Call and related Calls. Applications which need to know about changes to a Call, may register as a listener to that Call; in order to do so the listener must implement this (CallListener) interface.

In order to register as a listener for events reported for all calls involving a specific Address, as soon as the Calls are created, an application may implement the CallListener interface and then register with the Address, via the Address.addCallListener method.

Connection Event and TerminalConnection Event listeners

The above discussion describes how an application may arrange to be notified of Call related state changes. An application may be notified, in addition, of Connection and TerminalConnection events, by registering as a listener for extensions of this interface - that is, as a ConnectionListener or as a TerminalConnectionListener.

In addition to the CallEvents and MetaEvents described above, listeners to these interfaces receive Connection events (ConnectionListener) or Connection and TerminalConnection events (TerminalConnectionListener).

These other interfaces extend the CallListener interface, to guarantee a Listener a synchronous stream of Meta, Call, Connection and TerminalConnection events in the expected order. To have a separate Connection Listener might leave an application open to race conditions, where events coming on separate listener threads could come in an unexpected order.

The cost to an application which cares only about Connection or TerminalConnection events is small, in that relatively few other events are reported on those interfaces; in addition, the programming burden is mitigated by convenient adapters (**naturally we need to provide adapters here! like CallListenerAdapter**).

State changes are reported as events to the Listener method corresponding to the type of event.

Adding a Listener to a Call

Applications must instantiate an object which implements the appropriate listener interface and then register the Call Listener using one of several mechanisms described below to receive all future events associated with the Call and (as requested) its Connections and TerminalConnections. Applications may add a listener to a Call via one of several mechanisms. They may directly add a listener via the Call.addCallListener method. Applications may also add listeners to Calls indirectly via the Address.addCallListener and Terminal.addCallListener methods. These methods add the given listener to the Call when the Call comes to the Address or Terminal. See the specifications for Call, Address, and Terminal for more information.

Call State Changes

In the core package, an event is delivered whenever the Call changes state. The methods which are called with an event to indicate when these state changes occur, are: `Call.callActiveEvent` and `Call.callInvalidEvent`.

Meta Events

Meta events (`MetaEvent`) pertaining to this Call are reported on this interface. In the core package, a meta event is delivered to this interface to provide higher-level context for a sequence of JTAPI "regular" events. The Meta events indicate appropriate times for an application to query the JTAPI model, and convey details about higher-level activities and operations (see `MetaEvent` for more information).

Call Listener Ending

At various times, the underlying implementation may not be able to provide events noting Call state changes. In these instances, the CallListener is sent an event via the `callEventTransmissionEnded` method. This indicates that the application will not receive further events associated with the Call object. This listener is no longer reported via the `Call.getCallListeners` method.

See Also:

`Call`, `CallEvent`, `ConnectionEvent`, `TerminalConnectionEvent`, `MetaEvent`,
`MultiCallMetaEvent`, `SingleCallMetaEvent`

Method Summary

void	<u>callActive</u> (<u>CallEvent</u> event) The callActive method is called to indicate that the state of the Call object has changed to Call.ACTIVE.
void	<u>callEventTransmissionEnded</u> (<u>CallEvent</u> event) The callEventTransmissionEnded method is called to indicate that the the application will no longer receive CallEvent events on the instance of the CallListener.
void	<u>callInvalid</u> (<u>CallEvent</u> event) The callInvalid method is called to indicate that the state of the Call object has changed to Call.INVALID.
void	<u>multiCallMetaMergeEnded</u> (<u>MetaEvent</u> event) This method indicates that a merge operation has ended, and that no further related events will be reported.
void	<u>multiCallMetaMergeStarted</u> (<u>MetaEvent</u> event) This method indicates that a merge operation is starting, and that related events will now be reported.
void	<u>multiCallMetaTransferEnded</u> (<u>MetaEvent</u> event) This method indicates that a merge operation has ended, and that no further related events will be reported.
void	<u>multiCallMetaTransferStarted</u> (<u>MetaEvent</u> event) This method indicates that a transfer operation is starting, and that related events will now be reported.
void	<u>singleCallMetaProgressEnded</u> (<u>MetaEvent</u> event) This method indicates that all events reported to convey the state change have been reported.
void	<u>singleCallMetaProgressStarted</u> (<u>MetaEvent</u> event) This method indicates that a state change has occurred for this Call entity, and that related events will now be reported.
void	<u>singleCallMetaSnapshotEnded</u> (<u>MetaEvent</u> event) This method indicates that the last event necessary to reconstruct the current state of this call has been reported.
void	<u>singleCallMetaSnapshotStarted</u> (<u>MetaEvent</u> event) This method indicates that JTAPI events will be reported to help the application reconstruct the current state of this call.

Method Detail

callActive

```
public void callActive(CallEvent event)
```

The `callActive` method is called to indicate that the state of the `Call` object has changed to `Call.ACTIVE`.

Parameters:

event - A `CallEvent` with `eventID` "Call.CALL_ACTIVE".

callInvalid

```
public void callInvalid(CallEvent event)
```

The `callInvalid` method is called to indicate that the state of the `Call` object has changed to `Call.INVALID`.

Parameters:

event - A `CallEvent` with `eventID` "Call.CALL_INVALID".

callEventTransmissionEnded

```
public void callEventTransmissionEnded(CallEvent event)
```

The `callEventTransmissionEnded` method is called to indicate that the application will no longer receive `CallEvent` events on the instance of the `CallListener`.

Parameters:

event - A `CallEvent` with `eventID` "Call.CALL_EVENT_TRANSMISSION_ENDED".

singleCallMetaProgressStarted

```
public void singleCallMetaProgressStarted(MetaEvent event)
```

This method indicates that a state change has occurred for this `Call` entity, and that related events will now be reported. See `SingleCallMetaEvent.SINGLECALL_META_PROGRESS_STARTED` for details.

Parameters:

event - A `MetaEvent` with `eventID` "SingleCallMetaEvent.SINGLECALL_META_PROGRESS_STARTED".

singleCallMetaProgressEnded

```
public void singleCallMetaProgressEnded(MetaEvent event)
```

This method indicates that all events reported to convey the state change have been reported. See `SingleCallMetaEvent.SINGLECALL_META_PROGRESS_ENDED` for details.

Parameters:

`event` - A `MetaEvent` with `eventID`

"SingleCallMetaEvent.SINGLECALL_META_PROGRESS_ENDED".

singleCallMetaSnapshotStarted

```
public void singleCallMetaSnapshotStarted(MetaEvent event)
```

This method indicates that JTAPI events will be reported to help the application reconstruct the current state of this call. See `SingleCallMetaEvent.SINGLECALL_META_SNAPSHOT_STARTED` for details.

Parameters:

`event` - A `MetaEvent` with `eventID`

"SingleCallMetaEvent.SINGLECALL_META_SNAPSHOT_STARTED".

singleCallMetaSnapshotEnded

```
public void singleCallMetaSnapshotEnded(MetaEvent event)
```

This method indicates that the last event necessary to reconstruct the current state of this call has been reported. See `SingleCallMetaEvent.SINGLECALL_META_SNAPSHOT_ENDED` for details.

Parameters:

`event` - A `MetaEvent` with `eventID`

"SingleCallMetaEvent.SINGLECALL_META_SNAPSHOT_ENDED".

multiCallMetaMergeStarted

```
public void multiCallMetaMergeStarted(MetaEvent event)
```

This method indicates that a merge operation is starting, and that related events will now be reported. See `MultiCallMetaEvent.MULTICALL_META_MERGE_STARTED` for details.

Parameters:

`event` - A `MetaEvent` with `eventID`

"MultiCallMetaEvent.MULTICALL_META_MERGE_STARTED".

multiCallMetaMergeEnded

```
public void multiCallMetaMergeEnded(MetaEvent event)
```

This method indicates that a merge operation has ended, and that no further related events will be reported. See `MultiCallMetaEvent.MULTICALL_META_MERGE_ENDED` for details.

Parameters:

`event` - A `MetaEvent` with `eventID`

"MultiCallMetaEvent.MULTICALL_META_MERGE_ENDED".

multiCallMetaTransferStarted

```
public void multiCallMetaTransferStarted(MetaEvent event)
```

This method indicates that a transfer operation is starting, and that related events will now be reported. See `MultiCallMetaEvent.MULTICALL_META_TRANSFER_STARTED` for details.

Parameters:

`event` - A `MetaEvent` with `eventID`

"MultiCallMetaEvent.MULTICALL_META_TRANSFER_STARTED".

multiCallMetaTransferEnded

```
public void multiCallMetaTransferEnded(MetaEvent event)
```

This method indicates that a merge operation has ended, and that no further related events will be reported. See `MultiCallMetaEvent.MULTICALL_META_TRANSFER_ENDED` for details.

Parameters:

`event` - A `MetaEvent` with `eventID`

"MultiCallMetaEvent.MULTICALL_META_TRANSFER_ENDED".

javax.telephony

Interface CallObserver

All Known Subinterfaces:

[CallControlCallObserver](#)

public interface **CallObserver**

The `CallObserver` interface reports all changes which happen to the `Call` object and all of the `Connection` and `TerminalConnection` objects which are part of the `Call`.

Introduction

These changes are reported as events to the `CallObserver.callChangedEvent()` method. Applications must instantiate an object which implements this interface and then add the observer to the call using one of several mechanisms described below to receive all future events associated with the `Call` and its `Connections` and `TerminalConnections`.

The `CallObserver.callChangedEvent()` method receives an array of events which all must extend the `CalEv` interface. Since several changes may happen to a single JTAPI object at once, a list of events is needed to convey those changes which happen at the same time. Applications iterate through the array of events provided.

Adding an Observer to a Call

Applications may add an observer to a `Call` via one of several mechanisms. They may directly add an observer via the `Call.addObserver()` method. Applications may also add observers to `Calls` indirectly via the `Address.addCallObserver()` and `Terminal.addCallObserver()` methods. These methods add the given observer to the `Call` when the `Call` comes to the `Address` or `Terminal`. See the specifications for `Call`, `Address`, and `Terminal` for more information.

Call State Changes

In the core package, an event is delivered whenever the `Call` changes state. The event interfaces which correspond to these state changes for the core package are: `CallActiveEv` and `CallInvalidEv`.

Connection Events

All events pertaining to the `Connection` object are reported on this interface. `Connection` events extend the `ConnEv` event, which in turn, extends the `CalEv` event. In the core package, an event is delivered to this interface whenever the `Connection` changes state.

TerminalConnection Events

All events pertaining to the TerminalConnection object are reported on this interface. TerminalConnection events extend the TermConnEv interface, which in turn, extends the CallEv interface. In the core package, an event is delivered to this interface whenever the TerminalConnection changes state.

Call Observation Ending

At various times, the underlying implementation may not be able to observe the Call. In these instances, the CallObserver is sent an CallObservationEndedEv event. This indicates that the application will not receive further events associated with the Call object. This observer is no longer reported via the Call.getObservers() method.

See Also:

[CallEv](#), [ConnEv](#), [TermConnEv](#), [CallObservationEndedEv](#), [CallActiveEv](#), [CallInvalidEv](#), [ConnAlertingEv](#), [ConnConnectedEv](#), [ConnCreatedEv](#), [ConnDisconnectedEv](#), [ConnFailedEv](#), [ConnInProgressEv](#), [ConnUnknownEv](#), [TermConnActiveEv](#), [TermConnCreatedEv](#), [TermConnDroppedEv](#), [TermConnPassiveEv](#), [TermConnRinginEv](#), [TermConnUnknownEv](#)

Method Summary

void	callChangedEvent (CallEv [] eventList) Reports all events associated with the Call object.
------	---

Method Detail

callChangedEvent

```
public void callChangedEvent(CallEv[] eventList)
```

Reports all events associated with the Call object. This method passes an array of CallEv objects as its arguments which correspond to the list of events representing the changes to the Call object as well as changes to all of the Connection and TerminalConnection objects associated with this Call.

Parameters:

eventList - The list of Call events.

javax.telephony

Interface TerminalConnectionListener

public interface **TerminalConnectionListener**
extends ConnectionListener

This interface is an extension of the `ConnectionListener` interface, and reports state changes of the `Call`, its `Connections` and its `TerminalConnections`.

Introduction

As is true for the `CallListener` interface, specific `TerminalConnection` object state changes are reported to specific corresponding `TerminalConnectionListener` methods. The `eventID` carried by the event (a `TerminalConnectionEvent`) corresponds to the method called, and indicates which specific state change occurred.

Adding a Listener to a Call

Applications may add a `TerminalConnectionListener` to a `Call` via one of several mechanisms. They may directly add a `TerminalConnectionListener` by specifying an object which implements that interface, as an argument to the `Call.addCallListener()` method. Applications may also add a `TerminalConnectionListener` to `Calls` indirectly by passing this listener object to either the `Address.addCallListener()` or `Terminal.addCallListener()` methods. These methods add the given listener to the `Call` when the `Call` comes to the `Address` or `Terminal`. See the specifications for `Call`, `Address`, and `Terminal` for more information.

Call State Changes and TerminalConnection Events

In the core package, an event is delivered whenever the `Call` changes state. An event is delivered to a method of this interface specifically when a `TerminalConnection` object changes state. The method called, and the event ID reported, indicate the new `TerminalConnection` state. The methods in this interface are: `terminalConnectionActive`, `terminalConnectionCreated`, `terminalConnectionDropped`, `terminalConnectionPassive`, `terminalConnectionRinging`, and `terminalConnectionUnknown`.

TerminalConnection Events

All events pertaining to the `TerminalConnection` object are reported on this interface. `TerminalConnection` events implement the `TerminalConnectionEvent` interface, which in turn, extends the `CallEvent` interface. `TerminalConnectionEvents` have an event ID which identifies the new state of the affected `TerminalConnection` object. In the core package, an event is delivered to this interface whenever the `TerminalConnection` changes state.

See Also:

[TerminalConnection](#), [CallEvent](#), [ConnectionEvent](#),
[TerminalConnectionEvent](#), [MetaEvent](#), [CallListener](#), [ConnectionListener](#)

Method Summary

void	<u>terminalConnectionActive</u> (<u>TerminalConnectionEvent</u> event) The terminalConnectionActive method is called to report that the state of the TerminalConnection object has changed to TerminalConnection.ACTIVE.
void	<u>terminalConnectionCreated</u> (<u>TerminalConnectionEvent</u> event) The terminalConnectionCreated method is called to report that a new TerminalConnection object has been created.
void	<u>terminalConnectionDropped</u> (<u>TerminalConnectionEvent</u> event) The terminalConnectionDropped method is called to report that the state of the TerminalConnection object has changed to TerminalConnection.DROPPED.
void	<u>terminalConnectionPassive</u> (<u>TerminalConnectionEvent</u> event) The terminalConnectionPassive method is called to report that the state of the TerminalConnection object has changed to TerminalConnection.PASSIVE.
void	<u>terminalConnectionRinging</u> (<u>TerminalConnectionEvent</u> event) The terminalConnectionRinging method is called to report that the state of the TerminalConnection object has changed to TerminalConnection.RINGING.
void	<u>terminalConnectionUnknown</u> (<u>TerminalConnectionEvent</u> event) The terminalConnectionUnknown method is called to report that the state of the TerminalConnection object has changed to TerminalConnection.UNKNOWN.

Methods inherited from interface javax.telephony.[ConnectionListener](#)

[connectionAlerting](#), [connectionConnected](#), [connectionCreated](#),
[connectionDisconnected](#), [connectionFailed](#), [connectionInProgress](#),
[connectionUnknown](#)

Methods inherited from interface javax.telephony.[CallListener](#)

[callActive](#), [callEventTransmissionEnded](#), [callInvalid](#),
[multiCallMetaMergeEnded](#), [multiCallMetaMergeStarted](#),
[multiCallMetaTransferEnded](#), [multiCallMetaTransferStarted](#),
[singleCallMetaProgressEnded](#), [singleCallMetaProgressStarted](#),
[singleCallMetaSnapshotEnded](#), [singleCallMetaSnapshotStarted](#)

Method Detail

terminalConnectionActive

```
public void terminalConnectionActive(TerminalConnectionEvent event)
```

The `terminalConnectionActive` method is called to report that the state of the `TerminalConnection` object has changed to `TerminalConnection.ACTIVE`.

Parameters:

`event` - A `TerminalConnectionEvent` with `eventID` `TerminalConnection.ACTIVE`.

terminalConnectionCreated

```
public void terminalConnectionCreated(TerminalConnectionEvent event)
```

The `terminalConnectionCreated` method is called to report that a new `TerminalConnection` object has been created.

Parameters:

`event` - A `TerminalConnectionEvent` with `eventID` `TerminalConnection.IDLE`.

terminalConnectionDropped

```
public void terminalConnectionDropped(TerminalConnectionEvent event)
```

The `terminalConnectionDropped` method is called to report that the state of the `TerminalConnection` object has changed to `TerminalConnection.DROPPED`.

Parameters:

`event` - A `TerminalConnectionEvent` with `eventID` `TerminalConnection.DROPPED`.

terminalConnectionPassive

```
public void terminalConnectionPassive(TerminalConnectionEvent event)
```

The `terminalConnectionPassive` method is called to report that the state of the `TerminalConnection` object has changed to `TerminalConnection.PASSIVE`.

Parameters:

`event` - A `TerminalConnectionEvent` with `eventID` `TerminalConnection.PASSIVE`.

terminalConnectionRinging

public void **terminalConnectionRinging**(TerminalConnectionEvent event)

The `terminalConnectionRinging` method is called to report that the state of the `TerminalConnection` object has changed to `TerminalConnection.RINGING`.

Parameters:

event - A `TerminalConnectionEvent` with eventID `TerminalConnection.RINGING`.

terminalConnectionUnknown

public void **terminalConnectionUnknown**(TerminalConnectionEvent event)

The `terminalConnectionUnknown` method is called to report that the state of the `TerminalConnection` object has changed to `TerminalConnection.UNKNOWN`.

Parameters:

event - A `TerminalConnectionEvent` with eventID `terminalConnection.UNKNOWN`.

javax.telephony

Interface Connection

All Known Subinterfaces:

CallControlConnection

public interface **Connection**

A `Connection` represents a link (i.e. an association) between a Call object and an Address object.

Introduction

The purpose of a `Connection` object is to describe the relationship between a `Call` object and an `Address` object. A `Connection` object exists if the `Address` is a part of the telephone call. Each `Connection` has a *state* which describes the particular stage of the relationship between the `Call` and `Address`. These states and their meanings are described below. Applications use the `Connection.getCall()` and `Connection.getAddress()` methods to obtain the `Call` and `Address` associated with this `Connection`, respectively.

From one perspective, an application may view a `Call` only in terms of the `Address/Connection` objects which are part of the `Call`. This is termed a *logical* view of the `Call` because it ignores the details provided by the `Terminal` and `TerminalConnection` objects which are also associated with a `Call`. In many instances, simple applications (such as an *outcall* program) may only need to concern itself with the logical view. In this logical view, a telephone call is views as two or more endpoint addresses in communication. The `Connection` object describes the state of each of these endpoint addresses with respect to the `Call`.

Calls and Addresses

`Connection` objects are immutable in terms of their `Call` and `Address` references. In other words, the `Call` and `Address` object references do not change throughout the lifetime of the `Connection` object instance. The same `Connection` object may not be used in another telephone call. The existence of a `Connection` implies that its `Address` is associated with its `Call` in the manner described by the `Connection`'s state.

Although a `Connection`'s `Address` and `Call` references remain valid throughout the lifetime of the `Connection` object, the same is not true for the `Call` and `Address` object's references to this `Connection`. Particularly, when a `Connection` moves into the `Connection.DISCONNECTED` state, it is no longer listed by the `Call.getConnections()` and `Address.getConnections()` methods. Typically, when a `Connection` moves into the `Connection.DISCONNECTED` state, the application loses its references to it to facilitate its garbage collection.

TerminalConnections

Connections objects are containers for zero or more TerminalConnection objects. Connection objects are containers for zero or more TerminalConnection objects. Connection objects represent the relationship between the Call and the Address, whereas TerminalConnection objects represent the relationship between the Connection and the Terminal. The relationship between the Call and the Address may be viewed as a logical view of the Call. The relationship between a Connection and a Terminal represents the physical view of the Call, i.e. at which Terminal the telephone calls terminates. The TerminalConnection object specification provides further information.

Connection States

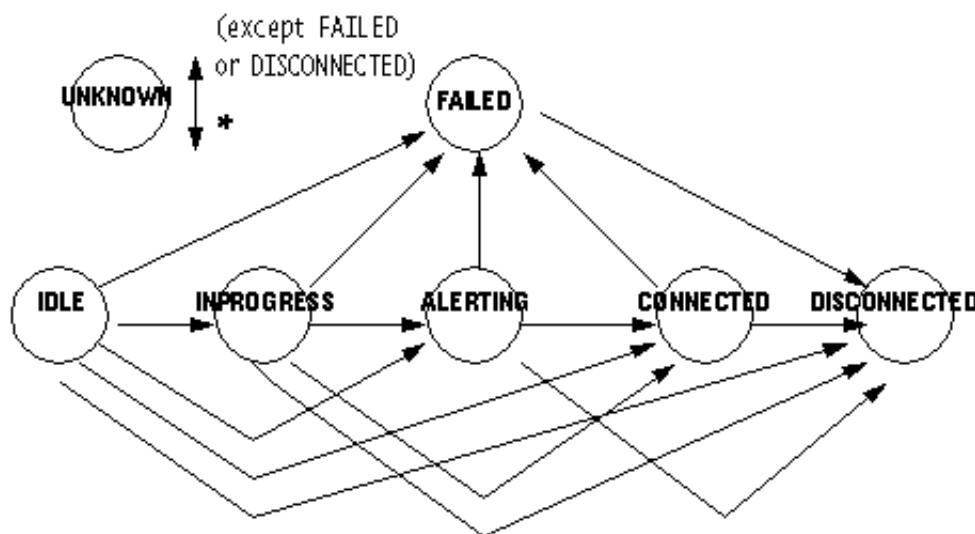
Below is a description of each Connection state in real-world terms. These real-world descriptions have no bearing on the specifications of methods, they only serve to provide a more intuitive understanding of what is going on. Several methods in this specification state pre-conditions based upon the state of the Connection.

<code>Connection.IDLE</code>	This state is the initial state for all new Connections. Connections which are in the <code>Connection.IDLE</code> state are not actively part of a telephone call, yet their references to the Call and Address objects are valid. Connections typically do not stay in the <code>Connection.IDLE</code> state for long, quickly transitioning to other states.
<code>Connection.DISCONNECTED</code>	This state implies it is no longer part of the telephone call, although its references to Call and Address still remain valid. A Connection in this state is interpreted as once previously belonging to this telephone call.
<code>Connection.INPROGRESS</code>	This state implies that the Connection, which represents the destination end of a telephone call, is in the process of contacting the destination side. Under certain circumstances, the Connection may not progress beyond this state. Extension packages elaborate further on this state in various situations.
<code>Connection.ALERTING</code>	This state implies that the Address is being notified of an incoming call.
<code>Connection.CONNECTED</code>	This state implies that a Connection and its Address is actively part of a telephone call. In common terms, two people talking to one another are represented by two Connections in the <code>Connection.CONNECTED</code> state.
<code>Connection.UNKNOWN</code>	This state implies that the implementation is unable to determine the current state of the Connection. Typically, methods are invalid on Connections which are in this state. Connections may move in and out of the <code>Connection.UNKNOWN</code> state at any time.
<code>Connection.FAILED</code>	This state indicates that a Connection to that end of the call has failed for some reason. One reason why a Connection would be in the <code>Connection.FAILED</code> state is because the party was busy.

Connection State Transitions

With these loose, real-world meanings in the back of one's mind, the Connection class defines a finite-state diagram which describes the allowable Connection state transitions. This finite-state diagram must be guaranteed by the implementation. Each method which causes a change in a Connection state must be consistent with this state diagram. This finite state diagram is below:

Note there is a general left-to-right progression of the state transitions. A Connection object may transition into and out of the `Connection.UNKNOWN` state at any time (hence, the asterisk qualifier next to its bidirectional transition arrow).



The Connection.disconnect() Method

The primary method supported on the core package's Connection interface is the `Connection.disconnect()` method. This method drops an entire Connection from a telephone call. The result of this method is to move the Connection object into the `Connection.DISCONNECTED` state. See the specification of the `Connection.disconnect()` method on this page for more detailed information.

Listeners and Events

All events pertaining to the Connection object are reported via the `CallListener` interface on the `Call` object associated with this Connection. In the core package, events are reported to a `CallListener` when a new Connection is created and whenever a Connection changes state. Listeners are added to `Call` objects via the `Call.addCallListener()` method and more indirectly via the `Address.addCallListener()` and `Terminal.addCallListener()` methods. See the specifications for the `Call`, `Address`, and `Terminal` interfaces for more information.

The following Connection-related events are defined in the core package. Each of these events extend the `ConnectionEvent` interface (which, in turn, extends the `CallEvent` interface).

<code>ConnectionCreated</code>	Indicates a new <code>Connection</code> has been created on a <code>Call</code> .
<code>ConnectionInProgress</code>	Indicates the <code>Connection</code> has moved into the <code>Connection.INPROGRESS</code> state.
<code>ConnectionAlerting</code>	Indicates the <code>Connection</code> has moved into the <code>Connection.ALERTING</code> state.
<code>ConnectionConnected</code>	Indicates the <code>Connection</code> has moved into the <code>Connection.CONNECTED</code> state.
<code>ConnectionDisconnected</code>	Indicates the <code>Connection</code> has moved into the <code>Connection.DISCONNECTED</code> state.
<code>ConnectionFailed</code>	Indicates the <code>Connection</code> has moved into the <code>Connection.FAILED</code> state.
<code>ConnectionUnknown</code>	Indicates the <code>Connection</code> has moved into the <code>Connection.UNKNOWN</code> state.

See Also:

[ConnectionEvent](#), [ConnectionListener](#)

Field Summary	
static int	<u>ALERTING</u> The <code>Connection.ALERTING</code> state implies that the Address is being notified of an incoming call.
static int	<u>CONNECTED</u> The <code>Connection.CONNECTED</code> state implies that a Connection and its Address is actively part of a telephone call.
static int	<u>DISCONNECTED</u> The <code>Connection.DISCONNECTED</code> state implies it is no longer part of the telephone call, although its references to Call and Address still remain valid.
static int	<u>FAILED</u> The <code>Connection.FAILED</code> state indicates that a Connection to that end of the call has failed for some reason.
static int	<u>IDLE</u> The <code>Connection.IDLE</code> state is the initial state for all new Connections.
static int	<u>INPROGRESS</u> The <code>Connection.INPROGRESS</code> state implies that the Connection, which represents the destination end of a telephone call, is in the process of contacting the destination side.
static int	<u>UNKNOWN</u> The <code>Connection.UNKNOWN</code> state implies that the implementation is unable to determine the current state of the Connection.

Method Summary	
<code>void</code>	<u>disconnect()</u> Drops a Connection from an active telephone call.
<code>Address</code>	<u>getAddress()</u> Returns the Address object associated with this Connection.
<code>Call</code>	<u>getCall()</u> Returns the Call object associated with this Connection.
<code>ConnectionCapabilities</code>	<u>getCapabilities()</u> Returns the dynamic capabilities for the instance of the Connection object.
<code>ConnectionCapabilities</code>	<u>getConnectionCapabilities(Terminal terminal, Address address)</u> Deprecated. Since JTAPI v1.2. This method has been replaced by the <i>Connection.getCapabilities()</i> method.
<code>int</code>	<u>getState()</u> Returns the current state of the Connection.
<code>TerminalConnection[]</code>	<u>getTerminalConnections()</u> Returns an array of TerminalConnection objects associated with this Connection.

Field Detail

IDLE

```
public static final int IDLE
```

The `Connection.IDLE` state is the initial state for all new Connections. Connections which are in the `Connection.IDLE` state are not actively part of a telephone call, yet their references to the `Call` and `Address` objects are valid. Connections typically do not stay in the `Connection.IDLE` state for long, quickly transitioning to other states.

INPROGRESS

```
public static final int INPROGRESS
```

The `Connection.INPROGRESS` state implies that the Connection, which represents the destination end of a telephone call, is in the process of contacting the destination side. Under certain circumstances, the Connection may not progress beyond this state. Extension packages elaborate further on this state in various situations.

ALERTING

public static final int **ALERTING**

The `Connection.ALERTING` state implies that the Address is being notified of an incoming call.

CONNECTED

public static final int **CONNECTED**

The `Connection.CONNECTED` state implies that a Connection and its Address is actively part of a telephone call. In common terms, two people talking to one another are represented by two Connections in the `Connection.CONNECTED` state.

DISCONNECTED

public static final int **DISCONNECTED**

The `Connection.DISCONNECTED` state implies it is no longer part of the telephone call, although its references to Call and Address still remain valid. A Connection in the `Connection.DISCONNECTED` state is interpreted as once previously belonging to this telephone call.

FAILED

public static final int **FAILED**

The `Connection.FAILED` state indicates that a Connection to that end of the call has failed for some reason. One reason why a Connection would be in the `Connection.FAILED` state is because the party was busy.

UNKNOWN

public static final int **UNKNOWN**

The `Connection.UNKNOWN` state implies that the implementation is unable to determine the current state of the Connection. Typically, method are invalid on Connections which are in the `Connection.UNKNOWN` state. Connections may move in and out of this state at any time.

Method Detail

getState

```
public int getState()
```

Returns the current state of the Connection. The return value will be one of states defined above.

Returns:

The current state of the Connection.

getCall

```
public Call getCall()
```

Returns the Call object associated with this Connection. This Call reference remains valid throughout the lifetime of the Connection object, despite the state of the Connection object. This Call reference does not change once the Connection object has been created.

Returns:

The call object associated with this Connection.

getAddress

```
public Address getAddress()
```

Returns the Address object associated with this Connection. This Address object reference remains valid throughout the lifetime of the Connection object despite the state of the Connection object. This Address reference does not change once the Connection object has been created.

Returns:

The Address associated with this Connection.

getTerminalConnections

```
public TerminalConnection[] getTerminalConnections()
```

Returns an array of TerminalConnection objects associated with this Connection. TerminalConnection objects represent the relationship between a Connection and a specific Terminal endpoint. There may be zero TerminalConnections associated with this Connection. In that case, this method returns null. Connection objects lose their reference to a TerminalConnection once the TerminalConnection moves into the `TerminalConnection.DROPPED` state.

Post-conditions:

1. Let TerminalConnection tc[] = this.getTerminalConnections()
2. tc == null or tc.length >= 1
3. For all i, tc[i].getState() != TerminalConnection.DROPPED

Returns:

An array of `TerminalConnection` objects associated with this `Connection`, null if there are no `TerminalConnections`.

disconnect

```
public void disconnect()  
    throws PrivilegeViolationException,  
           ResourceUnavailableException,  
           MethodNotSupportedException,  
           InvalidStateException
```

Drops a `Connection` from an active telephone call. The `Connection`'s `Address` is no longer associated with the telephone call. This method does not necessarily drop the entire telephone call, only the particular `Connection` on the telephone call. This method provides the ability to disconnect a specific party from a telephone call, which is especially useful in telephone calls consisting of three or more parties. Invoking this method may result in the entire telephone call being dropped, which is a permitted outcome of this method. In that case, the appropriate events are delivered to the application, indicating that more than just a single `Connection` has been dropped from the telephone call.

Allowable Connection States

The `Connection` object must be in one of several states in order for this method to be successfully invoked. These allowable states are: `Connection.CONNECTED`, `Connection.ALERTING`, `Connection.INPROGRESS`, or `Connection.FAILED`. If the `Connection` is not in one of these allowable states when this method is invoked, this method throws `InvalidStateException`. Having the `Connection` in one of the allowable states does not guarantee a successful invocation of this method.

Method Return Conditions

This method returns successfully only after the `Connection` has been disconnected from the telephone call and has transitioned into the `Connection.DISCONNECTED`. This method may return unsuccessfully by throwing one of the exceptions listed below. Note that this method waits (i.e. the invoking thread blocks) until either the `Connection` is actually disconnected from the telephone call or an error is detected and an exception thrown. Also, all of the `TerminalConnections` associated with this `Connection` are moved into the `TerminalConnection.DROPPED` state. As a result, they are no longer reported via the `Connection` by the `Connection.getTerminalConnections()` method.

As a result of this method returning successfully, one or more events are delivered to the application. These events are listed below:

1. A `ConnectionDisconnected` event for this `Connection`.
2. A `TerminalConnectionDropped` event for all `TerminalConnections` associated with this `Connection`.

Dropping Additional Connections

Additional Connections may be dropped indirectly as a result of this method. For example, dropping the destination Connection of a two-party Call may result in the entire telephone call being dropped. It is up to the implementation to determine which Connections are dropped as a result of this method. Implementations should not, however, drop additional Connections if it does not reflect the natural response of the underlying telephone hardware.

Dropping additional Connections implies that their TerminalConnections are dropped as well. Also, if all of the Connections on a telephone call are dropped as a result of this method, the Call will move into the `Call.INVALID` state. The following lists additional events which may be delivered to the application.

1. `ConnectionDisconnected`/`TerminalConnectionDropped` are delivered for all other Connections and TerminalConnections dropped indirectly as a result of this method.
2. `CallInvalid` if all of the Connections are dropped indirectly as a result of this method.

Pre-conditions:

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Connection.CONNECTED` or `Connection.ALERTING` or `Connection.INPROGRESS` or `Connection.FAILED`
3. Let TerminalConnection `tc[] = this.getTerminalConnections` (see post- conditions)

Post-conditions:

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Connection.DISCONNECTED`
3. For all `i`, `tc[i].getState() == TerminalConnection.DROPPED`
4. `this.getTerminalConnections() == null`.
5. `this` is not an element of `(this.getCall()).getConnections()`
6. `ConnectionDisconnected` is delivered for this Connection.
7. `TerminalConnectionDropped` is delivered for all TerminalConnections associated with this Connection.
8. `ConnectionDisconnected`/`TerminalConnectionDropped` are delivered for all other Connections and TerminalConnections dropped indirectly as a result of this method.
9. `CallInvalid` if all of the Connections are dropped indirectly as a result of this method.

Throws:

`PrivilegeViolationException` - The application does not have the authority or permission to disconnected the Connection. For example, the Address associated with this Connection may not be controllable in the Provider's domain.

`ResourceUnavailableException` - An internal resource required to drop a connection is not available.

`MethodNotSupportedException` - This method is not supported by the implementation.

`InvalidStateException` - Some object required for the successful invocation of this method is not in the proper state as given by this method's pre-conditions. For example, the Provider may not be in the `Provider.IN_SERVICE` state or the Connection may not be in one of the allowable states.

See Also:ConnectionListener, ConnectionEvent*getCapabilities*

```
public ConnectionCapabilities getCapabilities()
```

Returns the dynamic capabilities for the instance of the Connection object. Dynamic capabilities tell the application which actions are possible at the time this method is invoked based upon the implementations knowledge of its ability to successfully perform the action. This determination may be based upon the current state of the call model or some implementation-specific knowledge. These indications do not guarantee that a particular method can be successfully invoked, however.

The dynamic Connection capabilities require no additional arguments.

Returns:

The dynamic Connection capabilities.

getConnectionCapabilities

```
public ConnectionCapabilities getConnectionCapabilities(Terminal terminal,  
                                                         Address address)  
                                                         throws InvalidArgumentException,  
                                                         PlatformException
```

Deprecated. Since JTAPI v1.2. This method has been replaced by the *Connection.getCapabilities()* method.

Gets the ConnectionCapabilities object with respect to a Terminal and an Address. If null is passed as a Terminal parameter, the general/ provider-wide Connection capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Connection.getCapabilities()` method returns the dynamic Connection capabilities. This method now should simply invoke the `Connection.getCapabilities()` method.

Parameters:

`terminal` - This argument is ignored in JTAPI v1.2 and later.

`address` - This argument is ignored in JTAPI v1.2 and later.

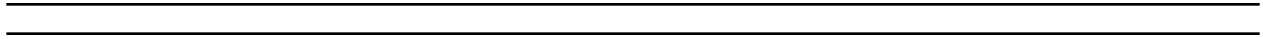
Returns:

The dynamic ConnectionCapabilities capabilities.

Throws:

InvalidArgumentException - This exception is never thrown in JTAPI v1.2 and later.

PlatformException - A platform-specific exception occurred.



javax.telephony

Interface ConnectionEvent

public interface **ConnectionEvent**
extends CallEvent

The `ConnectionEvent` interface is the base event interface for all Connection-related events. All events which pertain to the Connection object must extend this interface. This interface extends the `CallEvent` interface and therefore is reported via the `CallListener` interface.

The core package defines events which are reported when the Connection changes state. These events are: `CONNECTION_IN_PROGRESS`, `CONNECTION_ALERTING`, `CONNECTION_CONNECTED`, `CONNECTION_DISCONNECTED`, `CONNECTION_FAILED`, and `CONNECTION_UNKNOWN`. Also, the `CONNECTION_CREATED` is sent when a new Connection is created.

Each of these events is reported to the corresponding method for an application registered as a `ConnectionListener` or `TerminalConnectionListener`.

The `ConnectionEvent.getConnection` method on this interface returns the Connection associated with this Connection event.

See Also:

Connection, CallListener, CallEvent

Field Summary

static int	<u>CONNECTION_ALERTING</u> The CONNECTION_ALERTING event indicates that the state of the Connection object has changed to Connection.ALERTING.
static int	<u>CONNECTION_CONNECTED</u> The CONNECTION_CONNECTED event indicates that the state of the Connection object has changed to Connection.CONNECTED.
static int	<u>CONNECTION_CREATED</u> The CONNECTION_CREATED event indicates that a new Connection object has been created.
static int	<u>CONNECTION_DISCONNECTED</u> The CONNECTION_DISCONNECTED event indicates that the state of the Connection object has changed to Connection.DISCONNECTED.
static int	<u>CONNECTION_FAILED</u> The CONNECTION_FAILED interface indicates that the state of the Connection object has changed to Connection.FAILED.
static int	<u>CONNECTION_IN_PROGRESS</u> The CONNECTION_IN_PROGRESS event indicates that the state of the Connection object has changed to Connection.IN_PROGRESS.
static int	<u>CONNECTION_UNKNOWN</u> The CONNECTION_UNKNOWN event indicates that the state of the Connection object has changed to Connection.UNKNOWN.

Fields inherited from interface javax.telephony.CallEvent

CALL_ACTIVE, CALL_EVENT_TRANSMISSION_ENDED, CALL_INVALID

Fields inherited from interface javax.telephony.Event

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN

Method Summary

<u>Connection</u>	<u>getConnection()</u> Returns the Connection associated with this Connection event.
-------------------	--

Methods inherited from interface javax.telephony.CallEvent

getCall

Methods inherited from interface javax.telephony.Event

getCause, getID, getMetaEvent, getSource

Field Detail

CONNECTION_ALERTING

```
public static final int CONNECTION_ALERTING
```

The `CONNECTION_ALERTING` event indicates that the state of the `Connection` object has changed to `Connection.ALERTING`.

This constant indicates a specific event passed via a `ConnectionEvent` event, and is reported on the `CallListener` interface.

CONNECTION_CONNECTED

```
public static final int CONNECTION_CONNECTED
```

The `CONNECTION_CONNECTED` event indicates that the state of the `Connection` object has changed to `Connection.CONNECTED`.

This constant indicates a specific event passed via a `ConnectionEvent` event, and is reported on the `CallListener` interface.

CONNECTION_CREATED

```
public static final int CONNECTION_CREATED
```

The **CONNECTION_CREATED** event indicates that a new `Connection` object has been created.

This constant indicates a specific event passed via a `ConnectionEvent` event, and is reported on the `CallListener` interface.

CONNECTION_DISCONNECTED

```
public static final int CONNECTION_DISCONNECTED
```

The **CONNECTION_DISCONNECTED** event indicates that the state of the `Connection` object has changed to `Connection.DISCONNECTED`.

This constant indicates a specific event passed via a `ConnectionEvent` event, and is reported on the `CallListener` interface.

CONNECTION_FAILED

```
public static final int CONNECTION_FAILED
```

The **CONNECTION_FAILED** interface indicates that the state of the `Connection` object has changed to `Connection.FAILED`.

This constant indicates a specific event passed via a `ConnectionEvent` event, and is reported on the `CallListener` interface.

CONNECTION_IN_PROGRESS

```
public static final int CONNECTION_IN_PROGRESS
```

The **CONNECTION_IN_PROGRESS** event indicates that the state of the `Connection` object has changed to `Connection.IN_PROGRESS`.

This constant indicates a specific event passed via a `ConnectionEvent` event, and is reported on the `CallListener` interface.

CONNECTION_UNKNOWN

```
public static final int CONNECTION_UNKNOWN
```

The `CONNECTION_UNKNOWN` event indicates that the state of the `Connection` object has changed to `Connection.UNKNOWN`.

This constant indicates a specific event passed via a `ConnectionEvent` event, and is reported on the `CallListener` interface.

Method Detail

getConnection

```
public Connection getConnection()
```

Returns the `Connection` associated with this `Connection` event.

Returns:

The `Connection` associated with this event.

javax.telephony

Interface Event

All Known Subinterfaces:

[AddressEvent](#), [CallEvent](#), [ConnectionEvent](#), [MetaEvent](#), [MobileProviderEvent](#), [MobileRadioEvent](#),
[MultiCallMetaEvent](#), [ProviderEvent](#), [SingleCallMetaEvent](#), [TerminalConnectionEvent](#),
[TerminalEvent](#)

public interface **Event**

The **Event** interface is the parent of all JTAPI event interfaces.

Introduction

All JTAPI event interfaces extend this interface, either directly or indirectly. Event interfaces within each JTAPI package are organized in a hierarchical fashion. The architecture of the core package event hierarchy is described later.

The JTAPI event system notifies applications when changes in various JTAPI object occur. Each individual change in an object is represented by an event sent to the appropriate Listener.

In the spirit of the [JDK 1.1 Event model changes](#), JTAPI now has Listeners instead of Observers. In addition to the JDK 1.1 change in terminology, JTAPI event handling has been modified to make programming easier, by multiplexing specific events out to specific methods and by reducing the number of event objects involved. In addition, the memory footprint of the API has been lessened by reducing the number of interfaces defined, in order to accomodate smaller platforms.

JTAPI events will be delivered to a Listener one at a time, unlike the Observer model, where they are delivered in an array.

With the old observer event model, large scale activities such as 3rd party call transfer and conference, are awkward to detect and process.

With the new Listener model and meta events (see **MetaEvent**), an application has a listener interface with methods and events which convey, directly and with necessary details, what applications need to know to respond to higher-level events.

Event Identification

Event objects correspond to the JTAPI object which is undergoing a state change; the specific state change is conveyed to the application in two ways.

First, the provider reports the event to a particular method in a particular Listener interface to a listening object; generally the method corresponds to a particular state change.

Second, the event that is presented to the method has an identification integer which indicates the specific state change.

The `Event.getID()` method returns this identification number for each event. The actual event identification integer values that may be conveyed by the individual event object are defined in each of the specific event interfaces.

Cause Codes

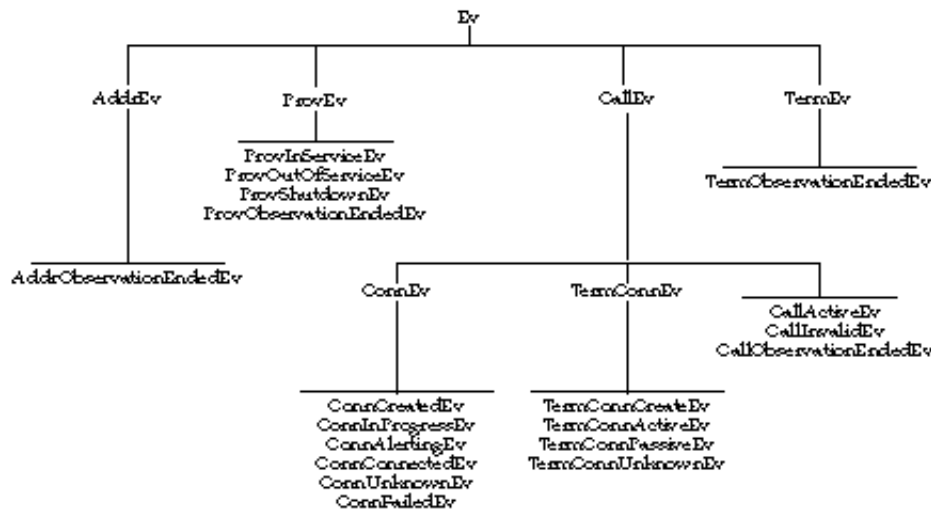
Each events carries a *cause* or a reason why the event happened. The `Event.getCause()` method returns this cause value. The different types of cause values are also defined in this interface.

Core Package Event Hierarchy

The core package defines a hierarchy of event interfaces. The base of this hierarchy is the `Event` interface. Directly extending this interface are those events interfaces for each object which supports an Listener: `ProviderEvent`, `CallEvent`, `AddressEvent`, and `TerminalEvent`.

Since Connection and TerminalConnection events are reported via the `CallListener` interface, the `ConnectionEvent` and `TerminalConnectionEvent` interfaces extends the `CallEvent` interface.

The following diagram illustrates the complete core package event structure.



JTAPI 1.2, Observer Model Meta Events

With JTAPI 1.2 & the observer model, "meta information" mechanisms (see `Event.java`) - event batches, meta codes, and the `Event.isNewMetaEvent()` method - were used

- A. to indicate what, and which, higher-level action an event belongs to
- B. to identify which events are associated with a particular update to the call model (implying synchronization points at which queries are reasonable)
- C. to convey, implicitly, multi-call event information.

Determining what and which higher-level action is happening with the observer mechanism is indirect - it must be concluded by looking at meta codes and possibly multiple events. Determining when the JTAPI events imply a consistent call state is conveyed by a change between events in the return value of the method "Event.isNewMetaEvent". Conveying information about multiple calls is done implicitly, across multiple events.

For applications to respond to higher-level actions, and to understand the scope of those actions, in the current JTAPI API, they must scan multiple events, inspect meta codes, inspect "isNewMetaEvent" "flag" values and, ultimately, guess that events are related.

Applications would be much simpler if they were given

- A. direct notification of high-level actions
- B. explicit descriptions of the scope of these actions (calls involved and events included)

JTAPI 1.3, Listener Model Meta Events

This semantic event proposal addresses these needs. It provides an interface with methods and events which convey, directly and with necessary details, what applications need to know to respond to higher-level events.

```

MetaEvent
  MultiCallMetaEvent extends MetaEvent
    constants
      MultiCallMetaMergeStart
      MultiCallMetaMergeEnd
      MultiCallMetaTransferStart
      MultiCallMetaTransferEnd
    methods
      Call[] getOldCalls()
      Call  getNewCall()

```

As for Listening for these events, we would add to CallListener

```

CallListener
  methods
    MultiCallMetaMergeStarted ( MetaEvent ev )
    MultiCallMetaMergeEnded ( MetaEvent ev )
    MultiCallMetaTransferStarted ( MetaEvent ev )
    MultiCallMetaTransferEnded ( MetaEvent ev )

```

and, to be able to map an event back to a meta-event, we'd add

```
CallEvent
method
    MetaEvent getMetaEvent()
```

which would return null if there were no meta-event associated with this event.

In addition to these interfaces and methods, the following operational principles would apply (currently just one):

meta events are reported before any corresponding normal events.

These changes are reflected in the current API.

Field Summary	
static int	<u>CAUSE CALL CANCELLED</u> Cause code indicating the user has terminated call without going on-hook.
static int	<u>CAUSE DEST NOT OBTAINABLE</u> Cause code indicating the destination is not available.
static int	<u>CAUSE INCOMPATIBLE DESTINATION</u> Cause code indicating that a call has encountered an incompatible destination.
static int	<u>CAUSE LOCKOUT</u> Cause code indicating that a call encountered inter-digit timeout while dialing.
static int	<u>CAUSE NETWORK CONGESTION</u> Cause code indicating call encountered network congestion.
static int	<u>CAUSE NETWORK NOT OBTAINABLE</u> Cause code indicating call could not reach a destination network.
static int	<u>CAUSE NEW CALL</u> Cause code indicating that a new call.
static int	<u>CAUSE NORMAL</u> Cause code indicating normal operation
static int	<u>CAUSE RESOURCES NOT AVAILABLE</u> Cause code indicating resources were not available.
static int	<u>CAUSE SNAPSHOT</u> Cause code indicating that the event is part of a snapshot of the current state of the call.
static int	<u>CAUSE UNKNOWN</u> Cause code indicating the cause was unknown

Method Summary	
int	<u>getCause()</u> Returns the cause associated with this event.
int	<u>getID()</u> Returns the id of event.
<u>MetaEvent</u>	<u>getMetaEvent()</u> Returns the MetaEvent associated with this event, or null.
java.lang.Object	<u>getSource()</u> Returns the event source of the event (see <code>java.util.EventObject</code>).

Field Detail

CAUSE_NORMAL

public static final int **CAUSE_NORMAL**

Cause code indicating normal operation

CAUSE_UNKNOWN

public static final int **CAUSE_UNKNOWN**

Cause code indicating the cause was unknown

CAUSE_CALL_CANCELLED

public static final int **CAUSE_CALL_CANCELLED**

Cause code indicating the user has terminated call without going on-hook.

CAUSE_DEST_NOT_OBTAINABLE

public static final int **CAUSE_DEST_NOT_OBTAINABLE**

Cause code indicating the destination is not available.

CAUSE_INCOMPATIBLE_DESTINATION

public static final int **CAUSE_INCOMPATIBLE_DESTINATION**

Cause code indicating that a call has encountered an incompatible destination.

CAUSE_LOCKOUT

public static final int **CAUSE_LOCKOUT**

Cause code indicating that a call encountered inter-digit timeout while dialing.

CAUSE_NEW_CALL

public static final int **CAUSE_NEW_CALL**

Cause code indicating that a new call.

CAUSE_RESOURCES_NOT_AVAILABLE

public static final int **CAUSE_RESOURCES_NOT_AVAILABLE**

Cause code indicating resources were not available.

CAUSE_NETWORK_CONGESTION

public static final int **CAUSE_NETWORK_CONGESTION**

Cause code indicating call encountered network congestion.

CAUSE_NETWORK_NOT_OBTAINABLE

public static final int **CAUSE_NETWORK_NOT_OBTAINABLE**

Cause code indicating call could not reach a destination network.

CAUSE_SNAPSHOT

public static final int **CAUSE_SNAPSHOT**

Cause code indicating that the event is part of a snapshot of the current state of the call.

Method Detail

getCause

```
public int getCause()
```

Returns the cause associated with this event. Every event has a cause. The various cause values are defined as public static final variables in this interface.

Returns:

The cause of the event.

getID

```
public int getID()
```

Returns the id of event. Every event has an id. The defined id of each event matches the object type of each event. The defined id allows applications to switch on event id rather than having to use multiple "if instanceof" statements.

Returns:

The id of the event.

getSource

```
public java.lang.Object getSource()
```

Returns the event source of the event (see `java.util.EventObject`).

Returns:

The object sending the event.

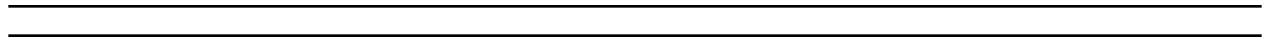
getMetaEvent

```
public MetaEvent getMetaEvent()
```

Returns the MetaEvent associated with this event, or null.

Returns:

The MetaEvent corresponding to this event, or null.



javax.telephony***Class InvalidArgumentException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.telephony.InvalidArgumentException

```

```

public class InvalidArgumentException
extends java.lang.Exception

```

An InvalidArgumentException indicates an argument passed to the method is invalid.

See Also:

[Serialized Form](#)

Constructor Summary**InvalidArgumentException()**

Constructor with no string.

InvalidArgumentException(java.lang.String s)

Constructor which takes a string description.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

InvalidArgumentException

public **InvalidArgumentException**()

Constructor with no string.

InvalidArgumentException

public **InvalidArgumentException**(java.lang.String s)

Constructor which takes a string description.

Parameters:

s - description of the faulty argument

javax.telephony***Class InvalidPartyException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.telephony.InvalidPartyException

```

```

public class InvalidPartyException
extends java.lang.Exception

```

An **InvalidPartyException** indicates that a party given as an argument to the method call was invalid. This may either be the originating party of a telephone call or the destination party of a telephone call.

See Also:

[Serialized Form](#)

Field Summary

static int	<u>DESTINATION PARTY</u> Indicates that the destination party was invalid.
static int	<u>ORIGINATING PARTY</u> Indicates that the originating party was invalid.
static int	<u>UNKNOWN PARTY</u> Indicates that the party was unknown.

Constructor Summary

<u>InvalidPartyException</u> (int type) Constructor with no string.	
<u>InvalidPartyException</u> (int type, java.lang.String s) Constructor which takes a string description.	

Method Summary

int	<u>getType</u> () Returns the type of party.
-----	---

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

ORIGINATING_PARTY

public static final int **ORIGINATING_PARTY**

Indicates that the originating party was invalid.

DESTINATION_PARTY

public static final int **DESTINATION_PARTY**

Indicates that the destination party was invalid.

UNKNOWN_PARTY

public static final int **UNKNOWN_PARTY**

Indicates that the party was unknown.

Constructor Detail

InvalidPartyException

```
public InvalidPartyException(int type)
```

Constructor with no string.

Parameters:

type - the type of party expected.

InvalidPartyException

```
public InvalidPartyException(int type,  
                             java.lang.String s)
```

Constructor which takes a string description.

Parameters:

type - type of exception

s - description of the fault

Method Detail

getType

```
public int getType()
```

Returns the type of party.

Returns:

The type of party.

javax.telephony

Class InvalidStateException

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.telephony.InvalidStateException

```

```

public class InvalidStateException
extends java.lang.Exception

```

An **InvalidStateException** indicates the current state of an object involved in the method invocation does not meet the acceptable pre-conditions for the method. Each method which changes the call model typically has a set of states in which the object must be as a pre-condition for the method. Each method documents the pre-condition states for objects. Typically, this method will succeed in the future once the object in question has reached the proper state.

This exception provides the application with the object in question and the state it is currently in.

See Also:

[Serialized Form](#)

Field Summary	
static int	<u>ADDRESS_OBJECT</u> The invalid object in question is the Address
static int	<u>CALL_OBJECT</u> The invalid object in question is the Call
static int	<u>CONNECTION_OBJECT</u> The invalid object in question is the Connection
static int	<u>PROVIDER_OBJECT</u> The invalid object in question is the Provider
static int	<u>TERMINAL_CONNECTION_OBJECT</u> The invalid object in question is the Terminal Connection
static int	<u>TERMINAL_OBJECT</u> The invalid object in question is the Terminal

Constructor Summary

InvalidStateException(java.lang.Object object, int type, int state)

Constructor with no string.

InvalidStateException(java.lang.Object object, int type, int state, java.lang.String s)

Constructor which takes a string description.

Method Summary

java.lang.Object	<u>getObject</u> () Returns the object which has the incorrect state.
int	<u>getObjectType</u> () Returns the type of object in question.
int	<u>getState</u> () Returns the state of the object.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

PROVIDER_OBJECT

```
public static final int PROVIDER_OBJECT
```

The invalid object in question is the Provider

CALL_OBJECT

public static final int **CALL_OBJECT**

The invalid object in question is the Call

CONNECTION_OBJECT

public static final int **CONNECTION_OBJECT**

The invalid object in question is the Connection

TERMINAL_OBJECT

public static final int **TERMINAL_OBJECT**

The invalid object in question is the Terminal

ADDRESS_OBJECT

public static final int **ADDRESS_OBJECT**

The invalid object in question is the Address

TERMINAL_CONNECTION_OBJECT

public static final int **TERMINAL_CONNECTION_OBJECT**

The invalid object in question is the Terminal Connection

Constructor Detail

InvalidStateException

```
public InvalidStateException(java.lang.Object object,
                             int type,
                             int state)
```

Constructor with no string.

Parameters:

object - instance associated with the invalid sate.

type - type of failure

state - current state at time of fault

InvalidStateException

```
public InvalidStateException(java.lang.Object object,  
                             int type,  
                             int state,  
                             java.lang.String s)
```

Constructor which takes a string description.

Parameters:

object - instance associated with the invalid state.

type - type of failure

state - current state at time of fault

s - description of the fault

Method Detail

getObjectType

```
public int getObjectType()
```

Returns the type of object in question.

Returns:

The type of object in question.

getObject

```
public java.lang.Object getObject()
```

Returns the object which has the incorrect state.

Returns:

The object which is in the wrong state.

getState

```
public int getState()
```

Returns the state of the object.

Returns:

The state of the object.

javax.telephony

Interface JtapiPeer

public interface **JtapiPeer**

The **JtapiPeer** interface represents a vendor's particular implementation of the Java Telephony API.

Introduction

Each JTAPI implementation vendor must implement this interface. The **JtapiPeer** object returned by the **JtapiPeerFactory.getJtapiPeer()** method determines which Providers are made available to the application.

Obtaining a Provider

Applications use the **JtapiPeer.getProvider()** method on this interface to obtain new Provider objects. Each implementation may support one or more different "services" (e.g. for different types of underlying network substrate). A list of available services can be obtained via the **JtapiPeer.getServices()** method.

Applications may also supply optional arguments to the Provider through the **JtapiPeer.getProvider()** method. These arguments are appended to the **providerString** argument passed to the **JtapiPeer.getProvider()** method. The **providerString** argument has the following format:

< service name > ; arg1 = val1; arg2 = val2; ...

Where < service name > is not optional, and each optional argument pair which follows is separated by a semi-colon. The keys for these arguments is implementation specific, except for two standard-defined keys:

1. login: provides the login user name to the Provider.
2. passwd: provides a password to the Provider.

See Also:

JtapiPeerFactory

Method Summary

java.lang.String	<u>getName</u> () Returns the name of this JtapiPeer object instance.
<u>Provider</u>	<u>getProvider</u> (java.lang.String providerString) Returns an instance of a Provider object given a string argument which contains the desired service name.
java.lang.String[]	<u>getServices</u> () Returns the services that this implementation supports.

Method Detail

getName

```
public java.lang.String getName()
```

Returns the name of this JtapiPeer object instance. This name is the same name used as an argument to *JtapiPeerFactory.getJtapiPeer()* method.

Returns:

The name of this JtapiPeer object instance.

getServices

```
public java.lang.String[] getServices()
```

Returns the services that this implementation supports. This method returns `null` if no services as supported.

Returns:

The services that this implementation supports.

getProvider

```
public Provider getProvider(java.lang.String providerString)  
    throws ProviderUnavailableException
```

Returns an instance of a **Provider** object given a string argument which contains the desired service name. Optional arguments may also be provided in this string, with the following format:

< service name > ; arg1 = val1; arg2 = val2; ...

Where < service name > is not optional, and each optional argument pair which follows is separated by a semi-colon. The keys for these arguments is implementation specific, except for two standard-defined keys:

1. login: provides the login user name to the Provider.
2. passwd: provides a password to the Provider.

If the argument is null, this method returns some default Provider as determined by the JtapiPeer object. The returned Provider is in the `Provider.OUT_OF_SERVICE` state.

Post-conditions:

1. `this.getProvider().getState() == Provider.OUT_OF_SERVICE`

Parameters:

`providerString` - The name of the desired service plus an optional arguments.

Returns:

An instance of the Provider object.

Throws:

`ProviderUnavailableException` - Indicates a Provider corresponding to the given string is unavailable.

javax.telephony

Class JtapiPeerFactory

```
java.lang.Object
|
+-- javax.telephony.JtapiPeerFactory
```

```
public class JtapiPeerFactory
    extends java.lang.Object
```

The `JtapiPeerFactory` class is a class by which applications obtain a Provider object.

Introduction

Applications use this class to first obtain a class which implements the `JtapiPeer` interface. The `JtapiPeer` interface represents a particular vendor's implementation of JTAPI. The term 'peer' is Java nomenclature for "a particular platform-specific implementation of a Java interface or API". This term has the same meaning for the Java Telephony API. Applications are not permitted to create an instance of the `JtapiPeerFactory` class. Through an installation procedure provided by each implementator, a `JtapiPeer` class is made available to an application environment. When applications have a `JtapiPeer` object for a particular platform-dependent implementation, they may obtain a Provider object via that interface. The details of that interface are discussed in the specification for the `JtapiPeer` interface.

Obtaining a JtapiPeer Object

Applications use the `JtapiPeerFactory.getJtapiPeer()` method to obtain a `JtapiPeer` object. The argument to this method is a classname which represents an object which implements the `JtapiPeer` interface. This object and the classname under which it can be found must be supplied by the vendor of the implementation. Note that this object is not a Provider, however, this interface is used to obtain Provider objects from that particular implementation.

The Java Telephony API places conventions on vendors on the classname they use for their `JtapiPeer` object. This class name *must* begin with the domain name assigned to the vendor in reverse order. Because the space of domain names is managed, this scheme ensures that collisions between two different vendor's implementations will not happen. For example, an implementation from Sun Microsystems's will have "com.sun" as the prefix to its `JtapiPeer` class. After the reversed domain name, vendors are free to choose any class hierarchy they desire.

Default JtapiPeer

Additionally, the vendor providing the `JtapiPeer` class may supply a `DefaultJtapiPeer.class` class file. When placed in the classpath of applications, this class (which must implement the `JtapiPeer` interface) becomes the default `JtapiPeer` object returned by the `JtapiPeerFactory.getJtapiPeer()` method. By convention the default class name must be `DefaultJtapiPeer`.

In basic environments, applications and users do not want the burden of finding out the class name in order to use a particular implementation. Therefore, the `JtapiPeerFactory` class supports a mechanism for applications to obtain the default implementation for their system. If applications use a `null` argument to the `JtapiPeerFactory.getJtapiPeer()` method, they will be returned the default installed implementation on their system if it exists.

Note: It is the responsibility of implementation vendors to supply a version of a `DefaultJtapiPeer` or some means to alias their peer implementation along with a means to place that `DefaultJtapiPeer` class in the application classpath.

See Also:

[JtapiPeer](#)

Method Summary

static <u>JtapiPeer</u>	<code>getJtapiPeer(java.lang.String jtapiPeerName)</code> Returns an instance of a <code>JtapiPeer</code> object given a fully qualified classname of the class which implements the <code>JtapiPeer</code> object.
---	---

Methods inherited from class java.lang.Object

`equals`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Method Detail

getJtapiPeer

```
public static JtapiPeer getJtapiPeer(java.lang.String jtapiPeerName)
                                   throws JtapiPeerUnavailableException
```

Returns an instance of a `JtapiPeer` object given a fully qualified classname of the class which implements the `JtapiPeer` object.

If no classname is provided (`null`), a default class named `DefaultJtapiPeer` is chosen as the classname to load. If it does not exist or is not installed in the CLASSPATH as the default, a `JtapiPeerUnavailableException` exception is thrown.

Parameters:

`jtapiPeerName` - The classname of the `JtapiPeer` object class.

Returns:

An instance of the JtapiPeer object.

Throws:

JtapiPeerUnavailableException - Indicates that the JtapiPeer specified by the classname is not available.

javax.telephony**Class JtapiPeerUnavailableException**

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.telephony.JtapiPeerUnavailableException

```

public class **JtapiPeerUnavailableException**
 extends java.lang.Exception

The JtapiPeerUnavailableException indicates that the JtapiPeer (i.e. a particular implementation of JTAPI) is unavailable on the current system.

See Also:

[Serialized Form](#)

Constructor Summary**JtapiPeerUnavailableException()**

Constructor with no string.

JtapiPeerUnavailableException(java.lang.String s)

Constructor which takes a string description.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

JtapiPeerUnavailableException

public **JtapiPeerUnavailableException()**

Constructor with no string.

JtapiPeerUnavailableException

public **JtapiPeerUnavailableException**(java.lang.String s)

Constructor which takes a string description.

Parameters:

s - description of the fault

javax.telephony

Interface MetaEvent

All Known Subinterfaces:

MultiCallMetaEvent, SingleCallMetaEvent

public interface **MetaEvent**
 extends Event

The **MetaEvent** interface is the base interface for all JTAPI Meta events. All Meta events must extend this interface.

An individual **MetaEvent** conveys, directly and with necessary details, what an application needs to know to respond to a higher-level JTAPI event.

Currently, meta events are defined in pairs, and they convey three types of information: first, the type of higher-level operation which has occurred; second, the beginning and end of the sequence of "normal" JTAPI events which were generated to convey the consequential JTAPI model state changes that occurred because of this higher-level operation; third, which JTAPI entities were involved in the higher-level operation.

A JTAPI implementation is alerted to changes in the state of the associated telephony platform, and reflects that state by sending a stream of JTAPI objects, delimited by MetaEvents. An application learns of the details of that state change by processing all the events between the starting and ending MetaEvents.

Generally the application may draw incorrect conclusions about the current state of the associated telephony platform if it decides to act before processing all the events delimited by the MetaEvents. Specifically, an application which wishes to submit queries to the JTAPI implementation about the current state of JTAPI entities should not submit the query until it has processed the matching "ending" MetaEvent.

The specific Meta event is indicated by the `Event.getID()` value returned by the event. The specific Meta event provides context information about the higher-level event.

The `Event.getMetaEvent` method returns the Meta event associated with a "normal" event (or returns null).

Fields inherited from interface javax.telephony.Event

<u>CAUSE CALL CANCELLED</u> , <u>CAUSE DEST NOT OBTAINABLE</u> , <u>CAUSE INCOMPATIBLE DESTINATION</u> , <u>CAUSE LOCKOUT</u> , <u>CAUSE NETWORK CONGESTION</u> , <u>CAUSE NETWORK NOT OBTAINABLE</u> , <u>CAUSE NEW CALL</u> , <u>CAUSE NORMAL</u> , <u>CAUSE RESOURCES NOT AVAILABLE</u> , <u>CAUSE SNAPSHOT</u> , <u>CAUSE UNKNOWN</u>

Methods inherited from interface javax.telephony.EventgetCause, getID, getMetaEvent, getSource

javax.telephony***Class MethodNotSupportedException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.telephony.MethodNotSupportedException

```

```

public class MethodNotSupportedException
extends java.lang.Exception

```

The `MethodNotSupportedException` indicates that the method which was invoked is not supported by the implementation.

See Also:

[Serialized Form](#)

Constructor Summary**MethodNotSupportedException()**

Constructor with no string.

MethodNotSupportedException(java.lang.String s)

Constructor which takes a string description.

Methods inherited from class java.lang.Throwable

`fillInStackTrace`, `getLocalizedMessage`, `getMessage`, `printStackTrace`, `printStackTrace`, `printStackTrace`, `toString`

Methods inherited from class java.lang.Object

`equals`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Detail

MethodNotSupportedException

public **MethodNotSupportedException**()

Constructor with no string.

MethodNotSupportedException

public **MethodNotSupportedException**(java.lang.String s)

Constructor which takes a string description.

Parameters:

s - description of the fault

javax.telephony

Interface SingleCallMetaEvent

public interface **SingleCallMetaEvent**
extends [MetaEvent](#)

The `SingleCallMetaEvent` interface is the base interface for all single-call Call Meta events. All single-call `MetaEvent`'s must extend this interface. Events which extend this interface are reported via the `CallListener` interface.

An individual `SingleCallMetaEvent` conveys, directly and with necessary details, what an application needs to know to respond to a higher-level call event that only effects a single call. The specific `SingleCallMetaEvent` event is indicated by the `Event.getID()` value returned by the event.

Since all these Meta events relate to a single JTAPI Call entity, this interface provides the method `getCall` to grant the application access to the affected Call.

The core package defines events which are reported when high-level actions occur. The event IDs (as returned by `Event.getID()`) are: `SINGLECALL_META_PROGRESS_STARTED`, `SINGLECALL_META_PROGRESS_ENDED`, `MULTICALL_META_TRANSFER_STARTED` and `MULTICALL_META_TRANSFER_ENDED`.

See Also:

[CallListener](#), [Call](#)

Field Summary

static int	<u>SINGLECALL_META_PROGRESS_ENDED</u> The SINGLECALL_META_PROGRESS_ENDED event indicates that the current call in the telephony platform has changed state, and all the events that were associated with that change have now been reported.
static int	<u>SINGLECALL_META_PROGRESS_STARTED</u> The SINGLECALL_META_PROGRESS_STARTED event indicates that the current call in the telephony platform has changed state, and events will follow which indicate the changes to this call.
static int	<u>SINGLECALL_META_SNAPSHOT_ENDED</u> The SINGLECALL_META_SNAPSHOT_ENDED event indicates that the JTAPI implementation has finished reporting a set of simulated state changes that in effect construct the current state of the call.
static int	<u>SINGLECALL_META_SNAPSHOT_STARTED</u> The SINGLECALL_META_SNAPSHOT_STARTED event indicates that the JTAPI implementation is reporting to the application the current state of the call on the associated telephony platform, by reporting a set of simulated state changes that in effect construct the current state of the call.

Fields inherited from interface javax.telephony.Event

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN

Method Summary

<u>Call</u>	<u>getCall()</u> Returns the Call object associated with the higher-level operation reported by this SingleCallMetaEvent event.
-------------	---

Methods inherited from interface javax.telephony.Event

getCause, getID, getMetaEvent, getSource

Field Detail

SINGLECALL_META_PROGRESS_STARTED

```
public static final int SINGLECALL_META_PROGRESS_STARTED
```

The `SINGLECALL_META_PROGRESS_STARTED` event indicates that the current call in the telephony platform has changed state, and events will follow which indicate the changes to this call.

This constant indicates a specific event passed via a `SingleCallMetaEvent` event, and is reported on the `CallListener` interface.

SINGLECALL_META_PROGRESS_ENDED

```
public static final int SINGLECALL_META_PROGRESS_ENDED
```

The `SINGLECALL_META_PROGRESS_ENDED` event indicates that the current call in the telephony platform has changed state, and all the events that were associated with that change have now been reported.

This constant indicates a specific event passed via a `SingleCallMetaEvent` event, and is reported on the `CallListener` interface.

SINGLECALL_META_SNAPSHOT_STARTED

```
public static final int SINGLECALL_META_SNAPSHOT_STARTED
```

The `SINGLECALL_META_SNAPSHOT_STARTED` event indicates that the JTAPI implementation is reporting to the application the current state of the call on the associated telephony platform, by reporting a set of simulated state changes that in effect construct the current state of the call. The events which follow convey that current state.

This constant indicates a specific event passed via a `SingleCallMetaEvent` event, and is reported on the `CallListener` interface.

SINGLECALL_META_SNAPSHOT_ENDED

```
public static final int SINGLECALL_META_SNAPSHOT_ENDED
```

The `SINGLECALL_META_SNAPSHOT_ENDED` event indicates that the JTAPI implementation has finished reporting a set of simulated state changes that in effect construct the current state of the call.

This constant indicates a specific event passed via a `SingleCallMetaEvent` event, and is reported on the `CallListener` interface.

Method Detail

getCall

```
public Call getCall()
```

Returns the `Call` object associated with the higher-level operation reported by this `SingleCallMetaEvent` event.

Returns:

The result `Call` associated with this event.

javax.telephony

Interface MultiCallMetaEvent

public interface **MultiCallMetaEvent**
 extends [MetaEvent](#)

The `MultiCallMetaEvent` interface is the base interface for all multiple-call Call Meta events (multi-call MetaEvent). All multi-call MetaEvent's must extend this interface. Events which extend this interface are reported via the `CallListener` interface.

An individual `MultiCallMetaEvent` conveys, directly and with necessary details, what an application needs to know to respond to a multiple-call higher-level call event. The specific `MultiCallMetaEvent` event is indicated by the `Event.getID()` value returned by the event.

The core package defines events which are reported when high-level actions occur. The event IDs (as returned by `Event.getID()`) are: `MULTICALL_META_MERGE_STARTED`, `MULTICALL_META_MERGE_ENDED`, `MULTICALL_META_TRANSFER_STARTED` and `MULTICALL_META_TRANSFER_ENDED`.

See Also:

[CallEvent](#), [MetaEvent](#), [CallListener](#), [Call](#)

Field Summary

static int	<u>MULTICALL META MERGE ENDED</u> The <code>MULTICALL_META_MERGE_ENDED</code> event indicates that calls have merged, and that all state change events resulting from this merge have been reported.
static int	<u>MULTICALL META MERGE STARTED</u> The <code>MULTICALL_META_MERGE_STARTED</code> event indicates that calls are merging, and events will follow which indicate the changes to those calls.
static int	<u>MULTICALL META TRANSFER ENDED</u> The <code>MULTICALL_META_TRANSFER_ENDED</code> event indicates that a transfer has completed, and that all state change events resulting from this transfer have been reported.
static int	<u>MULTICALL META TRANSFER STARTED</u> The <code>MULTICALL_META_TRANSFER_STARTED</code> event indicates that a transfer is occurring, and events will follow which indicate the changes to the affected calls.

Fields inherited from interface javax.telephony.Event

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN

Method Summary

<u>Call</u>	<u>getNewCall()</u> Returns the Call object associated with the result of the multiple-call operation reported by this MultiCallMetaEvent event.
<u>Call</u>	<u>getOldCalls()</u> Returns an array of Call objects which were considered input to the multiple-call operation reported by this MultiCallMetaEvent event.

Methods inherited from interface javax.telephony.Event

getCause, getID, getMetaEvent, getSource

Field Detail***MULTICALL_META_MERGE_STARTED***

```
public static final int MULTICALL_META_MERGE_STARTED
```

The MULTICALL_META_MERGE_STARTED event indicates that calls are merging, and events will follow which indicate the changes to those calls. The former calls involved are returned by `MultiCallMetaEvent.getOldCalls()` and the resulting call is returned by `MultiCallMetaEvent.getNewCall()`.

This constant indicates a specific event passed via a `MultiCallMetaEvent` event, and is reported on the `CallListener` interface.

MULTICALL_META_MERGE_ENDED

public static final int **MULTICALL_META_MERGE_ENDED**

The **MULTICALL_META_MERGE_ENDED** event indicates that calls have merged, and that all state change events resulting from this merge have been reported.

The former calls involved are returned by `MultiCallMetaEvent.getOldCalls()` and the resulting call is returned by `MultiCallMetaEvent.getNewCall()`.

This constant indicates a specific event passed via a `MultiCallMetaEvent` event, and is reported on the `CallListener` interface.

MULTICALL_META_TRANSFER_STARTED

public static final int **MULTICALL_META_TRANSFER_STARTED**

The **MULTICALL_META_TRANSFER_STARTED** event indicates that a transfer is occurring, and events will follow which indicate the changes to the affected calls. The former calls involved are returned by `MultiCallMetaEvent.getOldCalls()` and the resulting call is returned by `MultiCallMetaEvent.getNewCall()`.

This constant indicates a specific event passed via a `MultiCallMetaEvent` event, and is reported on the `CallListener` interface.

MULTICALL_META_TRANSFER_ENDED

public static final int **MULTICALL_META_TRANSFER_ENDED**

The **MULTICALL_META_TRANSFER_ENDED** event indicates that a transfer has completed, and that all state change events resulting from this transfer have been reported.

The former calls involved are returned by `MultiCallMetaEvent.getOldCalls()` and the resulting call is returned by `MultiCallMetaEvent.getNewCall()`.

This constant indicates a specific event passed via a `MultiCallMetaEvent` event, and is reported on the `CallListener` interface.

Method Detail

getNewCall

public Call **getNewCall()**

Returns the Call object associated with the result of the multiple-call operation reported by this MultiCallMetaEvent event.

Returns:

The result Call associated with this event.

getOldCalls

public Call **getOldCalls()**

Returns an array of Call objects which were considered input to the multiple-call operation reported by this MultiCallMetaEvent event.

Returns:

An array of old Calls associated with this event's operation.

javax.telephony

Class PlatformException

```

java.lang.Object
|
+-- java.lang.Throwable
    |
    +-- java.lang.Exception
        |
        +-- java.lang.RuntimeException
            |
            +-- javax.telephony.PlatformException

```

```

public class PlatformException
extends java.lang.RuntimeException

```

A **PlatformException** indicates an implementation-specific exception. The specific exceptions which implementations throw is documented in their release notes.

JTAPI v1.1.1 NOTE: PlatformException extends Java's RuntimeException. This permits it to be thrown from a JTAPI method without being declared in its signature. Note that no JTAPI methods declare PlatformException to be thrown. This is a change from v1.1, but does not affect applications.

Since PlatformException typically denotes some form of unrecoverable platform-dependent error, invoking the method again typically does not yield success. These types of exceptions are often best dealt with at a higher level, in a top-level "try-catch" block where the entire application could be restarted.

See Also:

[Serialized Form](#)

Constructor Summary

PlatformException()

Constructor with no string.

PlatformException(java.lang.String s)

Constructor which takes a string description.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object
--

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

PlatformException

public **PlatformException**()

Constructor with no string.

PlatformException

public **PlatformException**(java.lang.String s)

Constructor which takes a string description.

Parameters:

s - description of the fault.

javax.telephony***Class PrivilegeViolationException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.telephony.PrivilegeViolationException

```

public class **PrivilegeViolationException**
 extends java.lang.Exception

A **PrivilegeViolationException** indicates that an action pertaining to a certain object failed because the application did not have the proper security permissions to execute that command.

This class stores the type of privilege not available which is obtained via the **PrivilegeViolationException.getType()** method.

See Also:

[Serialized Form](#)

Field Summary

static int	<u>DESTINATION_VIOLATION</u> A privilege violation occurred on the destination.
static int	<u>ORIGINATOR_VIOLATION</u> A privilege violation occurred on the originator.
static int	<u>UNKNOWN_VIOLATION</u> A privilege violation occurred at an unknown place.

Constructor Summary

<u>PrivilegeViolationException</u> (int type) Constructor, takes a type but no string.	
<u>PrivilegeViolationException</u> (int type, java.lang.String s) Constructor, takes a type and a string.	

Method Summary

int	getType() Returns the type of privilege which is not available.
-----	--

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

ORIGINATOR_VIOLATION

public static final int **ORIGINATOR_VIOLATION**

A privilege violation occurred on the originator.

DESTINATION_VIOLATION

public static final int **DESTINATION_VIOLATION**

A privilege violation occurred on the destination.

UNKNOWN_VIOLATION

public static final int **UNKNOWN_VIOLATION**

A privilege violation occurred at an unknown place.

Constructor Detail

PrivilegeViolationException

```
public PrivilegeViolationException(int type)
```

Constructor, takes a type but no string.

Parameters:

type - kind of violation

PrivilegeViolationException

```
public PrivilegeViolationException(int type,  
                                   java.lang.String s)
```

Constructor, takes a type and a string.

Parameters:

type - kind of violation

s - description of the violation

Method Detail

getType

```
public int getType()
```

Returns the type of privilege which is not available.

Returns:

The type of privilege.

javax.telephony **Interface Provider**

All Known Subinterfaces:

MobileProvider, NetworkSelection

public interface **Provider**

A **Provider** represents the telephony software-entity that interfaces with a telephony subsystem.

Introduction

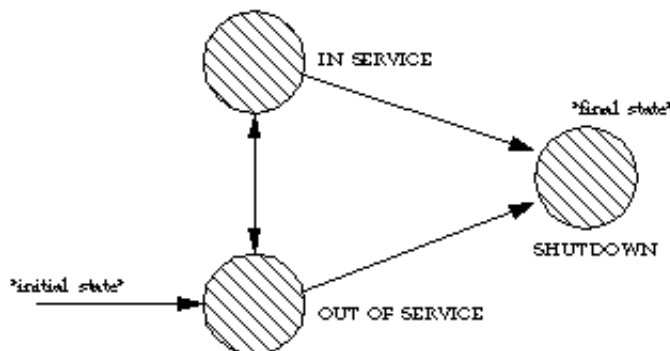
The telephony subsystem could be a PBX connected to a server machine, a telephony/fax card in a desktop machine or a networking technology such as IP or ATM.

Provider States

The Provider may either be in one of the following states: **Provider.IN_SERVICE**, **Provider.OUT_OF_SERVICE**, or **Provider.SHUTDOWN**. The Provider state represents whether any action on that Provider may be valid. The following tables describes each state:

Provider.IN_SERVICE	This state indicates that the Provider is currently alive and available for use.
Provider.OUT_OF_SERVICE	This state indicates that a Provider is temporarily not available for use. Many methods in the Java Telephony API are invalid when the Provider is in this state. Providers may come back in service at any time, however, the application can take no direct action to cause this change.
Provider.SHUTDOWN:	This state indicates that a Provider is permanently no longer available for use. Most methods in the Java Telephony API are invalid when the Provider is in this state. Applications may use the Provider.shutdown() method on this interface to cause a Provider to move into the Provider.SHUTDOWN state.

The following diagram shows the allowable state transitions for the Provider as defined by the core package.



Obtaining a Provider

A Provider is created and returned by the `JtapiPeer.getProvider()` method which is given a string to describe the desired Provider. This method sets up any needed communication paths between the application and the Provider. The string given is one of the services listed in the `JtapiPeer.getServices()`. JtapiPeers particular implementation on a system and may be obtained via the `JtapiPeerFactory` class.

Listeners and Events

Each time a state changes occurs on a Provider, the application is notified via an *event*. This event is reported via the `ProviderListener` interface. Applications instantiate objects which implement this interface and use the `Provider.addProviderListener()` method to begin the delivery of events. All Provider events reported via this interface extend the `ProviderEvent` interface. Therefore, applications may then query the event object returned for the specific state change, via the `Event.getID()` method. In the core package API, when the Provider changes state, a `ProviderEvent` is sent to the `ProviderListener`, having one of the following event ids: `PROVIDER_IN_SERVICE`, `PROVIDER_OUT_OF_SERVICE`, and `PROVIDER_SHUTDOWN`. A `ProviderEvent` with event id `PROVIDER_EVENT_TRANSMISSION_ENDED` is delivered to all `ProviderListeners` when the Provider becomes unobservable and is the final event delivered to the listener.

Call Objects and Providers

The Provider maintains knowledge of the calls currently associated with it. Applications may obtain an array of these Calls via the `Provider.getCalls()` method. A Provider may have Calls associated with it which were created before it came into existence. It is the responsibility of the implementation of the Provider to model and report all existing telephone calls which were created prior to the Provider's lifetime. The Provider maintains references to all calls until they move into the `Call.INVALID` state.

Applications may create new Calls using the `Provider.createCall()` method. A new Call is returned in the `Call.IDLE` state. Applications may then use this idle Call to place new telephone calls. Once created, this new Call object is returned via the `Provider.getCalls()` method.

The Provider's domain

The term *Provider's domain* refers to the collection of Address and Terminal objects which are local to the Provider, and typically, can be controlled by the Provider. For example, the domain of a Provider of a desktop system with an ISDN card are the Address(es) and Terminal(s) which represent that ISDN endpoint. The domain of a Provider for a PBX may be the Addresses and Terminals in that PBX. The Provider implementation controls access to Addresses and Terminals by limiting the domain it presents to the application.

Address and Terminal Objects

An Address object represents what we commonly think of as a "telephone number." In more rare implementations where the underlying network is not a telephony network, this address may represent something else, such as an IP address. Regardless, it represents a *logical* endpoint of a telephone call. A Terminal represents a physical hardware endpoint connected to the telephone network. An example of a Terminal is a telephone set, but a Terminal does not have to take the form of this limited and traditional example. Addresses and Terminals are in a many-to-many relationship. An Address may contain multiple Terminals, and Terminals may contain multiple Addresses. See the specifications for the Address and Terminal objects for more information.

Unlike Call objects, applications may not create Terminal or Address objects. The Provider begins with knowledge of certain Terminal and Address objects defined as its local domain. This list is static once the Provider is created. The Addresses and Terminals in the Provider's domain are reported via the `Provider.getAddresses()` and `Provider.getTerminals()` methods, respectively.

Other Addresses and Terminals may be created sometime during the operation of the Provider when the Provider learns of new instances of these objects. These new objects, however, represent Addresses and Terminals outside the Provider's domain. For example, if the Provider's domain is a PBX, the Provider will know about all Addresses and Terminals in this PBX when the Provider first starts. Any Addresses and Terminals it subsequently learns about are outside this PBX. These Address and Terminal objects outside this PBX are not reported via the `Provider.getTerminals()` and `Provider.getAddresses()` methods. Address and Terminal objects outside of the Provider's domain are referred to as *remote*.

Capabilities: Static and Dynamic

The Provider interface supports methods to return *static* capabilities for each of the Java Telephony call model objects. Static capabilities provide applications with information concerning the ability of the implementation to perform certain methods. These static capabilities indicate whether a method is implemented for a particular type of object and does not depend upon the particular instance of the object nor the current state of the call model. Those methods for which the static capability returns false will throw a `MethodNotSupportedException` when invoked. The static capability methods supported on this interface are: `Provider.getProviderCapabilities()`, `Provider.getCallCapabilities()`, `Provider.getAddressCapabilities()`, `Provider.getTerminalCapabilities()`, `Provider.getConnectionCapabilities()`, and `Provider.getTerminalConnectionCapabilities()`.

Dynamic capabilities tell the application which actions are possible at the time this method is invoked based upon the implementations knowledge of its ability to successfully perform the action. This determination may be based upon arguments passed to this method, the current state of the call model, or some implementation-specific knowledge. These indications do not guarantee that a particular method can be successfully invoked, however. This interface returns the dynamic capabilities for a Provider object via the `Provider.getCapabilities()` method. Note that this method is distinct from the static capability method `Provider.getProviderCapabilities()`.

Multiple Providers and Multiple Applications

It is not guaranteed or expected that objects (Call, Terminal, etc.) instantiated through one Provider will be usable with another Provider. Therefore, an application that uses two providers must keep all the objects relating to these providers separate. In the future, there may be a mechanism whereby a Provider may share objects with another Provider if they are speaking to the same telephony hardware, however, such capabilities are not available in this release.

Also, multiple applications may request and communicate with the same Provider implementation. Typically, since each application executes in its own object space, each will have its own instance of the Provider object. These two different Provider objects may, in fact, be proxies for a centralized Provider instance. Certain methods in JTAPI are specified to affect only the invoking applications and have no affect on others. The only example in the core package is the `Provider.shutdown()` method.

See Also:

[JtapiPeer](#), [JtapiPeerFactory](#), [ProviderListener](#)

Field Summary

static int	<u>IN_SERVICE</u> This state indicates that a Provider is currently available for use.
static int	<u>OUT_OF_SERVICE</u> This state indicates that a Provider is currently not available for use.
static int	<u>SHUTDOWN</u> This state indicates that a Provider is permanently no longer available for use.

Method Summary

	void	<u>addObserver</u> (<u>ProviderObserver</u> observer) Adds an observer to the Provider.
	void	<u>addProviderListener</u> (<u>ProviderListener</u> listener) Adds an listener to the Provider.
	<u>Call</u>	<u>createCall</u> () Creates and returns a new instance of the Call object.

<u>Address</u>	getAddress (java.lang.String number) Returns an Address object which corresponds to the telephone number string provided.
<u>AddressCapabilities</u>	getAddressCapabilities () Returns the static capabilities of the Address object.
<u>AddressCapabilities</u>	getAddressCapabilities (Terminal terminal) Deprecated. Since JTAPI v1.2. This method has been replaced by the Provider.getAddressCapabilities() method.
<u>Address[]</u>	getAddresses () Returns an array of Addresses associated with the Provider and within the Provider's domain.
<u>CallCapabilities</u>	getCallCapabilities () Returns the static capabilities of the Call object.
<u>CallCapabilities</u>	getCallCapabilities (Terminal terminal, Address address) Deprecated. Since JTAPI v1.2. This method has been replaced by the Provider.getCallCapabilities() method.
<u>Call[]</u>	getCalls () Returns an array of Call objects currently associated with the Provider.
<u>ProviderCapabilities</u>	getCapabilities () Returns the dynamic capabilities for the instance of the Provider object.
<u>ConnectionCapabilities</u>	getConnectionCapabilities () Returns the static capabilities of the Connection object.
<u>ConnectionCapabilities</u>	getConnectionCapabilities (Terminal terminal, Address address) Deprecated. Since JTAPI v1.2. This method has been replaced by the Provider.getConnectionCapabilities() method.
java.lang.String	getName () Returns the unique string name of this Provider.
<u>ProviderObserver[]</u>	getObservers () Returns a list of all ProviderObservers associated with this Provider object.
<u>ProviderCapabilities</u>	getProviderCapabilities () Returns the static capabilities of the Provider object.
<u>ProviderCapabilities</u>	getProviderCapabilities (Terminal terminal) Deprecated. Since JTAPI v1.2. This method has been replaced by the Provider.getProviderCapabilities() method.
<u>ProviderListener[]</u>	getProviderListeners () Returns a list of all ProviderListeners associated with this Provider object.
int	getState () Returns the current state of this Provider.

<u>Terminal</u>	getTerminal (java.lang.String name) Returns an instance of the Terminal class which corresponds to the given name.
<u>TerminalCapabilities</u>	getTerminalCapabilities () Returns the static capabilities of the Terminal object.
<u>TerminalCapabilities</u>	getTerminalCapabilities (Terminal terminal) Deprecated. Since JTAPI v1.2. This method has been replaced by the Provider.getTerminalCapabilities() method.
<u>TerminalConnectionCapabilities</u>	getTerminalConnectionCapabilities () Returns the static capabilities of the TerminalConnection object.
<u>TerminalConnectionCapabilities</u>	getTerminalConnectionCapabilities (Terminal terminal) Deprecated. Since JTAPI v1.2. This method has been replaced by the Provider.getTerminalConnectionCapabilities() method.
<u>Terminal</u> []	getTerminals () Returns an array of Terminals associated with the Provider and within the Provider's domain.
void	removeObserver (ProviderObserver observer) Removes the given observer from the Provider.
void	removeProviderListener (ProviderListener listener) Removes the given listener from the Provider.
void	shutdown () Instructs the Provider to shut itself down and perform all necessary cleanup.

Field Detail

IN_SERVICE

```
public static final int IN_SERVICE
```

This state indicates that a Provider is currently available for use.

OUT_OF_SERVICE

```
public static final int OUT_OF_SERVICE
```

This state indicates that a Provider is currently not available for use. Many methods in the Java Telephony API are invalid when the Provider is in this state. Providers may come back in service at any time, however, the application can take no direct action to cause this change.

SHUTDOWN

public static final int **SHUTDOWN**

This state indicates that a Provider is permanently no longer available for use. Most methods in the Java Telephony API are invalid when the Provider is in this state.

Method Detail

getState

public int **getState()**

Returns the current state of this Provider. The value is one of `Provider.IN_SERVICE`, `Provider.OUT_OF_SERVICE`, or `Provider.SHUTDOWN`.

Returns:

The current state of this Provider.

getName

public java.lang.String **getName()**

Returns the unique string name of this Provider. Each different Provider must have a unique string associated with it. This is the same string which the application passed to the `JtapiPeer.getProvider()` method to create this Provider instance.

Returns:

The unique String name of this Provider.

See Also:

[JtapiPeer](#)

getCalls

public [Call](#)[] **getCalls()**
throws [ResourceUnavailableException](#)

Returns an array of Call objects currently associated with the Provider. When a Call moves into the `Call.INVALID` state, the Provider loses its reference to this Call. Therefore, all Calls returned by this method must either be in the `Call.IDLE` or `Call.ACTIVE` state. This method returns null if the Provider has zero calls associated with it.

Post-conditions:

1. Let Calls calls[] = Provider.getCalls()
2. calls == null or calls.length >= 1
3. For all i, calls[i].getState() == Call.IDLE or Call.ACTIVE

Returns:

An array of Call objects currently associated with this Provider.

Throws:

ResourceUnavailableException - Indicates the number of calls present in the Provider is too great to return as a static array.

getAddress

```
public Address getAddress(java.lang.String number)
    throws InvalidArgumentException
```

Returns an Address object which corresponds to the telephone number string provided. If the provided name does not correspond to an Address known by the Provider and within the Provider's domain, InvalidArgumentException is thrown. In other words, the Address must appear in the list generated by `Provider.getAddresses()`.

Pre-conditions:

1. Let Address address = this.getAddress(number);
2. address is an element of this.getAddresses();

Post-conditions:

1. Let Address address = this.getAddress(number);
2. address is an element of this.getAddresses();

Parameters:

number - The telephone address string.

Returns:

The Address object corresponding to the given telephone number.

Throws:

InvalidArgumentException - The name, e.g. telephone number of the Address does not correspond to the name of any Address object known to the Provider or within the Provider's domain.

getAddresses

```
public Address[] getAddresses()
    throws ResourceUnavailableException
```

Returns an array of Addresses associated with the Provider and within the Provider's domain. This list is static (i.e. is does not change) after the Provider is first created. If no Address objects are associated with this Provider, then this method returns null.

Post-conditions:

1. Let Address[] addresses = this.getAddresses()
2. addresses == null or addresses.length >= 1

Returns:

An array of Addresses known by this provider.

Throws:

ResourceUnavailableException - Indicates the number of addresses present in the Provider is too great to return as a static array.

getTerminals

```
public Terminal[] getTerminals()
    throws ResourceUnavailableException
```

Returns an array of Terminals associated with the Provider and within the Provider's domain. Each Terminal possesses a unique name, which is assigned to it by the JTAPI implementation. If there are no Terminals associated with this Provider, then this method returns null.

Post-conditions:

1. Let Terminal[] terminals = this.getTerminals()
2. terminals == null or terminals.length >= 1

Returns:

An array of Terminals in the Provider's local domain.

Throws:

ResourceUnavailableException - Indicates the number of terminals present in the Provider is too great to return as a static array.

getTerminal

```
public Terminal getTerminal(java.lang.String name)
    throws InvalidArgumentException
```

Returns an instance of the Terminal class which corresponds to the given name. Each Terminal has a unique name associated with it, which is assigned to it by the JTAPI implementation. If no Terminal is available for the given name within the Provider's domain, this method throws the InvalidArgumentException. This Terminal must be in the array generated by `Provider.getTerminals()`.

Pre-conditions:

1. Let Terminal terminal = this.getTerminal(name);
2. terminals is an element of this.getTerminals();

Post-conditions:

1. Let Terminal terminal = this.getTerminal(name);
2. terminal is an element of this.getTerminals();

Parameters:

name - The name of desired Terminal object.

Returns:

The Terminal object associated with the given name.

Throws:

InvalidArgumentException - The name provided does not correspond to a name of any Terminal known to the Provider or within the Provider's domain.

shutdown

```
public void shutdown()
```

Instructs the Provider to shut itself down and perform all necessary cleanup. Applications invoke this method when they no longer intend to use the Provider, most often right before they exit. This method is intended to allow the Provider to perform any necessary cleanup which would not be taken care of when the Java objects are garbage collected. This method causes the Provider to move into the `Provider.SHUTDOWN` state, in which it will stay indefinitely.

If the Provider is already in the `Provider.SHUTDOWN` state, this method does nothing. The invocation of this method should not affect other applications which are using the same implementation of the Provider object.

Post-conditions:

1. `this.getState() == Provider.SHUTDOWN`

createCall

```
public Call createCall()
    throws ResourceUnavailableException,
           InvalidStateException,
           PrivilegeViolationException,
           MethodNotSupportedException
```

Creates and returns a new instance of the Call object. This new call object is in the `Call.IDLE` state and has no Connections. An exception is generated if a new call cannot be created for various reasons. This Provider must be in the `Provider.IN_SERVICE` state, otherwise an `InvalidStateException` is thrown.

Pre-conditions:

1. `this.getState() == Provider.IN_SERVICE`

Post-conditions:

1. `this.getState() == Provider.IN_SERVICE`
2. Let `Call call = this.createCall();`
3. `call.getState() == Call.IDLE.`
4. `call.getConnections() == null`
5. `call` is an element of `this.getCalls()`

Returns:

The new Call object.

Throws:

ResourceUnavailableException - An internal resource necessary to create a new Call object is unavailable.

InvalidStateException - The Provider is not in the `Provider.IN_SERVICE` state.

PrivilegeViolationException - The application does not have the proper authority to create a new telephone call object.

MethodNotSupportedException - The implementation does not support creating new Call objects.

addObserver

```
public void addObserver(ProviderObserver observer)
    throws ResourceUnavailableException,
           MethodNotSupportedException
```

Adds an observer to the Provider. Provider-related events are reported via the `ProviderObserver` interface. The Provider object will report events to this interface for the lifetime of the Provider object or until the observer is removed with the `Provider.removeObserver()` method or until the Provider is no longer observable.

If the Provider becomes unobservable, a `ProvObservationEndedEv` is delivered to the application as is final event. No further events are delivered to the observer unless it is explicitly re-added by the application. When an observer receives a `ProvObservationEndedEv` it is no longer reported via the `Provider.getObservers()` method.

This method is valid anytime and has no pre-conditions. Application must have the ability to add observers to Providers so they can monitor the changes in state in the Provider.

If an application attempts to add an instance of an observer already present on this Provider, then repeated attempts to add the instance of the observer will silently fail, i.e. multiple instances of an observer are not added and no exception will be thrown.

Post-conditions:

1. observer is an element of `this.getObservers()`

Parameters:

observer - The observer being added.

Throws:

MethodNotSupportedException - The observer cannot be added at this time.

ResourceUnavailableException - The resource limit for the numbers of observers has been exceeded.

getObservers

```
public ProviderObserver[] getObservers()
```

Returns a list of all ProviderObservers associated with this Provider object. If no observers exist on this Provider, then this method returns null.

Post-conditions:

1. Let `ProviderObserver[] observers = this.getObservers()`
2. `observers == null` or `observers.length >= 1`

Returns:

An array of ProviderObserver objects associated with this Provider.

removeObserver

```
public void removeObserver(ProviderObserver observer)
```

Removes the given observer from the Provider. The given observer will no longer receive events generated by this Provider object. The final event will be the `ProvObservationEndedEv` event and will no longer be listed by the `Provider.getObservers()` method.

Also, if an observer is not part of the Provider, then this method fails silently, i.e. no observer is removed and no exception is thrown.

Post-conditions:

1. `observer` is not an element of `this.getObservers()`
2. `ProvObservationEndedEv` is delivered to `observer`

Parameters:

`observer` - The observer which is being removed.

getProviderCapabilities

```
public ProviderCapabilities getProviderCapabilities()
```

Returns the static capabilities of the Provider object. The value of these capabilities will not change over the lifetime of the Provider. They represent the static abilities of the implementation to perform certain methods on the Provider object. For all capability methods which return false, the invocation of that method will always throw `MethodNotSupportedException`.

NOTE: This method is different from the `Provider.getCapabilities()`, method which returns the dynamic capabilities of a Provider object instance.

Returns:

The static capabilities of the Provider object.

getCallCapabilities

public CallCapabilities **getCallCapabilities()**

Returns the static capabilities of the Call object. The value of these capabilities will not change over the lifetime of the Provider. They represent the static abilities of the implementation to perform certain methods on the Call object. For all capability methods which return false, the invocation of that method will always throw `MethodNotSupportedException`.

Returns:

The static capabilities of the Call object.

getAddressCapabilities

public AddressCapabilities **getAddressCapabilities()**

Returns the static capabilities of the Address object. The value of these capabilities will not change over the lifetime of the Provider. They represent the static abilities of the implementation to perform certain methods on the Address object. For all capability methods which return false, the invocation of that method will always throw `MethodNotSupportedException`.

Returns:

The static capabilities of the Address object.

getTerminalCapabilities

public TerminalCapabilities **getTerminalCapabilities()**

Returns the static capabilities of the Terminal object. The value of these capabilities will not change over the lifetime of the Provider. They represent the static abilities of the implementation to perform certain methods on the Terminal object. For all capability methods which return false, the invocation of that method will always throw `MethodNotSupportedException`.

Returns:

The static capabilities of the Address object.

getConnectionCapabilities

public ConnectionCapabilities **getConnectionCapabilities()**

Returns the static capabilities of the Connection object. The value of these capabilities will not change over the lifetime of the Provider. They represent the static abilities of the implementation to perform certain methods on the Connection object. For all capability methods which return false, the invocation of that method will always throw `MethodNotSupportedException`.

Returns:

The static capabilities of the Connection object.

getTerminalConnectionCapabilities

```
public TerminalConnectionCapabilities getTerminalConnectionCapabilities()
```

Returns the static capabilities of the TerminalConnection object. The value of these capabilities will not change over the lifetime of the Provider. They represent the static abilities of the implementation to perform certain methods on the TerminalConnection object. For all capability methods which return false, the invocation of that method will always throw `MethodNotSupportedException`.

Returns:

The static capabilities of the TerminalConnection object.

getCapabilities

```
public ProviderCapabilities getCapabilities()
```

Returns the dynamic capabilities for the instance of the Provider object. Dynamic capabilities tell the application which actions are possible at the time this method is invoked based upon the implementations knowledge of its ability to successfully perform the action. This determination may be based upon argument passed to this method, the current state of the call model, or some implementation-specific knowledge. These indications do not guarantee that a particular method can be successfully invoked, however.

There are no arguments passed into this method for dynamic Provider capabilities

NOTE: This method is different from the `Provider.getProviderCapabilities()` method which returns the static capabilities for the Provider object.

Returns:

The dynamic Provider capabilities.

getProviderCapabilities

```
public ProviderCapabilities getProviderCapabilities(Terminal terminal)
                                                    throws InvalidArgumentException,
                                                    PlatformException
```

Deprecated. Since JTAPI v1.2. This method has been replaced by the `Provider.getProviderCapabilities()` method.

Returns the `ProviderCapabilities` object with respect to a `Terminal`. If null is passed as a `Terminal` parameter, the general/provider-wide `Provider` capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Provider.getProviderCapabilities()` method returns the static Provider capabilities. This method now should simply invoke the `Provider.getProviderCapabilities(void)` method.

Parameters:

`terminal` - This parameter is ignored in JTAPI v1.2 and later.

Returns:

The dynamic ProviderCapabilities capabilities.

Throws:

InvalidArgumentException - This exception is never thrown in JTAPI v1.2 and later.

PlatformException - A platform-specific exception occurred.

getCallCapabilities

```
public CallCapabilities getCallCapabilities(Terminal terminal,  
                                           Address address)  
    throws InvalidArgumentException,  
           PlatformException
```

Deprecated. Since JTAPI v1.2. This method has been replaced by the `Provider.getCallCapabilities()` method.

Gets the CallCapabilities object with respect to a Terminal and an Address. If null is passed as a Terminal/Address parameter, the general/provider-wide Call capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Provider.getCallCapabilities()` method returns the static Call capabilities. This method now should simply invoke the `Provider.getCallCapabilities(void)` method.

Parameters:

`terminal` - This argument is ignored in JTAPI v1.2 and later.

`address` - This argument is ignored in JTAPI v1.2 and later.

Returns:

The static CallCapabilities cpabilities.

Throws:

InvalidArgumentException - This exception is never thrown in JTAPI v1.2 and later.

PlatformException - A platform-specific exception occurred.

getConnectionCapabilities

```
public ConnectionCapabilities getConnectionCapabilities(Terminal terminal,  
                                                         Address address)  
    throws InvalidArgumentException,  
           PlatformException
```

Deprecated. *Since JTAPI v1.2. This method has been replaced by the `Provider.getConnectionCapabilities()` method.*

Gets the `ConnectionCapabilities` object with respect to a `Terminal` and an `Address`. If null is passed as a `Terminal/Address` parameter, the general/ provider-wide `Connection` capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Provider.getConnectionCapabilities()` method returns the static `Connection` capabilities. This method now should simply invoke the `Provider.getConnectionCapabilities(void)` method.

Parameters:

`terminal` - This argument is ignored in JTAPI v1.2 and later.
`address` - This argument is ignored in JTAPI v1.2 and later.

Returns:

The static `ConnectionCapabilities` capabilities

Throws:

`InvalidArgumentException` - This exception is never thrown in JTAPI v1.2 and later.
`PlatformException` - A platform-specific exception occurred.

getAddressCapabilities

```
public AddressCapabilities getAddressCapabilities(Terminal terminal)
                                         throws InvalidArgumentException,
                                              PlatformException
```

Deprecated. *Since JTAPI v1.2. This method has been replaced by the `Provider.getAddressCapabilities()` method.*

Gets the `AddressCapabilities` object with respect to a `Terminal`. If null is passed as a `Terminal` parameter, the general/provider-wide `Address` capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Provider.getAddressCapabilities()` method returns the static `Address` capabilities. This method now should simply invoke the `Provider.getAddressCapabilities(void)` method.

Parameters:

`terminal` - This argument is ignored in JTAPI v1.2 and later.

Throws:

`InvalidArgumentException` - This exception is never thrown in JTAPI v1.2 and later.
`PlatformException` - A platform-specific exception occurred.

getTerminalConnectionCapabilities

```
public TerminalConnectionCapabilities getTerminalConnectionCapabilities(Terminal terminal)
                                           throws InvalidArgumentException,
                                           PlatformException
```

Deprecated. Since JTAPI v1.2. This method has been replaced by the *Provider.getTerminalConnectionCapabilities()* method.

Gets the TerminalConnectionCapabilities of a Terminal. If null is passed as a Terminal parameter, the general/provider-wide TerminalConnection capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The *Provider.getTerminalConnectionCapabilities()* method returns the static TerminalConnection capabilities. This method now should simply invoke the *Provider.getTerminalConnectionCapabilities(void)* method.

Parameters:

terminal - This argument is ignored in JTAPI v1.2 and later. are being queried

Throws:

InvalidArgumentException - This exception is never thrown in JTAPI v1.2 and later.

PlatformException - A platform-specific exception occurred.

getTerminalCapabilities

```
public TerminalCapabilities getTerminalCapabilities(Terminal terminal)
                                           throws InvalidArgumentException,
                                           PlatformException
```

Deprecated. Since JTAPI v1.2. This method has been replaced by the *Provider.getTerminalCapabilities()* method.

Gets the TerminalCapabilities object with respect to a Terminal. If null is passed as a Terminal parameter, the general/provider-wide Terminal capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The *Provider.getTerminalCapabilities()* method returns the static Terminal capabilities. This method now should simply invoke the *Provider.getTerminalCapabilities(void)* method.

Parameters:

terminal - This argument is ignored in JTAPI v1.2 and later.

Throws:

InvalidArgumentException - This exception is never thrown in JTAPI v1.2 and later.

PlatformException - A platform-specific exception occurred.

addProviderListener

```
public void addProviderListener(ProviderListener listener)
                               throws ResourceUnavailableException,
                                       MethodNotSupportedException
```

Adds an listener to the Provider. Provider-related events are reported via the `ProviderListener` interface. The Provider object will report events to this interface for the lifetime of the Provider object or until the listener is removed with the `Provider.removeProviderListener()` method or until the Provider is no longer observable.

If the Provider becomes unobservable, a `ProviderEvent` with id `PROVIDER_EVENT_TRANSMISSION_ENDED` is delivered to the application as is final event. No further events are delivered to the listener unless it is explicitly re-added by the application. When an listener receives `PROVIDER_EVENT_TRANSMISSION_ENDED` it is no longer reported via the `Provider.getProviderListeners()` method.

This method is valid anytime and has no pre-conditions. Application must have the ability to add listeners to Providers so they can monitor the changes in state in the Provider.

If an application attempts to add an instance of an listener already present on this Provider, then repeated attempts to add the instance of the listener will silently fail, i.e. multiple instances of an listener are not added and no exception will be thrown.

Post-conditions:

1. listener is an element of `this.getProviderListeners()`

Parameters:

listener - The listener being added.

Throws:

MethodNotSupportedException - The listener cannot be added at this time.

ResourceUnavailableException - The resource limit for the numbers of listeners has been exceeded.

getProviderListeners

```
public ProviderListener[] getProviderListeners()
```

Returns a list of all `ProviderListeners` associated with this Provider object. If no listeners exist on this Provider, then this method returns null.

Post-conditions:

1. Let `ProviderListener[] listeners = this.getProviderListeners()`
2. `listeners == null` or `listeners.length >= 1`

Returns:

An array of `ProviderListener` objects associated with this Provider.

removeProviderListener

```
public void removeProviderListener(ProviderListener listener)
```

Removes the given listener from the Provider. The given listener will no longer receive events generated by this Provider object. The final event will have id `PROVIDER_EVENT_TRANSMISSION_ENDED`; subsequently, the listener will no longer be listed by the `Provider.getProviderListeners()` method.

Also, if the listener is not currently registered with the Provider, then this method fails silently, i.e. no listener is removed and no exception is thrown.

Post-conditions:

1. listener is not an element of `this.getProviderListeners()`
2. `ProviderEvent` with id `PROVIDER_EVENT_TRANSMISSION_ENDED` is delivered to listener

Parameters:

`listener` - The listener which is being removed.

javax.telephony **Interface ProviderEvent**

All Known Subinterfaces:

MobileProviderEvent

public interface **ProviderEvent**
extends Event

The `ProviderEvent` interface is the base interface for all Provider related events. All events which pertain to the Provider object must extend this interface. Events which extend this interface are reported via the `ProviderListener` interface.

The core package defines events which are reported when the Provider changes state. These events are: `PROVIDER_IN_SERVICE`, `PROVIDER_OUT_OF_SERVICE`, and `PROVIDER_SHUTDOWN`. Also, the core package defines the `PROVIDER_EVENT_TRANSMISSION_ENDED` event which is sent when the Provider becomes unobservable.

The `ProviderEvent.getProvider` method on this interface returns the Provider associated with the Provider event.

See Also:

Event, ProviderListener, Provider

Field Summary

static int	<u>PROVIDER_EVENT_TRANSMISSION_ENDED</u> The <code>PROVIDER_EVENT_TRANSMISSION_ENDED</code> event indicates that the application will no longer receive Provider events on the instance of the <code>ProviderListener</code> .
static int	<u>PROVIDER_IN_SERVICE</u> The <code>PROVIDER_IN_SERVICE</code> interface indicates that the state of the Provider object has changed to <code>Provider.IN_SERVICE</code> .
static int	<u>PROVIDER_OUT_OF_SERVICE</u> The <code>PROVIDER_OUT_OF_SERVICE</code> interface indicates that the state of the Provider object has changed to <code>Provider.OUT_OF_SERVICE</code> .
static int	<u>PROVIDER_SHUTDOWN</u> The <code>PROVIDER_SHUTDOWN</code> interface indicates that the state of the Provider object has changed to <code>Provider.SHUTDOWN</code> .

Fields inherited from interface javax.telephony.Event

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN

Method Summary

<u>Provider</u>	<u>getProvider()</u> Returns the Provider associated with this Provider event.
-----------------	--

Methods inherited from interface javax.telephony.Event

getCause, getID, getMetaEvent, getSource

Field Detail***PROVIDER_IN_SERVICE***

```
public static final int PROVIDER_IN_SERVICE
```

The PROVIDER_IN_SERVICE interface indicates that the state of the Provider object has changed to Provider.IN_SERVICE.

This constant indicates a specific event passed via a ProviderEvent event, and is reported on the ProviderListener interface.

PROVIDER_EVENT_TRANSMISSION_ENDED

```
public static final int PROVIDER_EVENT_TRANSMISSION_ENDED
```

The PROVIDER_EVENT_TRANSMISSION_ENDED event indicates that the application will no longer receive Provider events on the instance of the ProviderListener.

This constant indicates a specific event passed via a `ProviderEvent` event, and is reported on the `ProviderListener` interface.

PROVIDER_OUT_OF_SERVICE

```
public static final int PROVIDER_OUT_OF_SERVICE
```

The `PROVIDER_OUT_OF_SERVICE` interface indicates that the state of the `Provider` object has changed to `Provider.OUT_OF_SERVICE`.

This constant indicates a specific event passed via a `ProviderEvent` event, and is reported on the `ProviderListener` interface.

PROVIDER_SHUTDOWN

```
public static final int PROVIDER_SHUTDOWN
```

The `PROVIDER_SHUTDOWN` interface indicates that the state of the `Provider` object has changed to `Provider.SHUTDOWN`.

This constant indicates a specific event passed via a `ProviderEvent` event, and is reported on the `ProviderListener` interface.

Method Detail

getProvider

```
public Provider getProvider()
```

Returns the `Provider` associated with this `Provider` event.

Returns:

The `Provider` associated with this event.

javax.telephony

Interface ProviderListener

All Known Subinterfaces:

MobileProviderListener

public interface **ProviderListener**
extends java.util.EventListener

The **ProviderListener** interface reports all changes which happen to the **Provider** object.

These state changes are reported as events to the **ProviderListener** method corresponding to the type of event.

Applications must instantiate an object which implements this interface and then use the **Provider.addProviderListener()** method to register the object to receive all future events associated with the **Provider** object.

The **ProviderListener** methods each receive one event at a time, unlike the **ProviderObserver**.

Provider State Changes

In the core package, an event is delivered whenever the **Provider** changes state. The event interfaces which correspond to these state changes for the core package are: **providerInService**, **providerOutOfService**, and **providerShutdown**.

Provider Observation Ending

At various times, the underlying implementation may not be able to observe the **Provider**. In these instances, the **ProviderListener** is sent an **JProvObservationEndedEv** event, via the **ProviderListener.providerListenerEndedEvent()** method. This indicates that the application will not receive further events associated with the **Provider** object. This observer will no longer be reported via the **Provider.getProviderListeners** method.

See Also:

Provider, ProviderEvent

Method Summary

void	<u>providerEventTransmissionEnded</u> (<u>ProviderEvent</u> event) The <code>providerEventTransmissionEnded</code> method is called to indicate that the application will no longer receive Provider events on the instance of the <code>ProviderListener</code> .
void	<u>providerInService</u> (<u>ProviderEvent</u> event) The <code>providerInService</code> method is called to indicate that the state of the Provider object has changed to <code>Provider.IN_SERVICE</code> .
void	<u>providerOutOfService</u> (<u>ProviderEvent</u> event) The <code>providerOutOfService</code> method is called to indicate that the state of the Provider object has changed to <code>Provider.OUT_OF_SERVICE</code> .
void	<u>providerShutdown</u> (<u>ProviderEvent</u> event) The <code>providerShutdown</code> method is called to indicate that the state of the Provider object has changed to <code>Provider.SHUTDOWN</code> .

Method Detail

providerInService

```
public void providerInService(ProviderEvent event)
```

The `providerInService` method is called to indicate that the state of the Provider object has changed to `Provider.IN_SERVICE`.

Parameters:

a - `ProviderEvent` with event ID `ProviderEvent.PROVIDER_IN_SERVICE`.

providerEventTransmissionEnded

```
public void providerEventTransmissionEnded(ProviderEvent event)
```

The `providerEventTransmissionEnded` method is called to indicate that the application will no longer receive Provider events on the instance of the `ProviderListener`.

Parameters:

a - `ProviderEvent` with event ID
`ProviderEvent.PROVIDER_EVENT_TRANSMISSION_ENDED`.

providerOutOfService

```
public void providerOutOfService(ProviderEvent event)
```

The `providerOutOfService` method is called to indicate that the state of the Provider object has changed to `Provider.OUT_OF_SERVICE`.

Parameters:

a - ProviderEvent with event ID `ProviderEvent.PROVIDER_OUT_OF_SERVICE`.

providerShutdown

```
public void providerShutdown(ProviderEvent event)
```

The `providerShutdown` method is called to indicate that the state of the Provider object has changed to `Provider.SHUTDOWN`.

Parameters:

a - ProviderEvent with event ID `ProviderEvent.PROVIDER_SHUTDOWN`.

javax.telephony **Interface ProviderObserver**

public interface **ProviderObserver**

The **ProviderObserver** interface reports all changes which happen to the **Provider** object.

Introduction

These changes are reported as events to the **ProviderObserver.providerChangedEvent()** method. Applications must instantiate an object which implements this interface and then use the **Provider.addObserver()** method to register the object to receive all future events associated with the **Provider** object.

The **ProviderObserver.providerChangedEvent()** method receives an array of events which all must extend the **ProvEv** interface. Since several changes may happen to a single JTAPI object at once, a list of events is needed to convey those changes which happen at the same time. Applications iterate through the array of events provided.

Provider State Changes

In the core package, an event is delivered whenever the **Provider** changes state. The event interfaces which correspond to these state changes for the core package are: **ProvInServiceEv**, **ProvOutOfServiceEv**, and **ProvShutdownEv**.

Provider Observation Ending

At various times, the underlying implementation may not be able to observe the **Provider**. In these instances, the **ProviderObserver** is sent an **ProvObservationEndedEv** event. This indicates that the application will not receive further events associated with the **Provider** object. This observer will no longer be reported via the **Provider.getObservers()** method.

See Also:

[ProvEv](#), [ProvInServiceEv](#), [ProvOutOfServiceEv](#), [ProvShutdownEv](#),
[ProvObservationEndedEv](#)

Method Summary

void	<u>providerChangedEvent</u> (ProvEv[] eventList) Reports all events associated with the Provider object.
------	---

Method Detail

providerChangedEvent

```
public void providerChangedEvent(ProvEv[] eventList)
```

Reports all events associated with the Provider object. This method passes an array of ProvEv objects as its arguments which correspond to the list of events representing the changes to the Provider object.

Parameters:

eventList - The list of Provider events.

javax.telephony***Class ProviderUnavailableException***

```
java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--java.lang.RuntimeException
            |
            +--javax.telephony.ProviderUnavailableException
```

```
public class ProviderUnavailableException
    extends java.lang.RuntimeException
```

The **ProviderUnavailableException** indicates that the Provider is currently not available to the application. This exception extends Java's **RuntimeException**, and therefore can be thrown on any JTAPI method. It is typically thrown in two situations: when **JtapiPeer.getProvider()** is called or on any method when the Provider is in a **Provider.SHUTDOWN** state. Because this method extends **RuntimeException**, it can be thrown from any method without being declared.

This exception is thrown on **JtapiPeer.getProvider()** when the requested Provider is not available to the application for a number of reasons, including when an invalid service string or optional argument was given. If this exception is thrown on a random JTAPI method, it indicates that the method call is invalid because the Provider is not in the "in service" state.

This exception stores the reason for the failure which may be obtained via the **ProviderUnavailableException.getCause()** method.

See Also:

[Serialized Form](#)

Field Summary

static int	<u>CAUSE_INVALID_ARGUMENT</u> Constant definition for an invalid optional argument given to <code>JtapiPeer.getProvider()</code> .
static int	<u>CAUSE_INVALID_SERVICE</u> Constant definition for an invalid service string given to <code>JtapiPeer.getProvider()</code> .
static int	<u>CAUSE_NOT_IN_SERVICE</u> Constant definition for the Provider not in the "in service" state.
static int	<u>CAUSE_UNKNOWN</u> Constant definition for an unknown cause.

Constructor Summary

<u>ProviderUnavailableException()</u> Constructor with no cause and string.	
<u>ProviderUnavailableException(int cause)</u> Constructor which takes a cause string.	
<u>ProviderUnavailableException(int cause, java.lang.String s)</u> Constructor which takes both a string and a cause.	
<u>ProviderUnavailableException(java.lang.String s)</u> Constructor which takes a string description.	

Method Summary

int	<u>getCause()</u> Returns the cause for this exception.
-----	---

Methods inherited from class java.lang.Throwable

`fillInStackTrace`, `getLocalizedMessage`, `getMessage`, `printStackTrace`, `printStackTrace`, `printStackTrace`, `toString`

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail*CAUSE_UNKNOWN*

public static final int **CAUSE_UNKNOWN**

Constant definition for an unknown cause.

CAUSE_NOT_IN_SERVICE

public static final int **CAUSE_NOT_IN_SERVICE**

Constant definition for the Provider not in the "in service" state.

CAUSE_INVALID_SERVICE

public static final int **CAUSE_INVALID_SERVICE**

Constant definition for an invalid service string given to `JtapiPeer.getProvider()`.

CAUSE_INVALID_ARGUMENT

public static final int **CAUSE_INVALID_ARGUMENT**

Constant definition for an invalid optional argument given to `JtapiPeer.getProvider()`.

Constructor Detail*ProviderUnavailableException*

public **ProviderUnavailableException()**

Constructor with no cause and string.

ProviderUnavailableException

```
public ProviderUnavailableException(int cause)
```

Constructor which takes a cause string.

Parameters:

cause - reason code for this fault

ProviderUnavailableException

```
public ProviderUnavailableException(java.lang.String s)
```

Constructor which takes a string description.

Parameters:

s - description of the fault

ProviderUnavailableException

```
public ProviderUnavailableException(int cause,  
                                     java.lang.String s)
```

Constructor which takes both a string and a cause.

Parameters:

cause - reason code for the fault

s - description of the fault

Method Detail

getCause

```
public int getCause()
```

Returns the cause for this exception.

Returns:

The cause of this exception.

javax.telephony***Class ResourceUnavailableException***

```

java.lang.Object
|
+--java.lang.Throwable
    |
    +--java.lang.Exception
        |
        +--javax.telephony.ResourceUnavailableException

```

```

public class ResourceUnavailableException
extends java.lang.Exception

```

The `ResourceUnavailableException` indicates that a resource inside the system is not available to complete an operation. The type embodied in this exception further clarifies what is not available and is obtained via the `ResourceUnavailableException.getType()` method.

See Also:

[Serialized Form](#)

Field Summary

static int	<u>NO_DIALTONE</u> No dialtone detected.
static int	<u>OBSERVER_LIMIT_EXCEEDED</u> The number of observers existing already reached the limit.
static int	<u>ORIGINATOR_UNAVAILABLE</u> The originating device was not available for this action.
static int	<u>OUTSTANDING_METHOD_EXCEEDED</u> The internal resources to handle another method have been exceeded.
static int	<u>TRUNK_LIMIT_EXCEEDED</u> The number of trunks which are currently in use has been exceeded.
static int	<u>UNKNOWN</u> Indicates the specific reason is unspecified.
static int	<u>UNSPECIFIED_LIMIT_EXCEEDED</u> An internal resource, unspecified by the implementation, has been exceeded.
static int	<u>USER_RESPONSE</u> A user has not responded in the time allowed by an implementation.

Constructor Summary

ResourceUnavailableException(int type)

Constructor, takes a type but no string.

ResourceUnavailableException(int type, java.lang.String s)

Constructor, takes a type and a string.

Method Summary

int **getType**()

Returns the type of resource which was unavailable.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

UNKNOWN

public static final int **UNKNOWN**

Indicates the specific reason is unspecified.

ORIGINATOR_UNAVAILABLE

public static final int **ORIGINATOR_UNAVAILABLE**

The originating device was not available for this action.

OBSERVER_LIMIT_EXCEEDED

public static final int **OBSERVER_LIMIT_EXCEEDED**

The number of observers existing already reached the limit.

TRUNK_LIMIT_EXCEEDED

public static final int **TRUNK_LIMIT_EXCEEDED**

The number of trunks which are currently in use has been exceeded.

OUTSTANDING_METHOD_EXCEEDED

public static final int **OUTSTANDING_METHOD_EXCEEDED**

The internal resources to handle another method have been exceeded.

UNSPECIFIED_LIMIT_EXCEEDED

public static final int **UNSPECIFIED_LIMIT_EXCEEDED**

An internal resource, unspecified by the implementation, has been exceeded.

NO_DIALTONE

public static final int **NO_DIALTONE**

No dialtone detected.

USER_RESPONSE

public static final int **USER_RESPONSE**

A user has not responded in the time allowed by an implementation.

Constructor Detail

ResourceUnavailableException

```
public ResourceUnavailableException(int type)
```

Constructor, takes a type but no string.

ResourceUnavailableException

```
public ResourceUnavailableException(int type,  
                                     java.lang.String s)
```

Constructor, takes a type and a string.

Method Detail

getType

```
public int getType()
```

Returns the type of resource which was unavailable.

Returns:

The type of resource unavailable.

javax.telephony

Interface Terminal

All Known Subinterfaces:

[CallControlTerminal](#), [MobileTerminal](#), [PhoneTerminal](#)

public interface **Terminal**

A `Terminal` represents a physical hardware endpoint connected to the telephony domain.

Introduction

An example of a `Terminal` is a telephone set, but a `Terminal` does not have to take the form of this limited and traditional example. For example, computer workstations and hand-held devices are modeled as `Terminals` if they act as physical endpoints in a telephony network.

A `Terminal` object has a string *name* which is unique for all `Terminal` objects. The `Terminal` does not attempt to interpret this string in any way. This name is first assigned when the `Terminal` is created and does not change throughout the lifetime of the object. The method `Terminal.getName()` returns the name of the `Terminal` object. The name of the `Terminal` may not have any real-world interpretation. Typically, `Terminals` are chosen from a list of `Terminals` obtained from an `Address` object.

`Terminal` objects may be classified into two categories: *local* and *remote*. Local `Terminal` objects are those terminals which are part of the `Provider`'s local domain. These `Terminal` objects are created by the implementation of the `Provider` object when it is first instantiated. All of the `Provider`'s local terminals are reported via the `Provider.getTerminals()` method. Remote `Terminal` objects are those outside of the `Provider`'s domain which the `Provider` learns about during its lifetime through various happenings (e.g. an incoming call from a currently unknown address). Remote `Terminal` objects are **not** reported via the `Provider.getTerminals()` method. Note that applications never explicitly create new `Terminal` objects.

Address and Terminal objects

`Address` and `Terminal` objects exist in a many-to-many relationship. An `Address` object may have zero or more `Terminals` associated with it. For each `Terminal` associated with an `Address`, that `Terminal` must also reflect its association with the `Address`. Since the implementation creates `Address` (and `Terminal`) objects, it is responsible for insuring the correctness of these relationships. The `Terminals` associated with an `Address` is given by the `Address.getTerminals()` method.

An association between an `Address` and `Terminal` object indicates that the `Terminal` contains the `Address` object as one of its telephone number addresses. In many instances, a telephone set (represented by a `Terminal` object) has only one telephone number (represented by an `Address` object) associated with it. In more complex configurations, telephone sets may have several telephone numbers associated with them. Likewise, a telephone number may appear on more than one telephone set. For example, feature phones in PBX environments may exhibit this configuration.

Terminals and Call objects

Terminal objects represent the *physical* endpoints of a telephone call. With respect to a single Address endpoint on a Call, multiple physical Terminal endpoints may exist. Terminal objects are related to Call objects via the TerminalConnection object. TerminalConnection objects are associated with Call indirectly via Connections. A Terminal may be associated with a Call only if one of its Addresses is associated with the Call. The TerminalConnection object has a *state* which describes the current relationship between the Connection and the Terminal. Each Terminal object may be part of more than one telephone call, and in each case, is represented by a separate TerminalConnection object. The `Terminal.getTerminalConnections()` method returns all TerminalConnection object currently associated with the Terminal.

A Terminal object is associated with a Connection until the TerminalConnection moves into the `TerminalConnection.DROPPED` state. At that time, the TerminalConnection is no longer reported via the `Terminal.getTerminalConnections()` method. Therefore, the `Terminal.getTerminalConnections()` method never reports a TerminalConnection in the `TerminalConnection.DROPPED` state.

Existing Telephone Calls

The Java Telephony API specification states that the implementation is responsible for reporting all existing telephone calls when a Provider is first created. This implies that an Terminal object must report information regarding existing telephone calls to that Terminal. In other words, Terminal objects must report all TerminalConnection objects which represent existing telephone calls.

Terminal Listeners and Events

All changes in an Terminal object are reported via the TerminalListener interface. Applications instantiate an object which implements this interface and begins this delivery of events to this object using the `Terminal.addTerminalListener()` method. All Terminal-related events extend the `TerminalEvent` interface provided in the core package. Applications receive events on an listener until the listener is removed via the `Terminal.removeTerminalListener()` method or until the Terminal is no longer observable. In these instances, each TerminalListener receives a `TERMINAL_EVENT_TRANSMISSION_ENDED` as its final event.

Currently in the core package, the only Terminal-related event is `TERMINAL_EVENT_TRANSMISSION_ENDED`.

Call Listeners

At times, applications may want to monitor a particular Terminal for all Calls which come to that Terminal. For example, a desktop telephone application is only interested in telephone calls associated with a particular agent terminal. To achieve this sort of Terminal-based Call monitoring applications may *add* CallListeners to an Terminal via the `Terminal.addCallListener()` method.

When a CallListener is added to an Terminal, this listener instance is immediately added to all Calls at this Terminal and is added to all Calls which come to this Terminal in the future. These listeners remain on the telephone call as long as the Terminal is associated with the telephone call.

The specification of `Terminal.addCallListener()` contains more precise semantics.

See Also:

[TerminalListener](#), [CallListener](#)

Method Summary	
<code>void</code>	<u>addCallListener</u> (<u>CallListener</u> listener) Adds an listener to a Call object when this Terminal object first becomes part of that Call.
<code>void</code>	<u>addCallObserver</u> (<u>CallObserver</u> observer) Adds an observer to a Call object when this Terminal object first becomes part of that Call.
<code>void</code>	<u>addObserver</u> (<u>TerminalObserver</u> observer) Adds an observer to the Terminal.
<code>void</code>	<u>addTerminalListener</u> (<u>TerminalListener</u> listener) Adds an listener to the Terminal.
<code><u>Address</u>[]</code>	<u>getAddresses</u> () Returns an array of Address objects associated with this Terminal object.
<code><u>CallListener</u>[]</code>	<u>getCallListeners</u> () Returns a list of all CallListeners associated with this Terminal object.
<code><u>CallObserver</u>[]</code>	<u>getCallObservers</u> () Returns a list of all CallObservers associated with this Terminal object.
<code><u>TerminalCapabilities</u></code>	<u>getCapabilities</u> () Returns the dynamic capabilities for the instance of the Terminal object.
<code>java.lang.String</code>	<u>getName</u> () Returns the name of the Terminal.
<code><u>TerminalObserver</u>[]</code>	<u>getObservers</u> () Returns a list of all TerminalObservers associated with this Terminal object.
<code><u>Provider</u></code>	<u>getProvider</u> () Returns the Provider associated with this Terminal.
<code><u>TerminalCapabilities</u></code>	<u>getTerminalCapabilities</u> (<u>Terminal</u> terminal, <u>Address</u> address) Deprecated. Since JTAPI v1.2. This method has been replaced by the <code>Terminal.getCapabilities()</code> method.

<u>TerminalConnection</u> []	getTerminalConnections() Returns an array of TerminalConnection objects associated with this Terminal.
<u>TerminalListener</u> []	getTerminalListeners() Returns a list of all TerminalListeners associated with this Terminal object.
void	removeCallListener() <u>CallListener</u> listener Removes the given CallListener from the Terminal.
void	removeCallObserver() <u>CallObserver</u> observer Removes the given CallObserver from the Terminal.
void	removeObserver() <u>TerminalObserver</u> observer Removes the given observer from the Terminal.
void	removeTerminalListener() <u>TerminalListener</u> listener Removes the given listener from the Terminal.

Method Detail

getName

public java.lang.String **getName()**

Returns the name of the Terminal. Each Terminal possesses a unique name. This name is assigned by the implementation and may or may not carry a real-world interpretation.

Returns:

The name of the Terminal.

getProvider

public Provider **getProvider()**

Returns the Provider associated with this Terminal. This Provider object is valid throughout the lifetime of the Terminal and does not change once the Terminal is created.

Returns:

The Provider associated with this Terminal.

getAddresses

```
public Address[] getAddresses()
```

Returns an array of Address objects associated with this Terminal object. The Terminal object must have at least one Address object associated with it. This list does not change throughout the lifetime of the Terminal object.

Post-conditions:

1. Let Address[] addrs = this.getAddresses()
2. addrs.length >= 1

Returns:

An array of Address objects associated with this Terminal.

getTerminalConnections

```
public TerminalConnection[] getTerminalConnections()
```

Returns an array of TerminalConnection objects associated with this Terminal. Once a TerminalConnection is added to a Terminal, the Terminal maintains a reference until the TerminalConnection moves into the TerminalConnection.DROPPED state. Therefore, all TerminalConnections returned by this method will never be in the TerminalConnection.DROPPED state. If there are no TerminalConnections associated with this Terminal, this method returns null.

Post-conditions:

1. Let TerminalConnection tc[] = this.getTerminalConnections()
2. tc == null or tc.length >= 1
3. For all i, tc[i].getState() != TerminalConnection.DROPPED

Returns:

An array of TerminalConnection objects associated with this Terminal.

addObserver

```
public void addObserver(TerminalObserver observer)
    throws ResourceUnavailableException,
           MethodNotSupportedException
```

Adds an observer to the Terminal. The TerminalObserver reports all Terminal-related state changes as events. The Terminal object will report events to this TerminalObserver object for the lifetime of the Terminal object or until the observer is removed with the Terminal.removeObserver() or until the Terminal is no longer observable. In these instances, a TermObservationEndedEv is delivered to the observer as its final event. The observer will receive no events after TermObservationEndedEv unless the observer is explicitly re-added via the Terminal.addObserver() method. Also, once an observer receives an TermObservationEndedEv, it will no longer be reported via the Terminal.getObservers().

If an application attempts to add an instance of an observer already present on this Terminal, this attempt will silently fail, i.e. multiple instances of an observer are not added and no exception will be thrown.

Currently, only the `TermObservationEndedEv` event is defined by the core package and delivered to the `TerminalObserver`.

Post-conditions:

1. observer is an element of `this.getObservers()`

Parameters:

observer - The observer being added.

Throws:

`MethodNotSupportedException` - The observer cannot be added at this time

`ResourceUnavailableException` - The resource limit for the numbers of observers has been exceeded.

getObservers

```
public TerminalObserver[] getObservers()
```

Returns a list of all `TerminalObservers` associated with this Terminal object. If there are no observers associated with this Terminal, this method returns null.

Post-conditions:

1. Let `TerminalObserver[] obs = this.getObservers()`
2. `obs == null` or `obs.length >= 1`

Returns:

An array of `TerminalObserver` objects associated with this Terminal.

removeObserver

```
public void removeObserver(TerminalObserver observer)
```

Removes the given observer from the Terminal. If successful, the observer will no longer receive events generated by this Terminal object. As its final event, the `TerminalObserver` receives a `TermObservationEndedEv`.

If an observer is not part of the Terminal, then this method fails silently, i.e. no observer is removed and no exception is thrown.

Post-conditions:

1. A `TermObservationEndedEv` event is reported to the observer as its final event.
2. observer is not an element of `this.getObservers()`

Parameters:

observer - The observer which is being removed.

addCallObserver

```
public void addCallObserver(CallObserver observer)
    throws ResourceUnavailableException,
    MethodNotSupportedException
```

Adds an observer to a Call object when this Terminal object first becomes part of that Call. This method permits applications to select a Terminal object in which they are interested and automatically have the implementation attach an observer to all present and future Calls which come to this Terminal.

JTAPI v1.0 Semantics

In version 1.0 of the Java Telephony API specification, the application monitored the Terminal object for the TermCallAtTermEv event. This event indicated that a Call has come to this Terminal. Then, applications would manually add an observer to the Call. With this architecture, potentially dangerous race conditions existed while an application was adding an observer to the Call. As a result, this mechanism has been replaced in version 1.1.

JTAPI v1.1 Semantics

In version 1.1 of the specification, the TermCallAtTermEv event does not exist and this method replaces the functionality described above. Instead of monitoring for a TermCallAtTermEv event, the application simply uses the `Terminal.addCallObserver()` method, and observer will be added to new telephone calls at this Terminal automatically.

If an application attempts to add an instance of a call observer already present on this Terminal, these repeated attempts will silently fail, i.e. multiple instances of a call observer are not added and no exception will be thrown.

When a call observer is added to an Terminal with this method, the following behavior is exhibited by the implementation.

1. It is immediately associated with any existing calls at the Terminal and a snapshot of those calls are reported to the call observer. For each of these calls, the observer is reported via `Call.getObservers()`.
2. It is associated with all future calls which come to this Terminal. For each new call, the observer is reported via `Call.getObservers()`.

Note that the definition of the term ".. when a call is at a Terminal" means that the Call contains a Connection which contains a TerminalConnection with this Terminal as its Terminal.

Call Observer Lifetime

For all call observers which are present on Calls because of this method, the following behavior is exhibited with respect to the lifetime of the call.

1. The call observer receives events until the Call is no longer at this Terminal. In this case, the call observer will be re-applied to the Call if the Call returns to this Terminal at some point in the future.
2. The call observer is removed with `Call.removeObserver()`. Note that this only affects the instance of the call observer for that particular call. If the Call subsequently leaves and returns to the Terminal, the observer will be re-applied.
3. The Call can no longer be monitored by the implementation.
4. The Call moves into the `Call.INVALID` state.

When the CallObserver leaves the Call because of one of the reasons above, it receives a `CallObservationEndedEv` as its final event.

Call Observer on Multiple Addresses and Terminals

It is possible for an application to add CallObservers to more than one Address and Terminal (using `Address.addCallObserver()` and `Terminal.addCallObserver()`, respectively). The rules outlined above still apply, with the following additions:

1. A CallObserver is not added to a Call more than once, even if it has been added to more than one Address/Terminal which are present on the Call.
2. The CallObserver leaves the call only if ALL of the Addresses and Terminals on which the Call Observer was added leave the Call. If one of those Addresses/Terminals becomes part of the Call again, the call observer is re-applied to the Call.

Post-Conditions:

1. observer is an element of `this.getCallObservers()`
2. observer is an element of `Call.getObservers()` for each Call associated with the Connections from `this.getConnections()`.
3. An array of snapshot events are reported to the observer for existing calls associated with this Terminal.

Parameters:

observer - The observer being added.

Throws:

[MethodNotSupportedException](#) - The observer cannot be added at this time

[ResourceUnavailableException](#) - The resource limit for the numbers of observers has been exceeded.

See Also:

[Call](#)

getCallObservers

```
public CallObserver[] getCallObservers()
```

Returns a list of all CallObservers associated with this Terminal object. That is, it returns a list of CallObserver objects which have been added via the `Terminal.addCallObserver()` method. If there are no Call observers associated with this Terminal object, this method returns null.

Post-conditions:

1. Let `CallObserver[] obs = this.getCallObservers()`
2. `obs == null` or `obs.length >= 1`

Returns:

An array of `CallObserver` objects associated with this `Address`.

removeCallObserver

```
public void removeCallObserver(CallObserver observer)
```

Removes the given `CallObserver` from the `Terminal`. In other words, it removes a `CallObserver` which was added via the `Terminal.addCallObserver()` method. If successful, the observer will no longer be added to new `Calls` which are presented to this `Terminal`, however it does not affect `CallObservers` which have already been added at a `Call`.

Also, if the `CallObserver` is not part of the `Terminal`, then this method fails silently, i.e. no observer is removed and no exception is thrown.

Post-conditions:

1. `observer` is not an element of `this.getCallObservers()`

Parameters:

`observer` - The `CallObserver` which is being removed.

getCapabilities

```
public TerminalCapabilities getCapabilities()
```

Returns the dynamic capabilities for the instance of the `Terminal` object. Dynamic capabilities tell the application which actions are possible at the time this method is invoked based upon the implementation's knowledge of its ability to successfully perform the action. This determination may be based upon argument passed to this method, the current state of the call model, or some implementation-specific knowledge. These indications do not guarantee that a particular method will be successful when invoked, however.

The dynamic `Terminal` capabilities require no additional arguments.

Returns:

The dynamic `Terminal` capabilities.

getTerminalCapabilities

```
public TerminalCapabilities getTerminalCapabilities(Terminal terminal,  
                                                    Address address)  
    throws InvalidArgumentException,  
           PlatformException
```

Deprecated. *Since JTAPI v1.2. This method has been replaced by the `Terminal.getCapabilities()` method.*

Gets the `TerminalCapabilities` object with respect to a `Terminal` and an `Address`. If null is passed as a `Terminal` parameter, the general/provider- wide `Terminal` capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The `Terminal.getCapabilities()` method returns the dynamic `Terminal` capabilities. This method now should simply invoke the `Terminal.getCapabilities()` method.

Parameters:

`address` - This argument is ignored in JTAPI v1.2 and later.
`terminal` - This argument is ignored in JTAPI v1.2 and later.

Throws:

`InvalidArgumentException` - This exception is never thrown in JTAPI v1.2 and later.
`PlatformException` - A platform-specific exception occurred.

addTerminalListener

```
public void addTerminalListener(TerminalListener listener)
                               throws ResourceUnavailableException,
                                   MethodNotSupportedException
```

Adds an listener to the `Terminal`. The `TerminalListener` reports all `Terminal`-related state changes as events. The `Terminal` object will report events to this `TerminalListener` object for the lifetime of the `Terminal` object or until the listener is removed with the `Terminal.removeTerminalListener()` or until the `Terminal` is no longer observable. In these instances, a `TERMINAL_EVENT_TRANSMISSION_ENDED` is delivered to the listener as its final event. The listener will receive no events after `TERMINAL_EVENT_TRANSMISSION_ENDED` unless the listener is explicitly re-added via the `Terminal.addTerminalListener()` method. Also, once an listener receives an `TERMINAL_EVENT_TRANSMISSION_ENDED`, it will no longer be reported via the `Terminal.getTerminalListeners()`.

If an application attempts to add an instance of an listener already present on this `Terminal`, this attempt will silently fail, i.e. multiple instances of an listener are not added and no exception will be thrown.

Currently, only the `TERMINAL_EVENT_TRANSMISSION_ENDED` event is defined by the core package and delivered to the `TerminalListener`.

Post-conditions:

1. listener is an element of this.getTerminalListeners()

Parameters:

`listener` - The listener being added.

Throws:

MethodNotSupportedException - The listener cannot be added at this time

ResourceUnavailableException - The resource limit for the numbers of listeners has been exceeded.

getTerminalListeners

```
public TerminalListener[] getTerminalListeners()
```

Returns a list of all TerminalListeners associated with this Terminal object. If there are no listeners associated with this Terminal, this method returns null.

Post-conditions:

1. Let TerminalListener[] obs = this.getTerminalListeners()
2. obs == null or obs.length >= 1

Returns:

An array of TerminalListener objects associated with this Terminal.

removeTerminalListener

```
public void removeTerminalListener(TerminalListener listener)
```

Removes the given listener from the Terminal. If successful, the listener will no longer receive events generated by this Terminal object. As its final event, the TerminalListener receives a TERMINAL_EVENT_TRANSMISSION_ENDED.

If an listener is not part of the Terminal, then this method fails silently, i.e. no listener is removed and no exception is thrown.

Post-conditions:

1. A TERMINAL_EVENT_TRANSMISSION_ENDED event is reported to the listener as its final event.
2. listener is not an element of this.getTerminalListeners()

Parameters:

listener - The listener which is being removed.

addCallListener

```
public void addCallListener(CallListener listener)
    throws ResourceUnavailableException,
           MethodNotSupportedException
```

Adds an listener to a Call object when this Terminal object first becomes part of that Call. This method permits applications to select a Terminal object in which they are interested and automatically have the implementation attach an listener to all present and future Calls which come

to this Terminal.

JTAPI v1.0 Semantics

In version 1.0 of the Java Telephony API specification, the application monitored the Terminal object for the `TermCallAtTermEv` event. This event indicated that a Call has come to this Terminal. Then, applications would manually add an listener to the Call. With this architecture, potentially dangerous race conditions existed while an application was adding an listener to the Call. As a result, this mechanism has been replaced in version 1.1.

JTAPI v1.1 Semantics

In version 1.1 of the specification, the `TermCallAtTermEv` event does not exist and this method replaces the functionality described above. Instead of monitoring for a `TermCallAtTermEv` event, the application simply uses the `Terminal.addCallListener()` method, and listener will be added to new telephone calls at this Terminal automatically.

If an application attempts to add an instance of a call listener already present on this Terminal, these repeated attempts will silently fail, i.e. multiple instances of a call listener are not added and no exception will be thrown.

When a call listener is added to an Terminal with this method, the following behavior is exhibited by the implementation.

1. It is immediately associated with any existing calls at the Terminal and a snapshot of those calls are reported to the call listener. For each of these calls, the listener is reported via `Call.getTerminalListeners()`.
2. It is associated with all future calls which come to this Terminal. For each new call, the listener is reported via `Call.getTerminalListeners()`.

Note that the definition of the term ".. when a call is at a Terminal" means that the Call contains a Connection which contains a TerminalConnection with this Terminal as its Terminal.

Call Listener Lifetime

For all call listeners which are present on Calls because of this method, the following behavior is exhibited with respect to the lifetime of the call.

1. The call listener receives events until the Call is no longer at this Terminal. In this case, the call listener will be re-applied to the Call if the Call returns to this Terminal at some point in the future.
2. The call listener is removed with `Call.removeTerminalListener()`. Note that this only affects the instance of the call listener for that particular call. If the Call subsequently leaves and returns to the Terminal, the listener will be re-applied.
3. The Call can no longer be monitored by the implementation.
4. The Call moves into the `Call.INVALID` state.

When the CallListener leaves the Call because of one of the reasons above, it receives a `CALL_EVENT_TRANSMISSION_ENDED` as its final event.

Call Listener on Multiple Addresses and Terminals

It is possible for an application to add CallListeners to more than one Address and Terminal (using `Address.addCallListener()` and `Terminal.addCallListener()`, respectively). The rules outlined above still apply, with the following additions:

1. A CallListener is not added to a Call more than once, even if it has been added to more than one Address/Terminal which are present on the Call.
2. The CallListener leaves the call only if ALL of the Addresses and Terminals on which the Call Listener was added leave the Call. If one of those Addresses/Terminals becomes part of the Call again, the call listener is re-applied to the Call.

Call Listener Event level of Granularity

The application shall receive the event reports associated with the methods which are defined in the CallListener interface that the application implements - either the CallListener, ConnectionListener or TerminalConnectionListener interface. Note that the CallListener, ConnectionListener and TerminalConnectionListener form a hierarchy of interfaces, and so provide successively increasing amounts of call information and successively finer-grained event granularity.

Post-Conditions:

1. listener is an element of `this.getCallListeners()`
2. listener is an element of `Call.getTerminalListeners()` for each Call associated with the Connections from `this.getConnections()`.
3. An array of snapshot events are reported to the listener for existing calls associated with this Terminal.

Parameters:

listener - The listener being added.

Throws:

MethodNotSupportedException - The listener cannot be added at this time

ResourceUnavailableException - The resource limit for the numbers of listeners has been exceeded.

See Also:

Call

getCallListeners

```
public CallListener[] getCallListeners()
```

Returns a list of all CallListeners associated with this Terminal object. That is, it returns a list of CallListener objects which have been added via the `Terminal.addCallListener()` method. If there are no Call listeners associated with this Terminal object, this method returns null.

Post-conditions:

1. Let `CallListener[] obs = this.getCallListeners()`
2. `obs == null` or `obs.length >= 1`

Returns:

An array of `CallListener` objects associated with this `Terminal`.

removeCallListener

```
public void removeCallListener(CallListener listener)
```

Removes the given `CallListener` from the `Terminal`. In other words, it removes a `CallListener` which was added via the `Terminal.addCallListener()` method. If successful, the listener will no longer be added to new `Calls` which are presented to this `Terminal`, however it does not affect `CallListeners` which have already been added at a `Call`.

Also, if the `CallListener` is not part of the `Terminal`, then this method fails silently, i.e. no listener is removed and no exception is thrown.

Post-conditions:

1. listener is not an element of `this.getCallListeners()`

Parameters:

`listener` - The `CallListener` which is being removed.

javax.telephony

Interface TerminalConnection

All Known Subinterfaces:

[CallControlTerminalConnection](#)

public interface **TerminalConnection**

The `TerminalConnection` represents the relationship between a `Connection` and a `Terminal`.

Introduction

A `TerminalConnection` object must always be associated with a `Connection` object and a `Terminal` object. The `Connection` and the `Terminal` objects associated with the `TerminalConnection` do not change throughout the lifetime of the `TerminalConnection`. Applications obtain the `Connection` and `Terminal` associated with the `TerminalConnection` via the `TerminalConnection.getConnection()` and `TerminalConnection.getTerminal()` methods, respectively.

Because a `TerminalConnection` is associated with a `Connection`, it there is also associated with some `Call`. The `TerminalConnection` describes the specific relationship between a physical `Terminal` endpoint with respect to an `Address` on a `Call`. `TerminalConnections` provide a *physical* view of a `Call`. For a particular `Address` endpoint on a `Call`, there may be zero or more `Terminals` at which the `Call` terminates. The `TerminalConnection` describes each specific `Terminal` on the `Call` associated with a particular `Address` endpoint on the `Call`. Many simple applications may not care about which specific `Terminals` are on the `Call` at a particular `Address` endpoint. In these cases, the logical view provided by `Connections` are sufficient.

Requirements for TerminalConnections

In order for a `Terminal` to be on a `Call` and associated with a `Connection`, the `Terminal` must be associated with the `Address` object endpoint of the `Connection`. That is, for each `TerminalConnection` on a `Connection`, the `Connection`'s `Address` must be associated with the `TerminalConnection`'s `Terminal`. The following predicates illustrates this necessary relationship:

1. Let `address = connection.getAddress();`
2. Let `tc[] = connection.getTerminalConnections();`
3. For all `i` in `tc[]`, let `terminal[i] = tc[i].getTerminal();`
4. Assert for all `i`: `address` is an element of `terminal[i].getAddresses();`
5. Assert for all `i`: `terminal[i]` is an element of `address.getTerminals();`

TerminalConnection States

The TerminalConnection has a *state* which describes the current relationship between a Terminal and a Connection. TerminalConnection states are distinct from Connection states. Connection states describe the relationship between an entire Address endpoint and a Call, whereas the TerminalConnection state describes the relationship between one of the Terminals at the endpoint Address on the Call with respect to its Connection. Different Terminals on a Call which are associated with the same Connection may be in different states. Furthermore, the state of the TerminalConnection has a dependency and specific relationship to the state of its Connection, as described later on.

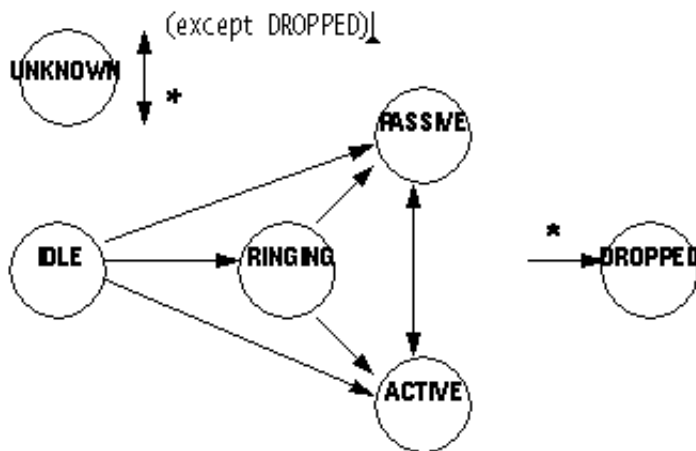
The TerminalConnection interface in the core package has six states defined in real-world terms below:

<code>TerminalConnection.IDLE</code>	This state is the initial state for all TerminalConnections. TerminalConnection objects do not stay in this state for long. They typically transition into another state quickly.
<code>TerminalConnection.RINGING</code>	This state indicates the a Terminal is ringing, indicating that the Terminal has an incoming Call.
<code>TerminalConnection.PASSIVE</code>	This state indicates that a Terminal is part of a telephone call but not in an active fashion. This may imply that a resource of the Terminal is being used and may limit actions on the Terminal.
<code>TerminalConnection.ACTIVE</code>	This state indicates that a Terminal is actively part of a telephone call. This usually implies that the party speaking on that Terminal is part of the telephone call.
<code>TerminalConnection.DROPPED</code>	This state indicates that a particular Terminal has permanently left the telephone call.
<code>TerminalConnection.UNKNOWN</code>	This state indicates that the implementation is unable to determine the state of the TerminalConnection. TerminalConnections may transition into and out of this state at any time.

When a TerminalConnection moves into the `TerminalConnection.DROPPED` state, it is no longer associated with its Connection and Terminal. That is, both of these objects lose their references to the TerminalConnection. However, the TerminalConnection still maintains its references to the Connection and Terminal object for application reference. That is, when a TerminalConnection moves into the `TerminalConnection.DROPPED` state, the methods `TerminalConnection.getConnection()` and `TerminalConnection.getTerminal()` still return valid objects.

TerminalConnection state transitions

Similar to the Connection, there is a finite-state diagram which describes the allowable state transitions of a TerminalConnection. The implementation must guarantee these state transitions. The specifications of methods which affect the state of the TerminalConnections also obey these state transitions. This state diagram is below:



Note the TerminalConnection may transition into the `TerminalConnection.DROPPED` state from any state, and into and out of the `TerminalConnection.UNKNOWN` state from any state.

Relationship between Connections and TerminalConnections

As mentioned previously, the state of the Connection determines the following about TerminalConnections:

- Whether TerminalConnections may exist on a Connection.
- The allowable states of the TerminalConnections if they exist.

These properties about Connections and TerminalConnections are guaranteed by the implementation. This relationship is further illustrated in the description of such methods as `Call.connect()`, `Connection.disconnect()`, and `TerminalConnection.answer()`. The following chart defines the specific relationship between Connection states and TerminalConnections.

<i>If the Connection is in state...</i>	<i>... then the TerminalConnection is</i>
<code>Connection.IDLE</code>	No TerminalConnections may exist on this Connection, that is, the <code>Connection.getTerminalConnections()</code> method returns null.
<code>Connection.INPROGRESS</code>	No TerminalConnections may exist on this Connection, that is, the <code>Connection.getTerminalConnections()</code> method returns null.
<code>Connection.ALERTING</code>	Zero or more TerminalConnections may exist on this Connection, and each must be in the <code>TerminalConnection.RINGING</code> state.
<code>Connection.CONNECTED</code>	Zero or more TerminalConnections may exist on this Connection, and each must be in the <code>TerminalConnection.PASSIVE</code> or the <code>TerminalConnection.ACTIVE</code> state. Note that when TerminalConnections must into the <code>TerminalConnection.DROPPED</code> state they are no longer associated with the Connection.
<code>Connection.DISCONNECTED</code>	No TerminalConnections may exist on this Connection, that is, the <code>Connection.getTerminalConnections()</code> method returns null. Note that all TerminalConnections previously associated with this Connection will move into the <code>TerminalConnection.DROPPED</code> state.
<code>Connection.FAILED</code>	No TerminalConnections may exist on this Connection, that is, the <code>Connection.getTerminalConnections()</code> method returns null. Note that all TerminalConnections previously associated with this Connection will move into the <code>TerminalConnection.DROPPED</code> state.
<code>Connection.UNKNOWN</code>	Zero or more TerminalConnections may exist on this Connection, and each must be in the <code>TerminalConnection.UNKNOWN</code> state.

The TerminalConnection.answer() Method

The primary method supported on the core package's TerminalConnection interface is the `TerminalConnection.answer()` method. This method answers a telephone call at a Terminal. This method moves the TerminalConnection into the `TerminalConnection.ACTIVE` state upon success. The TerminalConnection must be in the `TerminalConnection.RINGING` state when this method is invoked.

Listeners and Events

All events pertaining to the TerminalConnection object are reported via the `CallListener` interface on the Call object associated with this TerminalConnection. In the core package, events are reported to a CallListener when a new TerminalConnection is created and whenever a TerminalConnection changes state. Listeners are added to Call objects via the `Call.addCallListener()` method and more indirectly via the `Address.addCallListener()` and `Terminal.addCallListener()` methods. See the specifications for the Call, Address, and Terminal interfaces for more information.

The following TerminalConnection-related events are defined in the core package. Each of these events extend the `TerminalConnectionEvent` interface (which, in turn, extends the `CallEvent` interface).

<code>TerminalConnectionCreated</code>	Indicates a new <code>TerminalConnection</code> has been created on a <code>Connection</code> .
<code>TerminalConnectionRinging</code>	Indicates the <code>TerminalConnection</code> has moved into the <code>TerminalConnection.RINGING</code> state.
<code>TerminalConnectionActive</code>	Indicates the <code>TerminalConnection</code> has moved into the <code>TerminalConnection.ACTIVE</code> state.
<code>TerminalConnectionPassive</code>	Indicates the <code>TerminalConnection</code> has moved into the <code>TerminalConnection.PASSIVE</code> state.
<code>TerminalConnectionDropped</code>	Indicates the <code>TerminalConnection</code> has moved into the <code>TerminalConnection.DROPPED</code> state.
<code>TerminalConnectionUnknown</code>	Indicates the <code>TerminalConnection</code> has moved into the <code>TerminalConnection.UNKNOWN</code> state.

See Also:

[`CallListener`](#), [`TerminalConnectionListener`](#), [`TerminalListener`](#), [`TerminalConnectionEvent`](#), [`CallEvent`](#)

Field Summary	
static int	<u>ACTIVE</u> This state indicates that a Terminal is actively part of a telephone call.
static int	<u>DROPPED</u> This state indicates that a particular Terminal has permanently left the telephone call.
static int	<u>IDLE</u> This state is the initial state for all <code>TerminalConnection</code> objects.
static int	<u>PASSIVE</u> This state indicates that a Terminal is part of a telephone call but not in an active fashion.
static int	<u>RINGING</u> This state indicates the a Terminal is ringing, indicating that the Terminal has an incoming Call.
static int	<u>UNKNOWN</u> This state indicates that the implementation is unable to determine the state of the <code>TerminalConnection</code> .

Method Summary	
void	<u>answer()</u> Answers an incoming telephone call on this TerminalConnection.
<u>TerminalConnectionCapabilities</u>	<u>getCapabilities()</u> Returns the dynamic capabilities for the instance of the TerminalConnection object.
<u>Connection</u>	<u>getConnection()</u> Returns the Connection object associated with this TerminalConnection.
int	<u>getState()</u> Returns the state of the TerminalConnection object.
<u>Terminal</u>	<u>getTerminal()</u> Returns the Terminal associated with this TerminalConnection object.
<u>TerminalConnectionCapabilities</u>	<u>getTerminalConnectionCapabilities(Terminal terminal, Address address)</u> Deprecated. Since JTAPI v1.2. This method has been replaced by the TerminalConnection.getCapabilities() method.

Field Detail

IDLE

public static final int **IDLE**

This state is the initial state for all TerminalConnection objects.

RINGING

public static final int **RINGING**

This state indicates the a Terminal is ringing, indicating that the Terminal has an incoming Call.

PASSIVE

public static final int **PASSIVE**

This state indicates that a Terminal is part of a telephone call but not in an active fashion. This may imply that a resource of the Terminal is being used and may limit actions on the Terminal.

ACTIVE

public static final int **ACTIVE**

This state indicates that a Terminal is actively part of a telephone call. This usually implies that the party speaking on that Terminal is party of the telephone call.

DROPPED

public static final int **DROPPED**

This state indicates that a particular Terminal has permanently left the telephone call.

UNKNOWN

public static final int **UNKNOWN**

This state indicates that the implementation is unable to determine the state of the TerminalConnection.

Method Detail

getState

public int **getState()**

Returns the state of the TerminalConnection object.

Returns:

The current state of the TerminalConnection object.

getTerminal

public Terminal **getTerminal()**

Returns the Terminal associated with this TerminalConnection object. A TerminalConnection's reference to its Terminal remains valid for the lifetime of the object, even if the Terminal loses its references to this TerminalConnection object. Also, this reference does not change once the TerminalConnection object has been created.

Returns:

The Terminal object associated with this TerminalConnection.

getConnection

```
public Connection getConnection()
```

Returns the `Connection` object associated with this `TerminalConnection`. A `TerminalConnection`'s reference to the `Connection` remains valid throughout the lifetime of the `TerminalConnection`. Also, this reference does not change once the `TerminalConnection` object has been created.

Returns:

The `Connections` associated with this `TerminalConnection`.

answer

```
public void answer()  
    throws PrivilegeViolationException,  
           ResourceUnavailableException,  
           MethodNotSupportedException,  
           InvalidStateException
```

Answers an incoming telephone call on this `TerminalConnection`. This method waits (i.e. the invoking thread blocks) until the telephone call has been answered at the endpoint before returning. When this method returns successfully, the state of this `TerminalConnection` object is `TerminalConnection.ACTIVE`.

Allowable TerminalConnection States

The `TerminalConnection` must be in the `TerminalConnection.RINGING` state when this method is invoked. According to the specification of the `TerminalConnection` object, this state implies the associated `Connection` object is also in the `Connection.ALERTING` state. There may be more than one `TerminalConnection` on the `Connection` which are in the `TerminalConnection.RINGING` state. In fact, if the `Connection` is in the `Connection.ALERTING` state, all of these `TerminalConnections` must be in the `TerminalConnection.RINGING` state. Any of these `TerminalConnections` may invoke this method to answer the telephone call.

Multiple TerminalConnections

The underlying telephone hardware determines the resulting state of other `TerminalConnection` objects after the telephone call has been answered by one of the `TerminalConnections`. The other `TerminalConnection` object may either move into the `TerminalConnection.PASSIVE` state or the `TerminalConnection.DROPPED` state. If a `TerminalConnection` moves into the `TerminalConnection.PASSIVE` state, it remains part of the telephone call, but not actively so. It may have the ability to join the call in the future. If a `TerminalConnection` moves into the `TerminalConnection.DROPPED` state, it is removed from the telephone call and will never have the ability to join the call in the future. The appropriate events are delivered to the application indicates into which of these two states the other `TerminalConnection` objects have moved.

Events

The following events are reported to applications via the CallListener interface as a result of the successful outcome of this method:

1. TerminalConnectionActive for the TerminalConnection which invoked this method.
2. ConnectionConnected for the Connection associated with the TerminalConnection.
3. TerminalConnectionPassive or TerminalConnectionActive for other TerminalConnections associated with the Connection.

Pre-conditions:

1. ((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE
2. this.getState() == TerminalConnection.RINGING
3. (this.getConnection()).getState() == Connection.ALERTING

Post-conditions:

1. ((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE
2. this.getState() == TerminalConnection.ACTIVE
3. (this.getConnection()).getState() == Connection.CONNECTED
4. TerminalConnectionActive for the TerminalConnection which invoked this method.
5. ConnectionConnected for the Connection associated with the TerminalConnection.
6. TerminalConnectionPassive or TerminalConnectionActive for other TerminalConnections associated with the Connection.

Throws:

PrivilegeViolationException - The application did not have proper authority to answer the telephone call. For example, the Terminal associated with the TerminalConnection may not be in the Provider's local domain.

ResourceUnavailableException - The necessary resources to answer the telephone call were not available when the method was invoked.

MethodNotSupportedException - This method is currently not supported by this implementation.

InvalidStateException - An object was not in the proper state, violating the pre-conditions of this method. For example, the Provider was not in the Provider.IN_SERVICE state or the TerminalConnection was not in the TerminalConnection.RINGING state.

See Also:

TerminalConnectionListener, ConnectionListener,
TerminalConnectionEvent, ConnectionEvent

getCapabilities

```
public TerminalConnectionCapabilities getCapabilities()
```

Returns the dynamic capabilities for the instance of the TerminalConnection object. Dynamic capabilities tell the application which actions are possible at the time this method is invoked based upon the implementations knowledge of its ability to successfully perform the action. This determination may be based upon argument passed to this method, the current state of the call model, or some implementation- specific knowledge. These indications do not guarantee that a particular

method will be successful when invoked, however.

The dynamic TerminalConnection capabilities require no additional arguments.

Returns:

The dynamic TerminalConnection capabilities.

getTerminalConnectionCapabilities

```
public TerminalConnectionCapabilities getTerminalConnectionCapabilities(Terminal terminal,  
                                                                           Address address)  
    throws InvalidArgumentException,  
           PlatformException
```

Deprecated. *Since JTAPI v1.2. This method has been replaced by the TerminalConnection.getCapabilities() method.*

Gets the TerminalConnectionCapabilities object with respect to a Terminal and an Address. If null is passed as a Terminal parameter, the general/ provider-wide Terminal Connection capabilities are returned.

Note: This method has been replaced in JTAPI v1.2. The TerminalConnection.getCapabilities() method returns the dynamic TerminalConnection capabilities. This method now should simply invoke the TerminalConnection.getCapabilities() method.

Parameters:

address - This argument is ignored in JTAPI v1.2 and later.
terminal - This argument is ignored in JTAPI v1.2 and later.

Returns:

The static TerminalConnectionCapabilities capabilities.

Throws:

InvalidArgumentException - This exception is never thrown in JTAPI v1.2 and later.
PlatformException - A platform-specific exception occurred.

javax.telephony

Interface TerminalConnectionEvent

public interface **TerminalConnectionEvent**
extends [CallEvent](#)

The `TerminalConnectionEvent` interface is the base event interface for all `TerminalConnection`-related events. All events which pertain to the `TerminalConnection` object must extend this interface. This interface extends the `CallEvent` interface and therefore is reported via the `CallListener` interface.

The core package defines events which are reported when the `TerminalConnection` changes state. These events are: `TERMINAL_CONNECTION_RINGING`, `TERMINAL_CONNECTION_ACTIVE`, `TERMINAL_CONNECTION_PASSIVE`, `TERMINAL_CONNECTION_DROPPED`, and `TERMINAL_CONNECTION_UNKNOWN`. Also, a `TERMINAL_CONNECTION_CREATED` is sent when a new `TerminalConnection` is created.

The `TerminalConnectionEvent.getTerminalConnection()` method on this interface returns the `TerminalConnection` associated with this `TerminalConnectionEvent`.

See Also:

[TerminalConnection](#), [CallListener](#), [CallEvent](#), [TerminalConnectionListener](#)

Field Summary

static int	<u>TERMINAL_CONNECTION_ACTIVE</u> The <code>TERMINAL_CONNECTION_ACTIVE</code> event indicates that the state of the <code>TerminalConnection</code> object has changed to <code>TerminalConnection.ACTIVE</code> .
static int	<u>TERMINAL_CONNECTION_CREATED</u> The <code>TERMINAL_CONNECTION_DROPPED</code> event indicates that a new <code>TerminalConnection</code> object has been created.
static int	<u>TERMINAL_CONNECTION_DROPPED</u> The <code>TERMINAL_CONNECTION_DROPPED</code> event indicates that the state of the <code>TerminalConnection</code> object has changed to <code>TerminalConnection.DROPPED</code> .
static int	<u>TERMINAL_CONNECTION_PASSIVE</u> The <code>TERMINAL_CONNECTION_PASSIVE</code> event indicates that the state of the <code>TerminalConnection</code> object has changed to <code>TerminalConnection.PASSIVE</code> .
static int	<u>TERMINAL_CONNECTION_RINGING</u> The <code>TERMINAL_CONNECTION_RINGING</code> event indicates that the state of the <code>TerminalConnection</code> object has changed to <code>TerminalConnection.RINGING</code> .
static int	<u>TERMINAL_CONNECTION_UNKNOWN</u> The <code>TERMINAL_CONNECTION_UNKNOWN</code> event indicates that the state of the <code>TerminalConnection</code> object has changed to <code>TerminalConnection.UNKNOWN</code> .

Fields inherited from interface javax.telephony.CallEvent

CALL_ACTIVE, CALL_EVENT_TRANSMISSION_ENDED, CALL_INVALID

Fields inherited from interface javax.telephony.Event

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN

Method Summary

<u>TerminalConnection</u>	<u>getTerminalConnection()</u> Returns the <code>TerminalConnection</code> associated with this event.
---------------------------	--

Methods inherited from interface javax.telephony.CallEventgetCall**Methods inherited from interface javax.telephony.Event**getCause, getID, getMetaEvent, getSource**Field Detail***TERMINAL_CONNECTION_ACTIVE*

```
public static final int TERMINAL_CONNECTION_ACTIVE
```

The `TERMINAL_CONNECTION_ACTIVE` event indicates that the state of the `TerminalConnection` object has changed to `TerminalConnection.ACTIVE`.

This constant indicates a specific event passed via a `TerminalConnectionEvent` event, and is reported on the `CallListener` interface.

TERMINAL_CONNECTION_CREATED

```
public static final int TERMINAL_CONNECTION_CREATED
```

The `TERMINAL_CONNECTION_DROPPED` event indicates that a new `TerminalConnection` object has been created.

This constant indicates a specific event passed via a `TerminalConnectionEvent` event, and is reported on the `CallListener` interface.

TERMINAL_CONNECTION_DROPPED

```
public static final int TERMINAL_CONNECTION_DROPPED
```

The `TERMINAL_CONNECTION_DROPPED` event indicates that the state of the `TerminalConnection` object has changed to `TerminalConnection.DROPPED`.

This constant indicates a specific event passed via a `TerminalConnectionEvent` event, and is reported on the `CallListener` interface.

TERMINAL_CONNECTION_PASSIVE

`public static final int TERMINAL_CONNECTION_PASSIVE`

The `TERMINAL_CONNECTION_PASSIVE` event indicates that the state of the `TerminalConnection` object has changed to `TerminalConnection.PASSIVE`.

This constant indicates a specific event passed via a `TerminalConnectionEvent` event, and is reported on the `CallListener` interface.

TERMINAL_CONNECTION_RINGING

`public static final int TERMINAL_CONNECTION_RINGING`

The `TERMINAL_CONNECTION_RINGING` event indicates that the state of the `TerminalConnection` object has changed to `TerminalConnection.RINGING`. This interface extends the `TerminalConnectionEvent` interface and is reported via the `CallListener` interface.

TERMINAL_CONNECTION_UNKNOWN

`public static final int TERMINAL_CONNECTION_UNKNOWN`

The `TERMINAL_CONNECTION_UNKNOWN` event indicates that the state of the `TerminalConnection` object has changed to `TerminalConnection.UNKNOWN`.

This constant indicates a specific event passed via a `TerminalConnectionEvent` event, and is reported on the `CallListener` interface.

Method Detail

getTerminalConnection

`public TerminalConnection getTerminalConnection()`

Returns the `TerminalConnection` associated with this event.

Returns:

The `TerminalConnection` associated with this event.

javax.telephony **Interface TerminalEvent**

public interface **TerminalEvent**
extends [Event](#)

The `TerminalEvent` interface is the base interface for all Terminal- related events. All events which pertain to the Terminal object must extend this interface. Events which extend this interface are reported via the `TerminalListener` interface.

The only event defined in the core package for the Terminal is the `TERMINAL_EVENT_TRANSMISSION_ENDED`.

The `TerminalEvent.getTerminal()` method on this interface returns the Terminal associated with the Terminal event.

See Also:

[Event](#), [TerminalListener](#), [Terminal](#)

Field Summary

static int	<u>TERMINAL_EVENT_TRANSMISSION_ENDED</u> The <code>TERMINAL_EVENT_TRANSMISSION_ENDED</code> event indicates that the application will no longer receive Terminal events on the instance of the <code>TerminalListener</code> .
------------	--

Fields inherited from interface [javax.telephony.Event](#)

[CAUSE_CALL_CANCELLED](#), [CAUSE_DEST_NOT_OBTAINABLE](#),
[CAUSE_INCOMPATIBLE_DESTINATION](#), [CAUSE_LOCKOUT](#),
[CAUSE_NETWORK_CONGESTION](#), [CAUSE_NETWORK_NOT_OBTAINABLE](#),
[CAUSE_NEW_CALL](#), [CAUSE_NORMAL](#), [CAUSE_RESOURCES_NOT_AVAILABLE](#),
[CAUSE_SNAPSHOT](#), [CAUSE_UNKNOWN](#)

Method Summary

Terminal	<u>getTerminal()</u> Returns the Terminal associated with this Terminal event.
--------------------------	--

Methods inherited from interface `javax.telephony.Event``getCause`, `getID`, `getMetaEvent`, `getSource`**Field Detail*****TERMINAL_EVENT_TRANSMISSION_ENDED***

```
public static final int TERMINAL_EVENT_TRANSMISSION_ENDED
```

The `TERMINAL_EVENT_TRANSMISSION_ENDED` event indicates that the application will no longer receive Terminal events on the instance of the `TerminalListener`.

This constant indicates a specific event passed via a `TerminalEvent` event, and is reported on the `TerminalListener` interface.

Method Detail***getTerminal***

```
public Terminal getTerminal()
```

Returns the Terminal associated with this Terminal event.

Returns:

The Terminal associated with this event.

javax.telephony **Interface TerminalListener**

public interface **TerminalListener**
extends java.util.EventListener

The `TerminalListener` interface reports all changes which happen to the Terminal object.

Introduction

These changes are reported as events to the `TerminalListener()`. Applications must instantiate an object which implements this interface and then use the `Terminal.addListener()` method to register the object to receive all future events associated with the Terminal object.

Terminal Listener Ending

At various times, the underlying implementation may not be able to observe the Terminal. In these instances, the TerminalListener is sent an `terminalListenerEnded` event. This indicates that the application will not receive further events associated with the Terminal object. This listener is no longer reported via the `Terminal.getListeners()` method.

See Also:

[TerminalEvent](#)

Method Summary

void	<u>terminalListenerEnded</u> (<u>TerminalEvent</u> event) The <code>terminalListenerEnded</code> event indicates that the application will no longer receive Terminal events on the instance of the <code>TerminalListener</code> .
------	--

Method Detail

terminalListenerEnded

public void **terminalListenerEnded**([TerminalEvent](#) event)

The `terminalListenerEnded` event indicates that the application will no longer receive Terminal events on the instance of the `TerminalListener`.

This constant indicates a specific event passed via a `TerminalEvent` event, and is reported on the `TerminalListener` interface.

See Also:

[TerminalEvent](#)

javax.telephony **Interface TerminalObserver**

All Known Subinterfaces:

[CallControlTerminalObserver](#), [PhoneTerminalObserver](#)

public interface **TerminalObserver**

The `TerminalObserver` interface reports all changes which happen to the Terminal object.

Introduction

These changes are reported as events to the `TerminalObserver.terminalChangedEvent()` method. Applications must instantiate an object which implements this interface and then use the `Terminal.addObserver()` method to register the object to receive all future events associated with the Terminal object.

The `TerminalObserver.terminalChangedEvent()` method receives an array of events which all must extend the `TermEv` interface. Since several changes may happen to a single JTAPI object at once, a list of events is needed to convey those changes which happen at the same time. Applications iterate through the array of events provided.

Terminal Observation Ending

At various times, the underlying implementation may not be able to observe the Terminal. In these instances, the `TerminalObserver` is sent an `TermObservationEndedEv` event. This indicates that the application will not receive further events associated with the Terminal object. This observer is no longer reported via the `Terminal.getObservers()` method.

See Also:

[TermEv](#), [TermObservationEndedEv](#)

Method Summary	
void	<u>terminalChangedEvent</u> (<u>TermEv[]</u> eventList) Reports all events associated with the Terminal object.

Method Detail

terminalChangedEvent

```
public void terminalChangedEvent(TermEv[] eventList)
```

Reports all events associated with the Terminal object. This method passes an array of TermEv objects as its arguments which correspond to the list of events representing the changes to the Terminal object.

Parameters:

eventList - The list of Terminal events.

Package javax.telephony.callcontrol

The JTAPI advanced Call Control package provides more detailed information about the core call model and supplies an extended set of states for the Connection and TerminalConnection objects.

See:

Description

Interface Summary	
<u>CallControlAddress</u>	Introduction
<u>CallControlAddressObserver</u>	The CallControlAddressObserver interface reports all events for the CallControlAddress interface.
<u>CallControlCall</u>	Introduction
<u>CallControlCallObserver</u>	The CallControlCallObserver interface reports all events for the CallControlCall interface.
<u>CallControlConnection</u>	Introduction
<u>CallControlTerminal</u>	Introduction
<u>CallControlTerminalConnection</u>	Introduction
<u>CallControlTerminalObserver</u>	The CallControlTerminalObserver interface reports all events for the CallControlTerminal interface.

Class Summary	
<u>CallControlForwarding</u>	The CallControlForwarding class represents a <i>forwarding instruction</i> .

Package javax.telephony.callcontrol Description

The JTAPI advanced Call Control package provides more detailed information about the core call model and supplies an extended set of states for the Connection and TerminalConnection objects.

Java Telephony API - Call Control Extension Package

Introduction

This document provides an overview of the Call Control Extension Package to the Java Telephony API. The Call Control Package extends the Core API by providing more advanced features and more detailed information about the call model. The more advanced features take the form of additional methods on the call control objects: Call, Connection, Terminal, Address, and TerminalConnection. The Call Control Package provides more detailed information about the call model, using an extended set of states on the Connection and TerminalConnection objects.

1. Call Control Package Methods This section briefly summarizes the methods and features made available by the Call Control Package. It provides this summary for each of the interfaces in this package.
2. Call Control Package Extended States The detailed set of states available in the Call Control Package are described in this section of the overview. The Call Control Package extends both the core Connection states and the core TerminalConnection states.
3. Object and State Diagram for the telephone calls Similar to the presentation of the timeline of objects and states for the Call.connect() method in the overview document for the core API, this section presents an analysis of what applications might expect to see in terms of objects and their states when a telephone call is placed. It gives the most accurate and clearest view of when certain objects are created and their likely state transitions.
4. OutCall.java Code Example This section provides a code example of an application that places an outgoing telephone call. Although it invokes Call.connect(), a core method, it monitors the advanced states associated with the Call Control Package.
5. InCall.java Code Example This section provides a code example of answering a telephone call incoming to a Terminal. This example shows how an application uses the accept(), reject(), or redirect() methods in addition to the core TerminalConnection.answer() method.

Call Control Package Methods

This section outlines, for each interface defined in the Call Control Package, the methods and features provided. It is meant to provide a general overview of the additional features of each object, allowing application developers to ascertain whether the Call Control Package satisfies their requirements.

The CallControlCall Interface

The CallControlCall interfaces provides additional methods extending the core Call interface. The primary methods this extension interface provides are: conference(), transfer(), drop(), and consult(). This conference() method, as commonly known, conferences two calls together into a single call. When conference() returns control to the application, all of the parties on each of the two individual calls are connected to one another. The transfer() method moves the parties of one call to another call, where one party typically drops off of the call. The drop() method provides a convenience functionality for the core disconnect() method located on Connections. The drop() method drops the entire call rather than an

individual Connection. The consult() method begins a consultation call from an existing call. Applications perform the consult() action in order to eventually conference() or transfer() the two calls.

The CallControlAddress Interface

The CallControlAddress interface provides additional methods extending the core Address interface. The primary methods this extension interface provides are: setForwarding(), setDoNotDisturb(), and setMessageWaiting(). These methods do not pertain to an existing call, rather they provide instructions to the Provider pertaining to the designated Address. The setForwarding() method instructs the switch to forward all incoming calls to that address according to certain rules. The setDoNotDisturb() method instructs the switch not to allow any incoming calls to that Address to alert at the Address. The setMessageWaiting() method informs the switch that a message is waiting at this address.

The CallControlConnection Interface

The CallControlConnection interface provides additional methods extending the core Connection interface. The primary methods this extension interface provides are: accept(), reject(), redirect(), and park(). The accept() method accepts an incoming call offered to this Address, resulting in the call alerting at the Address. Conversely, the reject() method rejects an incoming call offered to this address. This rejected call never alerts at the Address and dies. The redirect() method transfers an incoming call which is either offered to an Address or already alerting at an Address to another Address. The redirect() method is similar to the transfer() method on the CallControlCall interface, except the redirect() method is used on Connections before they are connected and active in a call. The park() method "parks" a Connection at a destination Address. "Parking" a Connection moves the Connection from its current Address into a special "waiting" state on the destination Address.

The CallControlTerminal Interface

The CallControlTerminal interface provides additional methods extending the core Terminal interface. The primary method this extension interface provides is the pickup() method. The pickup() method "picks up" a Connection from a number of situations. Applications may use the pickup() method at a Terminal for a Connection which was previously "parked". Applications may also use the pickup() method to answer telephony calls which are not alerting at their Terminal, but at a different Address entirely.

The CallControlTerminalConnection Interface

The CallControlTerminalConnection interface provides additional methods extending the core TerminalConnection interface. The primary methods this extension interface provides are: hold(), unhold(), join(), and leave(). The hold() method, as commonly known, places a TerminalConnection on hold with respect to the telephone call it is part of. The unhold() method takes a TerminalConnection off hold and makes it active once again. The join() method takes a TerminalConnection which is bridged in a telephone call and makes it active. The leave() method takes an active TerminalConnection in a telephone call and returns it to the bridged state.

Call Control Package Extended States

The Call Control Package provides both additional features and additional information about the current state of telephone calls in the call model. This additional information takes the form of an expanded set of states for the Connection and TerminalConnection objects, above the set of states provided by the core package for each of these objects. This expanded set of states allows the implementation to describe more accurately to the application what is going on in a telephone call. Most of the methods in the package rely upon these expanded states.

If an application wants to use the Call Control Package, it is expected that it will only monitor the states provided by the Call Control Package, and not monitor the core states. In several cases, the Call Control Package does not expand upon a state and uses the exact states as in the core with the same meaning.

As mentioned above, the Call Control Package expands upon the states in the core Connection and TerminalConnection objects. Listed below are the states found in the CallControlConnection and CallControlTerminalConnection interfaces that expand upon their respective core states. With each state is a brief, real-world description of their meaning.

CallControlConnection States

IDLE state

The IDLE state carries the same meaning as the Connection's IDLE state. It is the initial state for all Connections and is transitory in nature. Most Connections do not stay in the IDLE state for long.

INITIATED state

The INITIATED state represents that an originating Connection is involved with placing a telephone call, however the telephone corresponding to the originating side has not been taken off-hook yet.

DIALING state

The DIALING state represents that an origination Connection is involved with placing a telephone call and has begun dialing the digits of the destination address but has not yet completed.

ESTABLISHED

The ESTABLISHED state represents that a Connection is actively part of a telephone call. This Connection's CONNECTED state most closely corresponds to this state, however Connection's CONNECTED is more general than ESTABLISHED.

OFFERED state

The OFFERED state indicates for a destination Connection that a Call is being offered to an address. Typically, applications either accept or reject this offered call.

QUEUED state

The QUEUED state indicates that a Connection is waiting at an Address, however, is not actively part of a telephone call. Applications may send a Connection to another Address in the QUEUED state to wait to be picked up. Another example of when Connections enter the QUEUED state is when the destination is busy and a "queueing" feature is turned on allowing the incoming Call to wait until the destination is no longer busy.

NETWORK_REACHED state

The NETWORK_REACHED state indicates that a Call has reached the network on the destination end, however no confirmation that the actual destination has been reached. This may be the last state reached by a destination Connection and applications often treat this as if the Connection is in the ESTABLISHED state.

NETWORK_ALERTING state

The NETWORK_ALERTING state indicates that a Connection which was previously in the NETWORK_REACHED state is now alerting at the destination. When the destination answers the telephone call, the Connection will move into the ESTABLISHED state from this state.

ALERTING state

The ALERTING state has the same meaning as the Connection's ALERTING state. It implies that a telephone call is alerting at its destination.

DISCONNECTED state

The DISCONNECTED state has the same meaning as the Connection's DISCONNECTED state. It implies that a Connection is no longer active in a telephone call. It is the final state for Connections in the Call Control Package as well as in the core.

FAILED state

The FAILED state has the same meaning as the Connection's FAILED state. It implies that a Connection has failed for some reason and is no longer able to perform additional actions in the Call.

UNKNOWN state

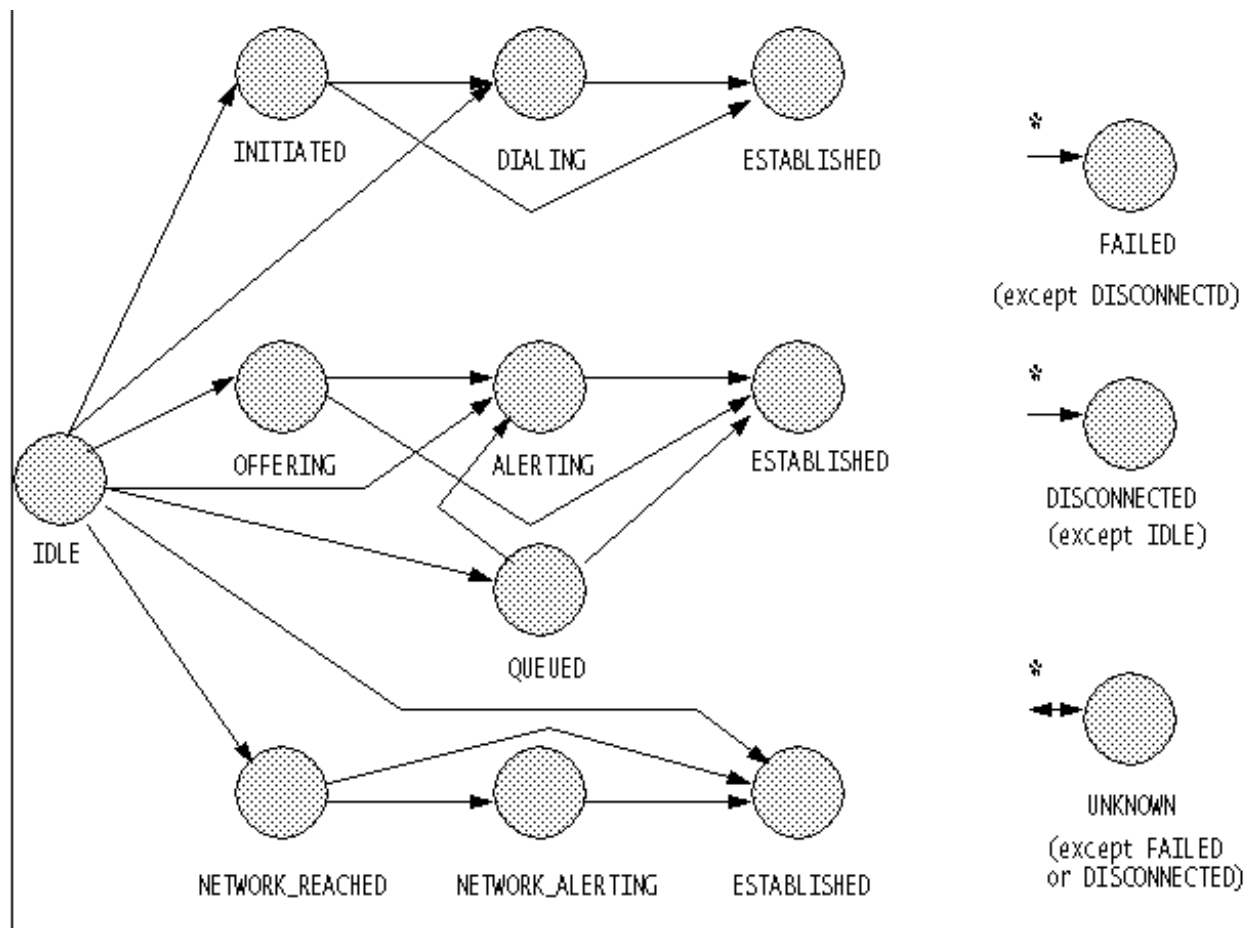
The UNKNOWN state has the same meaning as the Connection's UNKNOWN state. It implies that the state of a Connection is not known to the Provider as the application cannot take any action on this Connection.

State Transitions for the CallControlConnection Interface

In the overview of the core Connection states, a finite state machine described the allowable transitions from one Connection state to the other. This state diagram provides insight to the application developer on how Connections may move from one state to another. The following diagram is the analogous state

diagram for the CallControlConnection states. Because the CallControlConnection interface possesses more states than the core, the state diagram is more complex. Applications that perform only the simple tasks provided by the core need not concern themselves with this more complicated set of states.

To make the state diagram more clear, the ESTABLISHED state has been replicated three times. All of the transitions into the ESTABLISHED state can be viewed as the union of all of the transitions into each separate ESTABLISHED state below. Similarly, all of the transitions out of the ESTABLISHED state can be viewed as the union of all of the transitions out of each separate ESTABLISHED state below. The FAILED, DISCONNECTED, and UNKNOWN states are unique because they have transitions into, and out of, virtually every other state. The asterisk (*) next to the transition arrows denote this. There are several exceptions, however. There are no transitions out of the DISCONNECTED state and the IDLE state may not transition into the DISCONNECTED state. All states may transition into the FAILED state except for the DISCONNECTED state. Neither FAILED nor DISCONNECTED states may transition into the UNKNOWN state. Also, the FAILED state may transition only to the DISCONNECTED.



Relationship between Connection and CallControlConnection States

In addition to the allowable transitions of Connection states in the Call Control Package, the mapping of the Connection states to the CallControlConnection states is important. This mapping implies that when the CallControlConnection state is a certain states, it implies the Connection state must be a certain state. Although applications should only monitor one set of states, it is necessary to provide this mapping to maintain consistency.

The following chart describes the state of the Connection in the Call Control Package (on the right) and the state of the Connection in the core package (on the left). Note that applications may obtain the core Connection state via the method `Connection.getState()` and applications may obtain the Call Control Package Connection states via the `CallControlConnection.getCallControlState()` method.

CONNECTION STATE RELATIONSHIP	
Core	CallControl
IDLE	IDLE
INPROGRESS	OFFERING QUEUED
ALERTING	ALERTING
CONNECTED	ESTABLISHED NETWORK_REACHED NETWORK_ALERTING INITIATED DIALING
DISCONNECTED	DISCONNECTED
FAILED	FAILED
UNKNOWN	UNKNOWN

CallControlTerminalConnection States

IDLE state

The IDLE state has the same meaning as in the TerminalConnection. It is the initial state for all CallControlTerminalConnection objects.

RINGING state

The RINGING state has the same meaning as the TerminalConnection's RINGING state. It implies that a Terminal is ringing because of an incoming telephone call.

BRIDGED state

The BRIDGED state indicates that a Terminal is bridged into an active telephone call. Although the Terminal is not actively part of the telephone call it may become so by joining the call. Although not actively part of the telephone call, the resource associated with this bridge on the Terminal is being used.

INUSE state

The INUSE state indicates that a Terminal is not actively part of a call nor bridged into a call, but still part of the telephone call. Although the Terminal cannot join the Call, the resource on the Terminal associated with this Call is still being used.

TALKING state

The TALKING state indicates that a Terminal is actively part of a telephone call. It is similar to the TerminalConnection's TALKING state, however that state has a more general definition. The TALKING state implies that Terminal is talking on the telephone call versus being held.

HELD state

The HELD state indicates that a Terminal is actively part of a telephone call, however it is currently on hold.

DROPPED state

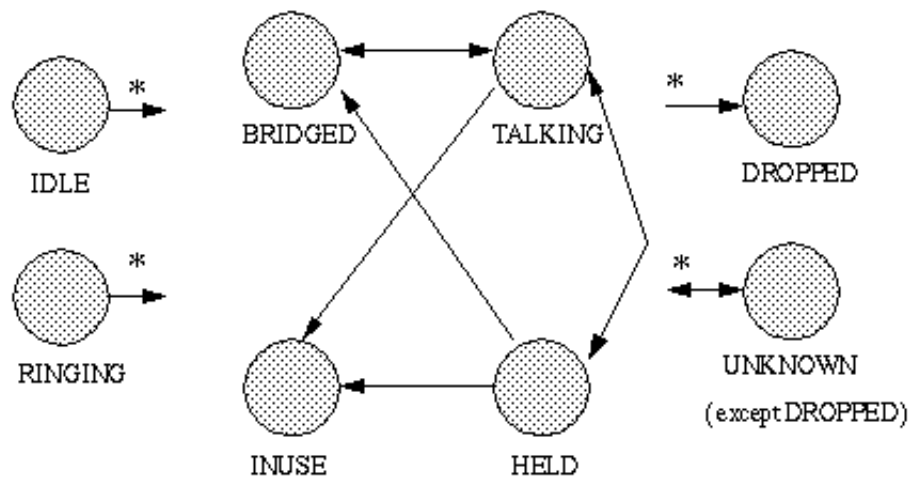
The DROPPED state has the same meaning as the TerminalConnection's DROPPED state. The Terminal was once part of a telephone call but no longer is. As in the core, this state is the final state for all TerminalConnections in the Call Control Package.

UNKNOWN state

The UNKNOWN state has the same meaning as the TerminalConnection's UNKNOWN state.

State Transitions for the CallControlTerminalConnection Interface

In the overview of the core TerminalConnection states, a finite state machine described the allowable transitions from one TerminalConnection state to the other. This state diagram illustrates how TerminalConnections may move from one state to another. The following diagram is the analogous state diagram for the CallControlTerminalConnection states.



Relationship between TerminalConnection and CallControlTerminalConnection States

In addition to the allowable transitions of TerminalConnection states in the Call Control Package, the mapping of the TerminalConnection states to the CallControlTerminalConnection states is important. This mapping implies that when the CallControlConnection state is a certain states, it implies the TerminalConnection state must be a certain state. Although applications should only monitor one set of states, it is necessary to provide this mapping to maintain consistency.

The following chart describes the state of the TerminalConnection in the Call Control Package (on the right) and the state of the TerminalConnection in the core package (on the left). Note that applications may obtain the core TerminalConnection state via the method TerminalConnection.getState() and applications may obtain the Call Control Package TerminalConnection states via the CallControlTerminalConnection.getCallControlState() method.

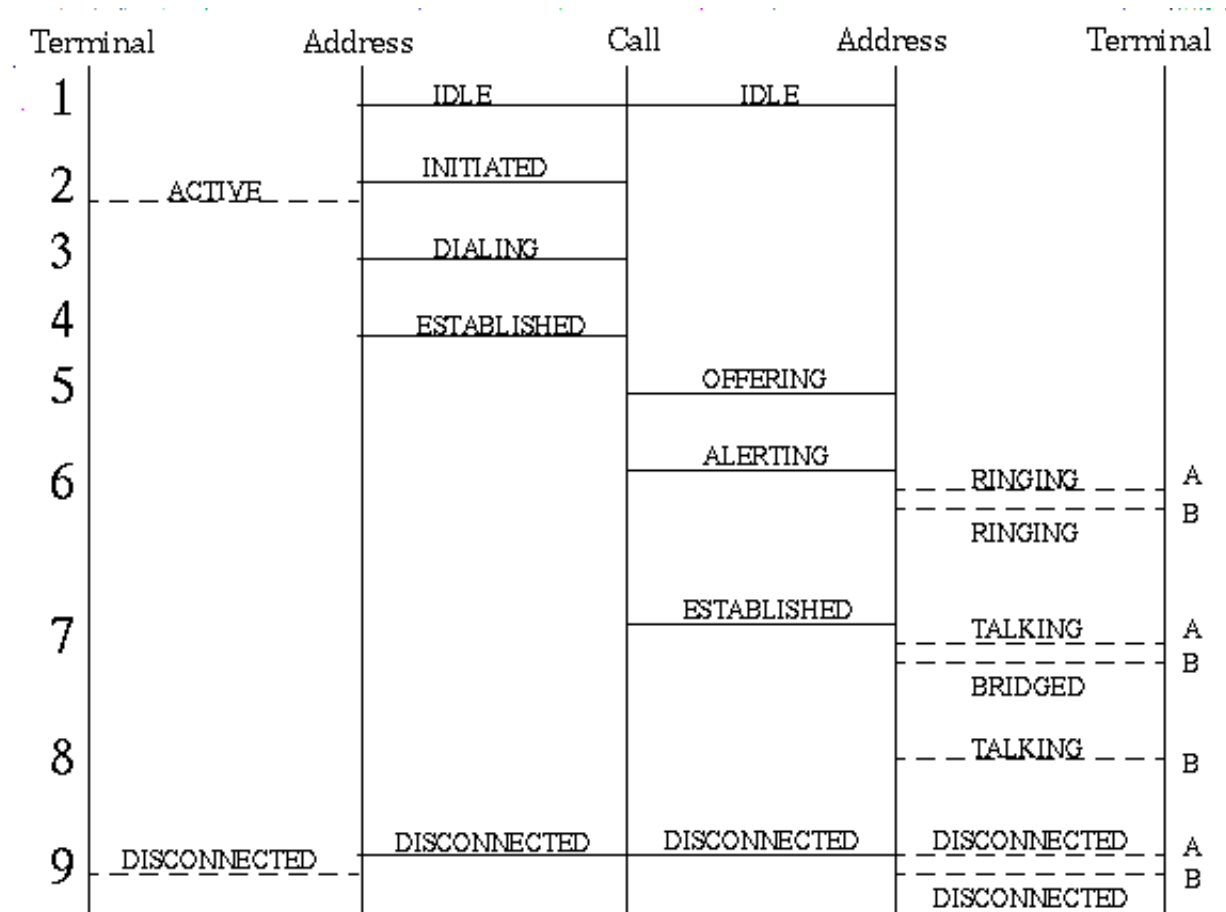
TERMINAL CONNECTION STATE RELATIONSHIP	
CORE	CALLCONTROL
IDLE	IDLE
PASSIVE	BRIDGED IN USE
ACTIVE	TALKING HELD
DROPPED	DROPPED
UNKNOWN	UNKNOWN

Timeline for Telephone Calls

The Overview document for the core Java Telephony API presented a timeline diagram of the state transitions for Connections and TerminalConnections in a likely scenario resulting from a `Call.connect()` invocation. These diagrams are the best means to understand the API during different situations. In this section, a similar diagram is presented describing the outcome of placing a telephone call in terms of the `CallControl` states.

In the diagram below, time is the vertical axis where time increases going down the page. The solid vertical lines denote various objects: `Call`, `Terminal`, and `Address`. The solid horizontal lines between the `Call` and `Address` represent a `Connection` and the dotted horizontal lines between the `Address` and `Terminal` represent `TerminalConnections`. Each discrete change in the call model is denoted by a number along the left-hand side of the page. More than one change may occur to the call model during a single discrete step; the application sees these changes as happening all at once.

In the scenario below, the originating side has one `Terminal` associated with it and the destination side has two `Terminals` (denoted in the diagram as 'A' and 'B') associated with it. The switch is configured to bridge all `Terminals` that are not actively part of a telephone call but share an `address` which is active. This scenario shows how a telephone call is placed and completed to one of the destination `Terminals`. Then, the second terminal joins the call using the `join()` method from this package. Finally, the originator hangs up his phone and the entire call becomes disconnected.



OutCall Code Example

The following code example shows placing a telephone call using the core `Call.connect()` method while examining states in the Call Control extension package.

```
import javax.telephony.*;
import javax.telephony.events.*;
import javax.telephony.callcontrol.*;
import javax.telephony.callcontrol.events.*;

/*
 * The MyCallCtlOutCallObserver class implements the CallControlCallObserver
 * interface and receives all events associated with the Call.
 */

public class MyCallCtlOutCallObserver implements CallControlCallObserver {

    public void callChangedEvent(CallEv[] evlist) {

        for (int i = 0; i < evlist.length; i++) {

            if (evlist[i] instanceof ConnEv) {
```

```

String name = null;
try {
    Connection connection = ((ConnEv)evlist[i]).getConnection();
    Address addr = connection.getAddress();
    name = addr.getName();
} catch (Exception excp) {
    // Handle Exceptions
}

String msg = "Connection to Address: " + name + " is ";

if (evlist[i].getID() == CallCtlConnInitiatedEv.ID) {
    System.out.println(msg + "INITIATED");
}
else if (evlist[i].getID() == CallCtlConnDialingEv.ID) {
    System.out.println(msg + "DIALING");
}
else if (evlist[i].getID() == CallCtlConnEstablishedEv.ID) {
    System.out.println(msg + "ESTABLISHED");
}
else if (evlist[i].getID() == CallCtlConnAlertingEv.ID) {
    System.out.println(msg + "ALERTING");
}
else if (evlist[i].getID() == CallCtlConnDisconnectedEv.ID) {
    System.out.println(msg + "DISCONNECTED");
}
}
}
}
}
}

```

```

-----

import javax.telephony.*;
import javax.telephony.events.*;
import javax.telephony.callcontrol.*;
import javax.telephony.callcontrol.events.*;

/*
 * Places a telephone call from 476111 to 5551212
 */
public class CallCtlOutcall {

    public static final void main(String args[]) {

        /*
         * Create a provider by first obtaining the default implementation of
         * JTAPI and then the default provider of that implementation.
         */
        Provider myprovider = null;
        try {
            JtapiPeer peer = JtapiPeerFactory.getJtapiPeer(null);
            myprovider = peer.getProvider(null);
        } catch (Exception excp) {
            System.out.println("Can't get Provider: " + excp.toString());
        }
    }
}

```



```

        System.exit(0);
    }

    /*
     * We need to get the appropriate objects associated with the
     * originating side of the telephone call. We ask the Address for a list
     * of Terminals on it and arbitrarily choose one.
     */
    Address origaddr = null;
    Terminal origterm = null;
    try {
        origaddr = myprovider.getAddress("4761111");

        /* Just get some Terminal on this Address */
        Terminal[] terminals = origaddr.getTerminals();
        if (terminals == null) {
            System.out.println("No Terminals on Address.");
            System.exit(0);
        }
        origterm = terminals[0];
    } catch (Exception excp) {
        // Handle exceptions;
    }

    /*
     * Create the telephone call object and add an observer.
     */
    Call mycall = null;
    try {
        mycall = myprovider.createCall();
        mycall.addObserver(new MyCallCtlOutCallObserver());
    } catch (Exception excp) {
        // Handle exceptions
    }

    /*
     * Place the telephone call.
     */
    try {
        Connection c[] = mycall.connect(origterm, origaddr, "5551212");
    } catch (Exception excp) {
        // Handle all Exceptions
    }
}
}

```

InCall Code Example

The following code example illustrates how an application answers a Call at a particular Terminal. It shows how application accept incoming calls offered to it. This code example closely resembles the core InCall code example.

```

import javax.telephony.*;
import javax.telephony.events.*;
import javax.telephony.callcontrol.*;
import javax.telephony.callcontrol.events.*;

/*
 * The MyCallCtlInCallObserver class implements the CallControlCallObserver and
 * receives all Call-related events.
 */

public class MyCallCtlInCallObserver implements CallControlCallObserver {

    public void callChangedEvent(CallEv[] evlist) {

        for (int i = 0; i < evlist.length; i++) {

            if (evlist[i] instanceof TermConnEv) {
                TerminalConnection termconn = null;
                String name = null;

                try {
                    TermConnEv tcev = (TermConnEv)evlist[i];
                    Terminal term = termconn.getTerminal();
                    termconn = tcev.getTerminalConnection();
                    name = term.getName();
                } catch (Exception excp) {
                    // Handle exceptions.
                }

                String msg = "TerminalConnection to Terminal: " + name + " is ";

                if (evlist[i].getID() == CallCtlTermConnTalkingEv.ID) {
                    System.out.println(msg + "TALKING");
                }
                else if (evlist[i].getID() == CallCtlTermConnRingingEv.ID) {
                    System.out.println(msg + "RINGING");

                    /* Answer the telephone Call using "inner class" thread */

                    try {
                        final TerminalConnection _tc = termconn;
                        Runnable r = new Runnable() {
                            public void run() {
                                try{
                                    _tc.answer();
                                } catch (Exception excp){
                                    // Handle answer exceptions
                                }
                            }
                        };
                        Thread T = new Thread(r);
                        T.start();
                    } catch (Exception excp) {
                        // Handle Exceptions;
                    }
                }
                else if (evlist[i].getID() == CallCtlTermConnDroppedEv.ID) {

```

```

        System.out.println(msg + "DROPPED");
    }
}
}
}
}

```

```

-----

import javax.telephony.*;
import javax.telephony.events.*;
import javax.telephony.callcontrol.*;
import javax.telephony.callcontrol.events.*;
import MyCallCtlInCallObserver;

/*
 * Create a provider and monitor a particular terminal for an incoming call.
 */
public class CallCtlIncall {

    public static final void main(String args[]) {

        /*
         * Create a provider by first obtaining the default implementation of
         * JTAPI and then the default provider of that implementation.
         */
        Provider myprovider = null;
        try {
            JtapiPeer peer = JtapiPeerFactory.getJtapiPeer(null);
            myprovider = peer.getProvider(null);
        } catch (Exception excp) {
            System.out.println("Can't get Provider: " + excp.toString());
            System.exit(0);
        }

        /*
         * Get the terminal we wish to monitor and add a call observer to that
         * Terminal. This will place a call observer on all call which come to
         * that terminal. We are assuming that Terminals are named after some
         * primary telephone number on them.
         */
        try {
            Terminal terminal = myprovider.getTerminal("4761111");
            terminal.addCallObserver(new MyCallCtlInCallObserver());
        } catch (Exception excp) {
            System.out.println("Can't get Terminal: " + excp.toString());
            System.exit(0);
        }
    }
}

```

Hierarchy For Package javax.telephony.callcontrol

Package Hierarchies:

[All Packages](#)

Class Hierarchy

- class java.lang.Object
 - class javax.telephony.callcontrol.[CallControlForwarding](#)

Interface Hierarchy

- interface javax.telephony.[Address](#)
 - interface javax.telephony.callcontrol.[CallControlAddress](#)
 - interface javax.telephony.[AddressObserver](#)
 - interface javax.telephony.callcontrol.[CallControlAddressObserver](#)
 - interface javax.telephony.[Call](#)
 - interface javax.telephony.callcontrol.[CallControlCall](#)
 - interface javax.telephony.[CallObserver](#)
 - interface javax.telephony.callcontrol.[CallControlCallObserver](#)
 - interface javax.telephony.[Connection](#)
 - interface javax.telephony.callcontrol.[CallControlConnection](#)
 - interface javax.telephony.[Terminal](#)
 - interface javax.telephony.callcontrol.[CallControlTerminal](#)
 - interface javax.telephony.[TerminalConnection](#)
 - interface javax.telephony.callcontrol.[CallControlTerminalConnection](#)
 - interface javax.telephony.[TerminalObserver](#)
 - interface javax.telephony.callcontrol.[CallControlTerminalObserver](#)
-

javax.telephony.callcontrol

Interface CallControlAddress

public interface **CallControlAddress**
extends [Address](#)

Introduction

The `CallControlAddress` interface extends the core `Address` interface. It provides additional features on the `Address`. Applications may query an `Address` object using the `instanceof` operator to see whether it supports this interface.

Address Forwarding

This interface supports methods which permit applications to modify and query the forwarding characteristics of an `Address`. The forwarding characteristics determine how incoming telephone calls to this `Address` should be handled, if any special handling is desired. A switching domain will honor those instructions to the extent that other possibly higher priority instructions allow.

Each `Address` may have zero or more *forwarding instructions*. Each instruction describes how the switching domain should handle incoming telephone calls to an `Address` under different circumstances. Examples of forwarding instructions are "forward all calls to x9999" or "forward all calls to x7777 when no one answers." Each forwarding instruction is represented by an instance of the `CallControlForwarding` class. A switching domain will honor forwarding instructions to the extent that other (possibly higher priority) instructions allow.

Applications assign a list of forwarding instructions via the the `CallControlAddress.setForwarding()` method. To obtain the current, effective forwarding instructions, applications invoke the `CallControlAddress.getForwarding()` method. To cancel all forwarding instructions, applications use the `CallControlAddress.cancelForwarding()` method.

Do Not Disturb and Message Waiting

The `CallControlAddress` interface defines two additional attributes: *do-not-disturb* and *message-waiting*.

The do-not-disturb feature gives the means to notify the telephony platform that an `Address` does not want to receive incoming telephone calls. That is, if this feature is activated, the underlying telephony platform will not alert this `Address` to incoming telephone calls. Applications use the `CallControlAddress.setDoNotDisturb()` method to activate or deactivate this feature and the `CallControlAddress.getDoNotDisturb()` method to return the current state of this attribute.

Note that the `CallControlTerminal` interface also has a do-not-disturb attribute. This gives the ability to control the do not disturb property at either the Address level (e.g. a phone number) or at the Terminal level (e.g. an individual phone.)

The message-waiting attribute indicates whether there are messages waiting for a human user of the Address. These messages may either be maintained by an application or some telephony platform. Applications inform the telephony hardware of the message waiting status, and typically the hardware displays a visible indicator (e.g. an LED) to users. Applications use the `CallControlAddress.setMessageWaiting()` method to activate or deactivate this feature and the `CallControlAddress.getMessageWaiting()` method to return the current state of this attribute.

Observers and Events

All events pertaining to the `CallControlAddress` interface are reported via the `AddressObserver.addressChangedEvent()` method. The application observer object must also implement the `CallControlCallObserver` interface to express interest in the call control package events.

The following are the events associated with this interface:

<code>CallCtlAddrDoNotDisturbEv</code>	Indicates the do-not-disturb feature of this Address has changed.
<code>CallCtlAddrForwardEv</code>	Indicates the forwarding characteristics of this Address has changed.
<code>CallCtlAddrMessageWaitingEv</code>	Indicates the message waiting characteristics of this Address has changed.

See Also:

`CallControlTerminal`, `CallControlForwarding`,
`CallControlAddressObserver`, `CallCtlAddrDoNotDisturbEv`,
`CallCtlAddrForwardEv`, `CallCtlAddrMessageWaitingEv`

Method Summary	
void	<u>cancelForwarding()</u> Cancels all of the forwarding instructions on this Address.
boolean	<u>getDoNotDisturb()</u> Returns true if the do-not-disturb feature is activated, false otherwise.
<u>CallControlForwarding[]</u>	<u>getForwarding()</u> Returns an array of forwarding instructions currently effective for this Address.
boolean	<u>getMessageWaiting()</u> Returns true if the message waiting is activated, false otherwise.
void	<u>setDoNotDisturb(boolean enable)</u> Specifies whether the do-not-disturb feature should be activated or deactivated for this Address.
void	<u>setForwarding(CallControlForwarding[] instructions)</u> Sets the forwarding characteristics for this Address.
void	<u>setMessageWaiting(boolean enable)</u> Specifies whether the message-waiting indicator should be activated or deactivated for this Address.

Methods inherited from interface <u>javax.telephony.Address</u>
<u>addAddressListener</u> , <u>addCallListener</u> , <u>addCallObserver</u> , <u>addObserver</u> , <u>getAddressCapabilities</u> , <u>getAddressListeners</u> , <u>getCallListeners</u> , <u>getCallObservers</u> , <u>getCapabilities</u> , <u>getConnections</u> , <u>getName</u> , <u>getObservers</u> , <u>getProvider</u> , <u>getTerminals</u> , <u>removeAddressListener</u> , <u>removeCallListener</u> , <u>removeCallObserver</u> , <u>removeObserver</u>

Method Detail

setForwarding

```
public void setForwarding(CallControlForwarding[] instructions)
    throws MethodNotSupportedException,
           InvalidStateException,
           InvalidArgumentException
```


Sets the forwarding characteristics for this Address. This forwarding command supplants all previous forwarding instructions if it succeeds, otherwise the forwarding state at the time of the command remains unchanged. This method takes an array of `CallControlForwarding` objects. Each object describes a different forwarding. This method waits until all forwarding instructions have been set or until an error occurs and an exception is thrown.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getForwarding() == instructions`
3. `CallCtlAddrForwardEv` is delivered for this Address

Parameters:

`instructions` - An array of Address forwarding instructions

Throws:

`MethodNotSupportedException` - This method is not supported by the given implementation.

`InvalidStateException` - The Provider is not "in service".

`InvalidArgumentException` - An invalid set of forwarding instructions were given as a parameter.

See Also:

`CallCtlAddrForwardEv`

getForwarding

```
public CallControlForwarding[] getForwarding()
                                throws MethodNotSupportedException
```

Returns an array of forwarding instructions currently effective for this Address. If there are no effective forwarding instructions, this method returns null.

Returns:

An array of Address forwarding instructions, null if there are none.

Throws:

`MethodNotSupportedException` - This method is not supported by the given implementation.

cancelForwarding

```
public void cancelForwarding()
           throws MethodNotSupportedException,
                  InvalidStateException
```

Cancels all of the forwarding instructions on this Address. When this method completes, the `CallControlAddress.getForwarding()` method will return null. This method waits until all forwarding instructions have been cancelled or until an error occurs and an exception is thrown.

Pre-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getForwarding == null
3. CallCtlAddrForwardEv is delivered for this Address.

Throws:

MethodNotSupportedException - This method is not supported by the given implementation.

InvalidStateException - The Provider is not "in service".

See Also:

CallCtlAddrForwardEv

getDoNotDisturb

```
public boolean getDoNotDisturb()
           throws MethodNotSupportedException
```

Returns true if the do-not-disturb feature is activated, false otherwise.

Returns:

True if do-not-disturb feature is activated, false if it is deactivated.

Throws:

MethodNotSupportedException - This method is not supported by the given implementation.

setDoNotDisturb

```
public void setDoNotDisturb(boolean enable)
           throws MethodNotSupportedException,
                  InvalidStateException
```

Specifies whether the do-not-disturb feature should be activated or deactivated for this Address. This feature only affects whether or not calls will be accepted at this Address. Note that the do-not-disturb feature on all Terminals associated with this Address are independent of this Terminal's do-not-disturb setting. If 'enable' is true, do-not-disturb is activated if not already activated. If 'enable' is false, do-not-disturb is deactivated if not already deactivated.

Pre-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getDoNotDisturb() == enable
3. CallCtlAddrDoNotDisturbEv is delivered for this Address

Parameters:

`enable` - True to activate do-not-disturb, false to deactivate.

Throws:

[MethodNotSupportedException](#) - This method is not supported by the given implementation.

[InvalidStateException](#) - The Provider is not "in service".

See Also:

[CallCtlAddrDoNotDisturbEv](#)

getMessageWaiting

```
public boolean getMessageWaiting()  
    throws MethodNotSupportedException
```

Returns true if the message waiting is activated, false otherwise.

Returns:

True if message-waiting is activated, false if it is deactivated.

Throws:

[MethodNotSupportedException](#) - This method is not supported by the given implementation.

setMessageWaiting

```
public void setMessageWaiting(boolean enable)  
    throws MethodNotSupportedException,  
           InvalidStateException
```

Specifies whether the message-waiting indicator should be activated or deactivated for this Address. If 'enable' is true, message-waiting is activated if not already activated. If 'enable' is false, message-waiting is deactivated if not already deactivated.

Pre-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getMessageWaiting() == enable
3. CallCtlAddrMessageWaitingEv is delivered for this Address

Parameters:

`enable` - True to activate message-waiting, false to deactivate.

Throws:

[MethodNotSupportedException](#) - This method is not supported by the given implementation.

[InvalidStateException](#) - The Provider is not "in service".

See Also:

[CallCtlAddrMessageWaitingEv](#)

javax.telephony.callcontrol **Interface CallControlAddressObserver**

public interface **CallControlAddressObserver**
extends [AddressObserver](#)

The `CallControlAddressObserver` interface reports all events for the `CallControlAddress` interface. Applications implement this interface to receive `CallControlAddress`-related events. All events are reported via the `AddressObserver.addressChangedEvent()` method. This interface, therefore, allows applications to signal to the implementation that they are interested in call control package events. This interface defines no additional methods.

All events must extend the `CallCtlAddrEv` event interface, which in turn, extends the core `AddrEv` interface.

The following are those events which are associated with this interface:

<code>CallCtlAddrDoNotDisturbEv</code>	Indicates the do not disturb characteristics of this Address has changed.
<code>CallCtlAddrForwardEv</code>	Indicates the forwarding characteristics of this Address has changed.
<code>CallCtlAddrMessageWaitingEv</code>	Indicates the message waiting characteristics of this Address has changed.

See Also:

[AddressObserver](#), [AddrEv](#), [CallControlAddress](#), [CallCtlAddrEv](#),
[CallCtlAddrDoNotDisturbEv](#), [CallCtlAddrForwardEv](#),
[CallCtlAddrMessageWaitingEv](#)

Methods inherited from interface javax.telephony.AddressObserver

addressChangedEvent

javax.telephony.callcontrol **Interface CallControlCall**

public interface **CallControlCall**
extends Call

Introduction

The `CallControlCall` interface extends the core `Call` interface. This interface provides additional methods on a `Call`.

Additional Call Information

This interface supports methods which return additional information regarding the `Call`. Specifically, it returns the *calling address*, *calling terminal*, *called address*, and *last redirected address* information.

The calling address, as returned by the `CallControlCall.getCallingAddress()` method is the `Address` which originally placed the `Call`. The calling terminal, as returned by the `CallControlCall.getCallingTerminal()` method is the `Terminal` which originally placed the `Call`. The called `Address`, as returned by the `CallControlCall.getCalledAddress()` method is the `Address` to which the `Call` was originally placed. The last redirected address, as returned by the `CallControlCall.getLastRedirectedAddress()` method is the `Address` to which this `Call` was placed before the current destination `Address`. For example, if a `Call` was forwarded from one `Address` to another, then the first `Address` is the last redirected `Address` for this call.

Each of these methods returns `null` if their values are unknown at the present time. During the lifetime of a `Call`, an implementation may learn this additional information, and return different values for some or all of these methods as a result.

Conferencing Telephone Calls

The conferencing feature supported by this interface permits two telephone calls to be "merged". That is, the two `Calls` are merged into a single `Call` with the union of all of the participants of the two `Calls` being placed on the single `Call`.

Applications invoke the `CallControlCall.conference()` method to perform the conferencing feature. This method is given the "second" `Call` as an argument. All participants are moved from the second `Call` to the `Call` on which the method is invoked. The second `Call` moves into the `Call.INVALID` state as a result.

In order for the conferencing feature to happen, there must be a common participant to both `Calls`, as represented by a single `Terminal` and two `TerminalConnections`, one on each of the two `Calls`. These two `TerminalConnections` are known as the *conference controllers*. In the real-world, one of the two telephone calls must be on hold with respect to the controlling `Terminal`, and hence, the `TerminalConnection` on the second `Call` must be in the `CallControlTerminalConnection.HELD` state. The two conference

controlling TerminalConnections are merged into one as a result of this method.

Applications may control which TerminalConnection acts as the conference controller via the `CallControlCall.setConferenceController()` method. The `CallControlCall.getConferenceController()` method returns the current conference controller, `null` if there is none. If no conference controller is set, the implementation chooses a suitable TerminalConnection when the conferencing feature is invoked.

Transferring Telephone Calls

The transfer feature supported by this interface permits one Call to be "moved" to another Call. That is, all of the participants from one Call are moved to another Call, except for the transferring participant which drops off from both Calls.

Applications invoke the `CallControlCall.transfer()` method to perform the transfer feature. There are two overloaded versions of this method. The first method takes a second Call as an argument. This method acts similarly to `CallControlCall.conference()`, except the two TerminalConnections on each Call with a common Terminal are removed from both Calls. The second version takes a string telephone address as an argument. This method removes the transfer controller participant while placing the telephone call to the designated address. This latter version of the transfer feature is often known as a *single-step transfer*.

In order for the transfer feature to happen, there must be a participant which acts as the *transfer controller*. The transfer controller is a TerminalConnection around which the transfer is performed. In the first version of the `CallControlCall.transfer()` method, the transfer controller must be present on each of the two Calls and share a common Terminal. In the second version, the transfer controller only applies to the Call object on which the method is invoked (since there is no second Call involved). In both cases, the transfer controller participant is no longer part of any Call once the transfer feature is complete.

Applications may control which TerminalConnection acts as the transfer controller via the `CallControlCall.setTransferController()` method. The `CallControlCall.getTransferController()` method returns the current transfer controller, `null` if there is none. If no transfer controller is set, the implementation chooses a suitable TerminalConnection when the conferencing feature is invoked.

Consultation Calls

Consultation Calls are special types of telephony calls created (often temporarily) for a specific purpose. Consultation calls are created if a user wants to "consult" with another party briefly while currently on a Call, or are created for the purpose of conferencing or transferring with a Call. Consequently, consultation calls are always associated with another existing Call.

Applications invoke the `CallControlCall.consult()` method to perform the consultation feature. The instance on which the method is invoked is always the "idle" Call on which the consultation takes place. There are two overloaded versions of this method. The first method takes a TerminalConnection and a string telephone address as arguments. This consultation telephone call is associated with the Call of the TerminalConnection argument. This method places a telephone call from the same originating endpoint specified by the TerminalConnection argument to the designated telephone address string. The

second version of this method only takes a `TerminalConnection` as an argument, and permits applications to use the `CallControlConnection.addToAddress()` method to dial the destination address string.

Additional CallControlCall Methods

The `CallControlCall.addParty()` method adds a single party to a `Call` given some telephone address string. The `CallControlCall.drop()` disconnects all parties from the `Call` and moves it into the `Call.INVALID` state. The `CallControlCall.offHook()` method takes an originating Address and Terminal pair "off hook" and permits applications to dial destination address digits one-by-one.

Observers and Events

All events pertaining to the `CallControlCall` interface are reported via the `CallObserver.callChangedEvent()` method. The application observer object must also implement the `CallControlCallObserver` interface to express interest in the call control package events. Applications receive events pertaining to the `CallControlConnection` and `CallControlTerminalConnection` interfaces via this observer as well.

All `CallControlCall`-related events must extend the `CallCtlCalEv` interface. There are no specific events pertaining to the `CallControlCall` interface, however.

See Also:

[Call](#), [CallObserver](#), [CallControlCallObserver](#), [CallCtlCalEv](#)

Fields inherited from interface javax.telephony.Call

`ACTIVE`, `IDLE`, `INVALID`

Method Summary

<u>Connection</u>	addParty (java.lang.String newParty) Adds an additional party to an existing Call.
void	conference (<u>Call</u> otherCall) Merges two Calls together, resulting in the union of the participants of both Calls being placed on a single Call.
<u>Connection</u>	consult (<u>TerminalConnection</u> tc) Creates a consultation between this Call and an active Call.
<u>Connection</u> []	consult (<u>TerminalConnection</u> tc, java.lang.String dialedDigits) Creates a consultation between this Call and an active Call.

void	<u>drop()</u> Drops the entire Call.
<u>Address</u>	<u>getCalledAddress()</u> Returns the called Address associated with this Call.
<u>Address</u>	<u>getCallingAddress()</u> Returns the calling Address associated with this call.
<u>Terminal</u>	<u>getCallingTerminal()</u> Returns the calling Terminal associated with this Call.
<u>TerminalConnection</u>	<u>getConferenceController()</u> Returns the TerminalConnection which currently acts as the conference controller.
boolean	<u>getConferenceEnable()</u> Return true if conferencing is enabled, false otherwise.
<u>Address</u>	<u>getLastRedirectedAddress()</u> Returns the last redirected Address associated with this Call.
<u>TerminalConnection</u>	<u>getTransferController()</u> Returns the TerminalConnection which currently acts as the transfer controller.
boolean	<u>getTransferEnable()</u> Return true if transferring is enabled, false otherwise.
<u>Connection</u>	<u>offHook()</u> (<u>Address</u> origaddress, <u>Terminal</u> origterminal) Takes the originating end of a Call off-hook.
void	<u>setConferenceController()</u> (<u>TerminalConnection</u> tc) Sets the TerminalConnection which acts as the conference controller for the Call.
void	<u>setConferenceEnable()</u> (boolean enable) Controls whether the Call is permitted or able to perform the conferencing feature.
void	<u>setTransferController()</u> (<u>TerminalConnection</u> tc) Sets the TerminalConnection which acts as the transfer controller for the Call.
void	<u>setTransferEnable()</u> (boolean enable) Controls whether the Call is permitted or able to perform the transferring feature.
void	<u>transfer()</u> (<u>Call</u> otherCall) This method moves all participants from one Call to another, with the exception of a selected common participant.

<u>Connection</u>	<u>transfer</u> (java.lang.String address) This overloaded version of this method transfers all participants currently on this Call, with the exception of the transfer controller participant, to another telephone address.
-------------------	---

Methods inherited from interface javax.telephony.Call

addCallListener, addObserver, connect, getCallCapabilities, getCallListeners, getCapabilities, getConnections, getObservers, getProvider, getState, removeCallListener, removeObserver

Method Detail

getCallingAddress

public Address **getCallingAddress**()

Returns the calling Address associated with this call. The calling Address is the Address which originally placed the telephone call. If the calling address is unknown or not yet known, this method returns null.

Returns:

The calling Address.

getCallingTerminal

public Terminal **getCallingTerminal**()

Returns the calling Terminal associated with this Call. The calling Terminal is the Terminal which originally placed the telephone call. If the calling Terminal is unknown or not yet known, this method returns null.

Returns:

The calling Terminal.

getCalledAddress

public Address **getCalledAddress**()

Returns the called Address associated with this Call. The called Address is the Address to which the call had been originally placed. If the called address is unknown or not yet known, this method returns null.

Returns:

The called Address.

getLastRedirectedAddress

public Address **getLastRedirectedAddress()**

Returns the last redirected Address associated with this Call. The last redirected Address is the Address at which the current Call was placed immediately before the current Address. This is common if a Call is forwarded to several Addresses before being answered. If the last redirected address is unknown or not yet known, this method returns null.

Returns:

The last redirected Address for this telephone Call.

addParty

public Connection **addParty**(java.lang.String newParty)
 throws InvalidStateException,
 InvalidPartyException,
 MethodNotSupportedException,
 PrivilegeViolationException,
 ResourceUnavailableException

Adds an additional party to an existing Call. This is sometimes called a "single-step conference" because a party is conferenced into a Call directly. The telephone address string provided as the argument must be complete and valid.

States of the Existing Connections

The Call must have at least two Connections in the `CallControlConnection.ESTABLISHED` state. An additional restriction requires that at most one other Connection may be in either the `CallControlConnection.QUEUED`, `CallControlConnection.OFFERED`, or `CallControlConnection.ALERTING` state.

Some telephony platforms impose restrictions on the number of Connections in a particular state. For instance, it is common to restrict the number of "alerting" Connections to at most one. As a result, this method requires that at most one other Connections is in the "queued", "offering", or "alerting" state. (Note that the first two states correspond to the core Connection "in progress" state). Although some systems may not enforce this requirement, for consistency, JTAPI specifies implementations must uphold the conservative requirement.

The New Connection

This method creates and returns a new `Connection` representing the new party. This `Connection` must at least be in the `CallControlConnection.IDLE` state. Its state may have progressed beyond "idle" before this method returns, and should be reflected by an event. This new `Connection` will progress as any normal destination `Connection` on a `Call`. Typical scenarios for this `Connection` are described by the `Call.connect()` method.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.ACTIVE`
3. Let `c[] = call.getConnections()` where `c.length >= 2`
4. `c[i].getCallControlState() == CallControlConnection.ESTABLISHED` for at least two `i`
5. `c[j].getCallControlState() == CallControlConnection.QUEUED`, `CallControlConnection.OFFERED`, or `CallControlConnection.ALERTING` for at most one `c[j]`

Post-conditions:

1. Let `connection` be the `Connection` created and returned
2. `(this.getProvider()).getState() == Provider.IN_SERVICE`
3. `this.getState() == Call.ACTIVE`
4. `connection.getCallControlState()` at least `CallControlConnection.IDLE`
5. `ConnCreatedEv` is delivered for `connection`

Parameters:

`newParty` - The telephone address of the party to be added.

Returns:

The new `Connection` associated with the added party.

Throws:

[InvalidStateException](#) - Either the `Provider` is not "in service", the `Call` is not "active" or the proper conditions on the `Connections` does not exist, as designated by the pre-conditions for this method.

[InvalidPartyException](#) - The destination address string is not valid and/or complete.

[MethodNotSupportedException](#) - This method is not supported by the implementation.

[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method.

[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

See Also:

[ConnCreatedEv](#)

drop

```
public void drop()
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Drops the entire Call. This method is equivalent to using the `Connection.disconnect()` method on each Connection which is part of the Call. Typically, each Connection on the Call will move into the `CallControlConnection.DISCONNECTED` state, each TerminalConnection will move into the `CallControlTerminalConnection.DROPPED` state, and the Call will move into the `Call.INVALID` state.

There are some Connections for which the application does not possess the proper authority to disconnect. In this case, this method performs no action on these Connections. These Connections may disconnect naturally as a result of disconnecting other Connections, however. This method returns when it can successfully disconnect as many methods as it can. The application is notified via events whether the entire Call was successfully dropped.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.ACTIVE`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. Let `c[] = this.getConnections()` before this method is invoked
3. If `c[i].getCallControlState == CallControlConnection.DISCONNECTED`, for all `i`, then `this.getState() == Call.INVALID`
4. `CallCtlConnDisconnectedEv/ConnDisconnectedEv` is delivered for all disconnected Connections
5. `CallCtlTermConnDroppedEv/TermConnDroppedEv` is delivered for all dropped TerminalConnections
6. `CallInvalidEv` is delivered if all Connections were disconnected

Throws:

[InvalidStateException](#) - Either the Provider was not "in service" or the Call was not "active".

[MethodNotSupportedException](#) - This method is not supported by the implementation.

[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method and it can drop none of the Connections.

[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

See Also:

[ConnDisconnectedEv](#), [TermConnDroppedEv](#), [CallInvalidEv](#),
[CallCtlConnDisconnectedEv](#), [CallCtlTermConnDroppedEv](#)

offHook

```
public Connection offHook(Address origaddress,
                        Terminal origterminal)
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Takes the originating end of a Call off-hook. This method permits applications to simply take the originating terminal of a Call off-hook so that users may manually dial telephone number digits or applications may supply digits with the `CallControlConnection.addToAddress()` method. This is in contrast to the `Call.connect()` method which requires the complete destination address string.

This method takes the originating Address and Terminal as arguments. This Call must be in the `Call.IDLE` state. This method creates and returns a Connection to the originating Address in the `CallControlConnection.INITIATED` state. This method also creates a TerminalConnection in the `CallControlTerminalConnection.TALKING` state and associated with the new Connection and originating Terminal.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.ACTIVE`
3. Let connection be the Connection created and returned
4. Let terminalconnection be the TerminalConnection created
5. `connection.getCallControlState() == CallControlConnection.INITIATED`
6. Let `tc[] = c.getTerminalConnections()` where `tc.length == 1`
7. `tc[0] == terminalconnection`
8. `tc[0].getCallControlState() == CallControlTerminalConnection.TALKING`
9. `tc[0]` element of `origterminal.getTerminalConnections()`
10. `connection` element of `origaddress.getConnections()`
11. `connection` element of `this.getConnections()`
12. `ConnCreatedEv` is delivered for connection
13. `TermConnCreatedEv` is delivered for terminalconnection
14. `CallActiveEv` is delivered for this Call
15. `CallCtlConnInitiatedEv/ConnConnectedEv` is delivered for connection
16. `CallCtlTermConnTalkingEv/TermConnActiveEv` is delivered for terminalconnection

Parameters:

`origaddress` - The originating Address object.
`origterminal` - The originating Terminal object.

Returns:

The Connection associated with the originating end of the telephone call.

Throws:

[InvalidStateException](#) - Either the Provider was not "in service" or the Call was not "idle".

[MethodNotSupportedException](#) - This method is not supported by the implementation.

[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method

[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

See Also:

[ConnCreatedEv](#), [TermConnCreatedEv](#), [CallActiveEv](#),
[CallCtlConnInitiatedEv](#), [CallCtlTermConnTalkingEv](#)

conference

```
public void conference(Call otherCall)
    throws InvalidStateException,
           InvalidArgumentException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Merges two Calls together, resulting in the union of the participants of both Calls being placed on a single Call. This method takes a Call as an argument, referred to hereafter as the "second" Call. All of the participants from the second call are moved to the Call on which this method is invoked.

The Conference Controller

In order for the conferencing feature to happen, there must be a common participant to both Calls, as represented by a single Terminal and two TerminalConnections, one on each of the two Calls. These two TerminalConnections are known as the *conference controllers*. In the real-world, one of the two Calls must be on hold with respect to the controlling Terminal, and hence, the TerminalConnection on the second Call must be in the `CallControlTerminalConnection.HELD` state. The two conference controlling TerminalConnections are merged into one as a result of this method.

Applications may control which TerminalConnection acts as the conference controller via the `CallControlCall.setConferenceController()` method. The `CallControlCall.getConferenceController()` method returns the current conference controller, null if there is none. If no conference controller is set, the implementation chooses a suitable TerminalConnection when the conferencing feature is invoked.

The Telephone Call Argument

All of the participants from the second Call, passed as the argument to this method, are "moved" to the Call on which this method was invoked. That is, new Connections and TerminalConnections are created on this Call which are found on the second Call. Those Connections and TerminalConnections on the second Call are removed from the Call and the Call moves into the `Call.INVALID` state.

The conference controller TerminalConnections are merged into one on this Call. That is, the existing TerminalConnection controller on this Call is left unchanged, while the TerminalConnection on the second Call is removed from that Call.

Other Shared Participants

There may exist Address and Terminals which are part of both telephone call in addition to the designated conference controller. In these instances, those participants which are shared between both Calls are merged into one. That is, the Connections and TerminalConnections on this Call are left unchanged. The corresponding Connections and TerminalConnections on the second Call are removed from that Call.

Pre-conditions:

1. Let tc1 be the conference controller on this Call
2. Let connection1 = tc1.getConnection()
3. Let tc2 be the conference controller on otherCall
4. (this.getProvider()).getState() == Provider.IN_SERVICE
5. this.getState() == Call.ACTIVE
6. tc1.getTerminal() == tc2.getTerminal()
7. tc1.getCallControlState() == CallControlTerminalConnection.TALKING
8. tc2.getCallControlState() == CallControlTerminalConnection.HELD
9. this != otherCall

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getState() == Call.ACTIVE
3. otherCall.getState() == INVALID
4. Let c[] be the Connections to be merged from otherCall
5. Let tc[] be the TerminalConnections to be merged from otherCall
6. Let new(c) be the set of new Connections created on this Call
7. Let new(tc) be the set of new TerminalConnections created on this Call
8. new(c) element of this.getConnections()
9. new(c).getCallState() == c.getCallState()
10. new(tc) element of (this.getConnections()).getTerminalConnections()
11. new(tc).getCallState() == tc.getCallState()
12. c[i].getCallControlState() == CallControlConnection.DISCONNECTED for all i
13. tc[i].getCallControlState() == CallControlTerminalConnection.DROPPED for all i
14. CallInvalidEv is delivered for otherCall
15. CallCtlConnDisconnectedEv/ConnDisconnectedEv is delivered for all c[i]
16. CallCtlTermConnDroppedEv/TermConnDroppedEv is delivered for all tc[i]
17. ConnCreatedEv is delivered for all new(c)
18. TermConnCreatedEv is delivered for all new(tc)
19. Appropriate events are delivered for all new(c) and new(tc)

Parameters:

otherCall - The second Call which to merge with this Call object.

Throws:

InvalidArgumentException - The Call object provided is not valid for the conference
InvalidStateException - Either the Provider is not "in service", the Call is not "active", or the conference controllers are not in the proper state.
MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

See Also:

ConnCreatedEv, TermConnCreatedEv, ConnDisconnectedEv,
TermConnDroppedEv, CallInvalidEv, CallCtlConnDisconnectedEv,
CallCtlTermConnDroppedEv

transfer

```
public void transfer(Call otherCall)
    throws InvalidStateException,
           InvalidArgumentException,
           InvalidPartyException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

This method moves all participants from one Call to another, with the exception of a selected common participant. This method takes a Call as an argument, referred to hereafter as the "second" Call. All of the participants, with the exception for the selected common participant, from the second call are moved to the Call on which this method is invoked.

The Transfer Controller

In order for the transfer feature to happen, there must be a participant which acts as the transfer controller. The transfer controller is a TerminalConnection around which the transfer is placed. This transfer controller must be present on each of the two Calls and share a common Terminal. The transfer controller participant is no longer part of any Call once this transfer feature is complete. The transfer controllers on each of the two Calls must be in either of the `CallControlTerminalConnection.TALKING` or `CallControlTerminalConnection.HELD` state.

Applications may control which TerminalConnection acts as the transfer controller via the `CallControlCall.setTransferController()` method. The `CallControlCall.getTransferController()` method returns the current transfer controller, null if there is none. If no transfer controller is set, the implementation chooses a suitable TerminalConnection when the transfer feature is invoked.

The Telephone Call Argument

All of the participants from the second Call, passed as the argument to this method, are "moved" to the Call on which this method is invoked, with the exception of the transfer controller. That is, new Connections and TerminalConnections are created on this Call which are found on the second Call. Those Connections and TerminalConnections on the second Call are removed from the Call and the Call moves into the `Call.INVALID` state.

The transfer controller TerminalConnections are dropped from both Calls. They move into the `CallControlTerminalConnection.DROPPED` state.

Other Shared Participants

There may exist Address and Terminals which are part of both Calls in addition to the designated transfer controller. In these instances, those participants which are shared between both Calls are merged into one. That is, the Connections and TerminalConnections on this Call are left unchanged. The corresponding Connections and TerminalConnections on the second Call are removed from that Call.

Pre-conditions:

1. Let tc1 be the transfer controller on this Call
2. Let tc2 be the transfer controller on otherCall
3. `this != otherCall`
4. `(this.getProvider()).getState() == Provider.IN_SERVICE`
5. `this.getState() == Call.ACTIVE`
6. `otherCall.getState() == Call.ACTIVE`
7. `tc1.getCallControlState() == CallControlTerminalConnection.TALKING` or `CallControlTerminalConnection.HELD`
8. `tc2.getCallControlState() == CallControlTerminalConnection.TALKING` or `CallControlTerminalConnection.HELD`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.ACTIVE`
3. `otherCall.getState() == Call.INVALID`
4. `tc1.getCallControlState() == CallControlTerminalConnection.DROPPED`
5. `tc2.getCallControlState() == CallControlTerminalConnection.DROPPED`
6. Let `c[]` be the Connections to be transferred from otherCall
7. Let `tc[]` be the TerminalConnections to be transferred from otherCall
8. Let `new(c)` be the set of new Connections created on this Call
9. Let `new(tc)` be the set of new TerminalConnections created on this Call
10. `new(c)` element of `this.getConnections()`
11. `new(c).getCallState() == c.getCallState()`
12. `new(tc)` element of `(this.getConnections()).getTerminalConnections()`
13. `new(tc).getCallState() == tc.getCallState()`
14. `c[i].getCallControlState() == CallControlConnection.DISCONNECTED` for all `i`
15. `tc[i].getCallControlState() == CallControlTerminalConnection.DROPPED` for all `i`
16. `CallInvalidEv` is delivered for otherCall
17. `CallCtlTermConnDroppedEv/TermConnDroppedEv` is delivered for tc1, tc2
18. `CallCtlConnDisconnectedEv/ConnDisconnectedEv` is delivered for all `c[i]`
19. `CallCtlTermConnDroppedEv/TermConnDroppedEv` is delivered for all `tc[i]`
20. `ConnCreatedEv` is delivered for all `new(c)`
21. `TermConnCreatedEv` is delivered for all `new(tc)`
22. Appropriate events are delivered for all `new(c)` and `new(tc)`

Parameters:

`otherCall` - The other Call which to transfer to this Call.

Throws:

[InvalidArgumentException](#) - The TerminalConnection controlling the transfer is not valid or does not exist or the other Call is not valid.

[InvalidStateException](#) - Either the Provider is not "in service", the Call are not "active", or the transfer controllers are not "talking" or "held".

[InvalidPartyException](#) - The other Call given as the argument is not a valid Call to conference with.

[MethodNotSupportedException](#) - This method is not supported by the implementation.

[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method.

[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

See Also:

[ConnCreatedEv](#), [TermConnCreatedEv](#), [ConnDisconnectedEv](#),
[TermConnDroppedEv](#), [CallInvalidEv](#), [CallCtlConnDisconnectedEv](#),
[CallCtlTermConnDroppedEv](#)

transfer

```
public Connection transfer(java.lang.String address)
    throws InvalidArgumentException,
           InvalidStateException,
           InvalidPartyException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

This overloaded version of this method transfers all participants currently on this Call, with the exception of the transfer controller participant, to another telephone address. This is often called a "single-step transfer" because the transfer feature places another telephone call and performs the transfer at one time. The telephone address string given as the argument to this method must be valid and complete.

The Transfer Controller

The transfer controller for this version of this method represents the participant on this Call around which the transfer is taking place and who drops off the Call once the transfer feature has completed. The transfer controller is a TerminalConnection which must be in the `CallControlTerminalConnection.TALKING` state.

Applications may control which TerminalConnection acts as the transfer controller via the `CallControlCall.setTransferController()` method. The `CallControlCall.getTransferController()` method returns the current transfer controller, null if there is none. If no transfer controller is set, the implementation chooses a suitable TerminalConnection when the transfer feature is invoked.

When the transfer feature is invoked, the transfer controller moves into the `CallControlTerminalConnection.DROPPED` state. If it is the only `TerminalConnection` associated with its `Connection`, then its `Connection` moves into the `CallControlConnection.DISCONNECTED` state as well.

The New Connection

This method creates and returns a new `Connection` representing the party to which the `Call` was transferred. Note that this `Connections` may be `null` in the case the `Call` has been transferred outside of the `Provider`'s domain and can no longer be tracked. This `Connection` must at least be in the `CallControlConnection.IDLE` state. Its state may have progressed beyond "idle" before this method returns, and should be reflected by an event. This new `Connection` will progress as any normal destination `Connection` on a telephone call. Typical scenarios for this `Connection` are described by the `Call.connect()` method.

Pre-conditions:

1. Let `tc` be the transfer controller on this `Call`
2. `(this.getProvider()).getState() == Provider.IN_SERVICE`
3. `this.getState() == Call.ACTIVE`
4. `tc.getCallControlState() == CallControlTerminalConnection.TALKING`

Post-conditions:

1. Let `newconnection` be the `Connection` created and returned
2. Let `connection == tc.getConnection()`
3. `(this.getProvider()).getState() == Provider.IN_SERVICE`
4. `this.getState() == Call.ACTIVE`
5. `tc.getCallControlState() == CallControlTerminalConnection.DROPPED`
6. If `connection.getTerminalConnections().length == 1`, then `connection.getCallControlState() == CallControlConnection.DISCONNECTED`
7. `newconnection` is an element of `this.getConnections()`, if not null.
8. `newconnection.getCallControlState()` at least `CallControlConnection.IDLE`, if not null.
9. `ConnCreatedEv` is delivered for `newconnection`
10. `CallCtlTermConnDroppedEv/TermConnDroppedEv` is delivered for `tc`
11. `CallCtlConnDisconnectedEv/ConnDisconnectedEv` is delivered for `connection` if no other `TerminalConnections`

Parameters:

`address` - The destination telephone address string to where the `Call` is being transferred.

Returns:

The new `Connection` associated with the destination or null.

Throws:

`InvalidArgumentException` - The `TerminalConnection` provided as controlling the transfer is not valid or part of this `Call`.

`InvalidStateException` - Either the `Provider` is not "in service", the `Call` is not "active", or the transfer controller is not "talking".

`InvalidPartyException` - The destination address is not valid and/or complete.

`MethodNotSupportedException` - This method is not supported by the implementation.

`PrivilegeViolationException` - The application does not have the proper authority to invoke this

method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

See Also:

ConnCreatedEv, ConnDisconnectedEv, TermConnDroppedEv,
CallCtlConnDisconnectedEv, CallCtlTermConnDroppedEv

setConferenceController

```
public void setConferenceController(TerminalConnection tc)
                                   throws InvalidArgumentException,
                                           InvalidStateException,
                                           MethodNotSupportedException,
                                           ResourceUnavailableException
```

Sets the TerminalConnection which acts as the conference controller for the Call. The conference controller represents the participant in the Call around which a conference takes place.

Typically, when two Calls are conferenced together, a single participant is part of both Calls. This participant is represented by a TerminalConnection on each Call, each of which shares the same Terminal.

If the designated TerminalConnection is not part of this Call, an exception is thrown. If the TerminalConnection leaves the Call in the future, the implementation resets the conference controller to null.

Pre-conditions:

1. Let connections[] = this.getConnections()
2. (this.getProvider()).getState() == Provider.IN_SERVICE
3. this.getState() == Call.ACTIVE
4. tc element of connections[i].getTerminalConnections() for all i
5. tc.getCallControlState() != CallControlTerminalConnection.DROPPED

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getState() == Call.ACTIVE
3. Let connections[] = this.getConnections()
4. tc element of connections[i].getTerminalConnections() for all i
5. tc.getCallControlState() != CallControlTerminalConnection.DROPPED
6. tc == this.getConferenceController()

Parameters:

tc - The TerminalConnection to use as the conference controller

Throws:

InvalidArgumentException - The TerminalConnection provided is not associated with this Call.

InvalidStateException - Either the Provider is not "in service" or the Call is not "active".

MethodNotSupportedException - This method is not supported by the implementation.

ResourceUnavailableException - An internal resource necessary for the successful invocation of

this method is not available.

getConferenceController

```
public TerminalConnection getConferenceController()
```

Returns the `TerminalConnection` which currently acts as the conference controller. The conference controller represents the participant in the telephone around which a conference takes place.

When a Call is initially created, the conference controller is set to `null`. This method returns non-null only if the application has previously set the conference controller. If the current conference controller leaves the Call, the conference controller is reset to `null`.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() != Call.INVALID`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() != Call.INVALID`
3. Let `tc = this.getConferenceController()`
4. Let `connections[] = this.getConnections()`
5. `tc` element of `connections[i].getTerminalConnections()` for all `i`, if `tc` is not null
6. `tc.getCallControlState() != CallControlTerminalConnection.DROPPED`, if `tc` is not null

Returns:

The current TerminalConnection acting as the conference controller, null if one is not set.

setTransferController

[illegible]

Sets the TerminalConnection which acts as the transfer controller for the Call. The transfer controller represents the participant in the Call around which a transfer takes place.

If the designated `TerminalConnection` is not part of this `Call`, an exception is thrown. If the `TerminalConnection` leaves the `Call` in the future, the implementation resets the transfer controller to `null`.

Pre-conditions:

1. Let connections[] = this.getConnections()
2. (this.getProvider()).getState() == Provider.IN_SERVICE
3. this.getState() == Call.ACTIVE

4. tc element of connections[i].getTerminalConnections() for all i
5. tc.getCallControlState() != CallControlTerminalConnection.DROPPED

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getState() == Call.ACTIVE
3. Let connections[] = this.getConnections()
4. tc element of connections[i].getTerminalConnections() for all i
5. tc.getCallControlState() != CallControlTerminalConnection.DROPPED
6. tc == this.getTransferController()

Parameters:

tc - The TerminalConnection to use as the transfer controller

Throws:

InvalidArgumentException - The TerminalConnection provided is not associated with this Call.

InvalidStateException - Either the Provider is not "in service", Call was not "active", or the TerminalConnection argument was "dropped".

MethodNotSupportedException - This method is not supported by the implementation.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

getTransferController

public TerminalConnection **getTransferController()**

Returns the TerminalConnection which currently acts as the transfer controller. The transfer controller represents the participant in the Call around which a transfer takes place.

When a Call is initially created, the transfer controller is set to `null`. This method returns non-null only if the application has previously set the transfer controller. If the current transfer controller leaves the telephone call, the transfer controller is reset to `null`.

Pre-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getState() != Call.INVALID

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getState() != Call.INVALID
3. Let tc = this.getTransferController()
4. Let connections[] = this.getConnections()
5. tc element of connections[i].getTerminalConnections() for all i, if tc is not null
6. tc.getCallControlState() != CallControlTerminalConnection.DROPPED, if tc is not null

Returns:

The current TerminalConnection acting as the transfer controller, null if one is not set.

setConferenceEnable

```
public void setConferenceEnable(boolean enable)
    throws InvalidArgumentException,
           InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException
```

Controls whether the Call is permitted or able to perform the conferencing feature. The boolean argument provided indicates whether conferencing should be turned on (true) or off (false). This method throws an exception if the boolean argument is true and the implementation does not support the conferencing feature. This method must be invoked when the Call is in the `Call.IDLE` state.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`
3. `enable == this.getConferenceEnable()`

Parameters:

`enable` - True turns conferencing on, false turns conferencing off.

Throws:

InvalidArgumentException - Conferencing cannot be turned on as requested by a true argument.

InvalidStateException - Either the Provider is not "in service" or the Call is not "idle".

MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

getConferenceEnable

```
public boolean getConferenceEnable()
```

Return true if conferencing is enabled, false otherwise. Applications may use this method initially to obtain the default value set by the implementation and may attempt to change this value using the `CallControlCall.setConferenceEnable()` method.

Returns:

True if conferencing is enabled, false otherwise.

setTransferEnable

```
public void setTransferEnable(boolean enable)
    throws InvalidArgumentException,
           InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException
```

Controls whether the Call is permitted or able to perform the transferring feature. The boolean argument provided indicates whether transferring should be turned on (true) or off (false). This method throws an exception if the boolean argument is true and the implementation does not support the transferring feature. This method must be invoked when the Call is in the `Call.IDLE` state.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`
3. `enable == this.getConferenceEnable()`

Parameters:

`enable` - True turns transferring on, false turns transferring off.

Throws:

InvalidArgumentException - Transferring cannot be turned on as requested by the true argument.

InvalidStateException - Either the Provider is not "in service" or the Call is not "idle".

MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

getTransferEnable

```
public boolean getTransferEnable()
```

Return true if transferring is enabled, false otherwise. Applications may use this method initially to obtain the default value set by the implementation and may attempt to change this value using the `CallControlCall.setTransferEnable()` method.

Returns:

True if transferring is enabled, false otherwise.

consult

```
public Connection[] consult(TerminalConnection tc,
                             java.lang.String dialedDigits)
    throws InvalidStateException,
           InvalidArgumentException,
           MethodNotSupportedException,
           ResourceUnavailableException,
           PrivilegeViolationException,
           InvalidPartyException
```

Creates a consultation between this Call and an active Call. A consultation Call is a new, idle Call which is associated with a particular existing Call and often created for a particular purpose. For example, the consultation Call may be used simply to "consult" with another party or to conference or transfer with its associated Call. This method establishes a special relationship between the two Calls which extends down to the telephony platform level. Most often, this feature is directly provided by the underlying telephony platform.

This Call Object

The instance on which this method is invoked is used as the Call on which the consultation takes place. This Call must be in `Call.IDLE` and is created with the `Provider.createCall()` method.

The TerminalConnection Argument

The `TerminalConnection` argument provides two pieces of information. The first piece of information is the other active Call to which this idle Call is associated. The Call associated with the `TerminalConnection` argument must be in the `Call.ACTIVE` state.

The second piece of information given by the `TerminalConnection` argument is the originating endpoint from which to place a telephone call on this idle Call. That is, the Address and Terminal associated with the `TerminalConnection` argument are used as the originating endpoint for the telephone call. The state of the `TerminalConnection` must be `CallControlTerminalConnection.TALKING` and this method first moves it into the `CallControlTerminalConnection.HELD` in order to place a telephone call on this idle Call.

The Destination Address String

A telephone call is placed to the destination telephone address string given as the second argument to this method. This address string must be valid and complete.

The Consultation Purpose

As mentioned above, the purpose of creating a consultation Call is often to perform a transfer or conference action on these two Calls. If the methods `CallControlCall.setConferenceEnable()` and `CallControlCall.setTransferEnable()` are supported as indicated by the `CallControlCallCapabilities` interface, applications must specify the purpose of the consultation Call by first telling the telephony platform if they intend to perform a transfer and/or conference action.

If the `CallControlCall.setConferenceEnable()` and the `CallControlCall.setTransferEnable()` methods are not supported as indicated by this interface's capabilities, applications permit the telephony platform to use the static, default values reported by the `CallControlCall.getConferenceEnable()` and the `CallControlCall.getTransferEnable()` methods.

The Telephone Call

A telephone call is placed on this `Call` and an array of two `Connections` are returned representing the originating and destination participants of the `Call`. The `Call` progresses in the same way as if the `Call` was placed using the `Call.connect()` method. The description of that method describes different scenarios under which the state of the call progresses.

Pre-conditions:

1. `this.getProvider().getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`
3. `tc.getCallControlState() == CallControlTerminalConnection.TALKING`
4. `(tc.getCall()).getState() == Call.ACTIVE`

Post-conditions:

1. Let `connections[]` be the two `Connections` created and returned
2. `this.getProvider().getState() == Provider.IN_SERVICE`
3. `this.getState() == Call.ACTIVE`
4. `tc.getCallControlState() == CallControlTerminalConnection.HELD`
5. `(tc.getCall()).getState() == Call.ACTIVE`
6. Let `c[] = this.getConnections()` such that `c.length == 2`
7. `c[0].getCallControlState()` at least `CallControlConnection.IDLE`
8. `c[1].getCallControlState()` at least `CallControlConnection.IDLE`
9. `c == connection`
10. `CallActiveEv` is delivered for this `Call`
11. `ConnCreatedEv` are delivered for both `connections[i]`
12. `CallCtlTermConnHeldEv` is delivered for `tc`

Parameters:

`tc` - The controlling `TerminalConnection` for the consultation call.

`dialedDigits` - The destination telephone address string to which a telephone call is being placed.

Returns:

The two new `Connections` in the `Call`.

Throws:

`ResourceUnavailableException` - An internal resource necessary for the successful invocation of this method is unavailable.

`PrivilegeViolationException` - The application does not have the proper authority to place a consultation telephone call.

`InvalidArgumentException` - The `TerminalConnection` given is not a valid originating endpoint for a `Call`.

`InvalidPartyException` - The destination address string is not valid and/or complete.

`InvalidStateException` - Either the `Provider` is not "in service", the `Call` is not "idle", the other

Call is not "active", or the TerminalConnection is not "talking".

MethodNotSupportedException - The implementation does not support this method.

See Also:

CallActiveEv, ConnCreatedEv, CallCtlTermConnHeldEv

consult

```
public Connection consult(TerminalConnection tc)
    throws InvalidStateException,
           InvalidArgumentException,
           MethodNotSupportedException,
           ResourceUnavailableException,
           PrivilegeViolationException
```

Creates a consultation between this Call and an active Call. A consultation call is a new, idle Call which is associated with a particular existing Call and often created for a particular purpose. For example, the consultation call may be used simply to "consult" with another party or to conference or transfer with its associated Call. This method establishes a special relationship between the two Calls which extends down to the telephony platform level. Most often, this feature is directly provided by the underlying telephony platform.

This overloaded version of this method has a single difference with the other version of the `CallControlCall.consult()` method. This method does not take a destination telephone address string as an argument.

This method creates and returns a single Connection which is in the `CallControlConnection.INITIATED` state. Applications may use the `CallControlConnection.addToAddress()` method to dial the destination address digits.

Pre-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getState() == Call.IDLE`
3. `tc.getCallControlState() == CallControlTerminalConnection.TALKING`
4. `(tc.getCall()).getState() == Call.ACTIVE`

Post-conditions:

1. Let connection be the connection created and returned
2. `(this.getProvider()).getState() == Provider.IN_SERVICE`
3. `this.getState() == Call.IDLE`
4. `tc.getCallControlState() == CallControlTerminalConnection.HELD`
5. `(tc.getCall()).getState() == Call.ACTIVE`
6. `connection == c`
7. Let `c = this.getConnections()` such that `c.length == 1`
8. `c[0].getCallControlState() == CallControlConnection.INITIATED`
9. `ConnCreatedEv` is delivered for connection
10. `CallCtlConnInitiatedEv/ConnConnectedEv` is delivered for connection
11. `CallActiveEv` is delivered for this Call

Parameters:

tc - The controlling TerminalConnection for the consultation call.

Returns:

The Connection in the "initiated" state.

Throws:

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is unavailable.

PrivilegeViolationException - The application does not have the proper authority to place a consultation telephone call.

InvalidArgumentException - The TerminalConnection given is not a valid originating endpoint for a Call.

InvalidStateException - Either the Provider is not "in service", the Call is not "idle", the other Call is not "active", or the TerminalConnection is not "talking".

MethodNotSupportedException - The implementation does not support this method.

See Also:

CallActiveEv, ConnCreatedEv, ConnConnectedEv, CallCtlConnInitiatedEv

javax.telephony.callcontrol

Interface CallControlCallObserver

public interface **CallControlCallObserver**
 extends [CallObserver](#)

The `CallControlCallObserver` interface reports all events for the `CallControlCall` interface. It also reports events for the `CallControlConnection` and the `CallControlTerminalConnection` interfaces. Applications implement this interface to receive these events. All events are reported via the `CallObserver.callChangedEvent()` method. This interface, therefore, allows applications to signal to the implementation that they are interested in call control package events. This interface defines no additional methods.

All events must extend the `CallCtlCalEv` event interface, which in turn, extends the core `CalEv` interface.

The `CallControlConnection` state events defined in this package are:

`CallCtlConnOfferedEv`, `CallCtlConnQueuedEv`, `CallCtlConnAlertingEv`,
`CallCtlConnInitiatedEv`, `CallCtlConnDialingEv`, `CallCtlConnNetworkReachedEv`,
`CallCtlConnNetworkAlertingEv`, `CallCtlConnFailedEv`,
`CallCtlConnEstablishedEv`, `CallCtlConnUnknownEv`, and
`CallCtlConnDisconnectedEv`.

The `CallControlTerminalConnection` state events defined in this package are:

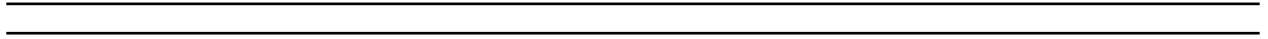
`CallCtlTermConnBridgedEv`, `CallCtlTermConnDroppedEv`, `CallCtlTermConnHeldEv`,
`CallCtlTermConnInUseEv`, `CallCtlTermConnRingingEv`,
`CallCtlTermConnTalkingEv`, and `CallCtlTermConnUnknownEv`.

See Also:

[CallObserver](#), [CalEv](#), [Connection](#), [TerminalConnection](#), [CallCtlCalEv](#),
[CallCtlConnEv](#), [CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#),
[CallCtlConnDisconnectedEv](#), [CallCtlConnEstablishedEv](#),
[CallCtlConnFailedEv](#), [CallCtlConnInitiatedEv](#),
[CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#),
[CallCtlConnOfferedEv](#), [CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#),
[CallCtlTermConnEv](#), [CallCtlTermConnRingingEv](#), [CallCtlTermConnTalkingEv](#),
[CallCtlTermConnHeldEv](#), [CallCtlTermConnBridgedEv](#),
[CallCtlTermConnInUseEv](#), [CallCtlTermConnDroppedEv](#),
[CallCtlTermConnUnknownEv](#)

Methods inherited from interface javax.telephony.CallObserver

callChangedEvent



javax.telephony.callcontrol **Interface CallControlConnection**

public interface **CallControlConnection**
extends Connection

Introduction

The `CallControlConnection` interface extends the core `Connection` interface and provides additional features and greater detail about the `Connection` state. Applications may query a `Connection` object using the `instanceof` operator to see whether it supports this interface.

CallControlConnection State

This interface defines states for the `Connection` which provide greater detail beyond the states defined in the `Connection` interface. The states defined by this interface are related to the states defined in the core package in certain, specific ways, as defined below. Applications may obtain the `CallControlConnection` state via the `getCallControlState()` method defined on this interface. This method returns one of the integer state constants defined in this interface.

Below is a description of each `CallControlConnection` state in real-world terms. These real-world descriptions only serve to provide a more intuitive understanding of what is going on. Several methods in this specification state pre-conditions based upon the `CallControlConnection` state. Some of these states are identical to those defined in the core package.

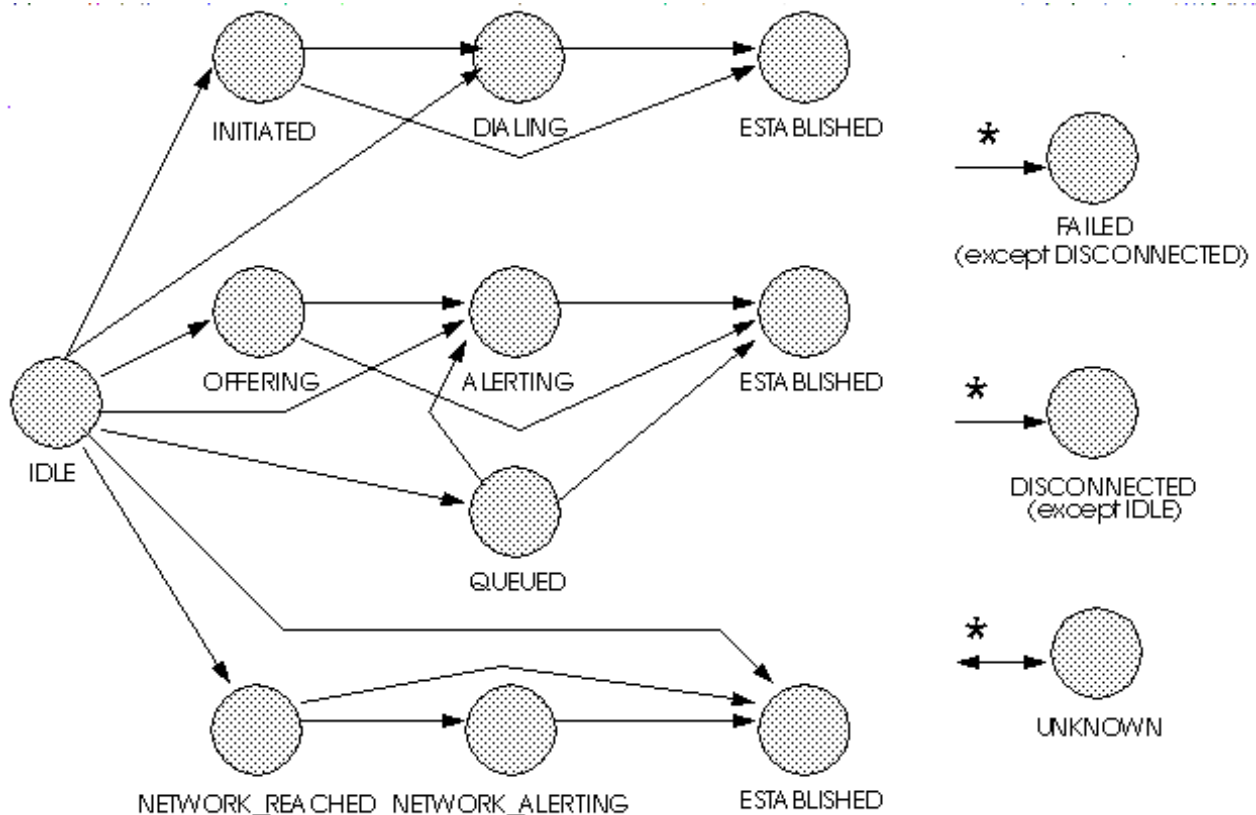
<code>CallControlConnection.IDLE</code>	This state has the same definition as in the core package. It is the initial <code>CallControlConnection</code> state for all new <code>Connections</code> . <code>Connections</code> typically do not stay in this state for long, but quickly transition to another state.
<code>CallControlConnection.OFFERED</code>	This state indicates that an incoming call is being offered to the <code>Address</code> associated with the <code>Connection</code> . Typically, applications must either accept or reject this offered call before the called party is alerted to the incoming telephone call.
<code>CallControlConnection.QUEUED</code>	This state indicates that a <code>Connection</code> is queued at the particular <code>Address</code> associated with the <code>Connection</code> . For example, some telephony platforms permit the "queueing" of incoming telephone call to an <code>Address</code> when the <code>Address</code> is busy.

<code>CallControlConnection.NETWORK_REACHED</code>	This state indicates that an outgoing telephone call has reached the network. Applications may not receive further events about this leg of the telephone call, depending upon the ability of the telephone network to provide additional progress information. Applications must decide whether to treat this as a connected telephone call.
<code>CallControlConnection.NETWORK_ALERTING</code>	This state indicates that an outgoing telephone call is alerting at the destination end, which was previously only known to have reached the network. Typically, Connections transition into this state from the <code>CallControlConnection.NETWORK_REACHED</code> state. This state results from additional progress information being sent from the telephone network.
<code>CallControlConnection.ALERTING</code>	This state has the same definition as in the core package. It implies that the Address is being notified of an incoming call.
<code>CallControlConnection.INITIATED</code>	This state indicates the originating end of a telephone call has begun the process of placing a telephone call, but the dialing of the destination telephone address has not yet begun. Typically, a telephone associated with the Address has gone "off-hook".
<code>CallControlConnection.DIALING</code>	This state indicates the originating end of a telephone call has begun the process of dialing a destination telephone address, but has not yet completed dialing.
<code>CallControlConnection.ESTABLISHED</code>	This state is similar to that of <code>Connection.CONNECTED</code>. It indicates that the endpoint has reached its final, active state in the telephone call.
<code>CallControlConnection.DISCONNECTED</code>	This state has the same definition as in the core package. It implies the Connection object is no longer part of the telephone call.
<code>CallControlConnection.FAILED</code>	This state has the same definition as in the core package. It indicates that a particular leg of a telephone call has failed for some reason, perhaps because the party was busy.
<code>CallControlConnection.UNKNOWN</code>	This state has the same definition as in the core package. It indicates the implementation is unable to determine the current call control package state of the Connection object. Typically, methods are invalid on this object when it is in this state.

State Transitions

Similar to the `Connection` state transition diagram, the `CallControlConnection` state must transition according to rules illustrated in the finite state diagram below. An implementation must guarantee that `CallControlConnection` state must abide by this transition diagram.

Note there is a general left-to-right progression of the state transitions. The asterisk next to a state transition, as in the core package, implies a transition to/from another state, except where noted.



Core vs. CallControl Package States

There is a strong relationship between the `CallControlConnection` states and the `Connection` states. If an implementation supports the call control package, it must ensure this relationship is properly maintained.

Since the states defined in the `CallControlConnection` interface provide more detail to the states defined in the `Connection` interface, each state in the `Connection` interface corresponds to a state defined in the `CallControlConnection` interface. Conversely, each `CallControlConnection` state corresponds to exactly one `Connection` state. This arrangement permits applications to view either the core state or the call control state and still see a consistent view.

The following table outlines the relationship between the core package Connection states and the call control package Connection states.

<i>If the call control package state is...</i>	<i>then the core package state must be...</i>
CallControlConnection.IDLE	Connection.IDLE
CallControlConnection.QUEUED	Connection.INPROGRESS
CallControlConnection.OFFERED	Connection.INPROGRESS
CallControlConnection.ALERTING	Connection.ALERTING
CallControlConnection.INITIATED	Connection.CONNECTED
CallControlConnection.DIALING	Connection.CONNECTED
CallControlConnection.NETWORK_REACHED	Connection.CONNECTED
CallControlConnection.NETWORK_ALERTING	Connection.CONNECTED
CallControlConnection.ESTABLISHED	Connection.CONNECTED
CallControlConnection.DISCONNECTED	Connection.DISCONNECTED
CallControlConnection.FAILED	Connection.FAILED
CallControlConnection.UNKNOWN	Connection.UNKNOWN

Observers and Events

All events pertaining to the CallControlConnection interface are reported via the CallObserver.callChangedEvent() method. The application observer object must also implement the CallControlCallObserver interface to express interest in the call control package events.

Observers which are registered on a Call receive events when the CallControlConnection state changes. Note that when the CallControlConnection state changes, it sometimes results in the Connection state changing (according to the table above). In these instances, both the proper call control and core package events are delivered to the observer.

The CallControlConnection state events defined in this package are:

CallCtlConnOfferedEv, CallCtlConnQueuedEv, CallCtlConnAlertingEv, CallCtlConnInitiatedEv, CallCtlConnDialingEv, CallCtlConnNetworkReachedEv, CallCtlConnNetworkAlertingEv, CallCtlConnFailedEv, CallCtlConnEstablishedEv, CallCtlConnUnknownEv, and CallCtlConnDisconnectedEv.

See Also:

[Connection](#), [CallObserver](#), [CallControlCallObserver](#), [CallCtlCalEv](#), [CallCtlConnEv](#), [CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#), [CallCtlConnDisconnectedEv](#), [CallCtlConnEstablishedEv](#), [CallCtlConnFailedEv](#), [CallCtlConnInitiatedEv](#), [CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#), [CallCtlConnOfferedEv](#), [CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#)

Field Summary

static int	<u>ALERTING</u> The <code>CallControlConnection.ALERTING</code> state has the same definition as in the core package.
static int	<u>DIALING</u> The <code>CallControlConnection.DIALING</code> state indicates the originating end of a telephone call has begun the process of dialing a destination telephone address, but has not yet completed.
static int	<u>DISCONNECTED</u> The <code>CallControlConnection.DISCONNECTED</code> state has the same definition as in the core package.
static int	<u>ESTABLISHED</u> The <code>CallControlConnection.ESTABLISHED</code> state is similar to that of <code>Connection.CONNECTED</code> .
static int	<u>FAILED</u> The <code>CallControlConnection.FAILED</code> state has the same definition as in the core package.
static int	<u>IDLE</u> The <code>CallControlConnection.IDLE</code> state has the same definition as in the core package.
static int	<u>INITIATED</u> The <code>CallControlConnection.INITIATED</code> state indicates the originating end of a telephone call has begun the process of placing a telephone call, but the dialing of the destination telephone address has not yet begun.
static int	<u>NETWORK_ALERTING</u> The <code>CallControlConnection.NETWORK_ALERTING</code> state indicates that an outgoing telephone call is alerting at the destination end, which was previously only known to have reached the network.
static int	<u>NETWORK_REACHED</u> The <code>CallControlConnection.NETWORK_REACHED</code> state indicates that an outgoing telephone call has reached the network.
static int	<u>OFFERED</u> The <code>CallControlConnection.OFFERED</code> state indicates than an incoming call is being offered to the Address associated with the Connection.
static int	<u>OFFERING</u> Deprecated. Since JTAPI v1.2.

static int	<u>QUEUED</u> The <code>CallControlConnection.QUEUED</code> state indicates that a <code>Connection</code> is queued at the particular <code>Address</code> associated with the <code>Connection</code> .
static int	<u>UNKNOWN</u> The <code>CallControlConnection.UNKNOWN</code> state has the same definition as in the core package.

Fields inherited from interface javax.telephony.Connection

ALERTING, CONNECTED, DISCONNECTED, FAILED, IDLE, INPROGRESS, UNKNOWN

Method Summary

void	<u>accept()</u> Accepts a telephone call incoming to an <code>Address</code> .
void	<u>addToAddress</u> (java.lang.String additionalAddress) Appends additional address information onto an existing <code>Connection</code> .
int	<u>getCallControlState()</u> Returns the current call control state of the <code>Connection</code> .
<u>Connection</u>	<u>park</u> (java.lang.String destinationAddress) Parks a <code>Connection</code> at a destination telephone address.
<u>Connection</u>	<u>redirect</u> (java.lang.String destinationAddress) Redirects an incoming telephone call at an <code>Address</code> to another telephone address.
void	<u>reject()</u> Rejects a telephone call incoming to an <code>Address</code> .

Methods inherited from interface javax.telephony.Connection

disconnect, getAddress, getCall, getCapabilities,
getConnectionCapabilities, getState, getTerminalConnections

Field Detail

IDLE

```
public static final int IDLE
```

The `CallControlConnection.IDLE` state has the same definition as in the core package. It is the initial `CallControlConnection` state for all new `Connections`. `Connections` typically do not stay in this state for long, quickly transitioning to another state.

OFFERED

```
public static final int OFFERED
```

The `CallControlConnection.OFFERED` state indicates than an incoming call is being offered to the `Address` associated with the `Connection`. Typically, applications must either accept or reject this offered call before the called party is alerted to the incoming telephone call.

Since:
JTAPI v1.2

QUEUED

```
public static final int QUEUED
```

The `CallControlConnection.QUEUED` state indicates that a `Connection` is queued at the particular `Address` associated with the `Connection`. A queued call is not active on a call. For example, some telephony platforms permit the "queueing" of incoming telephone call to an `Address` when the `Address` is busy.

ALERTING

```
public static final int ALERTING
```

The `CallControlConnection.ALERTING` state has the same definition as in the core package. It means that the `Address` is being notified of an incoming call.

INITIATED

```
public static final int INITIATED
```

The `CallControlConnection.INITIATED` state indicates the originating end of a telephone call has begun the process of placing a telephone call, but the dialing of the destination telephone address has not yet begun. Typically, a telephone associated with the `Address` has gone "off-hook".

DIALING

public static final int **DIALING**

The `CallControlConnection.DIALING` state indicates the originating end of a telephone call has begun the process of dialing a destination telephone address, but has not yet completed.

NETWORK_REACHED

public static final int **NETWORK_REACHED**

The `CallControlConnection.NETWORK_REACHED` state indicates that an outgoing telephone call has reached the network. Applications may not receive further events about this leg of the telephone call, depending upon the ability of the telephone network to provide additional progress information. Applications must decide whether to treat this as a connected telephone call.

NETWORK_ALERTING

public static final int **NETWORK_ALERTING**

The `CallControlConnection.NETWORK_ALERTING` state indicates that an outgoing telephone call is alerting at the destination end, which was previously only known to have reached the network. Typically, Connections transition into this state from the `CallControlConnection.NETWORK_REACHED` state. This state results from additional progress information being sent from a telephone network that was capable of transmitting that information.

ESTABLISHED

public static final int **ESTABLISHED**

The `CallControlConnection.ESTABLISHED` state is similar to that of `Connection.CONNECTED`. It indicates that the endpoint has reached its final, active state in the telephone call.

DISCONNECTED

public static final int **DISCONNECTED**

The `CallControlConnection.DISCONNECTED` state has the same definition as in the core package. It indicates that the `Connection` object is no longer part of the telephone call.

FAILED

public static final int **FAILED**

The `CallControlConnection.FAILED` state has the same definition as in the core package. It indicates that a `Connection` is no longer able to participate in a call. It is a final state in the life of a `Connection`. It indicates that a particular leg of a telephone call has failed for some reason, perhaps because the party was busy.

UNKNOWN

public static final int **UNKNOWN**

The `CallControlConnection.UNKNOWN` state has the same definition as in the core package. It indicates that the state of the `Connection` is not known to its Provider. Typically, methods are invalid on this object when it is in this state.

OFFERING

public static final int **OFFERING**

Deprecated. *Since JTAPI v1.2.*

The `CallControlConnection.OFFERING` state has been deprecated in JTAPI v1.2. It has the same meaning as the new `CallControlConnection.OFFERED` state. This constant has been replaced to be more tense-consistent with the event name.

Method Detail

getCallControlState

public int **getCallControlState()**

Returns the current call control state of the `Connection`. The return value will be one of integer state constants defined above.

Returns:

The current call control state of the `Connection`.

accept

```
public void accept()
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Accepts a telephone call incoming to an Address. Telephone calls into an Address may first be *offered* to that address for acceptance before the standard notion of "alerting" takes place. This method is valid on a Connection in the `CallControlConnection.OFFERED` state. If successful, this method moves the Connection into the `CallControlConnection.ALERTING` state. This method waits until the telephone call has been accepted or an error occurs and an exception is thrown.

Pre-conditions:

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlConnection.OFFERED`

Post-conditions:

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlConnection.ALERTING`
3. `CallCtlConnAlertingEv/ConnAlertingEv` is delivered for this Connection

Throws:

[InvalidStateException](#) - Either the Provider is not "in service" or the Connection is not "offered".

[MethodNotSupportedException](#) - This method is not supported by the implementation.

[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method.

[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

See Also:

[ConnAlertingEv](#), [CallCtlConnAlertingEv](#)

reject

```
public void reject()
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Rejects a telephone call incoming to an Address. Telephone calls into an Address may first be *offered* to that address for acceptance before the standard notion of "alerting" takes place. This method is valid on a Connection in the `CallControlConnection.OFFERED` state. If successful, this method moves the Connection into the `CallControlConnection.DISCONNECTED` state. This method waits until the telephone call has been rejected or an error occurs and an exception is thrown.

Pre-conditions:

1. ((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE
2. this.getCallControlState() == CallControlConnection.OFFERED

Post-conditions:

1. ((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE
2. this.getCallControlState() == CallControlConnection.DISCONNECTED
3. CallCtlConnDisconnectedEv/ConnDisconnectedEv is delivered for this Connection

Throws:

InvalidStateException - Either the Provider is not "in service" or the Connection is not "offered".

MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

See Also:

ConnDisconnectedEv, CallCtlConnDisconnectedEv

redirect

```
public Connection redirect(java.lang.String destinationAddress)
    throws InvalidStateException,
           InvalidPartyException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Redirects an incoming telephone call at an Address to another telephone address. This method is very similar to the transfer feature, however, applications may invoke this method before first answering the telephone call. This method redirects the telephone call to another telephone address string provided as the argument to this method. This telephone address string must be valid and complete.

This Connection must either be in the `CallControlConnection.OFFERED` state or the `CallControlConnection.ALERTING` state. If successful, this method moves the Connection to the `CallControlConnection.DISCONNECTED` state. Additionally, any TerminalConnections associated with this Connection will move to the `CallControlTerminalConnection.DROPPED` state.

A new Connection is created and returned corresponding to the new destination leg of the telephone call. Note that this Connection may be `null` in the case the Call has been redirected outside of the Provider's domain and can no longer be tracked. The new Connection (if not null) must at least be in the `CallControlConnection.IDLE` state. The Connection may progress beyond this state before this method returns, which should be reflected by the proper events. This Connection behaves similarly to the destination Connection as described in `Call.connect()` and undergoes similar possible state change scenarios.

Pre-conditions:

1. ((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE
2. this.getCallControlState() == CallControlConnection.OFFERED or CallControlConnection.ALERTING
3. destinationAddress must be a valid and complete telephone address

Post-conditions:

1. Let newconnection be the new Connection created and returned
2. ((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE
3. this.getCallControlState() == CallControlConnection.DISCONNECTED
4. newconnection element of (this.getCall()).getConnections()
5. newconnection.getCallControlState() at least CallControlConnection.IDLE
6. Let tc[] = this.getTerminalConnections() before this method is invoked
7. tc[i].getCallControlState() == CallControlTerminalConnection.DROPPED, for all i.
8. CallCtlConnDisconnected/ConnDisconnectedEv is delivered for this Connection
9. CallCtlTermConnDroppedEv/TermConnDroppedEv is delivered for all tc[i]
10. ConnCreatedEv is delivered for newconnection

Parameters:

destinationAddress - The Connection is redirected to this telephone address

Returns:

The Connection associated with the new leg of the Call.

Throws:

InvalidStateException - Either the Provider is not "in service" or the Connection is not "offered" or "alerting".

InvalidPartyException - The destination address to which this call is redirected is not valid and/or complete.

MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

See Also:

ConnCreatedEv, ConnDisconnectedEv, TermConnDroppedEv,
CallCtlConnDisconnectedEv, CallCtlTermConnDroppedEv

addToAddress

```
public void addToAddress(java.lang.String additionalAddress)
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Appends additional address information onto an existing Connection. This method is used when part of a telephone address string has been dialed and additional addressing information is needed in order to complete the dialing process and place the telephone call. The additional addressing information is provided as the argument to this method.

This Connection must either be in the `CallControlConnection.DIALING` state or the `CallControlConnection.INITIATED` state. If successful, this moves the Connection into one of two states. If the information provided completes the addressing information, as determined by the telephony platform, the Connection moves into the `CallControlConnection.ESTABLISHED` state and the telephone call is placed. If additional addressing information is still required, the Connection moves into the `CallControlConnection.DIALING` state if not already there.

Pre-conditions:

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlConnection.DIALING` or `CallControlConnection.INITIATED`

Post-conditions Outcome 1: The addressing information is complete.

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlConnection.ESTABLISHED`
3. `CallCtlConnEstablishedEv` is delivered for this Connection

Post-conditions Outcome 2: The addressing information is not complete.

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlConnection.DIALING`
3. `CallCtlConnDialingEv` is delivered for this Connection

Parameters:

`additionalAddress` - The additional addressing information.

Throws:

`InvalidStateException` - Either the Provider is not "in service" or the Connection is not "initiated" or "dialing".

`MethodNotSupportedException` - This method is not supported by the implementation.

`PrivilegeViolationException` - The application does not have the proper authority to invoke this method.

`ResourceUnavailableException` - An internal resource necessary for the successful invocation of this method is not available.

See Also:

`CallCtlConnDialingEv`, `CallCtlConnEstablishedEv`

park

```
public Connection park(java.lang.String destinationAddress)
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           InvalidPartyException,
           ResourceUnavailableException
```

Parks a Connection at a destination telephone address. This method is similar to the transfer feature, except the Connection at the new destination Address is in a special *queued* state. Parking a Connection at a destination Address drops the Connection from the Call and creates and returns a new Connection at the specified destination address in the `CallControlConnection.QUEUED`

state.

The new destination telephone address string is given as an argument to this method and must be a valid and complete telephone address. The `CallControlTerminal.pickup()` method permits applications to "unpark" the new Connection.

The Connection must be in the `CallControlConnection.ESTABLISHED` state. If this method is successful, this Connection moves to the `CallControlConnection.DISCONNECTED` state. All of its associated TerminalConnections move to the `CallControlTerminalConnection.DROPPED` state.

Pre-conditions:

1. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlConnection.ESTABLISHED`
3. destinationAddress must be a valid and complete telephone address.

Post-conditions:

1. Let newconnection be the new Connection created and returned
2. `((this.getCall()).getProvider()).getState() == Provider.IN_SERVICE`
3. newconnection element of `(this.getCall()).getConnections()`
4. `newconnection.getCallControlState() == CallControlConnection.QUEUED`
5. `this.getCallControlState() == CallControlConnection.DISCONNECTED`
6. Let `tc[] = this.getTerminalConnections()` before this method is invoked
7. `tc[i].getCallControlState() == CallControlTerminalConnection.DROPPED`, for all i
8. ConnCreatedEv is delivered for newconnection
9. CallCtlConnQueuedEv/ConnInProgressEv is delivered for newconnection
10. CallCtlConnDisconnected/ConnDisconnectedEv is delivered for this Connection
11. CallCtlTermConnDroppedEv/TermConnDroppedEv is delivered for all `tc[i]`

Parameters:

destinationAddress - The telephone address string at which this connection is to be parked.

Returns:

The new Connection which is parked at the new destination Address.

Throws:

[InvalidStateException](#) - Either the Provider was not "in service" or the Connection was not "established".

[MethodNotSupportedException](#) - This method is not supported by the implementation.

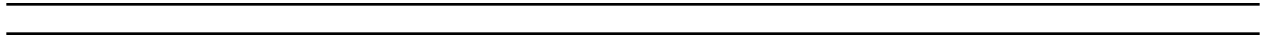
[InvalidPartyException](#) - The party to which to party the Connection is invalid.

[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method.

[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

See Also:

[ConnCreatedEv](#), [ConnInProgressEv](#), [ConnDisconnectedEv](#),
[TermConnDroppedEv](#), [CallCtlConnQueuedEv](#), [CallCtlConnDisconnectedEv](#),
[CallCtlTermConnDroppedEv](#)



javax.telephony.callcontrol **Class CallControlForwarding**

```
java.lang.Object
|
+-- javax.telephony.callcontrol.CallControlForwarding
```

```
public class CallControlForwarding
extends java.lang.Object
```

The `CallControlForwarding` class represents a *forwarding instruction*. This instruction tells how the platform should forward incoming telephone calls to a specific address. There are several attributes to a forwarding instruction.

The first attribute is its *type*. The forwarding instruction's type tells the platform when to forward the call. There are currently three types of instructions: telling the platform to always forward incoming calls, telling the platform to forward incoming calls when the address is busy, and telling the platform to forward incoming calls when no one answers.

The second attribute of a forwarding instruction is its *filter*. The filter indicates which type of incoming calls this forwarding instruction should apply to. This forwarding instruction can apply to all calls, to external calls only, to internal calls only, or to a specific calling address.

Field Summary	
static int	<u>ALL CALLS</u> Forwarding filter: apply instruction to all incoming calls.
static int	<u>EXTERNAL CALLS</u> Forwarding filter: apply instruction to calls originating from outside the provider domain.
static int	<u>FORWARD ON BUSY</u> Forwarding type: forward calls on busy.
static int	<u>FORWARD ON NOANSWER</u> Forwarding type: forward calls on no answer.
static int	<u>FORWARD UNCONDITIONALLY</u> Forwarding type: forward calls unconditionally.
static int	<u>INTERNAL CALLS</u> Forwarding filter: apply instruction to calls originating from the provider domain.
static int	<u>SPECIFIC ADDRESS</u> Forwarding filter: apply instruction to calls originating from a specific address.

Constructor Summary

CallControlForwarding(java.lang.String destAddress)

This constructor is the default constructor, which only takes the address to apply this forwarding instruction.

CallControlForwarding(java.lang.String destAddress, int type)

This constructor takes the address to apply this forwarding instruction and the type of forwarding for all incoming calls.

CallControlForwarding(java.lang.String destAddress, int type, boolean internalCalls)

This constructor takes the address to apply this forwarding instruction, the type of forwarding desired for this address, and a boolean flag indicating whether this instruction should apply to internal (true) or external (false) calls.

CallControlForwarding(java.lang.String destAddress, int type, java.lang.String caller)

This constructor takes an address to apply the forwarding instruction for a specific incoming telephone call, identified by a string address.

Method Summary

java.lang.String	<u>getDestinationAddress</u> ()	Returns the destination address of this forwarding instruction.
int	<u>getFilter</u> ()	Returns the filter of this forwarding instruction.
java.lang.String	<u>getSpecificCaller</u> ()	If the filter for this forwarding instruction is SPECIFIC_ADDRESS, then this method returns that calling address to which this filter applies.
int	<u>getType</u> ()	Returns the type of this forwarding instruction, either unconditionally, upon no answer, or upon busy.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

ALL_CALLS

```
public static final int ALL_CALLS
```

Forwarding filter: apply instruction to all incoming calls.

INTERNAL_CALLS

```
public static final int INTERNAL_CALLS
```

Forwarding filter: apply instruction to calls originating from the provider domain.

EXTERNAL_CALLS

```
public static final int EXTERNAL_CALLS
```

Forwarding filter: apply instruction to calls originating from outside the provider domain.

SPECIFIC_ADDRESS

```
public static final int SPECIFIC_ADDRESS
```

Forwarding filter: apply instruction to calls originating from a specific address.

FORWARD_UNCONDITIONALLY

```
public static final int FORWARD_UNCONDITIONALLY
```

Forwarding type: forward calls unconditionally.

FORWARD_ON_BUSY

```
public static final int FORWARD_ON_BUSY
```

Forwarding type: forward calls on busy.

FORWARD_ON_NOANSWER

```
public static final int FORWARD_ON_NOANSWER
```

Forwarding type: forward calls on no answer.

Constructor Detail

CallControlForwarding

```
public CallControlForwarding(java.lang.String destAddress)
```

This constructor is the default constructor, which only takes the address to apply this forwarding instruction. The forwarding instruction forwards all calls unconditionally.

Parameters:

`destAddress` - destination address for the call forwarding operation

CallControlForwarding

```
public CallControlForwarding(java.lang.String destAddress,  
                             int type)
```

This constructor takes the address to apply this forwarding instruction and the type of forwarding for all incoming calls.

Parameters:

`destAddress` - destination address for the call forwarding operation

`type` - the type of the forwarding operation

CallControlForwarding

```
public CallControlForwarding(java.lang.String destAddress,  
                             int type,  
                             boolean internalCalls)
```

This constructor takes the address to apply this forwarding instruction, the type of forwarding desired for this address, and a boolean flag indicating whether this instruction should apply to internal (true) or external (false) calls.

Parameters:

`destAddress` - destination address for the call forwarding operation

`type` - the type of the forwarding operation

`internalCalls` - flag for internal call forwarding

CallControlForwarding

```
public CallControlForwarding(java.lang.String destAddress,  
                             int type,  
                             java.lang.String caller)
```

This constructor takes an address to apply the forwarding instruction for a specific incoming telephone call, identified by a string address. It also takes the type of forwarding desired for this specific address.

Parameters:

`destAddress` - destination address for the call forwarding operation

`type` - the type of the forwarding operation

`caller` - the address of the incoming caller

Method Detail

getDestinationAddress

```
public java.lang.String getDestinationAddress()
```

Returns the destination address of this forwarding instruction.

Returns:

The destination address of this forwarding instruction.

getType

```
public int getType()
```

Returns the type of this forwarding instruction, either unconditionally, upon no answer, or upon busy.

Returns:

The type of this forwarding instruction.

getFilter

```
public int getFilter()
```

Returns the filter of this forwarding instruction. The filter indicates which calls should trigger this forwarding instruction. Filters include: applying this instruction to all calls, to only internal calls, to only external call, or for calls from a specific address.

Returns:

The filter for this forwarding instruction.

getSpecificCaller

public java.lang.String **getSpecificCaller()**

If the filter for this forwarding instruction is SPECIFIC_ADDRESS, then this method returns that calling address to which this filter applies. If the filter is something other than SPECIFIC_ADDRESS, this method returns null.

Returns:

The specific address for this forwarding instruction.

javax.telephony.callcontrol

Interface CallControlTerminal

public interface **CallControlTerminal**
extends Terminal

Introduction

The `CallControlTerminal` interface extends the core `Terminal` interface with features like the ability to pickup a call at a terminal and the ability to specify that this terminal should not be disturbed. Applications may query a `Terminal` object using the `instanceof` operator to see whether it supports this interface.

Do Not Disturb

The `CallControlTerminal` interface defines the do not disturb attribute. The do not disturb attribute indicates to the telephony platform that this `Terminal` does not want to be bothered with incoming telephone calls. That is, if this feature is activated, the underlying telephone platform will not ring this `Terminal` for incoming telephone calls. Applications use the `CallControlTerminal.setDoNotDisturb()` method to activate or deactivate this feature and the `CallControlTerminal.getDoNotDisturb()` method to return the current state of this attribute.

Note that the `CallControlAddress` interface also carries the do not disturb attribute. This attribute associated with each are maintained independently. Maintaining a separate do not disturb attribute at terminal and address allows for control over the do not disturb feature at either the terminal or address level.

Picking Up Telephone Calls

The pickup feature permits applications to answer telephone calls which are not ringing at a particular `Terminal`. This feature is often used when a call is "queued" at an `Address` or a telephone is ringing at a `Terminal` across a room. Both the `pickup()` and the `pickupFromGroup()` methods defined on this interface provide this pickup feature to the application.

The `CallControlTerminal.pickup()` method has three versions and permits applications to answer a `Call` at a designated `Address` or `Terminal`. The `CallControlTerminal.pickupFromGroup()` method permits applications to answer a `Call` at some other `Terminal` in the same "pickup group".

Observers and Events

All events pertaining to the `CallControlTerminal` interface are reported via the `TerminalObserver.terminalChangedEvent()` method. The application observer object must also implement the `CallControlCalObserver` interface to express interest in the call control package events.

The following are those events associated with this interface:

`CallCtlTermDoNotDisturbEv` Indicates the do-not-disturb attribute of this Terminal has changed.

See Also:

[CallControlAddress](#), [CallControlTerminalObserver](#),
[CallCtlTermDoNotDisturbEv](#)

Method Summary

boolean	<code>getDoNotDisturb()</code> Returns true if the do-not-disturb feature is activated, false otherwise.
<u>TerminalConnection</u>	<code>pickup(<u>Address</u> pickupAddress, <u>Address</u> terminalAddress)</code> This method "picks up" a Call at this Terminal.
<u>TerminalConnection</u>	<code>pickup(<u>Connection</u> pickupConnection, <u>Address</u> terminalAddress)</code> This method "picks up" a Call at this Terminal.
<u>TerminalConnection</u>	<code>pickup(<u>TerminalConnection</u> pickTermConn, <u>Address</u> terminalAddress)</code> This method "picks up" a Call at this Terminal.
<u>TerminalConnection</u>	<code>pickupFromGroup(<u>Address</u> terminalAddress)</code> This method "picks up" a Call at this Terminal.
<u>TerminalConnection</u>	<code>pickupFromGroup(java.lang.String pickupGroup, <u>Address</u> terminalAddress)</code> This method "picks up" a Call at this Terminal.
void	<code>setDoNotDisturb(boolean enable)</code> Specifies whether the do-not-disturb feature should be activated or deactivated for this Terminal.

Methods inherited from interface javax.telephony.Terminal

addCallListener, addCallObserver, addObserver, addTerminalListener,
getAddresses, getCallListeners, getCallObservers, getCapabilities,
getName, getObservers, getProvider, getTerminalCapabilities,
getTerminalConnections, getTerminalListeners, removeCallListener,
removeCallObserver, removeObserver, removeTerminalListener

Method Detail*getDoNotDisturb*

```
public boolean getDoNotDisturb()
           throws MethodNotSupportedException
```

Returns true if the do-not-disturb feature is activated, false otherwise.

Returns:

True if do-not-disturb is activated, false if it is deactivated

Throws:

MethodNotSupportedException - This method is not supported by the given implementation.

setDoNotDisturb

```
public void setDoNotDisturb(boolean enable)
           throws MethodNotSupportedException,  

                InvalidStateException
```

Specifies whether the do-not-disturb feature should be activated or deactivated for this Terminal. This feature only affects whether or not calls will be accepted at this Terminal. The setting of this attribute does not affect the do-not-disturb attribute associated with all Addresses associated with this Terminal. If 'enable' is true, do-not-disturb is activated if not already so. If 'enable' is false, do-not-disturb is deactivated if not already so.

Pre-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE

Post-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. this.getDoNotDisturb() == enable
3. CallCtlTermDoNotDisturbEv is delivered for this Terminal

Parameters:

`enable` - True to activated do-not-disturb, false to deactivate.

Throws:

MethodNotSupportedException - This method is not supported by the given implementation.

InvalidStateException - The Provider is not "in service".

See Also:

CallCtlTermDoNotDisturbEv

pickup

```
public TerminalConnection pickup(Connection pickupConnection,  
                                Address terminalAddress)  
    throws InvalidArgumentException,  
           InvalidStateException,  
           MethodNotSupportedException,  
           PrivilegeViolationException,  
           ResourceUnavailableException
```

This method "picks up" a Call at this Terminal. Picking up a Call is analogous to answering a Call at this Terminal (i.e. `TerminalConnection.answer()`), except the Call typically is not ringing at this Terminal. For example, this method is used to answer a "queued" Call or a Call which is ringing at another Terminal across the room.

This method takes a `Connection` and an `Address` as arguments. The `Connection` argument represents the destination end of the telephone call to be picked up. This `Connection` must be in either the `CallControlConnection.QUEUED` state or the `CallControlConnection.ALERTING` state. The `Address` argument chooses the `Address` associated with this Terminal on which to pick up the Call. A new `TerminalConnection` is created and returned which is in the `CallControlTerminalConnection.TALKING` state and associated with this Terminal.

The Address and Connection Arguments

The relationship between the `Address` and `Connection` arguments affects the resulting behavior of this method. There are two different situations: if the given `Connection` is associated with the given `Address`, and if the given `Connection` is not associated with the given `Address` (i.e. via the `Connection.getAddress()` method).

If the given `Connection` is associated with the given `Address`, this implies that the `Connection` was in the `CallControlConnection.QUEUED` state, or the Terminal did not ring for some reason even though the `Connection` is in the `CallControlConnection.ALERTING` state. In this case, the `Connection` moves to the `CallControlConnection.ESTABLISHED` state and the new `TerminalConnection` created is associated with the `Connection`.

If the given `Connection` is not associated with the given `Address`, this implies that the call is alerting at an entirely different endpoint from this Terminal. This scenario permits applications to pick up a telephone call which is ringing across the room. In this case, the `Connection` moves to the `CallControlConnection.DISCONNECTED` state. A new `Connection` is created and associated with the `Address` argument. It is in the `CallControlConnection.ESTABLISHED` state. The new `TerminalConnection` created is associated with this new `Connection`.

Pre-conditions:

1. (this.getProvider()).getState() == Provider.IN_SERVICE
2. (pickupConnection.getCall()).getState() == Call.ACTIVE
3. pickupConnection.getCallControlState() == CallControlConnection.QUEUED or CallControlConnection.ALERTING
4. terminaladdress element of this.getAddresses()

Post-conditions Outcome 1: If pickupConnection is associated with terminaladdress (i.e. pickupConnection.getAddress() == terminaladdress)

1. Let tc be the new TerminalConnection created and returned
2. tc.getCallControlState() == CallControlTerminalConnection.TALKING
3. pickupConnection.getCallControlState() == CallControlConnection.ESTABLISHED
4. tc.getConnection() == pickupConnection
5. TermConnCreatedEv is delivered for tc
6. CallCtlTermConnTalkingEv/TermConnActiveEv is delivered for tc
7. CallCtlConnEstablishedEv/ConnConnectedEv is delivered for pickupConnection

Post-conditions Outcome 2: If pickupConnection is not associated with terminaladdress (i.e. pickupConnection.getAddress() != terminaladdress)

1. Let tc be the new TerminalConnection created and returned
2. Let connection be the new Connection created
3. Let call = pickupConnection.getCall()
4. tc.getCallControlState() == CallControlTerminalConnection.TALKING
5. connection.getCallControlState() == CallControlConnection.ESTABLISHED
6. connection.getAddress() == terminaladdress
7. connection.getCall() == call
8. tc.getConnection() == connection
9. pickupConnection.getCallControlState() == CallControlConnection.DISCONNECTED
10. TermConnCreatedEv is delivered for tc
11. CallCtlTermConnTalkingEv/TermConnActiveEv is delivered for tc
12. ConnCreatedEv is delivered for connection
13. CallCtlConnEstablishedEv/ConnConnectedEv is delivered for connection
14. CallCtlConnDisconnectedEv/ConnDisconnectedEv is delivered for pickupConnection

Parameters:

`pickupConnection` - The Connection to be picked up

`terminalAddress` - The Address associated with the Terminal

Returns:

The new TerminalConnection associated with the Terminal

Throws:

InvalidArgumentException - Either the Connection or Address given as arguments is not valid.
InvalidStateException - Either the Provider is not "in service" or the Connection is not "queued" or "alerting".

MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

See Also:

[ConnCreatedEv](#), [TermConnCreatedEv](#), [ConnConnectedEv](#),
[ConnDisconnectedEv](#), [TermConnActiveEv](#), [CallCtlTermConnTalkingEv](#),
[CallCtlConnEstablishedEv](#), [CallCtlConnDisconnectedEv](#)

pickup

```
public TerminalConnection pickup(TerminalConnection pickTermConn,  

                                   Address terminalAddress)  

    throws InvalidArgumentException,  

           InvalidStateException,  

           MethodNotSupportedException,  

           PrivilegeViolationException,  

           ResourceUnavailableException
```

This method "picks up" a Call at this Terminal. Picking up a Call is analogous to answering a Call at this Terminal (i.e. `TerminalConnection.answer()`), except the Call typically is not ringing at this Terminal. For example, this method is used to answer a "queued" Call or a Call which is ringing at another Terminal across the room.

This method is similar to the `CallControlTerminal.pickup(Connection, Address)` method except an explicit `TerminalConnection` is given. Since an explicit `TerminalConnection` is given, this implies its `Connection` must be "alerting" since "queued" `Connections` may not have any associated `TerminalConnections`.

Parameters:

`pickTermConn` - The `TerminalConnection` to be picked up
`terminalAddress` - The `Address` associated with the Terminal

Returns:

The new `TerminalConnection` associated with the Terminal

Throws:

[InvalidArgumentException](#) - Either the `Connection` or `Address` given as arguments is not valid.

[InvalidStateException](#) - Either the Provider is not "in service" or the `Connection` is not "queued".

[MethodNotSupportedException](#) - This method is not supported by the implementation.

[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method.

[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

pickup

```
public TerminalConnection pickup(Address pickupAddress,  

                                   Address terminalAddress)  

    throws InvalidArgumentException,
```

InvalidStateException,
MethodNotSupportedException,
PrivilegeViolationException,
ResourceUnavailableException

This method "picks up" a Call at this Terminal. Picking up a Call is analogous to answering a Call at this Terminal (i.e. `TerminalConnection.answer()`), except the Call typically is not ringing at this Terminal. For example, this method is used to answer a "queued" Call or a Call which is ringing at another Terminal across the room.

This method is similar to the `CallControlTerminal.pickup(Connection, Address)` method except an Address is given instead of an explicit Connection. This method permits the implementation to choose a suitable Connection associated with 'pickupAddress'.

Parameters:

`pickupAddress` - The Address which is to be picked up

`terminalAddress` - The Address associated with the Terminal

Returns:

The new `TerminalConnection` associated with the Terminal

Throws:

InvalidArgumentException - Either the Connection or Address given as arguments is not valid.
InvalidStateException - Either the Provider is not "in service" or the Connection is not "queued" or "alerting".

MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

pickupFromGroup

```
public TerminalConnection pickupFromGroup(java.lang.String pickupGroup,
                                           Address terminalAddress)
    throws InvalidArgumentException,
           InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

This method "picks up" a Call at this Terminal. Picking up a Call is analogous to answering a Call at this Terminal (i.e. `TerminalConnection.answer()`), except the Call typically is not ringing at this Terminal. For example, this method is used to answer a "queued" Call or a Call which is ringing at another Terminal across the room.

This method takes two arguments: a string "pickup group" code and an Address associated with this Terminal. The Address argument chooses the Address associated with this Terminal on which to pick up the Call. A new `TerminalConnection` is created and returned which is in the `CallControlTerminalConnection.TALKING` state and associated with this Terminal.

The Pickup Group Code

The application designates the Call to pick up via a string code rather than giving a specific Connection endpoint. An administrator of the underlying telephony platform can create groups of endpoints associated with a particular group code. From the code, the implementations decides which particular Connection is to be picked up. Once the Connection has been determined, this method behavior similarly to the `CallControlTerminal.pickup(Connection, Address)` method.

Parameters:

`pickupGroup` - The pickup group
`terminalAddress` - The Address associated with the Terminal

Returns:

The new TerminalConnection associated with the Terminal

Throws:

[InvalidArgumentException](#) - Either the Connection or Address given as arguments is not valid.
[InvalidStateException](#) - Either the Provider is not "in service" or the Connection is not "queued" or "alerting".
[MethodNotSupportedException](#) - This method is not supported by the implementation.
[PrivilegeViolationException](#) - The application does not have the proper authority to invoke this method.
[ResourceUnavailableException](#) - An internal resource necessary for the successful invocation of this method is not available.

pickupFromGroup

```
public TerminalConnection pickupFromGroup(Address terminalAddress)
                                throws InvalidArgumentException,
                                       InvalidStateException,
                                       MethodNotSupportedException,
                                       PrivilegeViolationException,
                                       ResourceUnavailableException
```

This method "picks up" a Call at this Terminal. Picking up a Call is analogous to answering a Call at this Terminal (i.e. `TerminalConnection.answer()`), except the Call typically is not ringing at this Terminal. For example, this method is used to answer a "queued" Call or a Call which is ringing at another Terminal across the room.

This method takes a single argument: an Address associated with this Terminal. The Address argument chooses the Address associated with this Terminal on which to pick up the Call. A new TerminalConnection is created and returned which is in the `CallControlTerminalConnection.TALKING` state and associated with this Terminal.

The Pickup Group Code

This method does not take the pickup group code as an argument. Instead, the implementation chooses a suitable Call to be picked up. This Call should have a Connection in either the `CallControlConnection.ALERTING` state or the `CallControlConnection.QUEUED` state. The Address associated with this Connection should belong to the same pickup group as the Address given as the argument. Once the Connection has been determined, this method behavior similarly to the `CallControlTerminal.pickup(Connection, Address)` method.

Parameters:

`terminalAddress` - The Address associated with the Terminal

Returns:

The new `TerminalConnection` associated with the Terminal

Throws:

`InvalidArgumentException` - Either the Connection or Address given as arguments is not valid.

`InvalidStateException` - Either the Provider is not "in service" or the Connection is not "queued" or "alerting".

`MethodNotSupportedException` - This method is not supported by the implementation.

`PrivilegeViolationException` - The application does not have the proper authority to invoke this method.

`ResourceUnavailableException` - An internal resource necessary for the successful invocation of this method is not available.

javax.telephony.callcontrol ***Interface CallControlTerminalConnection***

public interface **CallControlTerminalConnection**
extends TerminalConnection

Introduction

The `CallControlTerminalConnection` interface extends the core `TerminalConnection` interface with additional features and greater detail about the `TerminalConnection` state. Applications may query a `TerminalConnection` object using the `instanceof` operator to see whether it supports this interface.

CallControlTerminalConnection State

This interface defines states for the `TerminalConnection` which provide greater detail beyond the states defined in the `TerminalConnection` interface. These states are related to the states defined in the core package in certain, specific ways, as defined below. Applications may obtain the `CallControlTerminalConnection` state via the `getCallControlState()` method defined on this interface. This method returns one of the integer state constants defined in this interface.

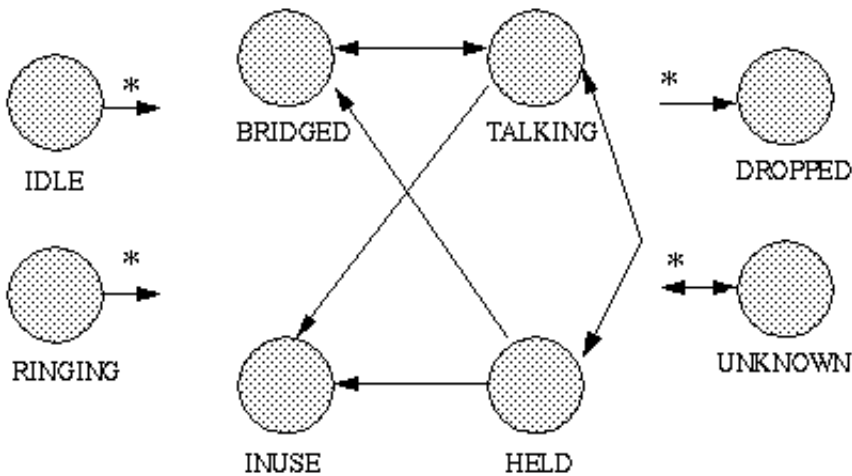
Below is a description of each `CallControlTerminalConnection` state in real-world terms. These real-world descriptions only serve to provide a more intuitive understanding of what is going on. Several methods in this specification state pre-conditions based upon the `CallControlTerminalConnection` state. Some of these states are identical to those defined in the core package.

<code>CallControlTerminalConnection.IDLE</code>	This state has the same definition as in the core package. It is the initial <code>CallControlTerminalConnection</code> state for all new <code>TerminalConnection</code> objects. <code>TerminalConnections</code> typically do not stay in this state for long, quickly transitioning to another state.
<code>CallControlTerminalConnection.RINGING</code>	This state has the same definition as in the core package. It indicates that the associated <code>Terminal</code> is ringing and has an incoming telephone call.
<code>CallControlTerminalConnection.TALKING</code>	This state indicates that the <code>Terminal</code> is actively part of a <code>Call</code>, is typically "off-hook", and the party is communicating on the telephone call.
<code>CallControlTerminalConnection.HELD</code>	This state indicates that a <code>Terminal</code> is part of a <code>Call</code>, but is on hold. Other <code>Terminals</code> which are on the same <code>Call</code> and associated with the same <code>Connection</code> may or may not also be in this state.
<code>CallControlTerminalConnection.BRIDGED</code>	This state indicates that a <code>Terminal</code> is currently bridged into a <code>Call</code>. A <code>Terminal</code> may typically join a <code>Call</code> when it is bridged. A bridged <code>Terminal</code> is part of the telephone call, but not active. Typically, some hardware resource is occupied while a <code>Terminal</code> is bridged into a <code>Call</code>.
<code>CallControlTerminalConnection.INUSE</code>	This state indicates that a <code>Terminal</code> hardware resource is currently in use. The terminal is not available for the <code>Call</code> associated with this <code>Terminal Connection</code>, that is the <code>Terminal</code> may not join the call. This state is similar to the <code>CallControlTerminalConnection.BRIDGED</code> state except that the <code>Terminal</code> may not join the <code>Call</code>.
<code>CallControlTerminalConnection.DROPPED</code>	This state has the same definition as in the core package. It indicates that a particular <code>Terminal</code> has permanently left the <code>Call</code>.
<code>CallControlTerminalConnection.UNKNOWN</code>	This state has the same definition as in the core package. It indicates that the implementation is unable to determine the state of the <code>TerminalConnection</code>. <code>TerminalConnections</code> may transition into and out of this state at any time.

State Transitions

Similar to the `TerminalConnection` state transition diagram, the `CallControlTerminalConnection` state must transition according to the rules illustrated in the finite state diagram below. The implementation must guarantee that the `CallControlTerminalConnection` state abides by this transition diagram.

The asterisk next to a state transition, as in the core package, implies a transition to/from another state as designated by the direction of the transition arrow.



Core vs. CallControl Package States

There is a strong relationship between the `TerminalConnection` states and the `CallControlTerminalConnection` states. If an implementation supports the call control package, it must ensure this relationship is properly maintained.

Since the states defined in the `CallControlTerminalConnection` interface provide more detail to the states defined in the `TerminalConnection` interface, each state in the `TerminalConnection` interface corresponds to a state defined in the `CallControlTerminalConnection` interface. Conversely, each `CallControlTerminalConnection` state corresponds to exactly one `TerminalConnection` state. This arrangement permits applications to view either the core state or the call control state and still see a consistent view.

The following table outlines the relationship between the core package `TerminalConnection` states and the call control package `TerminalConnection` states.

If the call control package state is...***then the core package state must be...***

<code>CallControlTerminalConnection.IDLE</code>	<code>TerminalConnection.IDLE</code>
<code>CallControlTerminalConnection.RINGING</code>	<code>TerminalConnection.RINGING</code>
<code>CallControlTerminalConnection.TALKING</code>	<code>TerminalConnection.ACTIVE</code>
<code>CallControlTerminalConnection.HELD</code>	<code>TerminalConnection.ACTIVE</code>
<code>CallControlTerminalConnection.INUSE</code>	<code>TerminalConnection.PASSIVE</code>
<code>CallControlTerminalConnection.BRIDGED</code>	<code>TerminalConnection.PASSIVE</code>
<code>CallControlTerminalConnection.DROPPED</code>	<code>TerminalConnection.DROPPED</code>
<code>CallControlTerminalConnection.UNKNOWN</code>	<code>TerminalConnection.UNKNOWN</code>

Observers and Events

All events pertaining to the `CallControlTerminalConnection` interface are reported via the `CallObserver.callChangedEvent()` method. The application observer object must also implement the `CallControlCallObserver` interface to express interest in the call control package events. Applications receive `TerminalConnection`-related events in the call control package when the call control state changes.

Observers which are registered on a Call receive events when the `CallControlTerminalConnection` state changes. Note that when the `CallControlTerminalConnection` state changes, it sometimes results in the `TerminalConnection` state changing (according to the table above). In these instances, both the proper call control and core package events are delivered to the observer.

The `CallControlTerminalConnection` state events defined in this package are: `CallCtlTermConnBridgedEv`, `CallCtlTermConnDroppedEv`, `CallCtlTermConnHeldEv`, `CallCtlTermConnInUseEv`, `CallCtlTermConnRingingEv`, `CallCtlTermConnTalkingEv`, and `CallCtlTermConnUnknownEv`.

See Also:

[TerminalConnection](#), [CallObserver](#), [CallControlCallObserver](#),
[CallCtlCalEv](#), [CallCtlTermConnEv](#), [CallCtlTermConnRingingEv](#),
[CallCtlTermConnTalkingEv](#), [CallCtlTermConnHeldEv](#),
[CallCtlTermConnBridgedEv](#), [CallCtlTermConnInUseEv](#),
[CallCtlTermConnDroppedEv](#), [CallCtlTermConnUnknownEv](#)

Field Summary

static int	<u>BRIDGED</u> The <code>CallControlTerminalConnection.BRIDGED</code> state indicates that a Terminal is currently bridged into a Call.
static int	<u>DROPPED</u> The <code>CallControlTerminalConnection.DROPPED</code> state has the same definition as in the core package.
static int	<u>HELD</u> The <code>CallControlTerminalConnection.HELD</code> state indicates that a Terminal is part of a Call, but is on hold.
static int	<u>IDLE</u> The <code>CallControlTerminalConnection.IDLE</code> state has the same definition as in the core package.
static int	<u>INUSE</u> The <code>CallControlTerminalConnection.INUSE</code> state indicates that a Terminal is part of a Call, but is not active.
static int	<u>RINGING</u> The <code>CallControlTerminalConnection.RINGING</code> state has the same definition as in the core package.
static int	<u>TALKING</u> The <code>CallControlTerminalConnection.TALKING</code> state indicates that a Terminal is actively part of a Call, is typically "off-hook", and the party is communicating on the telephone call.
static int	<u>UNKNOWN</u> The <code>CallControlTerminalConnection.UNKNOWN</code> state has the same definition as in the core package.

Fields inherited from interface javax.telephony.TerminalConnection

ACTIVE, DROPPED, IDLE, PASSIVE, RINGING, UNKNOWN

Method Summary

int	<u>getCallControlState()</u> Returns the call control state of the TerminalConnection.
void	<u>hold()</u> Places a TerminalConnection on hold with respect to the Call of which it is a part.
void	<u>join()</u> Makes a currently bridged TerminalConnection active on a Call.
void	<u>leave()</u> Places a currently active TerminalConnection in a <i>bridged</i> state on a Call.
void	<u>unhold()</u> Takes a TerminalConnection off hold with respect to the Call of which it is a part.

Methods inherited from interface javax.telephony.TerminalConnection

answer, getCapabilities, getConnection, getState, getTerminal,
getTerminalConnectionCapabilities

Field Detail

IDLE

```
public static final int IDLE
```

The `CallControlTerminalConnection.IDLE` state has the same definition as in the core package. It is the initial `CallControlTerminalConnection` state for all new TerminalConnections. TerminalConnections typically do not stay in this state for long, but quickly transition to another state.

RINGING

```
public static final int RINGING
```

The `CallControlTerminalConnection.RINGING` state has the same definition as in the core package. It indicates that the associated Terminal is ringing and has an incoming telephone call.

TALKING

```
public static final int TALKING
```

The `CallControlTerminalConnection.TALKING` state indicates that a Terminal is actively part of a Call, is typically "off-hook", and the party is communicating on the telephone call.

HELD

```
public static final int HELD
```

The `CallControlTerminalConnection.HELD` state indicates that a Terminal is part of a Call, but is on hold. Other Terminals which are on the same Call and associated with the same Connection may or may not also be in this state.

BRIDGED

```
public static final int BRIDGED
```

The `CallControlTerminalConnection.BRIDGED` state indicates that a Terminal is currently bridged into a Call. A Terminal may typically join a telephone call when it is bridged. A bridged Terminal is part of the telephone call, but not active. Typically, some hardware resource is occupied while a Terminal is bridged into a Call.

INUSE

```
public static final int INUSE
```

The `CallControlTerminalConnection.INUSE` state indicates that a Terminal is part of a Call, but is not active. It may not Call, however the resource on the Terminal is currently in use. This state is similar to the `CallControlTerminalConnection.BRIDGED` state however, the Terminal may not join the Call.

DROPPED

```
public static final int DROPPED
```

The `CallControlTerminalConnection.DROPPED` state has the same definition as in the core package. It indicates that a particular Terminal has permanently left the Call.

UNKNOWN

public static final int **UNKNOWN**

The `CallControlTerminalConnection.UNKNOWN` state has the same definition as in the core package. It indicates that the implementation is unable to determine the state of the `TerminalConnection`. `TerminalConnections` may transition into and out of this state at any time.

Method Detail

getCallControlState

public int **getCallControlState()**

Returns the call control state of the `TerminalConnection`. The return values will be one of the integer state constants defined above.

Returns:

The current call control state of the `TerminalConnection`.

hold

public void **hold()**
 throws `InvalidStateException`,
 `MethodNotSupportedException`,
 `PrivilegeViolationException`,
 `ResourceUnavailableException`

Places a `TerminalConnection` on hold with respect to the Call of which it is a part. Many Terminals may be on the same Call and associated with the same Connection. Any one of them may go "on hold" at any time, provided they are active in the Call. The `TerminalConnection` must be in the `CallControlTerminalConnection.TALKING` state. This method returns when the `TerminalConnection` has moved to the `CallControlTerminalConnection.HELD` state, or until an error occurs and an exception is thrown.

Pre-conditions:

1. `(this.getTerminal()).getProvider().getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.TALKING`

Post-conditions:

1. `(this.getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.HELD`
3. `CallCtlTermConnHeldEv` is delivered for this `TerminalConnection`

Throws:

`InvalidStateException` - Either the Provider is not "in service" or the `TerminalConnection` is not "talking".

`MethodNotSupportedException` - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

See Also:

CallCtlTermConnHeldEv

unhold

```
public void unhold()
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Takes a TerminalConnection off hold with respect to the Call of which it is a part. Many Terminals may be on the same Call and associated with the same Connection. Any one of them may go "on hold" at any time, provided they are active in the Call. The TerminalConnection must be in the `CallControlTerminalConnection.HELD` state. This method returns successfully when the TerminalConnection moves into the `CallControlTerminalConnection.TALKING` state or until an error occurs and an exception is thrown.

Pre-conditions:

1. `((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.HELD`

Post-conditions:

1. `((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.TALKING`
3. `CallCtlTermConnTalkingEv` is delivered for this TerminalConnection

Throws:

InvalidStateException - Either the Provider is not "in service" or the TerminalConnection is not "held".

MethodNotSupportedException - This method is not supported by the implementation.

PrivilegeViolationException - The application does not have the proper authority to invoke this method.

ResourceUnavailableException - An internal resource necessary for the successful invocation of this method is not available.

See Also:

CallCtlTermConnTalkingEv

join

```
public void join()
```

```
throws InvalidStateException,
       MethodNotSupportedException,
       PrivilegeViolationException,
       ResourceUnavailableException
```

Makes a currently bridged `TerminalConnection` active on a Call. Other Terminals, which share the same Address as this Terminal, may be active on the same Call. This situation is known as *bridging*. The `TerminalConnection` must be in the `CallControlTerminalConnection.BRIDGED` state. This method returns which the `TerminalConnection` has moved to the `CallControlTerminalConnection.TALKING` state or until an error occurs and an exception is thrown.

Pre-conditions:

1. `((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.BRIDGED`

Post-conditions:

1. `((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.TALKING`
3. `CallCtlTermConnTalkingEv/TermConnActiveEv` is delivered for this `TerminalConnection`

Throws:

`InvalidStateException` - Either the Provider is not "in service" or the `TerminalConnection` is not "bridged".

`MethodNotSupportedException` - This method is not supported by the implementation.

`PrivilegeViolationException` - The application does not have the proper authority to invoke this method.

`ResourceUnavailableException` - An internal resource necessary for the successful invocation of this method is not available.

See Also:

`TermConnActiveEv`, `CallCtlTermConnTalkingEv`

leave

```
public void leave()
    throws InvalidStateException,
           MethodNotSupportedException,
           PrivilegeViolationException,
           ResourceUnavailableException
```

Places a currently active `TerminalConnection` in a *bridged* state on a Call. Other Terminals, which share the same Address as this Terminal, may remain active on the same Call. This situation where Terminals share an Address on a call is known as *bridging*. The `TerminalConnection` on which this method is invoked must be in the `CallControlTerminalConnection.TALKING` state.

There are two possible outcomes of this method depending upon the number of remaining, active `TerminalConnections` on the Call. If there are other active `TerminalConnections`, then this `TerminalConnection` moves into the `CallControlTerminalConnection.BRIDGED` state and this method returns. If there are no other active `TerminalConnections`, then this `TerminalConnection`

moves into the `CallControlTerminalConnection.DROPPED` state. Its associated `Connection` moves into the `CallControlConnection.DISCONNECTED` state, i.e. the entire endpoint leaves the telephone call. This method waits until one of these two outcomes occurs or until an error occurs and an exception is thrown.

Pre-conditions:

1. `((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.TALKING`

Post-conditions Outcome 1: There are no other active `TerminalConnections` on this `Call` (no others which are either "held" or "talking")

1. `((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE`
2. Let `connection = this.getConnection()`
3. Let `tc[] = connection.getTerminalConnections()` before this method is invoked
4. `tc[i].getCallControlState() == CallControlTerminalConnection.DROPPED`, for all `i`
5. `connection.getCallControlState() == CallControlConnection.DISCONNECTED`
6. `CallCtlTermConnDroppedEv/TermConnDroppedEv` is delivered for all `tc[i]`
7. `CallCtlConnDisconnectedEv/ConnDisconnectedEv` is delivered for `connection`

Post-conditions Outcome 2: There are other active `TerminalConnections` on this `Call` (others which are either "held" or "talking")

1. `((this.getTerminal()).getProvider()).getState() == Provider.IN_SERVICE`
2. `this.getCallControlState() == CallControlTerminalConnection.BRIDGED`
3. `CallCtlTermConnBridgedEv/TermConnPassiveEv` is delivered for this `TerminalConnection`

Throws:

`InvalidStateException` - Either the `Provider` is not "in service" or the `TerminalConnection` is not "talking".

`MethodNotSupportedException` - This method is not supported by the implementation.

`PrivilegeViolationException` - The application does not have the proper authority to invoke this method.

`ResourceUnavailableException` - An internal resource necessary for the successful invocation of this method is not available.

See Also:

`TermConnPassiveEv`, `TermConnDroppedEv`, `ConnDisconnectedEv`,
`CallCtlTermConnBridgedEv`, `CallCtlTermConnDroppedEv`,
`CallCtlConnDisconnectedEv`

javax.telephony.callcontrol

Interface CallControlTerminalObserver

public interface **CallControlTerminalObserver**
 extends [TerminalObserver](#)

The `CallControlTerminalObserver` interface reports all events for the `CallControlTerminal` interface. Applications implement this interface to receive `CallControlTerminal`-related events. All events are reported via the `TerminalObserver.terminalChangedEvent()` method. This interface, therefore, allows applications to signal to the implementation that they are interested in call control package events. This interface defines no additional methods.

All events must extend the `CallCtlTermEv` event interface, which in turn, extends the core `TermEv` interface.

The following are those events which are associated with this interface:

`CallCtlTermDoNotDisturbEv` Indicates the Do Not Disturb characteristics of this Terminal has changed.

See Also:

[TerminalObserver](#), [TermEv](#), [CallControlTerminal](#), [CallCtlTermEv](#),
[CallCtlTermDoNotDisturbEv](#)

Methods inherited from interface javax.telephony. TerminalObserver
--

terminalChangedEvent

Package javax.telephony.callcontrol.capabilities

The JTAPI Call Control Capabilities package provides the interfaces that inform the an application about which features were included in the current JTAPI implementation and which features are currently valid throughout a call transaction.

See:

Description

Interface Summary	
<u><i>CallControlAddressCapabilities</i></u>	The CallControlAddressCapabilities interface extends the core AddressCapabilities interface.
<u><i>CallControlCallCapabilities</i></u>	The CallControlCallCapabilities interface extends the core CallCapabilities interface.
<u><i>CallControlConnectionCapabilities</i></u>	The CallControlConnectionCapabilities interface extends the core ConnectionCapabilities interface.
<u><i>CallControlTerminalCapabilities</i></u>	The CallControlTerminalCapabilities interface extends the core TerminalCapabilities interface.
<u><i>CallControlTerminalConnectionCapabilities</i></u>	The CallControlTerminalConnectionCapabilities interface extends the core TerminalConnectionCapabilities interface.

Package javax.telephony.callcontrol.capabilities Description

The JTAPI Call Control Capabilities package provides the interfaces that inform the an application about which features were included in the current JTAPI implementation and which features are currently valid throughout a call transaction.

Hierarchy For Package javax.telephony.callcontrol.capabilities

Package Hierarchies:

All Packages

Interface Hierarchy

- interface javax.telephony.capabilities.**AddressCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlAddressCapabilities**
 - interface javax.telephony.capabilities.**CallCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlCallCapabilities**
 - interface javax.telephony.capabilities.**ConnectionCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlConnectionCapabilities**
 - interface javax.telephony.capabilities.**TerminalCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlTerminalCapabilities**
 - interface javax.telephony.capabilities.**TerminalConnectionCapabilities**
 - interface
javax.telephony.callcontrol.capabilities.**CallControlTerminalConnectionCapabilities**
-
-

javax.telephony.callcontrol.capabilities

Interface CallControlAddressCapabilities

public interface **CallControlAddressCapabilities**
 extends [AddressCapabilities](#)

The `CallControlAddressCapabilities` interface extends the core `AddressCapabilities` interface. This interface provides methods to reflect the capabilities of the methods on the `CallControlAddress` interface.

The `Provider.getAddressCapabilities()` method returns the static `Address` capabilities, and the `Address.getCapabilities()` method returns the dynamic `Address` capabilities. The object returned from each of these methods can be queried with the `instanceof` operator to check if it supports this interface. This same interface is used to reflect both static and dynamic `Address` capabilities.

See Also:

[Provider](#), [Address](#), [AddressCapabilities](#)

Method Summary

boolean	<u>canCancelForwarding()</u> Returns true if the application can cancel the forwarding on this <code>Address</code> , false otherwise.
boolean	<u>canGetDoNotDisturb()</u> Returns true if the application can obtain the do not disturb state, false otherwise.
boolean	<u>canGetForwarding()</u> Returns true if the application can obtain the current forwarding status on this <code>Address</code> , false otherwise.
boolean	<u>canGetMessageWaiting()</u> Returns true if the application can obtain the message waiting state, false otherwise.
boolean	<u>canSetDoNotDisturb()</u> Returns true if the application can set the do not disturb state, false otherwise.
boolean	<u>canSetForwarding()</u> Returns true if the application can set the forwarding on this <code>Address</code> , false otherwise.
boolean	<u>canSetMessageWaiting()</u> Returns true if the application can set the message waiting state, false otherwise.

Methods inherited from interface javax.telephony.capabilities.AddressCapabilitiesisObservable**Method Detail***canSetForwarding*

```
public boolean canSetForwarding()
```

Returns true if the application can set the forwarding on this Address, false otherwise.

Returns:

True if the application can set the forwarding on this Address, false otherwise.

canGetForwarding

```
public boolean canGetForwarding()
```

Returns true if the application can obtain the current forwarding status on this Address, false otherwise.

Returns:

True if the application can obtain the current forwarding status on this Address, false otherwise.

canCancelForwarding

```
public boolean canCancelForwarding()
```

Returns true if the application can cancel the forwarding on this Address, false otherwise.

Returns:

True if the application can cancel the forwarding on this Address, false otherwise.

canGetDoNotDisturb

```
public boolean canGetDoNotDisturb()
```

Returns true if the application can obtain the do not disturb state, false otherwise.

Returns:

True if the application can obtain the do not disturb state, false otherwise.

canSetDoNotDisturb

public boolean **canSetDoNotDisturb()**

Returns true if the application can set the do not disturb state, false otherwise.

Returns:

True if the application can set the do not disturb state, false otherwise.

canGetMessageWaiting

public boolean **canGetMessageWaiting()**

Returns true if the application can obtain the message waiting state, false otherwise.

Returns:

True if the application can obtain the message waiting state, false otherwise.

canSetMessageWaiting

public boolean **canSetMessageWaiting()**

Returns true if the application can set the message waiting state, false otherwise.

Returns:

True if the application can set the message waiting state, false otherwise.

javax.telephony.callcontrol.capabilities

Interface CallControlCallCapabilities

public interface **CallControlCallCapabilities**
 extends [CallCapabilities](#)

The **CallControlCallCapabilities** interface extends the core **CallCapabilities** interface. This interface provides methods to reflect the capabilities of the methods on the **CallControlCall** interface.

The **Provider.getCallCapabilities()** method returns the static Call capabilities, and the **Call.getCapabilities()** method returns the dynamic Call capabilities. The object returned from each of these methods can be queried with the **instanceof** operator to check if it supports this interface. This same interface is used to reflect both static and dynamic Call capabilities.

See Also:

[Provider](#), [Call](#), [CallCapabilities](#)

Method Summary

boolean	canAddParty() Returns true if the application can invoke the add party feature, false otherwise.
boolean	canConference() Returns true if the application can invoke the conference feature, false otherwise.
boolean	canConsult() Deprecated. <i>Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the canConsult(TerminalConnection, String) method.</i>
boolean	canConsult(TerminalConnection tc) Returns true if the application can invoke the overloaded consult feature which takes a TerminalConnection as an argument, false otherwise.
boolean	canConsult(TerminalConnection tc, java.lang.String destination) Returns true if the application can invoke the overloaded consult feature which takes a TerminalConnection and string as arguments, false otherwise.
boolean	canDrop() Returns true if the application can invoke the drop feature, false otherwise.
boolean	canOffHook() Returns true if the application can invoke the off hook feature, false otherwise.

boolean	<u>canSetConferenceController()</u> Returns true if the application can set the conference controller, false otherwise.
boolean	<u>canSetConferenceEnable()</u> Returns true if the application can invoke the set conferencing enabling feature, false otherwise.
boolean	<u>canSetTransferController()</u> Returns true if the application can set the transfer controller, false otherwise.
boolean	<u>canSetTransferEnable()</u> Returns true if the application can invoke the set transferring enabling feature, false otherwise.
boolean	<u>canTransfer()</u> Deprecated. Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the <i>canTransfer(Call)</i> method.
boolean	<u>canTransfer(Call call)</u> Returns true if the application can invoke the overloaded transfer feature which takes a Call as an argument, false otherwise.
boolean	<u>canTransfer(java.lang.String destination)</u> Returns true if the application can invoke the overloaded transfer feature which takes a destination string as an argument, false otherwise.

Methods inherited from interface javax.telephony.capabilities.CallCapabilities

canConnect, isObservable

Method Detail

canDrop

```
public boolean canDrop()
```

Returns true if the application can invoke the drop feature, false otherwise.

Returns:

True if the application can invoke the drop feature, false otherwise.

canOffHook

public boolean **canOffHook()**

Returns true if the application can invoke the off hook feature, false otherwise.

Returns:

true if the application can invoke the off hook feature, false otherwise.

canSetConferenceController

public boolean **canSetConferenceController()**

Returns true if the application can set the conference controller, false otherwise.

Returns:

true if the application can set the conference controller, false otherwise.

canSetTransferController

public boolean **canSetTransferController()**

Returns true if the application can set the transfer controller, false otherwise.

Returns:

true if the application can set the transfer controller, false otherwise.

canSetTransferEnable

public boolean **canSetTransferEnable()**

Returns true if the application can invoke the set transferring enabling feature, false otherwise. The value returned by this method is independent of the ability of the application to invoke the transfer feature.

Applications are not required to inform the implementation of the purpose of the consultation call and may rely upon the default value returned by the `CallControlCall.getTransferEnable()` method.

Returns:

True if the application can invoke the set transferring enabling feature, false otherwise.

canSetConferenceEnable

public boolean **canSetConferenceEnable**()

Returns true if the application can invoke the set conferencing enabling feature, false otherwise. The value returned by this method is independent of the ability of the application to invoke the conference feature.

Applications are not required to inform the implementation of the purpose of the consultation call and may rely upon the default value returned by the `CallControlCall.getConferenceEnable()` method.

Returns:

True if the application can invoke the set conferencing enabling feature, false otherwise.

canTransfer

public boolean **canTransfer**()

Deprecated. *Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the `canTransfer(Call)` method.*

Returns true if the application can invoke the transfer feature, false otherwise.

Note: This method has been replaced in JTAPI 1.2 with overloaded versions. These versions permit applications to give typed argument to obtain the capabilities for a particular overloaded version of the `CallControlCall.transfer()` method.

Returns:

True if the application can invoke the transfer feature, false otherwise.

canTransfer

public boolean **canTransfer**(Call call)

Returns true if the application can invoke the overloaded transfer feature which takes a `Call` as an argument, false otherwise.

The argument provided is for typing purposes only. The particular instance of the object given is ignored and not used to determine the capability outcome is any way.

Parameters:

`call` - This argument is used for typing information to determine the overloaded version of the `transfer()` method.

Returns:

True if the application can invoke the transfer feature which takes a `Call` as an argument, false otherwise.

Since:
JTAPI v1.2

canTransfer

public boolean **canTransfer**(java.lang.String destination)

Returns true if the application can invoke the overloaded transfer feature which takes a destination string as an argument, false otherwise.

The argument provided is for typing purposes only. The particular instance of the object given is ignored and not used to determine the capability outcome is any way.

Parameters:
 destination - This argument is used for typing information to determine the overloaded version of the transfer() method.

Returns:
 True if the application can invoke the transfer feature which takes a destination string as an argument, false otherwise.

Since:
JTAPI v1.2

canConference

public boolean **canConference**()

Returns true if the application can invoke the conference feature, false otherwise.

Returns:
 True if the application can invoke the conference feature, false otherwise.

canAddParty

public boolean **canAddParty**()

Returns true if the application can invoke the add party feature, false otherwise.

Returns:
 True if the application can invoke the add party feature, false otherwise.

canConsult

```
public boolean canConsult()
```

Deprecated. *Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the `canConsult(TerminalConnection, String)` method.*

Returns true if the application can invoke the consult feature, false otherwise.

Note: This method has been replaced in JTAPI 1.2 with overloaded versions. These versions permit applications to give typed argument to obtain the capabilities for a particular overloaded version of the `CallControlCall.consult()` method.

Returns:

True if the application can invoke the consult feature, false otherwise.

canConsult

```
public boolean canConsult(TerminalConnection tc,  
                          java.lang.String destination)
```

Returns true if the application can invoke the overloaded consult feature which takes a `TerminalConnection` and string as arguments, false otherwise.

The arguments provided are for typing purposes only. The particular instances of the objects given are ignored and not used to determine the capability outcome is any way.

Parameters:

`tc` - This argument is used for typing information to determine the overloaded version of the `consult()` method.

`destination` - This argument is used for typing information to destination the overloaded version of the `consult()` method.

Returns:

True if the application can invoke the consult feature which takes a `TerminalConnection` and a string as arguments.

Since:

JTAPI v1.2

canConsult

```
public boolean canConsult(TerminalConnection tc)
```

Returns true if the application can invoke the overloaded consult feature which takes a `TerminalConnection` as an argument, false otherwise.

The arguments provided are for typing purposes only. The particular instances of the objects given are ignored and not used to determine the capability outcome is any way.

Parameters:

`tc` - This argument is used for typing information to determine the overloaded version of the `consult()` method.

Returns:

True if the application can invoke the consult feature which takes a `TerminalConnection` as an argument.

Since:

JTAPI v1.2

javax.telephony.callcontrol.capabilities **Interface CallControlConnectionCapabilities**

public interface **CallControlConnectionCapabilities**
extends ConnectionCapabilities

The `CallControlConnectionCapabilities` interface extends the core `ConnectionCapabilities` interface. This interface provides methods to reflect the capabilities of the methods on the

Method Summary

boolean	<u>canAccept</u> () Returns true if the application can invoke the accept feature, false otherwise.
boolean	<u>canAddToAddress</u> () Returns true if the application can invoke the add to address feature, false otherwise.
boolean	<u>canPark</u> () Returns true if the application can invoke the park feature, false otherwise.
boolean	<u>canRedirect</u> () Returns true if the application can invoke the redirect feature, false otherwise.
boolean	<u>canReject</u> () Returns true if the application can invoke the reject feature, false otherwise.

Methods inherited from interface javax.telephony.capabilities.ConnectionCapabilities

canDisconnect

Method Detail

canRedirect

public boolean **canRedirect**()

Returns true if the application can invoke the redirect feature, false otherwise.

Returns:

True if the application can invoke the redirect feature, false otherwise.

canAddToAddress

public boolean **canAddToAddress**()

Returns true if the application can invoke the add to address feature, false otherwise.

Returns:

True if the application can invoke the add to address feature, false otherwise.

canAccept

public boolean **canAccept**()

Returns true if the application can invoke the accept feature, false otherwise.

Returns:

True if the application can invoke the accept feature, false otherwise.

canReject

public boolean **canReject**()

Returns true if the application can invoke the reject feature, false otherwise.

Returns:

True if the application can invoke the reject feature, false otherwise.

canPark

public boolean **canPark**()

Returns true if the application can invoke the park feature, false otherwise.

Returns:

True if the application can invoke the park feature, false otherwise.

javax.telephony.callcontrol.capabilities
Interface CallControlTerminalCapabilities

public interface **CallControlTerminalCapabilities**
extends TerminalCapabilities

The `CallControlTerminalCapabilities` interface extends the core `TerminalCapabilities` interface. This interface provides methods to reflect the capabilities of the methods on the `CallControlTerminal` interface.

The `Provider.getTerminalCapabilities()` method returns the static Terminal capabilities, and the `Terminal.getCapabilities()` method returns the dynamic Terminal capabilities. The object returned from each of these methods can be queried with the `instanceof` operator to check if it supports this interface. This same interface is used to reflect both static and dynamic Terminal capabilities.

See Also:

Provider, Terminal, TerminalCapabilities

Method Summary

boolean	<u>canGetDoNotDisturb()</u> Returns true if the application can obtain the do not disturb state, false otherwise.
boolean	<u>canPickup()</u> Deprecated. Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the <i>canPickup(Connection, Address)</i> method.
boolean	<u>canPickup(Address address1, Address address2)</u> Returns true if the application can invoke the overloaded pickup feature which takes two Addresses as arguments, false otherwise.
boolean	<u>canPickup(Connection connection, Address address)</u> Returns true if the application can invoke the overloaded pickup feature which takes a Connection and an Address as arguments, false otherwise.
boolean	<u>canPickup(TerminalConnection tc, Address address)</u> Returns true if the application can invoke the overloaded pickup feature which takes a TerminalConnection and an Address as arguments, false otherwise.
boolean	<u>canPickupFromGroup()</u> Deprecated. Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the <i>canPickupFromGroup(String, Address)</i> method.
boolean	<u>canPickupFromGroup(Address address)</u> Returns true if the application can invoke the pickup from group feature which takes an Address as an argument, false otherwise.
boolean	<u>canPickupFromGroup(java.lang.String group, Address address)</u> Returns true if the application can invoke the pickup from group feature which takes a string pickup group code and an Address as arguments, false otherwise.
boolean	<u>canSetDoNotDisturb()</u> Returns true if the application can set the do not disturb state, false otherwise.

Methods inherited from interface `javax.telephony.capabilities.TerminalCapabilities`

isObservable

Method Detail

canGetDoNotDisturb

public boolean **canGetDoNotDisturb**()

Returns true if the application can obtain the do not disturb state, false otherwise.

Returns:

True if the application can obtain the do not disturb state, false otherwise.

canSetDoNotDisturb

public boolean **canSetDoNotDisturb**()

Returns true if the application can set the do not disturb state, false otherwise.

Returns:

True if the application can set the do not disturb state, false otherwise.

canPickup

public boolean **canPickup**()

Deprecated. Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the *canPickup(Connection, Address)* method.

Returns true if the application can invoke the pickup feature, false otherwise.

Note: This method has been replaced in JTAPI 1.2 with overloaded versions. These versions permit applications to give typed argument to obtain the capabilities for a particular overloaded version of the `CallControlTerminal.pickup()` method.

Returns:

True if the application can invoke the pickup feature, false otherwise.

canPickup

public boolean **canPickup**(Connection connection,
 Address address)

Returns true if the application can invoke the overloaded pickup feature which takes a `Connection` and an `Address` as arguments, false otherwise.

The arguments provided are for typing purposes only. The particular instances of the objects given are ignored and not used to determine the capability outcome is any way.

Parameters:

connection - This argument is used for typing information to determine the overloaded version of the pickup() method.

address - This argument is used for typing information to determine the overloaded version of the pickup() method.

Returns:

True if the application can invoke the pickup feature which takes a Connection and an Address as arguments, false otherwise.

Since:

JTAPI v1.2

canPickup

```
public boolean canPickup(TerminalConnection tc,
                        Address address)
```

Returns true if the application can invoke the overloaded pickup feature which takes a TerminalConnection and an Address as arguments, false otherwise.

The arguments provided are for typing purposes only. The particular instances of the objects given are ignored and not used to determine the capability outcome is any way.

Parameters:

tc - This argument is used for typing information to determine the overloaded version of the pickup() method.

address - This argument is used for typing information to determine the overloaded version of the pickup() method.

Returns:

True if the application can invoke the pickup feature which takes a TerminalConnection and an Address as arguments, false otherwise.

Since:

JTAPI v1.2

canPickup

```
public boolean canPickup(Address address1,
                        Address address2)
```

Returns true if the application can invoke the overloaded pickup feature which takes two Addresses as arguments, false otherwise.

The arguments provided are for typing purposes only. The particular instances of the objects given are ignored and not used to determine the capability outcome is any way.

Parameters:

address1 - This argument is used for typing information to determine the overloaded version of the pickup() method.

address2 - This argument is used for typing information to determine the overloaded version of the pickup() method.

Returns:

True if the application can invoke the pickup feature which takes two Addresses as arguments, false otherwise.

Since:

JTAPI v1.2

canPickupFromGroup

```
public boolean canPickupFromGroup()
```

Deprecated. *Since JTAPI v1.2. The default behavior of this method in JTAPI v1.2 and later should invoke the canPickupFromGroup(String, Address) method.*

Returns true if the application can invoke the pickup from group feature, false otherwise.

Note: This method has been replaced in JTAPI 1.2 with overloaded versions. These versions permit applications to give typed argument to obtain the capabilities for a particular overloaded version of the `CallControlTerminal.pickupFromGroup()` method.

Returns:

True if the application can invoke the pickup from group feature, false otherwise.

canPickupFromGroup

```
public boolean canPickupFromGroup(java.lang.String group,
                                   Address address)
```

Returns true if the application can invoke the pickup from group feature which takes a string pickup group code and an Address as arguments, false otherwise.

The arguments provided are for typing purposes only. The particular instances of the objects given are ignored and not used to determine the capability outcome is any way.

Parameters:

group - This argument is used for typing information to determine the overloaded version of the pickupFromGroup() method.

address - This argument is used for typing information to determine the overloaded version of the pickupFromGroup() method.

Returns:

True if the application can invoke the pickup from group feature which takes a string pickup group code and an Address as arguments, false otherwise.

Since:JTAPI v1.2

canPickupFromGroup

```
public boolean canPickupFromGroup(Address address)
```

Returns true if the application can invoke the pickup from group feature which takes an Address as an argument, false otherwise.

The arguments provided are for typing purposes only. The particular instances of the objects given are ignored and not used to determine the capability outcome in any way.

Parameters:

address - This argument is used for typing information to determine the overloaded version of the pickupFromGroup() method.

Returns:

True if the application can invoke the pickup from group feature which takes an Address as an argument, false otherwise.

Since:JTAPI v1.2

javax.telephony.callcontrol.capabilities

Interface CallControlTerminalConnectionCapabilities

public interface **CallControlTerminalConnectionCapabilities**
extends [TerminalConnectionCapabilities](#)

The `CallControlTerminalConnectionCapabilities` interface extends the core `TerminalConnectionCapabilities` interface. This interface provides methods to reflect the capabilities of the methods on the

Method Summary

boolean	canHold() Returns true if the application can invoke the hold feature, false otherwise.
boolean	canJoin() Returns true if the application can invoke the join feature, false otherwise.
boolean	canLeave() Returns true if the application can invoke the leave feature, false otherwise.
boolean	canUnhold() Returns true if the application can invoke the unhold feature, false otherwise.

Methods inherited from interface `javax.telephony.capabilities.TerminalConnectionCapabilities`

[canAnswer](#)

Method Detail

canHold

public boolean **canHold()**

Returns true if the application can invoke the hold feature, false otherwise.

Returns:

True if the application can invoke the hold feature, false otherwise.

canUnhold

public boolean **canUnhold()**

Returns true if the application can invoke the unhold feature, false otherwise.

Returns:

True if the application can invoke the unhold feature, false otherwise.

canJoin

public boolean **canJoin()**

Returns true if the application can invoke the join feature, false otherwise.

Returns:

True if the application can invoke the join feature, false otherwise.

canLeave

public boolean **canLeave()**

Returns true if the application can invoke the leave feature, false otherwise.

Returns:

True if the application can invoke the leave feature, false otherwise.

Package javax.telephony.callcontrol.events

The JTAPI Call Control Events package defines the specific Call Control state transition events.

See:

Description

Interface Summary	
<u><i>CallCtlAddrDoNotDisturbEv</i></u>	The <code>CallCtlAddrDoNotDisturbEv</code> interface indicates the state of the do not disturb feature has changed for the Address.
<u><i>CallCtlAddrEv</i></u>	The <code>CallCtlAddrEv</code> interface is the base interface for all call control package Address-related events.
<u><i>CallCtlAddrForwardEv</i></u>	The <code>CallCtlAddrForwardEv</code> interface indicates the state of the forward feature has changed for the Address.
<u><i>CallCtlAddrMessageWaitingEv</i></u>	The <code>CallCtlAddrMessageWaitingEv</code> interface indicates the state of the message waiting feature has changed for the Address.
<u><i>CallCtlCallEv</i></u>	The <code>CallCtlCallEv</code> interface is the base interface for all call control package Call-related events.
<u><i>CallCtlConnAlertingEv</i></u>	The <code>CallCtlConnAlertingEv</code> interface indicates that the call control package state of the Connection is now <code>CallControlConnection.ALERTING</code> .
<u><i>CallCtlConnDialingEv</i></u>	The <code>CallCtlConnDialingEv</code> interface indicates that the call control package state of the Connection is now <code>CallControlConnection.DIALING</code> .
<u><i>CallCtlConnDisconnectedEv</i></u>	The <code>CallCtlConnDisconnectedEv</code> interface indicates that the call control package state of the Connection is now <code>CallControlConnection.DISCONNECTED</code> .
<u><i>CallCtlConnEstablishedEv</i></u>	The <code>CallCtlConnEstablishedEv</code> interface indicates that the call control package state of the Connection is now <code>CallControlConnection.ESTABLISHED</code> .
<u><i>CallCtlConnEv</i></u>	The <code>CallCtlConnEv</code> interface is the base interface for all call control package Connection-related events.
<u><i>CallCtlConnFailedEv</i></u>	The <code>CallCtlConnFailedEv</code> interface indicates that the call control package state of the Connection is now <code>CallControlConnection.FAILED</code> .
<u><i>CallCtlConnInitiatedEv</i></u>	The <code>CallCtlConnInitiatedEv</code> interface indicates that the call control package state of the Connection is now <code>CallControlConnection.INITIATED</code> .

<u>CallCtlConnNetworkAlertingEv</u>	The CallCtlConnNetworkAlertingEv interface indicates that the call control package state of the Connection is now CallControlConnection.NETWORK_ALERTING.
<u>CallCtlConnNetworkReachedEv</u>	The CallCtlConnNetworkReachedEv interface indicates that the call control package state of the Connection is now CallControlConnection.NETWORK_REACHED.
<u>CallCtlConnOfferedEv</u>	The CallCtlConnOfferedEv interface indicates that the call control package state of the Connection is now CallControlConnection.OFFERING.
<u>CallCtlConnQueuedEv</u>	The CallCtlConnQueuedEv interface indicates that the call control package state of the Connection is now CallControlConnection.QUEUED.
<u>CallCtlConnUnknownEv</u>	The CallCtlConnUnknownEv interface indicates that the call control package state of the Connection is now CallControlConnection.UNKNOWN.
<u>CallCtlEv</u>	The CallCtlEv is the base event for all events in the call control package.
<u>CallCtlTermConnBridgedEv</u>	The CallCtlTermConnBridgedEv interface indicates that the call control package state of the TerminalConnection is now CallControlTerminalConnection.BRIDGED.
<u>CallCtlTermConnDroppedEv</u>	The CallCtlTermConnDroppedEv interface indicates that the call control package state of the TerminalConnection is now CallControlTerminalConnection.DROPPED.
<u>CallCtlTermConnEv</u>	The CallCtlTermConnEv interface is the base interface for all call control package TerminalConnection-related events.
<u>CallCtlTermConnHeldEv</u>	The CallCtlTermConnHeldEv interface indicates that the call control package state of the TerminalConnection is now CallControlTerminalConnection.HELD.
<u>CallCtlTermConnInUseEv</u>	The CallCtlTermConnInUseEv interface indicates that the call control package state of the TerminalConnection is now CallControlTerminalConnection.INUSE.
<u>CallCtlTermConnRingingEv</u>	The CallCtlTermConnRingingEv interface indicates that the call control package state of the TerminalConnection is now CallControlTerminalConnection.RINGING.
<u>CallCtlTermConnTalkingEv</u>	The CallCtlTermConnTalkingEv interface indicates that the call control package state of the TerminalConnection is now CallControlTerminalConnection.TALKING.
<u>CallCtlTermConnUnknownEv</u>	The CallCtlTermConnUnknownEv interface indicates that the call control package state of the TerminalConnection is now CallControlTerminalConnection.UNKNOWN.

<u>CallCtlTermDoNotDisturbEv</u>	The <code>CallCtlTermDoNotDisturbEv</code> interface indicates the state of the do not disturb feature has changed for the Terminal.
<u>CallCtlTermEv</u>	The <code>CallCtlTermEv</code> interface is the base interface for all call control package Terminal-related events.

Package javax.telephony.callcontrol.events Description

The JTAPI Call Control Events package defines the specific Call Control state transition events.

Hierarchy For Package javax.telephony.callcontrol.events

Package Hierarchies:

[All Packages](#)

Interface Hierarchy

- interface javax.telephony.events.[Ev](#)
 - interface javax.telephony.events.[AddrEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlAddrEv](#)(also extends javax.telephony.callcontrol.events.[CallCtlEv](#))
 - interface javax.telephony.callcontrol.events.[CallCtlAddrDoNotDisturbEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlAddrForwardEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlAddrMessageWaitingEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlAddrEv](#)(also extends javax.telephony.events.[AddrEv](#))
 - interface javax.telephony.callcontrol.events.[CallCtlAddrDoNotDisturbEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlAddrForwardEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlAddrMessageWaitingEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlCallEv](#)(also extends javax.telephony.events.[CallEv](#))
 - interface javax.telephony.callcontrol.events.[CallCtlConnEv](#)(also extends javax.telephony.events.[ConnEv](#))
 - interface javax.telephony.callcontrol.events.[CallCtlConnAlertingEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnDialingEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnDisconnectedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnEstablishedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnFailedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnInitiatedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnNetworkAlertingEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnNetworkReachedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnOfferedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnQueuedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlConnUnknownEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlTermConnEv](#)(also extends javax.telephony.events.[TermConnEv](#))
 - interface javax.telephony.callcontrol.events.[CallCtlTermConnBridgedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlTermConnDroppedEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlTermConnHeldEv](#)
 - interface javax.telephony.callcontrol.events.[CallCtlTermConnInUseEv](#)

- interface javax.telephony.callcontrol.events.**CallCtlTermConnRingingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnTalkingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnUnknownEv**
- interface javax.telephony.callcontrol.events.**CallCtlTermEv**(also extends javax.telephony.events.**TermEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermDoNotDisturbEv**
- interface javax.telephony.events.**CallEv**
 - interface javax.telephony.callcontrol.events.**CallCtlCallEv**(also extends javax.telephony.callcontrol.events.**CallCtlEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnEv**(also extends javax.telephony.events.**ConnEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDialingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDisconnectedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnEstablishedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnFailedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnInitiatedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkReachedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnOfferedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnQueuedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnUnknownEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnEv**(also extends javax.telephony.events.**TermConnEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnBridgedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnDroppedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnHeldEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnInUseEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnRingingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnTalkingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnUnknownEv**
 - interface javax.telephony.events.**ConnEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnEv**(also extends javax.telephony.callcontrol.events.**CallCtlCallEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDialingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDisconnectedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnEstablishedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnFailedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnInitiatedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkReachedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnOfferedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnQueuedEv**

- interface javax.telephony.callcontrol.events.**CallCtlConnUnknownEv**
 - interface javax.telephony.events.**TermConnEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnEv**(also extends javax.telephony.callcontrol.events.**CallCtlCallEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnBridgedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnDroppedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnHeldEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnInUseEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnRingingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnTalkingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnUnknownEv**
 - interface javax.telephony.events.**TermEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermEv**(also extends javax.telephony.callcontrol.events.**CallCtlEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermDoNotDisturbEv**
-
-

javax.telephony.callcontrol.events

Interface CallCtlAddrDoNotDisturbEv

public interface **CallCtlAddrDoNotDisturbEv**
extends [CallCtlAddrEv](#)

The `CallCtlAddrDoNotDisturbEv` interface indicates the state of the do not disturb feature has changed for the Address. This interface extends the `CallCtlAddrEv` interface and is reported via the `AddressObserver.addressChangedEvent()` method. The observer object must also implement the `CallControlAddressObserver` interface to signal it is interested in call control package events.

This interface supports a single method which returns the current state of the do not disturb feature.

See Also:

[Address](#), [AddressObserver](#), [CallControlAddress](#),
[CallControlAddressObserver](#), [CallCtlAddrEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Method Summary

boolean	<u>getDoNotDisturbState()</u> Returns true if the do not disturb feature is activated, false otherwise.
---------	---

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv

getCallControlCause

Methods inherited from interface javax.telephony.events.AddrEv

getAddress

Field Detail

ID

```
public static final int ID
```

Event id

Method Detail

getDoNotDisturbState

```
public boolean getDoNotDisturbState()
```

Returns true if the do not disturb feature is activated, false otherwise.

Returns:

True if the do not disturb feature is activated, false otherwise.

javax.telephony.callcontrol.events

Interface CallCtlAddrEv

All Known Subinterfaces:

[CallCtlAddrDoNotDisturbEv](#), [CallCtlAddrForwardEv](#), [CallCtlAddrMessageWaitingEv](#)

public interface **CallCtlAddrEv**
 extends [CallCtlEv](#), [AddrEv](#)

The `CallCtlAddrEv` interface is the base interface for all call control package Address-related events. All events which pertain to the `CallControlAddress` interface must extend this interface. Events which extend this interface are reported via the `AddressObserver.addressChangedEvent()` method. The observer object must also implement the `CallControlAddressObserver` interface to signal it is interested in call control package events. This interface extends both the `CallCtlEv` and `AddrEv` interfaces.

The events defined in the call control package for the Address are the `CallCtlAddrDoNotDisturbEv`, `CallCtlAddrForwardEv`, and `CallCtlAddrMessageWaitingEv` events.

This interface supports no additional methods.

See Also:

[Address](#), [AddressObserver](#), [AddrEv](#), [CallControlAddress](#),
[CallControlAddressObserver](#), [CallCtlAddrDoNotDisturbEv](#),
[CallCtlAddrForwardEv](#), [CallCtlAddrMessageWaitingEv](#), [CallCtlEv](#)

Fields inherited from interface javax.telephony.callcontrol.events. <u>CallCtlEv</u>
<u>CAUSE ALTERNATE</u> , <u>CAUSE BUSY</u> , <u>CAUSE CALL BACK</u> , <u>CAUSE CALL NOT ANSWERED</u> , <u>CAUSE CALL PICKUP</u> , <u>CAUSE CONFERENCE</u> , <u>CAUSE DO NOT DISTURB</u> , <u>CAUSE PARK</u> , <u>CAUSE REDIRECTED</u> , <u>CAUSE REORDER TONE</u> , <u>CAUSE TRANSFER</u> , <u>CAUSE TRUNKS BUSY</u> , <u>CAUSE UNHOLD</u>

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv

getCallControlCause

Methods inherited from interface javax.telephony.events.AddrEv

getAddress

javax.telephony.callcontrol.events

Interface CallCtlAddrForwardEv

public interface **CallCtlAddrForwardEv**
 extends [CallCtlAddrEv](#)

The `CallCtlAddrForwardEv` interface indicates the state of the forward feature has changed for the Address. This interface extends the `CallCtlAddrEv` interface and is reported via the `AddressObserver.addressChangedEvent()` method. The observer object must also implement the `CallControlAddressObserver` interface to signal it is interested in call control package events.

This interface supports a single method which returns the current state of the forwarding feature.

See Also:

[Address](#), [AddressObserver](#), [CallControlAddress](#),
[CallControlAddressObserver](#), [CallCtlAddrEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface [javax.telephony.callcontrol.events.CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Method Summary

<u>CallControlForwarding[]</u>	getForwarding() Returns the current forwarding on the Address.
--------------------------------	--

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv

getCallControlCause

Methods inherited from interface javax.telephony.events.AddrEv

getAddress

Field Detail

ID

```
public static final int ID
```

Event id

Method Detail

getForwarding

```
public CallControlForwarding[] getForwarding()
```

Returns the current forwarding on the Address.

Returns:

An array of CallControlForwarding objects.

javax.telephony.callcontrol.events***Interface CallCtlAddrMessageWaitingEv***

public interface **CallCtlAddrMessageWaitingEv**
extends [CallCtlAddrEv](#)

The `CallCtlAddrMessageWaitingEv` interface indicates the state of the message waiting feature has changed for the Address. This interface extends the `CallCtlAddrEv` interface and is reported via the `AddressObserver.addressChangedEvent()` method. The observer object must also implement the `CallControlAddressObserver` interface to signal it is interested in call control package events.

This interface supports a single method which returns the current state of the message waiting feature.

See Also:

[Address](#), [AddressObserver](#), [CallControlAddress](#),
[CallControlAddressObserver](#), [CallCtlAddrEv](#)

Field Summary	
static int	ID Event id

Fields inherited from interface javax.telephony.callcontrol.events. CallCtlEv
CAUSE ALTERNATE , CAUSE BUSY , CAUSE CALL BACK , CAUSE CALL NOT ANSWERED , CAUSE CALL PICKUP , CAUSE CONFERENCE , CAUSE DO NOT DISTURB , CAUSE PARK , CAUSE REDIRECTED , CAUSE REORDER TONE , CAUSE TRANSFER , CAUSE TRUNKS BUSY , CAUSE UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

boolean	<u>getMessageWaitingState()</u> Returns the current message waiting state of the Address.
---------	---

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv

getCallControlCause

Methods inherited from interface javax.telephony.events.AddrEv

getAddress

Field Detail

ID

```
public static final int ID
```

Event id

Method Detail

getMessageWaitingState

```
public boolean getMessageWaitingState()
```

Returns the current message waiting state of the Address.

Returns:

The message waiting state of the Address.

javax.telephony.callcontrol.events

Interface CallCtlCallEv

All Known Subinterfaces:

[CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#), [CallCtlConnDisconnectedEv](#),
[CallCtlConnEstablishedEv](#), [CallCtlConnEv](#), [CallCtlConnFailedEv](#), [CallCtlConnInitiatedEv](#),
[CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#), [CallCtlConnOfferedEv](#),
[CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#), [CallCtlTermConnBridgedEv](#),
[CallCtlTermConnDroppedEv](#), [CallCtlTermConnEv](#), [CallCtlTermConnHeldEv](#),
[CallCtlTermConnInUseEv](#), [CallCtlTermConnRingingEv](#), [CallCtlTermConnTalkingEv](#),
[CallCtlTermConnUnknownEv](#)

public interface **CallCtlCallEv**
 extends [CallCtlEv](#), [CallEv](#)

The `CallCtlCallEv` interface is the base interface for all call control package Call-related events. All events which pertain to the `CallControlCall` interface must extend this interface. Events which extend this interface are reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events. This interface extend both the `CallCtlEv` and core `CallEv` interfaces.

The `CallCtlConnEv` and `CallCtlTermConnEv` events extend this interface. This reflects the fact that all events pertaining to the `CallControlConnection` interface and the `CallControlTerminalConnection` interface are reported via the `CallControlCallObserver` interface.

Additional Call Information

This interface supports methods which return additional information regarding the telephone call. Specifically, it returns the *calling address*, *calling terminal*, *called address*, and *last redirected address* information. This information is returned by the `getCallingAddress()`, `getCallingTerminal()`, `getCalledAddress()`, and `getLastRedirectedAddress()` methods on this interface, respectively.

See Also:

[Call](#), [Address](#), [Terminal](#), [CallObserver](#), [CallEv](#), [CallControlCall](#),
[CallControlCallObserver](#), [CallCtlEv](#), [CallCtlConnEv](#), [CallCtlTermConnEv](#)

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE ALTERNATE, CAUSE BUSY, CAUSE CALL BACK, CAUSE CALL NOT ANSWERED,
CAUSE CALL PICKUP, CAUSE CONFERENCE, CAUSE DO NOT DISTURB, CAUSE PARK,
CAUSE REDIRECTED, CAUSE REORDER TONE, CAUSE TRANSFER,
CAUSE TRUNKS BUSY, CAUSE UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Method Summary

<u>Address</u>	<u>getCalledAddress()</u> Returns the called Address associated with this Call.
<u>Address</u>	<u>getCallingAddress()</u> Returns the calling Address associated with this call.
<u>Terminal</u>	<u>getCallingTerminal()</u> Returns the calling Terminal associated with this Call.
<u>Address</u>	<u>getLastRedirectedAddress()</u> Returns the last redirected Address associated with this Call.

Methods inherited from interface <code>javax.telephony.callcontrol.events.CallCtlEv</code>

<code>getCallControlCause</code>

Methods inherited from interface <code>javax.telephony.events.CallEv</code>
--

<code>getCall</code>

Method Detail

getCallingAddress

public Address **getCallingAddress()**

Returns the calling Address associated with this call. The calling Address is defined as the Address which placed the telephone call.

If the calling address is unknown or not yet known, this method returns null.

Returns:

The calling Address.

getCallingTerminal

public Terminal **getCallingTerminal()**

Returns the calling Terminal associated with this Call. The calling Terminal is defined as the Terminal which placed the telephone call.

If the calling Terminal is unknown or not yet know, this method returns null.

Returns:

The calling Terminal.

getCalledAddress

public Address **getCalledAddress()**

Returns the called Address associated with this Call. The called Address is defined as the Address to which the call has been originally placed.

If the called address is unknown or not yet known, this method returns null.

Returns:

The called Address.

getLastRedirectedAddress

public Address **getLastRedirectedAddress()**

Returns the last redirected Address associated with this Call. The last redirected Address is the Address at which the current telephone call was placed immediately before the current Address. This is common if a Call is forwarded to several Addresses before being answered.

If the the last redirected address is unknown or not yet known, this method returns null.

Returns:

The last redirected Address for this telephone Call.

javax.telephony.callcontrol.events

Interface CallCtlConnAlertingEv

public interface **CallCtlConnAlertingEv**
 extends [CallCtlConnEv](#)

The `CallCtlConnAlertingEv` interface indicates that the call control package state of the Connection is now `CallControlConnection.ALERTING`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEvgetConnection**Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv**getCallControlCause**Field Detail***ID*

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlConnDialingEv

public interface **CallCtlConnDialingEv**
extends [CallCtlConnEv](#)

The `CallCtlConnDialingEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.DIALING`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports the `getDigits()` method which returns the digits which have already been dialed.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface `javax.telephony.callcontrol.events.CallCtlEv`

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Method Summary

java.lang.String	getDigits() Returns the digits that have already been dialed.
------------------	---

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEv

getConnection

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv

getCallControlCause

Field Detail*ID*

```
public static final int ID
```

Event id

Method Detail*getDigits*

```
public java.lang.String getDigits()
```

Returns the digits that have already been dialed.

Returns:

The digits that have already been dialed.

javax.telephony.callcontrol.events***Interface CallCtlConnDisconnectedEv***

public interface **CallCtlConnDisconnectedEv**
 extends [CallCtlConnEv](#)

The `CallCtlConnDisconnectedEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.DISCONNECTED`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface [javax.telephony.callcontrol.events.CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEvgetConnection**Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv**getCallControlCause**Field Detail***ID*

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlConnEstablishedEv

public interface **CallCtlConnEstablishedEv**
 extends [CallCtlConnEv](#)

The `CallCtlConnEstablishedEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.ESTABLISHED`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEv

<u>getConnection</u>

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv
--

<u>getCallControlCause</u>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlConnEv

All Known Subinterfaces:

[CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#), [CallCtlConnDisconnectedEv](#),
[CallCtlConnEstablishedEv](#), [CallCtlConnFailedEv](#), [CallCtlConnInitiatedEv](#),
[CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#), [CallCtlConnOfferedEv](#),
[CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#)

public interface **CallCtlConnEv**
 extends [CallCtlCallEv](#), [ConnEv](#)

The `CallCtlConnEv` interface is the base interface for all call control package Connection-related events. All events which pertain to the `CallControlConnection` interface must extend this interface. Events which extend this interface are reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events. This interface extends both the `CallCtlCallEv` and core `ConnEv` interfaces.

A number of event interfaces defined in this package extend this interface. Each of these events conveys a change in the call control package state of the Connection. The event interfaces which extend this interface are: `CallCtlConnAlertingEv`, `CallCtlConnDialingEv`, `CallCtlConnDisconnectedEv`, `CallCtlConnEstablishedEv`, `CallCtlConnFailedEv`, `CallCtlConnInitiatedEv`, `CallCtlConnNetworkAlertingEv`, `CallCtlConnNetworkReachedEv`, `CODE>CallCtlConnOfferedEv`, `CallCtlConnQueuedEv`, and `CallCtlConnUnknownEv`

This interface supports no additional methods.

See Also:

[Connection](#), [CallObserver](#), [ConnEv](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlCallEv](#), [CallCtlConnAlertingEv](#),
[CallCtlConnDialingEv](#), [CallCtlConnDisconnectedEv](#),
[CallCtlConnEstablishedEv](#), [CallCtlConnFailedEv](#), [CallCtlConnInitiatedEv](#),
[CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#),
[CallCtlConnOfferedEv](#), [CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#)

Fields inherited from interface javax.telephony.callcontrol.events. <u>CallCtlEv</u>
<u>CAUSE ALTERNATE</u> , <u>CAUSE BUSY</u> , <u>CAUSE CALL BACK</u> , <u>CAUSE CALL NOT ANSWERED</u> , <u>CAUSE CALL PICKUP</u> , <u>CAUSE CONFERENCE</u> , <u>CAUSE DO NOT DISTURB</u> , <u>CAUSE PARK</u> , <u>CAUSE REDIRECTED</u> , <u>CAUSE REORDER TONE</u> , <u>CAUSE TRANSFER</u> , <u>CAUSE TRUNKS BUSY</u> , <u>CAUSE UNHOLD</u>

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.<u>ConnEv</u>
--

<u>getConnection</u>

Methods inherited from interface javax.telephony.callcontrol.events.<u>CallCtlEv</u>

<u>getCallControlCause</u>

javax.telephony.callcontrol.events

Interface CallCtlConnFailedEv

public interface **CallCtlConnFailedEv**
 extends [CallCtlConnEv](#)

The `CallCtlConnFailedEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.FAILED`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEvgetConnection**Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv**getCallControlCause**Field Detail***ID*

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlConnInitiatedEv

public interface **CallCtlConnInitiatedEv**
 extends CallCtlConnEv

The `CallCtlConnInitiatedEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.INITIATED`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

CallObserver, Connection, CallControlConnection,
CallControlCallObserver, CallCtlConnEv

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE_ALTERNATE, CAUSE_BUSY, CAUSE_CALL_BACK, CAUSE_CALL_NOT_ANSWERED,
CAUSE_CALL_PICKUP, CAUSE_CONFERENCE, CAUSE_DO_NOT_DISTURB, CAUSE_PARK,
CAUSE_REDIRECTED, CAUSE_REORDER_TONE, CAUSE_TRANSFER,
CAUSE_TRUNKS_BUSY, CAUSE_UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEv

<u>getConnection</u>

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv
--

<u>getCallControlCause</u>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events***Interface CallCtlConnNetworkAlertingEv***

public interface **CallCtlConnNetworkAlertingEv**
 extends [CallCtlConnEv](#)

The **CallCtlConnNetworkAlertingEv** interface indicates that the call control package state of the Connection is now **CallControlConnection.NETWORK_ALERTING**. This method **CallControlConnection.getCallControlState()** returns the current state. This interface extends the **CallCtlConnEv** interface and is reported via the **CallObserver.callChangedEvent()** method. The observer object must also implement the **CallControlCallObserver** interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEvgetConnection**Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv**getCallControlCause**Field Detail***ID*

```
public static final int ID
```

Event id

*javax.telephony.callcontrol.events****Interface CallCtlConnNetworkReachedEv***

public interface **CallCtlConnNetworkReachedEv**
 extends [CallCtlConnEv](#)

The `CallCtlConnNetworkReachedEv` interface indicates that the call control package state of the Connection is now `CallControlConnection.NETWORK_REACHED`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEvgetConnection**Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv**getCallControlCause**Field Detail***ID*

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlConnOfferedEv

public interface **CallCtlConnOfferedEv**
 extends CallCtlConnEv

The `CallCtlConnOfferedEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.OFFERING`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

CallObserver, Connection, CallControlConnection,
CallControlCallObserver, CallCtlConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE_ALTERNATE, CAUSE_BUSY, CAUSE_CALL_BACK, CAUSE_CALL_NOT_ANSWERED,
CAUSE_CALL_PICKUP, CAUSE_CONFERENCE, CAUSE_DO_NOT_DISTURB, CAUSE_PARK,
CAUSE_REDIRECTED, CAUSE_REORDER_TONE, CAUSE_TRANSFER,
CAUSE_TRUNKS_BUSY, CAUSE_UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEvgetConnection**Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv**getCallControlCause**Field Detail***ID*

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlConnQueuedEv

public interface **CallCtlConnQueuedEv**
 extends CallCtlConnEv

The `CallCtlConnQueuedEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.QUEUED`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports the `getNumberInQueue()` method which returns the number of `Connections` being queue along with this `Connection` at the same `Address`.

See Also:

CallObserver, Connection, CallControlConnection,
CallControlCallObserver, CallCtlConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE ALTERNATE, CAUSE BUSY, CAUSE CALL BACK, CAUSE CALL NOT ANSWERED,
CAUSE CALL PICKUP, CAUSE CONFERENCE, CAUSE DO NOT DISTURB, CAUSE PARK,
CAUSE REDIRECTED, CAUSE REORDER TONE, CAUSE TRANSFER,
CAUSE TRUNKS BUSY, CAUSE UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getNumberInQueue()</u> Returns the number of Connections which are queued at the Address of this Connection.
-----	--

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEv

getConnection

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv

getCallControlCause

Field Detail*ID*

```
public static final int ID
```

Event id

Method Detail*getNumberInQueue*

```
public int getNumberInQueue()
```

Returns the number of Connections which are queued at the Address of this Connection.

Returns:

The number of queued Connections.

javax.telephony.callcontrol.events

Interface CallCtlConnUnknownEv

public interface **CallCtlConnUnknownEv**
 extends [CallCtlConnEv](#)

The `CallCtlConnUnknownEv` interface indicates that the call control package state of the `Connection` is now `CallControlConnection.UNKNOWN`. This method `CallControlConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [Connection](#), [CallControlConnection](#),
[CallControlCallObserver](#), [CallCtlConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.ConnEvgetConnection**Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv**getCallControlCause**Field Detail***ID*

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlEv

All Known Subinterfaces:

[CallCtlAddrDoNotDisturbEv](#), [CallCtlAddrEv](#), [CallCtlAddrForwardEv](#),
[CallCtlAddrMessageWaitingEv](#), [CallCtlCallEv](#), [CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#),
[CallCtlConnDisconnectedEv](#), [CallCtlConnEstablishedEv](#), [CallCtlConnEv](#), [CallCtlConnFailedEv](#),
[CallCtlConnInitiatedEv](#), [CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#),
[CallCtlConnOfferedEv](#), [CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#),
[CallCtlTermConnBridgedEv](#), [CallCtlTermConnDroppedEv](#), [CallCtlTermConnEv](#),
[CallCtlTermConnHeldEv](#), [CallCtlTermConnInUseEv](#), [CallCtlTermConnRingingEv](#),
[CallCtlTermConnTalkingEv](#), [CallCtlTermConnUnknownEv](#), [CallCtlTermDoNotDisturbEv](#),
[CallCtlTermEv](#)

public interface **CallCtlEv**

extends [Ev](#)

The `CallCtlEv` is the base event for all events in the call control package. Each event in this package must extend this interface. This interface extends the core `Ev` interface.

This interface contains the `getCallControlCause()` method which returns a call control package specific cause for the event. Cause codes pertaining to this package are defined in this interface as well.

In the call control package, this interface is extended by the following interfaces: `CallCtlCallEv`, `CallCtlAddrEv`, and `CallCtlTermEv`.

See Also:

[Ev](#), [CallCtlCallEv](#), [CallCtlAddrEv](#), [CallCtlTermEv](#)

Field Summary

static int	<u>CAUSE_ALTERNATE</u> Cause code indicating that the call was put on hold and another retrieved in an atomic operation, typically on single line telephones.
static int	<u>CAUSE_BUSY</u> Cause code indicating that the call encountered a busy endpoint.
static int	<u>CAUSE_CALL_BACK</u> Cause code indicating the event is related to the callback feature.
static int	<u>CAUSE_CALL_NOT_ANSWERED</u> Cause code indicating that the call was not answered before a timer elapsed.
static int	<u>CAUSE_CALL_PICKUP</u> Cause code indicating that the call was redirected by the call pickup feature.
static int	<u>CAUSE_CONFERENCE</u> Cause code indicating the event is related to the conference feature.
static int	<u>CAUSE_DO_NOT_DISTURB</u> Cause code indicating the event is related to the do not disturb feature.
static int	<u>CAUSE_PARK</u> Cause code indicating the event is related to the park feature.
static int	<u>CAUSE_REDIRECTED</u> Cause code indicating the event is related to the redirect feature.
static int	<u>CAUSE_REORDER_TONE</u> Cause code indicating that the call encountered a reorder tone.
static int	<u>CAUSE_TRANSFER</u> Cause code indicating the event is related to the transfer feature.
static int	<u>CAUSE_TRUNKS_BUSY</u> Cause code indicating that the call encountered the "all trunks busy" condition.
static int	<u>CAUSE_UNHOLD</u> Cause code indicating the event is related to the unhold feature.

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getCallControlCause()</u> Returns the call control package cause associated with the event.
-----	--

Methods inherited from interface javax.telephony.events.Ev

getCause, getID, getMetaCode, getObserved, isNewMetaEvent

Field Detail*CAUSE_ALTERNATE*

```
public static final int CAUSE_ALTERNATE
```

Cause code indicating that the call was put on hold and another retrieved in an atomic operation, typically on single line telephones.

CAUSE_BUSY

```
public static final int CAUSE_BUSY
```

Cause code indicating that the call encountered a busy endpoint.

CAUSE_CALL_BACK

public static final int **CAUSE_CALL_BACK**

Cause code indicating the event is related to the callback feature.

CAUSE_CALL_NOT_ANSWERED

public static final int **CAUSE_CALL_NOT_ANSWERED**

Cause code indicating that the call was not answered before a timer elapsed.

CAUSE_CALL_PICKUP

public static final int **CAUSE_CALL_PICKUP**

Cause code indicating that the call was redirected by the call pickup feature.

CAUSE_CONFERENCE

public static final int **CAUSE_CONFERENCE**

Cause code indicating the event is related to the conference feature.

CAUSE_DO_NOT_DISTURB

public static final int **CAUSE_DO_NOT_DISTURB**

Cause code indicating the event is related to the do not disturb feature.

CAUSE_PARK

public static final int **CAUSE_PARK**

Cause code indicating the event is related to the park feature.

CAUSE_REDIRECTED

public static final int **CAUSE_REDIRECTED**

Cause code indicating the event is related to the redirect feature.

CAUSE_REORDER_TONE

public static final int **CAUSE_REORDER_TONE**

Cause code indicating that the call encountered a reorder tone.

CAUSE_TRANSFER

public static final int **CAUSE_TRANSFER**

Cause code indicating the event is related to the transfer feature.

CAUSE_TRUNKS_BUSY

public static final int **CAUSE_TRUNKS_BUSY**

Cause code indicating that the call encountered the "all trunks busy" condition.

CAUSE_UNHOLD

public static final int **CAUSE_UNHOLD**

Cause code indicating the event is related to the unhold feature.

Method Detail

getCallControlCause

public int **getCallControlCause()**

Returns the call control package cause associated with the event. The cause values are integer constants defined in this interface. This method may also returns the `Ev.CAUSE_NORMAL` and the `Ev.CAUSE_UNKNOWN` values as defined in the core package.

Returns:

The call control package cause of the event.

javax.telephony.callcontrol.events

Interface CallCtlTermConnBridgedEv

public interface **CallCtlTermConnBridgedEv**
extends CallCtlTermConnEv

The `CallCtlTermConnBridgedEv` interface indicates that the call control package state of the `TerminalConnection` is now `CallControlTerminalConnection.BRIDGED`. This method `CallControlTerminalConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlTermConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

CallObserver, TerminalConnection, CallControlTerminalConnection,
CallControlCallObserver, CallCtlTermConnEv

Field Summary

static int	ID
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE_ALTERNATE, CAUSE_BUSY, CAUSE_CALL_BACK, CAUSE_CALL_NOT_ANSWERED,
CAUSE_CALL_PICKUP, CAUSE_CONFERENCE, CAUSE_DO_NOT_DISTURB, CAUSE_PARK,
CAUSE_REDIRECTED, CAUSE_REORDER_TONE, CAUSE_TRANSFER,
CAUSE_TRUNKS_BUSY, CAUSE_UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.TermConnEv

<u>getTerminalConnection</u>

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv
--

<u>getCallControlCause</u>

Field Detail

ID

```
public static final int ID
```

Event id

*javax.telephony.callcontrol.events****Interface CallCtlTermConnDroppedEv***

public interface **CallCtlTermConnDroppedEv**

extends [CallCtlTermConnEv](#)

The `CallCtlTermConnDroppedEv` interface indicates that the call control package state of the `TerminalConnection` is now `CallControlTerminalConnection.DROPPED`. This method `CallControlTerminalConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlTermConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [TerminalConnection](#), [CallControlTerminalConnection](#),
[CallControlCallObserver](#), [CallCtlTermConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface <code>javax.telephony.events.TermConnEv</code>
--

<code>getTerminalConnection</code>

Methods inherited from interface <code>javax.telephony.callcontrol.events.CallCtlEv</code>

<code>getCallControlCause</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlTermConnEv

All Known Subinterfaces:

[CallCtlTermConnBridgedEv](#), [CallCtlTermConnDroppedEv](#), [CallCtlTermConnHeldEv](#),
[CallCtlTermConnInUseEv](#), [CallCtlTermConnRingingEv](#), [CallCtlTermConnTalkingEv](#),
[CallCtlTermConnUnknownEv](#)

public interface **CallCtlTermConnEv**
 extends [CallCtlCallEv](#), [TermConnEv](#)

The `CallCtlTermConnEv` interface is the base interface for all call control package TerminalConnection-related events. All events which pertain to the `CallControlTerminalConnection` interface must extend this interface. Events which extend this interface are reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events. This interface extends both the `CallCtlCallEv` and core `TermConnEv` interfaces.

A number of event interfaces defined in this package extend this interface. Each of these events conveys a change in the call control package state of the TerminalConnection. The event interfaces which extend this interface are: `CallCtlTermConnBridgedEv`, `CallCtlTermConnDroppedEv`, `CallCtlTermConnHeldEv`, `CallCtlTermConnInUseEv`, `CallCtlTermConnRingingEv`, `CallCtlTermConnTalkingEv`, and `CallCtlTermConnUnknownEv`

This interface supports no additional methods.

See Also:

[TerminalConnection](#), [CallObserver](#), [TermConnEv](#),
[CallControlTerminalConnection](#), [CallControlCallObserver](#), [CallCtlCallEv](#),
[CallCtlTermConnBridgedEv](#), [CallCtlTermConnDroppedEv](#),
[CallCtlTermConnHeldEv](#), [CallCtlTermConnInUseEv](#),
[CallCtlTermConnRingingEv](#), [CallCtlTermConnTalkingEv](#),
[CallCtlTermConnUnknownEv](#)

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv
<u>CAUSE ALTERNATE</u> , <u>CAUSE BUSY</u> , <u>CAUSE CALL BACK</u> , <u>CAUSE CALL NOT ANSWERED</u> , <u>CAUSE CALL PICKUP</u> , <u>CAUSE CONFERENCE</u> , <u>CAUSE DO NOT DISTURB</u> , <u>CAUSE PARK</u> , <u>CAUSE REDIRECTED</u> , <u>CAUSE REORDER TONE</u> , <u>CAUSE TRANSFER</u> , <u>CAUSE TRUNKS BUSY</u> , <u>CAUSE UNHOLD</u>

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.<u>TermConnEv</u>
--

<u>getTerminalConnection</u>

Methods inherited from interface javax.telephony.callcontrol.events.<u>CallCtlEv</u>

<u>getCallControlCause</u>

javax.telephony.callcontrol.events

Interface CallCtlTermConnHeldEv

public interface **CallCtlTermConnHeldEv**
 extends CallCtlTermConnEv

The `CallCtlTermConnHeldEv` interface indicates that the call control package state of the `TerminalConnection` is now `CallControlTerminalConnection.HELD`. This method `CallControlTerminalConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlTermConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

CallObserver, TerminalConnection, CallControlTerminalConnection,
CallControlCallObserver, CallCtlTermConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE ALTERNATE, CAUSE BUSY, CAUSE CALL BACK, CAUSE CALL NOT ANSWERED,
CAUSE CALL PICKUP, CAUSE CONFERENCE, CAUSE DO NOT DISTURB, CAUSE PARK,
CAUSE REDIRECTED, CAUSE REORDER TONE, CAUSE TRANSFER,
CAUSE TRUNKS BUSY, CAUSE UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface <code>javax.telephony.events.TermConnEv</code>
--

<code>getTerminalConnection</code>

Methods inherited from interface <code>javax.telephony.callcontrol.events.CallCtlEv</code>

<code>getCallControlCause</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlTermConnInUseEv

public interface **CallCtlTermConnInUseEv**
 extends CallCtlTermConnEv

The `CallCtlTermConnInUseEv` interface indicates that the call control package state of the `TerminalConnection` is now `CallControlTerminalConnection.INUSE`. This method `CallControlTerminalConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlTermConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

CallObserver, TerminalConnection, CallControlTerminalConnection,
CallControlCallObserver, CallCtlTermConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE_ALTERNATE, CAUSE_BUSY, CAUSE_CALL_BACK, CAUSE_CALL_NOT_ANSWERED,
CAUSE_CALL_PICKUP, CAUSE_CONFERENCE, CAUSE_DO_NOT_DISTURB, CAUSE_PARK,
CAUSE_REDIRECTED, CAUSE_REORDER_TONE, CAUSE_TRANSFER,
CAUSE_TRUNKS_BUSY, CAUSE_UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface <code>javax.telephony.events.TermConnEv</code>
--

<code>getTerminalConnection</code>

Methods inherited from interface <code>javax.telephony.callcontrol.events.CallCtlEv</code>

<code>getCallControlCause</code>

Field Detail

ID

```
public static final int ID
```

Event id

*javax.telephony.callcontrol.events****Interface CallCtlTermConnRingingEv***

public interface **CallCtlTermConnRingingEv**
 extends CallCtlTermConnEv

The `CallCtlTermConnRingingEv` interface indicates that the call control package state of the `TerminalConnection` is now `CallControlTerminalConnection.RINGING`. This method `CallControlTerminalConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlTermConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

CallObserver, TerminalConnection, CallControlTerminalConnection,
CallControlCallObserver, CallCtlTermConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE_ALTERNATE, CAUSE_BUSY, CAUSE_CALL_BACK, CAUSE_CALL_NOT_ANSWERED,
CAUSE_CALL_PICKUP, CAUSE_CONFERENCE, CAUSE_DO_NOT_DISTURB, CAUSE_PARK,
CAUSE_REDIRECTED, CAUSE_REORDER_TONE, CAUSE_TRANSFER,
CAUSE_TRUNKS_BUSY, CAUSE_UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface <code>javax.telephony.events.TermConnEv</code>
--

<code>getTerminalConnection</code>

Methods inherited from interface <code>javax.telephony.callcontrol.events.CallCtlEv</code>

<code>getCallControlCause</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events

Interface CallCtlTermConnTalkingEv

public interface **CallCtlTermConnTalkingEv**
 extends CallCtlTermConnEv

The `CallCtlTermConnTalkingEv` interface indicates that the call control package state of the `TerminalConnection` is now `CallControlTerminalConnection.TALKING`. This method `CallControlTerminalConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlTermConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

CallObserver, TerminalConnection, CallControlTerminalConnection,
CallControlCallObserver, CallCtlTermConnEv

Field Summary

static int	ID
	Event id

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

CAUSE_ALTERNATE, CAUSE_BUSY, CAUSE_CALL_BACK, CAUSE_CALL_NOT_ANSWERED,
CAUSE_CALL_PICKUP, CAUSE_CONFERENCE, CAUSE_DO_NOT_DISTURB, CAUSE_PARK,
CAUSE_REDIRECTED, CAUSE_REORDER_TONE, CAUSE_TRANSFER,
CAUSE_TRUNKS_BUSY, CAUSE_UNHOLD

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface javax.telephony.events.TermConnEv

<u>getTerminalConnection</u>

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv
--

<u>getCallControlCause</u>

Field Detail

ID

```
public static final int ID
```

Event id

*javax.telephony.callcontrol.events****Interface CallCtlTermConnUnknownEv***

public interface **CallCtlTermConnUnknownEv**
 extends [CallCtlTermConnEv](#)

The `CallCtlTermConnUnknownEv` interface indicates that the call control package state of the `TerminalConnection` is now `CallControlTerminalConnection.UNKNOWN`. This method `CallControlTerminalConnection.getCallControlState()` returns the current state. This interface extends the `CallCtlTermConnEv` interface and is reported via the `CallObserver.callChangedEvent()` method. The observer object must also implement the `CallControlCallObserver` interface to signal it is interested in call control package events.

This interface supports no additional methods.

See Also:

[CallObserver](#), [TerminalConnection](#), [CallControlTerminalConnection](#),
[CallControlCallObserver](#), [CallCtlTermConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.CallCtlEv

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlCallEv

getCalledAddress, getCallingAddress, getCallingTerminal,
getLastRedirectedAddress

Methods inherited from interface <code>javax.telephony.events.TermConnEv</code>
--

<code>getTerminalConnection</code>

Methods inherited from interface <code>javax.telephony.callcontrol.events.CallCtlEv</code>

<code>getCallControlCause</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.callcontrol.events***Interface CallCtlTermDoNotDisturbEv***

public interface **CallCtlTermDoNotDisturbEv**
 extends [CallCtlTermEv](#)

The `CallCtlTermDoNotDisturbEv` interface indicates the state of the do not disturb feature has changed for the Terminal. This interface extends the `CallCtlTermEv` interface and is reported via the `TerminalObserver.terminalChangedEvent()` method. The observer object must also implement the `CallControlTerminalObserver` interface to signal it is interested in call control package events.

This interface supports a single method which returns the current state of the do not disturb feature.

See Also:

[Terminal](#), [TerminalObserver](#), [CallControlTerminal](#),
[CallControlTerminalObserver](#), [CallCtlTermEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE_ALTERNATE](#), [CAUSE_BUSY](#), [CAUSE_CALL_BACK](#), [CAUSE_CALL_NOT_ANSWERED](#),
[CAUSE_CALL_PICKUP](#), [CAUSE_CONFERENCE](#), [CAUSE_DO_NOT_DISTURB](#), [CAUSE_PARK](#),
[CAUSE_REDIRECTED](#), [CAUSE_REORDER_TONE](#), [CAUSE_TRANSFER](#),
[CAUSE_TRUNKS_BUSY](#), [CAUSE_UNHOLD](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Method Summary

boolean	<u>getDoNotDisturbState()</u> Returns true if the do not disturb feature is activated, false otherwise.
---------	---

Methods inherited from interface javax.telephony.callcontrol.events.CallCtlEv

getCallControlCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail

ID

```
public static final int ID
```

Event id

Method Detail

getDoNotDisturbState

```
public boolean getDoNotDisturbState()
```

Returns true if the do not disturb feature is activated, false otherwise.

Returns:

True if the do not disturb feature is activated, false otherwise.

javax.telephony.callcontrol.events

Interface CallCtlTermEv

All Known Subinterfaces:

[CallCtlTermDoNotDisturbEv](#)

public interface **CallCtlTermEv**
extends [CallCtlEv](#), [TermEv](#)

The `CallCtlTermEv` interface is the base interface for all call control package Terminal-related events. All events which pertain to the `CallControlTerminal` interface must extend this interface. Events which extend this interface are reported via the `TerminalObserver.terminalChangedEvent()` method. The observer object must also implement the `CallControlTerminalObserver` interface to signal it is interested in call control package events. This interface extends both the `CallCtlEv` and `TermEv` interfaces.

The only event defined in the call control package for the Terminal is the `CallCtlTermDoNotDisturbEv`.

This interface supports no additional methods.

See Also:

[Terminal](#), [TerminalObserver](#), [TermEv](#), [CallControlTerminal](#),
[CallControlTerminalObserver](#), [CallCtlTermDoNotDisturbEv](#), [CallCtlEv](#)

Fields inherited from interface javax.telephony.callcontrol.events.[CallCtlEv](#)

[CAUSE ALTERNATE](#), [CAUSE BUSY](#), [CAUSE CALL BACK](#), [CAUSE CALL NOT ANSWERED](#),
[CAUSE CALL PICKUP](#), [CAUSE CONFERENCE](#), [CAUSE DO NOT DISTURB](#), [CAUSE PARK](#),
[CAUSE REDIRECTED](#), [CAUSE REORDER TONE](#), [CAUSE TRANSFER](#),
[CAUSE TRUNKS BUSY](#), [CAUSE UNHOLD](#)

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Fields inherited from interface [javax.telephony.events.Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface [javax.telephony.callcontrol.events.CallCtlEv](#)

[getCallControlCause](#)

Methods inherited from interface [javax.telephony.events.TermEv](#)

[getTerminal](#)

Package javax.telephony.capabilities

The JTAPI Core Capabilities package provides an interface to query the current JTAPI implementation to see which features are currently accessible.

See:

Description

Interface Summary	
<u>AddressCapabilities</u>	The AddressCapabilities interface represents the initial capabilities interface for the Address.
<u>CallCapabilities</u>	The CallCapabilities interface represents the initial capabilities interface for the Call.
<u>ConnectionCapabilities</u>	The ConnectionCapabilities interface represents the initial capabilities interface for the Connection.
<u>ProviderCapabilities</u>	The ProviderCapabilities interface represents the initial capabilities interface for the Provider.
<u>TerminalCapabilities</u>	The TerminalCapabilities interface represents the initial capabilities interface for the Terminal.
<u>TerminalConnectionCapabilities</u>	The TerminalConnectionCapabilities interface represents the initial capabilities interface for the TerminalConnection.

Package javax.telephony.capabilities Description

The JTAPI Core Capabilities package provides an interface to query the current JTAPI implementation to see which features are currently accessible.

Hierarchy For Package javax.telephony.capabilities

Package Hierarchies:

All Packages

Interface Hierarchy

- interface javax.telephony.capabilities.**AddressCapabilities**
 - interface javax.telephony.capabilities.**CallCapabilities**
 - interface javax.telephony.capabilities.**ConnectionCapabilities**
 - interface javax.telephony.capabilities.**ProviderCapabilities**
 - interface javax.telephony.capabilities.**TerminalCapabilities**
 - interface javax.telephony.capabilities.**TerminalConnectionCapabilities**
-
-

javax.telephony.capabilities **Interface AddressCapabilities**

All Known Subinterfaces:

[CallControlAddressCapabilities](#)

public interface **AddressCapabilities**

The `AddressCapabilities` interface represents the initial capabilities interface for the `Address`. This interface supports basic queries for the core package.

Applications obtain the static `Address` capabilities via the `Provider.getAddressCapabilities()` method, and the dynamic capabilities via the `Address.getCapabilities()` method. This interface is used to represent both static and dynamic capabilities.

Any package which extends the core `Address` interface should also extend this interface to provide additional capability queries for that particular package.

See Also:

[Provider](#), [Address](#)

Method Summary

boolean	<u>isObservable()</u> Returns true if this <code>Address</code> can be observed, false otherwise.
---------	---

Method Detail

isObservable

public boolean **isObservable()**

Returns true if this `Address` can be observed, false otherwise.

Returns:

True if this `Address` can be observed, false otherwise.

javax.telephony.capabilities **Interface CallCapabilities**

All Known Subinterfaces:

[CallControlCallCapabilities](#)

public interface **CallCapabilities**

The **CallCapabilities** interface represents the initial capabilities interface for the **Call**. This interface supports basic queries for the core package.

Applications obtain the static **Call** capabilities via the **Provider.getCallCapabilities()** method, and the dynamic capabilities via the **Call.getCapabilities()** method. This interface is used to represent both static and dynamic capabilities.

Any package which extends the core **Call** interface should also extend this interface to provide additional capability queries for that particular package.

See Also:

[Provider](#), [Call](#)

Method Summary

boolean	<u>canConnect()</u> Returns true if the application can invoke Call.connect() , false otherwise.
boolean	<u>isObservable()</u> Returns true if this Call can be observed, false otherwise.

Method Detail

canConnect

public boolean **canConnect()**

Returns true if the application can invoke **Call.connect()**, false otherwise.

Returns:

True if the application can perform a connect, false otherwise.

isObservable

public boolean **isObservable**()

Returns true if this Call can be observed, false otherwise.

Returns:

True if this Call can be observed, false otherwise.

javax.telephony.capabilities **Interface ConnectionCapabilities**

All Known Subinterfaces:CallControlConnectionCapabilities

public interface **ConnectionCapabilities**

The **ConnectionCapabilities** interface represents the initial capabilities interface for the **Connection**. This interface supports basic queries for the core package.

Applications obtain the static **Connection** capabilities via the **Provider.getConnectionCapabilities()** method, and the dynamic capabilities via the **Connection.getCapabilities()** method. This interface is used to represent both static and dynamic capabilities.

Any package which extends the core **Connection** interface should also extend this interface to provide additional capability queries for that particular package.

See Also:Provider, Connection

Method Summary

boolean	<u>canDisconnect()</u> Returns true if the application can invoke Connection.disconnect() perform a disconnect(), false otherwise.
---------	---

Method Detail

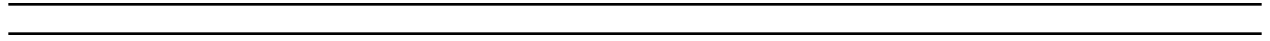
canDisconnect

```
public boolean canDisconnect()
```

Returns true if the application can invoke **Connection.disconnect()** perform a disconnect(), false otherwise.

Returns:

True if the application can disconnect, false otherwise.



javax.telephony.capabilities **Interface ProviderCapabilities**

public interface **ProviderCapabilities**

The **ProviderCapabilities** interface represents the initial capabilities interface for the Provider. This interface supports basic queries for the core package.

Applications obtain the static Provider capabilities via the **Provider.getProviderCapabilities()** method, and the dynamic capabilities via the **Provider.getCapabilities()** method. This interface is used to represent both static and dynamic capabilities.

Any package which extends the core **Provider** interface should also extend this interface to provide additional capability queries for that particular package.

See Also:

[Provider](#)

Method Summary

boolean	<u>isObservable()</u> Returns true if this Provider can be observed, false otherwise.
---------	---

Method Detail

isObservable

public boolean **isObservable()**

Returns true if this Provider can be observed, false otherwise.

Returns:

True if this Provider can be observed, false otherwise.

javax.telephony.capabilities **Interface TerminalCapabilities**

All Known Subinterfaces:

[CallControlTerminalCapabilities](#)

public interface **TerminalCapabilities**

The `TerminalCapabilities` interface represents the initial capabilities interface for the Terminal. This interface supports basic queries for the core package.

Applications obtain the static Terminal capabilities via the `Provider.getTerminalCapabilities()` method, and the dynamic capabilities via the `Terminal.getCapabilities()` method. This interface is used to represent both static and dynamic capabilities.

Any package which extends the core `Terminal` interface should also extend this interface to provide additional capability queries for that particular package.

See Also:

[Provider](#), [Terminal](#)

Method Summary

boolean	<code>isObservable()</code> Returns true if this Terminal is observable, false otherwise.
---------	---

Method Detail

isObservable

public boolean **`isObservable()`**

Returns true if this Terminal is observable, false otherwise.

Returns:

True if this Terminal is observable, false otherwise.

javax.telephony.capabilities **Interface TerminalConnectionCapabilities**

All Known Subinterfaces:

[CallControlTerminalConnectionCapabilities](#)

public interface **TerminalConnectionCapabilities**

The `TerminalConnectionCapabilities` interface represents the initial capabilities interface for the `TerminalConnection`. This interface supports basic queries for the core package.

Applications obtain the static `TerminalConnection` capabilities via the `Provider.getTerminalConnectionCapabilities()` method, and the dynamic capabilities via the `TerminalConnection.getCapabilities()` method. This interface is used to represent both static and dynamic capabilities.

Any package which extends the core `TerminalConnection` interface should also extend this interface to provide additional capability queries for that particular package.

See Also:

[Provider](#), [TerminalConnection](#)

Method Summary

boolean	<u>canAnswer()</u> Returns true if the application can invoke <code>TerminalConnection.answer()</code> , false otherwise.
---------	---

Method Detail

canAnswer

public boolean **canAnswer()**

Returns true if the application can invoke `TerminalConnection.answer()`, false otherwise.

Returns:

True if the application can answer, false otherwise.

Package javax.telephony.events

The JTAPI Core Events package defines the core call model state transistions in setting up and monitoring call progress.

See:

Description

Interface Summary	
<u>AddrEv</u>	The AddrEv interface is the base interface for all Address- related events.
<u>AddrObservationEndedEv</u>	The AddrObservationEndedEv event indicates that the application will no longer receive Address events on the instance of the AddressObserver.
<u>CallActiveEv</u>	Indicates that the state of the Call object has changed to Call.ACTIVE.
<u>CallEv</u>	The CallEv interface is the base interface for all Call-related events.
<u>CallInvalidEv</u>	The CallInvalidEv interface indicates that the state of the Call object has changed to Call.INVALID.
<u>CallObservationEndedEv</u>	The CallObservationEndedEv event indicates that the application will no longer receive Call events on the instance of the CallObserver This interface extends the CallEv interface and is reported on the CallObserver interface.
<u>ConnAlertingEv</u>	The ConnAlertingEv interface indicates that the state of the Connection object has changed to Connection.ALERTING.
<u>ConnConnectedEv</u>	The ConnConnectedEv interface indicates that the state of the Connection object has changed to Connection.CONNECTED.
<u>ConnCreatedEv</u>	The ConnUnknownEv interface indicates that a new Connection object has been created.
<u>ConnDisconnectedEv</u>	The ConnDisconnectedEv interface indicates that the state of the Connection object has changed to Connection.DISCONNECTED.
<u>ConnEv</u>	The ConnEv interface is the base event interface for all Connection-related events.
<u>ConnFailedEv</u>	The ConnFailedEv interface indicates that the state of the Connection object has changed to Connection.FAILED.
<u>ConnInProgressEv</u>	The ConnInProgressEv interface indicates that the state of the Connection object has changed to Connection.IN_PROGRESS.
<u>ConnUnknownEv</u>	The ConnUnknownEv interface indicates that the state of the Connection object has changed to Connection.UNKNOWN.

<u>Ev</u>	The parent of all JTAPI event interfaces.
<u>ProvEv</u>	The ProvEv interface is the base interface for all Provider- related events.
<u>ProvInServiceEv</u>	The ProvInServiceEv interface indicates that the state of the Provider object has changed to Provider.IN_SERVICE.
<u>ProvObservationEndedEv</u>	The ProvObservationEndedEv event indicates that the application will no longer receive Provider events on the instance of the ProviderObserver.
<u>ProvOutOfServiceEv</u>	The ProvOutOfServiceEv interface indicates that the state of the Provider object has changed to Provider.OUT_OF_SERVICE.
<u>ProvShutdownEv</u>	The ProvShutdownEv interface indicates that the state of the Provider object has changed to Provider.SHUTDOWN.
<u>TermConnActiveEv</u>	The TermConnActiveEv interface indicates that the state of the TerminalConnection object has changed to TerminalConnection.ACTIVE.
<u>TermConnCreatedEv</u>	The TermConnDroppedEv interface indicates that a new TerminalConnection object has been created.
<u>TermConnDroppedEv</u>	The TermConnDroppedEv interface indicates that the state of the TerminalConnection object has changed to TerminalConnection.DROPPED.
<u>TermConnEv</u>	The TermConnEv interface is the base event interface for all TerminalConnection-related events.
<u>TermConnPassiveEv</u>	The TermConnPassiveEv interface indicates that the state of the TerminalConnection object has changed to TerminalConnection.PASSIVE.
<u>TermConnRingingEv</u>	The TermConnRingingEv interface indicates that the state of the TerminalConnection object has changed to TerminalConnection.RINGING.
<u>TermConnUnknownEv</u>	The TermConnUnknownEv interface indicates that the state of the TerminalConnection object has changed to TerminalConnection.UNKNOWN.
<u>TermEv</u>	The TermEv interface is the base interface for all Terminal-related events.
<u>TermObservationEndedEv</u>	The TermObservationEndedEv event indicates that the application will no longer receive Terminal events on the instance of the TerminalObserver This interface extends the TermEv interface and is reported on the TerminalObserver interface.

Package javax.telephony.events Description

The JTAPI Core Events package defines the core call model state transistions in setting up and monitoring call progress.

Hierarchy For Package javax.telephony.events

Package Hierarchies:

All Packages

Interface Hierarchy

- interface javax.telephony.events.**Ev**
 - interface javax.telephony.events.**AddrEv**
 - interface javax.telephony.events.**AddrObservationEndedEv**
 - interface javax.telephony.events.**CallEv**
 - interface javax.telephony.events.**CallActiveEv**
 - interface javax.telephony.events.**CallInvalidEv**
 - interface javax.telephony.events.**CallObservationEndedEv**
 - interface javax.telephony.events.**ConnEv**
 - interface javax.telephony.events.**ConnAlertingEv**
 - interface javax.telephony.events.**ConnConnectedEv**
 - interface javax.telephony.events.**ConnCreatedEv**
 - interface javax.telephony.events.**ConnDisconnectedEv**
 - interface javax.telephony.events.**ConnFailedEv**
 - interface javax.telephony.events.**ConnInProgressEv**
 - interface javax.telephony.events.**ConnUnknownEv**
 - interface javax.telephony.events.**TermConnEv**
 - interface javax.telephony.events.**TermConnActiveEv**
 - interface javax.telephony.events.**TermConnCreatedEv**
 - interface javax.telephony.events.**TermConnDroppedEv**
 - interface javax.telephony.events.**TermConnPassiveEv**
 - interface javax.telephony.events.**TermConnRingingEv**
 - interface javax.telephony.events.**TermConnUnknownEv**
 - interface javax.telephony.events.**ProvEv**
 - interface javax.telephony.events.**ProvInServiceEv**
 - interface javax.telephony.events.**ProvObservationEndedEv**
 - interface javax.telephony.events.**ProvOutOfServiceEv**
 - interface javax.telephony.events.**ProvShutdownEv**
 - interface javax.telephony.events.**TermEv**
 - interface javax.telephony.events.**TermObservationEndedEv**

javax.telephony.events

Interface AddrEv

All Known Subinterfaces:

[AddrObservationEndedEv](#), [CallCtlAddrDoNotDisturbEv](#), [CallCtlAddrEv](#), [CallCtlAddrForwardEv](#),
[CallCtlAddrMessageWaitingEv](#)

public interface **AddrEv**
extends [Ev](#)

The **AddrEv** interface is the base interface for all Address- related events. All events which pertain to the Address object must extend this interface. Events which extend this interface are reported via the **AddressObserver** interface.

The only event defined in the core package for the Address is the **AddrObservationEndedEv**.

The **AddrEv.getAddress()** method on this interface returns the Address associated with the Address event.

See Also:

[AddrObservationEndedEv](#), [Ev](#), [AddressObserver](#), [Address](#)

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Method Summary

<u>Address</u>	<u>getAddress()</u> Returns the Address associated with this Address event.
--------------------------------	---

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Method Detail

getAddress

public Address **getAddress()**

Returns the Address associated with this Address event.

Returns:

The Address associated with this event.

*javax.telephony.events****Interface AddrObservationEndedEv***

public interface **AddrObservationEndedEv**

extends [AddrEv](#)

The `AddrObservationEndedEv` event indicates that the application will no longer receive Address events on the instance of the `AddressObserver`. This interface extends the `AddrEv` interface and is reported on the `AddressObserver` interface.

See Also:

[AddrEv](#), [AddressObserver](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[AddrEv](#)

[getAddress](#)

Methods inherited from interface javax.telephony.events.[Ev](#)

[getCause](#), [getID](#), [getMetaCode](#), [getObserved](#), [isNewMetaEvent](#)

Field Detail

ID`public static final int ID`Event id

javax.telephony.events

Interface CallActiveEv

public interface **CallActiveEv**
 extends CallEv

Indicates that the state of the Call object has changed to `Call.ACTIVE`. This interface extends the `CallEv` interface and is reported via the `CallObserver` interface.

See Also:

Call, CallObserver, CallEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.events.CallEv

getCall

Methods inherited from interface javax.telephony.events.Ev

getCause, getID, getMetaCode, getObserved, isNewMetaEvent

Field Detail

ID`public static final int ID`Event id

javax.telephony.events

Interface CallEv

All Known Subinterfaces:

[CallActiveEv](#), [CallCtlCallEv](#), [CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#),
[CallCtlConnDisconnectedEv](#), [CallCtlConnEstablishedEv](#), [CallCtlConnEv](#), [CallCtlConnFailedEv](#),
[CallCtlConnInitiatedEv](#), [CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#),
[CallCtlConnOfferedEv](#), [CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#),
[CallCtlTermConnBridgedEv](#), [CallCtlTermConnDroppedEv](#), [CallCtlTermConnEv](#),
[CallCtlTermConnHeldEv](#), [CallCtlTermConnInUseEv](#), [CallCtlTermConnRingingEv](#),
[CallCtlTermConnTalkingEv](#), [CallCtlTermConnUnknownEv](#), [CallInvalidEv](#),
[CallObservationEndedEv](#), [ConnAlertingEv](#), [ConnConnectedEv](#), [ConnCreatedEv](#),
[ConnDisconnectedEv](#), [ConnEv](#), [ConnFailedEv](#), [ConnInProgressEv](#), [ConnUnknownEv](#),
[TermConnActiveEv](#), [TermConnCreatedEv](#), [TermConnDroppedEv](#), [TermConnEv](#),
[TermConnPassiveEv](#), [TermConnRingingEv](#), [TermConnUnknownEv](#)

public interface **CallEv**
 extends [Ev](#)

The `CallEv` interface is the base interface for all Call-related events. All events which pertain to the `Call` object must extend this interface. Events which extend this interface are reported via the `CallObserver` interface.

The core package defines events which are reported when the `Call` changes state. These events are: `CallActiveEv` and `CallInvalidEv`. Also, the core package defines the `CallObservationEndedEv` event which is sent when the `Call` becomes unobservable.

The `ConnEv` and `TermConnEv` events extend this interface. This reflects the fact that all `Connection` and `TerminalConnection` events are reported via the `CallObserver` interface.

The `CallEv.getCall()` method on this interface returns the `Call` associated with the `Call` event.

See Also:

[CallActiveEv](#), [CallInvalidEv](#), [CallObservationEndedEv](#), [Ev](#), [ConnEv](#),
[TermConnEv](#), [CallObserver](#), [Call](#)

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

<u>Call</u>	getCall() Returns the Call object associated with this Call event.
-------------	--

Methods inherited from interface javax.telephony.events.Ev

getCause, getID, getMetaCode, getObserved, isNewMetaEvent

Method Detail*getCall*

public Call **getCall()**

Returns the Call object associated with this Call event.

Returns:

The Call associated with this event.

javax.telephony.events

Interface CallInvalidEv

public interface **CallInvalidEv**
 extends [CallEv](#)

The `CallInvalidEv` interface indicates that the state of the `Call` object has changed to `Call.INVALID`. This interface extends the `CallEv` interface and is reported via the `CallObserver` interface.

See Also:

[Call](#), [CallObserver](#), [CallEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface javax.telephony.events.[Ev](#)

[getCause](#), [getID](#), [getMetaCode](#), [getObserved](#), [isNewMetaEvent](#)

Field Detail

ID`public static final int ID`Event id

*javax.telephony.events***Interface CallObservationEndedEv**

public interface **CallObservationEndedEv**
 extends [CallEv](#)

The `CallObservationEndedEv` event indicates that the application will no longer receive Call events on the instance of the `CallObserver`. This interface extends the `CallEv` interface and is reported on the `CallObserver` interface.

See Also:

[CallEv](#), [CallObserver](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Method Summary

java.lang.Object	<u>getEndedObject()</u> This method returns the object which is responsible for the observation of the call ending.
------------------	---

Methods inherited from interface javax.telephony.events.CallEv`getCall`**Methods inherited from interface javax.telephony.events.Ev**`getCause, getID, getMetaCode, getObserved, isNewMetaEvent`**Field Detail***ID*

```
public static final int ID
```

Event id

Method Detail*getEndedObject*

```
public java.lang.Object getEndedObject()
```

This method returns the object which is responsible for the observation of the call ending. If this method returns a `Call`, the observation ended because either the `Call` could no longer be observed or the observer was removed via the `Call.removeObserver()` method. If this method returns either an `Address` or `Terminal`, then additional observers will not be added if the `Call` arrives at the returned `Address` or `Terminal`.

Returns:The object responsible for the observation ending for the `Call`.**Since:**

JTAPI v1.2

javax.telephony.events

Interface ConnAlertingEv

public interface **ConnAlertingEv**
 extends [ConnEv](#)

The ConnAlertingEv interface indicates that the state of the Connection object has changed to Connection.ALERTING. This interface extends the ConnEv interface and is reported via the CallObserver interface.

See Also:

[Connection](#), [CallObserver](#), [ConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[ConnEv](#)

[getConnection](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events

Interface ConnConnectedEv

public interface **ConnConnectedEv**
 extends ConnEv

The ConnConnectedEv interface indicates that the state of the Connection object has changed to Connection.CONNECTED. This interface extends the ConnEv interface and is reported via the CallObserver interface.

See Also:

Connection, CallObserver, ConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.events.ConnEv

getConnection

Methods inherited from interface javax.telephony.events.CallEv

getCall

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events

Interface ConnCreatedEv

public interface **ConnCreatedEv**
 extends ConnEv

The ConnUnknownEv interface indicates that a new Connection object has been created. This interface extends the ConnEv interface and is reported via the CalObserver interface.

See Also:

Connection, CalObserver, ConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.events.ConnEv

getConnection

Methods inherited from interface javax.telephony.events.CallEv

getCall

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events***Interface ConnDisconnectedEv***

public interface **ConnDisconnectedEv**

extends [ConnEv](#)

The ConnDisconnectedEv interface indicates that the state of the Connection object has changed to Connection.DISCONNECTED. This interface extends the ConnEv interface and is reported via the CallObserver interface.

See Also:

[Connection](#), [CallObserver](#), [ConnEv](#)

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[ConnEv](#)

[getConnection](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events

Interface ConnEv

All Known Subinterfaces:

[CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#), [CallCtlConnDisconnectedEv](#),
[CallCtlConnEstablishedEv](#), [CallCtlConnEv](#), [CallCtlConnFailedEv](#), [CallCtlConnInitiatedEv](#),
[CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#), [CallCtlConnOfferedEv](#),
[CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#), [ConnAlertingEv](#), [ConnConnectedEv](#),
[ConnCreatedEv](#), [ConnDisconnectedEv](#), [ConnFailedEv](#), [ConnInProgressEv](#), [ConnUnknownEv](#)

public interface **ConnEv**
 extends [CalEv](#)

The ConnEv interface is the base event interface for all Connection-related events. All events which pertain to the Connection object must extend this interface. This interface extends the CalEv interface and therefore is reported via the CalObserver interface.

The core package defines events which are reported when the Connection changes state. These events are: ConnInProgressEv, ConnAlertingEv, ConnConnectedEv, ConnDisconnectedEv, ConnFailedEv, and ConnUnknownEv. Also, the ConnCreatedEv is sent when a new Connection is created.

The ConnEv.getConnection() method on this interface returns the Connection associated with this Connection event.

See Also:

[Connection](#), [CalObserver](#), [CalEv](#), [ConnCreatedEv](#), [ConnInProgressEv](#),
[ConnAlertingEv](#), [ConnConnectedEv](#), [ConnDisconnectedEv](#), [ConnFailedEv](#),
[ConnUnknownEv](#)

Fields inherited from interface javax.telephony.events.Ev
CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE, CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT, CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE, CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE, CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY, META CALL ENDING, META CALL MERGING, META CALL PROGRESS, META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING, META SNAPSHOT, META UNKNOWN

Method Summary

<u>Connection</u>	getConnection() Returns the Connection associated with this Connection event.
-------------------	--

Methods inherited from interface javax.telephony.events.CallEv

getCall

Methods inherited from interface javax.telephony.events.Ev

getCause, getID, getMetaCode, getObserved, isNewMetaEvent

Method Detail

getConnection

public Connection **getConnection()**

Returns the Connection associated with this Connection event.

Returns:

The Connection associated with this event.

javax.telephony.events

Interface ConnFailedEv

public interface **ConnFailedEv**
 extends [ConnEv](#)

The `ConnFailedEv` interface indicates that the state of the `Connection` object has changed to `Connection.FAILED`. This interface extends the `ConnEv` interface and is reported via the `CallObserver` interface.

See Also:

[Connection](#), [CallObserver](#), [ConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface `javax.telephony.events.Ev`

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface `javax.telephony.events.ConnEv`

[getConnection](#)

Methods inherited from interface `javax.telephony.events.CallEv`

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events***Interface ConnInProgressEv***

public interface **ConnInProgressEv**
 extends [ConnEv](#)

The ConnInProgressEv interface indicates that the state of the Connection object has changed to Connection.IN_PROGRESS. This interface extends the ConnEv interface and is reported via the CallObserver interface.

See Also:

[Connection](#), [CallObserver](#), [ConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[ConnEv](#)

[getConnection](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events

Interface ConnUnknownEv

public interface **ConnUnknownEv**
 extends [ConnEv](#)

The ConnUnknownEv interface indicates that the state of the Connection object has changed to Connection.UNKNOWN. This interface extends the ConnEv interface and is reported via the CallObserver interface.

See Also:

[Connection](#), [CallObserver](#), [ConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[ConnEv](#)

[getConnection](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events

Interface Ev

All Known Subinterfaces:

[AddrEv](#), [AddrObservationEndedEv](#), [ButtonInfoEv](#), [ButtonPressEv](#), [CallActiveEv](#),
[CallCtlAddrDoNotDisturbEv](#), [CallCtlAddrEv](#), [CallCtlAddrForwardEv](#),
[CallCtlAddrMessageWaitingEv](#), [CallCtlCallEv](#), [CallCtlConnAlertingEv](#), [CallCtlConnDialingEv](#),
[CallCtlConnDisconnectedEv](#), [CallCtlConnEstablishedEv](#), [CallCtlConnEv](#), [CallCtlConnFailedEv](#),
[CallCtlConnInitiatedEv](#), [CallCtlConnNetworkAlertingEv](#), [CallCtlConnNetworkReachedEv](#),
[CallCtlConnOfferedEv](#), [CallCtlConnQueuedEv](#), [CallCtlConnUnknownEv](#), [CallCtlEv](#),
[CallCtlTermConnBridgedEv](#), [CallCtlTermConnDroppedEv](#), [CallCtlTermConnEv](#),
[CallCtlTermConnHeldEv](#), [CallCtlTermConnInUseEv](#), [CallCtlTermConnRingingEv](#),
[CallCtlTermConnTalkingEv](#), [CallCtlTermConnUnknownEv](#), [CallCtlTermDoNotDisturbEv](#),
[CallCtlTermEv](#), [CallEv](#), [CallInvalidEv](#), [CallObservationEndedEv](#), [ConnAlertingEv](#),
[ConnConnectedEv](#), [ConnCreatedEv](#), [ConnDisconnectedEv](#), [ConnEv](#), [ConnFailedEv](#),
[ConnInProgressEv](#), [ConnUnknownEv](#), [DisplayUpdateEv](#), [HookswitchStateEv](#), [LampModeEv](#),
[MicrophoneGainEv](#), [PhoneEv](#), [PhoneTermEv](#), [ProvEv](#), [ProvInServiceEv](#), [ProvObservationEndedEv](#),
[ProvOutOfServiceEv](#), [ProvShutdownEv](#), [RingerPatternEv](#), [RingerVolumeEv](#), [SpeakerVolumeEv](#),
[TermConnActiveEv](#), [TermConnCreatedEv](#), [TermConnDroppedEv](#), [TermConnEv](#),
[TermConnPassiveEv](#), [TermConnRingingEv](#), [TermConnUnknownEv](#), [TermEv](#),
[TermObservationEndedEv](#)

public interface **Ev**

The parent of all JTAPI event interfaces.

Introduction

The **Ev** interface is the parent of all JTAPI event interfaces. All JTAPI event interfaces extend this interface, either directly or indirectly. Event interfaces within each JTAPI package are organized in a hierarchical fashion. The architecture of the core package event hierarchy is described later.

The JTAPI event system notifies applications when changes in various JTAPI object occur. Each individual change in an object is represented by an event sent to the appropriate observer. Because several changes may happen to an object at once, events are delivered as a *batch*. A batch of events represents a series of events and changes to the call model which happened exactly at the same time. For this reason, events are delivered to observers as arrays.

Event IDs

Each event carries a corresponding identification integer. The **Ev.getID()** method returns this identification number for each event. The actual event identification integer is defined in each of the specific event interfaces. Each event interface must carry a unique id.

Cause Codes

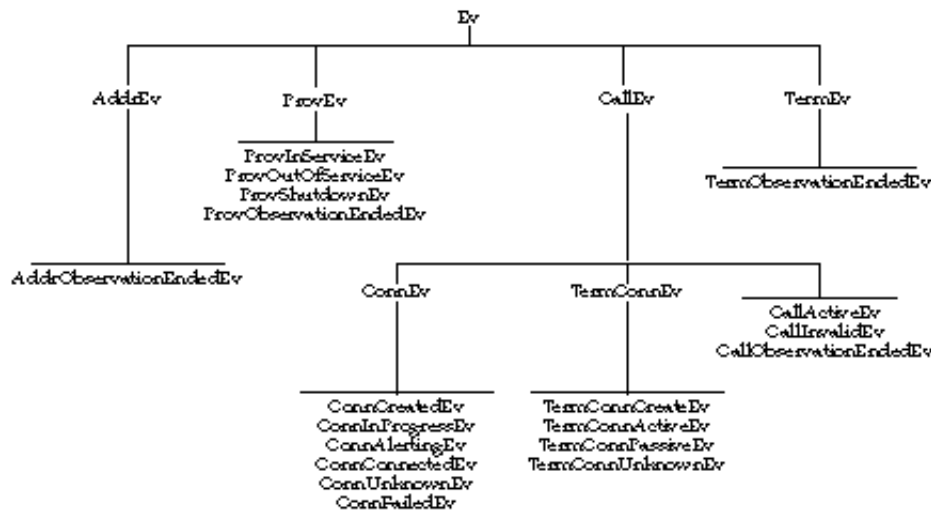
Each events carries a *cause* or a reason why the event happened. The `Ev.getCause()` method returns this cause value. The different types of cause values are also defined in this interface.

Core Package Event Hierarchy

The core package defines a hierarchy of event interfaces. The base of this hierarchy is the `Ev` interface. Directly extending this interface are those events interfaces for each object which supports an observer: `ProvEv`, `CallEv`, `AddrEv`, and `TermEv`.

Since Connection and TerminalConnection events are reported via the `CallObserver` interface, the `ConnEv` and `TermConnEv` interfaces extends the `CallEv` interface.

The following diagram illustrates the complete core package event structure.



Meta Codes

The `Ev.getMetaCode()` method returns the *meta code* for the event. Events are grouped together using meta codes to provide a higher-level description of an update to the call model. Since events represent singular changes in one particular object in the call model, it may be difficult for the application to infer a higher-level interpretation of several of these singular events. Meta codes exist on events to assist the application in this regard.

Events which belong to the same higher-level action and contain the same meta code are reported consecutively in an event batch sent to an observer. In fact, multiple meta code grouping of events may exist in a single event batch. In that case, the `Ev.isNewMetaEvent()` method is used to indicate the beginning of a new meta code event grouping. This method also indicates whether a meta code grouping exists across event batch boundaries. That is, events belonging to the same meta code grouping may be delivered in two contiguous event batches.

There are five types of meta codes which pertain to individual calls, and two which pertain to a mutli-call action, and two miscellaneous meta codes. The five meta codes which pertain to individual calls are:

<code>Ev.META_CALL_STARTING</code>	Indicates that a new active call has been presented to the application, either by an application creating a call and performing an action on it, or by an incoming call to an object being observed by the application.
<code>Ev.META_CALL_PROGRESS</code>	Indicates that the objects belonging to a call have changed state, with the exception of <code>Connection</code> moving to <code>Connection.DISCONNECTED</code> . For example, when a remote party answers a telephone call and the corresponding <code>Connection</code> moves into the <code>Connection.CONNECTED</code> state, this is the meta code associated with the resulting batch of events.
<code>Ev.META_CALL_ADDING_PARTY</code>	Indicates that a party has been added to the call. A "party" corresponds to a <code>Connection</code> being added. Note that if a <code>TerminalConnection</code> is added, it carries a meta code of <code>Ev.META_CALL_PROGRESS</code> .
<code>Ev.META_CALL_REMOVING_PARTY</code>	Indicates that a party (i.e. <code>Connection</code>) has been removed from the call by moving into the <code>Connection.DISCONNECTED</code> state.
<code>Ev.META_CALL_ENDING</code>	Indicates that an entire telephone call has ended, which implies the call has moved into the <code>Call.INVALID</code> state and all of its <code>Connections</code> have moved into the <code>Connection.DISCONNECTED</code> state.

The two meta codes pertaining to a mutli-call actions are as follows:

<code>Ev.META_CALL_MERGING</code>	Indicates that a party has moved from one call to another as part of the two calls merging. A common example is when two telephone calls are conferenced.
<code>Ev.META_CALL_TRANSFERRING</code>	Indicates that a party has moved from one call to another as part of one call being transferred to another. The differs from <code>Ev.META_CALL_MERGING</code> because a common party leaves both calls.

The two miscellaneous meta codes are as follows:

Ev.META_SNAPSHOT Indicates that the sequence of events are part of a "snapshot" given to the application to bring it up-to-date with the current state of the call model.

Ev.META_UNKNOWN Indicates that the meta code is unknown for the event.

Field Summary

static int	<u>CAUSE CALL CANCELLED</u> Cause code indicating the user has terminated call without going on-hook.
static int	<u>CAUSE DEST NOT OBTAINABLE</u> Cause code indicating the destination is not available.
static int	<u>CAUSE INCOMPATIBLE DESTINATION</u> Cause code indicating that a call has encountered an incompatible destination.
static int	<u>CAUSE LOCKOUT</u> Cause code indicating that a call encountered inter-digit timeout while dialing.
static int	<u>CAUSE NETWORK CONGESTION</u> Cause code indicating call encountered network congestion.
static int	<u>CAUSE NETWORK NOT OBTAINABLE</u> Cause code indicating call could not reach a destination network.
static int	<u>CAUSE NEW CALL</u> Cause code indicating that a new call.
static int	<u>CAUSE NORMAL</u> Cause code indicating normal operation
static int	<u>CAUSE RESOURCES NOT AVAILABLE</u> Cause code indicating resources were not available.
static int	<u>CAUSE SNAPSHOT</u> Cause code indicating that the event is part of a snapshot of the current state of the call.
static int	<u>CAUSE UNKNOWN</u> Cause code indicating the cause was unknown
static int	<u>META CALL ADDITIONAL PARTY</u> Meta code description for the addition of a party to call.
static int	<u>META CALL ENDING</u> Meta code description for the entire call ending.
static int	<u>META CALL MERGING</u> Meta code description for an action of merging two calls.

static int	<u>META_CALL_PROGRESS</u> Meta code description for the progress of a call.
static int	<u>META_CALL_REMOVING_PARTY</u> Meta code description for a party leaving the call.
static int	<u>META_CALL_STARTING</u> Meta code description for the initiation or starting of a call.
static int	<u>META_CALL_TRANSFERRING</u> Meta code description for an action of transferring one call to another.
static int	<u>META_SNAPSHOT</u> Meta code description for a snapshot of events.
static int	<u>META_UNKNOWN</u> Meta code is unknown.

Method Summary

int	<u>getCause()</u> Returns the cause associated with this event.
int	<u>getID()</u> Returns the id of event.
int	<u>getMetaCode()</u> Returns the meta code associated with this event.
java.lang.Object	<u>getObserved()</u> Deprecated. Since JTAPI v1.2 This interface no longer needs to supply this information and may return null.
boolean	<u>isNewMetaEvent()</u> Returns true when this event is the start of a meta code group.

Field Detail

CAUSE_NORMAL

```
public static final int CAUSE_NORMAL
```

Cause code indicating normal operation

CAUSE_UNKNOWN

public static final int **CAUSE_UNKNOWN**

Cause code indicating the cause was unknown

CAUSE_CALL_CANCELLED

public static final int **CAUSE_CALL_CANCELLED**

Cause code indicating the user has terminated call without going on-hook.

CAUSE_DEST_NOT_OBTAINABLE

public static final int **CAUSE_DEST_NOT_OBTAINABLE**

Cause code indicating the destination is not available.

CAUSE_INCOMPATIBLE_DESTINATION

public static final int **CAUSE_INCOMPATIBLE_DESTINATION**

Cause code indicating that a call has encountered an incompatible destination.

CAUSE_LOCKOUT

public static final int **CAUSE_LOCKOUT**

Cause code indicating that a call encountered inter-digit timeout while dialing.

CAUSE_NEW_CALL

public static final int **CAUSE_NEW_CALL**

Cause code indicating that a new call.

CAUSE_RESOURCES_NOT_AVAILABLE

public static final int **CAUSE_RESOURCES_NOT_AVAILABLE**

Cause code indicating resources were not available.

CAUSE_NETWORK_CONGESTION

public static final int **CAUSE_NETWORK_CONGESTION**

Cause code indicating call encountered network congestion.

CAUSE_NETWORK_NOT_OBTAINABLE

public static final int **CAUSE_NETWORK_NOT_OBTAINABLE**

Cause code indicating call could not reach a destination network.

CAUSE_SNAPSHOT

public static final int **CAUSE_SNAPSHOT**

Cause code indicating that the event is part of a snapshot of the current state of the call.

META_CALL_STARTING

public static final int **META_CALL_STARTING**

Meta code description for the initiation or starting of a call. This implies that the call is a new call and in the active state with at least one Connection added to it.

META_CALL_PROGRESS

public static final int **META_CALL_PROGRESS**

Meta code description for the progress of a call. This indicates an update in state of certain objects in the call, or the addition of TerminalConnections (but not Connections).

META_CALL_ADDITIONAL_PARTY

public static final int **META_CALL_ADDITIONAL_PARTY**

Meta code description for the addition of a party to call. This includes adding a connection to the call.

META_CALL_REMOVING_PARTY

```
public static final int META_CALL_REMOVING_PARTY
```

Meta code description for a party leaving the call. This includes exactly one `Connection` moving to the `Connection.DISCONNECTED` state.

META_CALL_ENDING

```
public static final int META_CALL_ENDING
```

Meta code description for the entire call ending. This includes the call going to `Call.INVALID`, all of the `Connections` moving to the `Connection.DISCONNECTED` state.

META_CALL_MERGING

```
public static final int META_CALL_MERGING
```

Meta code description for an action of merging two calls. This involves the removal of one party from one call and the addition of the same party to another call.

META_CALL_TRANSFERRING

```
public static final int META_CALL_TRANSFERRING
```

Meta code description for an action of transferring one call to another. This involves the removal of parties from one call and the addition to another call, and the common party dropping off completely.

META_SNAPSHOT

```
public static final int META_SNAPSHOT
```

Meta code description for a snapshot of events.

META_UNKNOWN

```
public static final int META_UNKNOWN
```

Meta code is unknown.

Method Detail

getCause

```
public int getCause()
```

Returns the cause associated with this event. Every event has a cause. The various cause values are defined as public static final variables in this interface.

Returns:

The cause of the event.

getMetaCode

```
public int getMetaCode()
```

Returns the meta code associated with this event. The meta code provides a higher-level description of the event.

Returns:

The meta code for this event.

isNewMetaEvent

```
public boolean isNewMetaEvent()
```

Returns true when this event is the start of a meta code group. This method is used to distinguish two contiguous groups of events bearing the same meta code.

Returns:

True if this event represents a new meta code grouping, false otherwise.

getID

```
public int getID()
```

Returns the id of event. Every event has an id. The defined id of each event matches the object type of each event. The defined id allows applications to switch on event id rather than having to use multiple "if instanceof" statements.

Returns:

The id of the event.

getObserved

public java.lang.Object **getObserved()**

Deprecated. *Since JTAPI v1.2 This interface no longer needs to supply this information and may return null.*

Returns the object that is being observed.

Note: Implementation need no longer supply this information. The `CallObsevationEndedEv.getObservedObject()` method has been added which returns related information. This method may return null in JTAPI v1.2 and later.

Returns:

The object that is being observed.

javax.telephony.events

Interface ProvEv

All Known Subinterfaces:

[ProvInServiceEv](#), [ProvObservationEndedEv](#), [ProvOutOfServiceEv](#), [ProvShutdownEv](#)

public interface **ProvEv**
extends [Ev](#)

The **ProvEv** interface is the base interface for all Provider- related events. All events which pertain to the Provider object must extend this interface. Events which extend this interface are reported via the [ProviderObserver](#) interface.

The core package defines events which are reported when the Provider changes state. These events are: [ProvInServiceEv](#), [ProvOutOfServiceEv](#), and [ProvShutdownEv](#). Also, the core package defines the [ProvObservationEndedEv](#) event which is sent when the Provider becomes unobservable.

The [ProvEv.getProvider\(\)](#) method on this interface returns the Provider associated with the Provider event.

See Also:

[ProvInServiceEv](#), [ProvOutOfServiceEv](#), [ProvShutdownEv](#),
[ProvObservationEndedEv](#), [Ev](#), [ProviderObserver](#), [Provider](#)

Fields inherited from interface [javax.telephony.events.Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Method Summary

Provider	getProvider() Returns the Provider associated with this Provider event.
--------------------------	---

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Method Detail

getProvider

public Provider **getProvider()**

Returns the Provider associated with this Provider event.

Returns:

The Provider associated with this event.

javax.telephony.events

Interface ProvInServiceEv

public interface **ProvInServiceEv**
 extends [ProvEv](#)

The `ProvInServiceEv` interface indicates that the state of the Provider object has changed to `Provider.IN_SERVICE`. This interface extends the `ProvEv` interface and is reported via the `ProviderObserver` interface.

See Also:

[Provider](#), [ProviderObserver](#), [ProvEv](#)

Field Summary

static int	<u>ID</u>
	Event id.

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[ProvEv](#)

[getProvider](#)

Methods inherited from interface javax.telephony.events.[Ev](#)

[getCause](#), [getID](#), [getMetaCode](#), [getObserved](#), [isNewMetaEvent](#)

Field Detail

ID`public static final int ID`

Event id.

javax.telephony.events***Interface ProvObservationEndedEv***

public interface **ProvObservationEndedEv**
 extends ProvEv

The ProvObservationEndedEv event indicates that the application will no longer receive Provider events on the instance of the ProviderObserver. This interface extends the ProvEv interface and is reported on the ProviderObserver interface.

See Also:

ProvEv, ProviderObserver

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.events.ProvEv

getProvider

Methods inherited from interface javax.telephony.events.Ev

getCause, getID, getMetaCode, getObserved, isNewMetaEvent

Field Detail

ID`public static final int ID`Event id

*javax.telephony.events****Interface ProvOutOfServiceEv***

public interface **ProvOutOfServiceEv**
 extends [ProvEv](#)

The ProvOutOfServiceEv interface indicates that the state of the Provider object has changed to Provider.OUT_OF_SERVICE. This interface extends the ProvEv interface and is reported via the ProviderObserver interface.

See Also:

[Provider](#), [ProviderObserver](#), [ProvEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[ProvEv](#)

[getProvider](#)

Methods inherited from interface javax.telephony.events.[Ev](#)

[getCause](#), [getID](#), [getMetaCode](#), [getObserved](#), [isNewMetaEvent](#)

Field Detail

ID`public static final int ID`Event id

javax.telephony.events

Interface ProvShutdownEv

public interface **ProvShutdownEv**
 extends [ProvEv](#)

The ProvShutdownEv interface indicates that the state of the Provider object has changed to Provider.SHUTDOWN. This interface extends the ProvEv interface and is reported via the ProviderObserver interface.

See Also:

[Provider](#), [ProviderObserver](#), [ProvEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[ProvEv](#)

[getProvider](#)

Methods inherited from interface javax.telephony.events.[Ev](#)

[getCause](#), [getID](#), [getMetaCode](#), [getObserved](#), [isNewMetaEvent](#)

Field Detail

ID`public static final int ID`Event id

javax.telephony.events***Interface TermConnActiveEv***

public interface **TermConnActiveEv**
 extends TermConnEv

The **TermConnActiveEv** interface indicates that the state of the **TerminalConnection** object has changed to **TerminalConnection.ACTIVE**. This interface extends the **TermConnEv** interface and is reported via the **CallObserver** interface.

See Also:

TerminalConnection, CallObserver, TermConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.events.TermConnEv

getTerminalConnection

Methods inherited from interface javax.telephony.events.CallEv

getCall

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

*javax.telephony.events****Interface TermConnCreatedEv***

public interface **TermConnCreatedEv**
 extends TermConnEv

The TermConnDroppedEv interface indicates that a new TerminalConnection object has been created. This interface extends the TermConnEv interface and is reported via the CalObserver interface.

See Also:

TerminalConnection, CalObserver, TermConnEv

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Methods inherited from interface javax.telephony.events.TermConnEv

getTerminalConnection

Methods inherited from interface javax.telephony.events.CallEv

getCall

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events***Interface TermConnDroppedEv***

public interface **TermConnDroppedEv**
 extends [TermConnEv](#)

The `TermConnDroppedEv` interface indicates that the state of the `TerminalConnection` object has changed to `TerminalConnection.DROPPED`. This interface extends the `TermConnEv` interface and is reported via the `CallObserver` interface.

See Also:

[TerminalConnection](#), [CallObserver](#), [TermConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[TermConnEv](#)

[getTerminalConnection](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events

Interface TermConnEv

All Known Subinterfaces:

[CallCtlTermConnBridgedEv](#), [CallCtlTermConnDroppedEv](#), [CallCtlTermConnEv](#),
[CallCtlTermConnHeldEv](#), [CallCtlTermConnInUseEv](#), [CallCtlTermConnRingingEv](#),
[CallCtlTermConnTalkingEv](#), [CallCtlTermConnUnknownEv](#), [TermConnActiveEv](#),
[TermConnCreatedEv](#), [TermConnDroppedEv](#), [TermConnPassiveEv](#), [TermConnRingingEv](#),
[TermConnUnknownEv](#)

public interface **TermConnEv**
 extends [CalEv](#)

The **TermConnEv** interface is the base event interface for all **TerminalConnection**-related events. All events which pertain to the **TerminalConnection** object must extend this interface. This interface extends the **CalEv** interface and therefore is reported via the **CalObserver** interface.

The core package defines events which are reported when the **TerminalConnection** changes state. These events are: **TermConnRingingEv**, **TermConnActiveEv**, **TermConnPassiveEv**, **TermConnDroppedEv**, and **TermConnUnknownEv**. Also, a **TermConnCreatedEv** is sent when a new **TerminalConnection** is created.

The **TermConnEv.getTerminalConnection()** method on this interface returns the **TerminalConnection** associated with this **TerminalConnection** event.

See Also:

[TerminalConnection](#), [CalObserver](#), [CalEv](#), [TermConnEv](#), [TermConnRingingEv](#),
[TermConnActiveEv](#), [TermConnPassiveEv](#), [TermConnDroppedEv](#),
[TermConnUnknownEv](#)

Fields inherited from interface **javax.telephony.events.Ev**

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Method Summary

<u>TerminalConnection</u>	getTerminalConnection() Returns the TerminalConnection associated with this event.
---------------------------	--

Methods inherited from interface javax.telephony.events.CallEv

getCall

Methods inherited from interface javax.telephony.events.Ev

getCause, getID, getMetaCode, getObserved, isNewMetaEvent

Method Detail

getTerminalConnection

public TerminalConnection **getTerminalConnection()**

Returns the TerminalConnection associated with this event.

Returns:

The TerminalConnection associated with this event.

*javax.telephony.events****Interface TermConnPassiveEv***

public interface **TermConnPassiveEv**

extends [TermConnEv](#)

The `TermConnPassiveEv` interface indicates that the state of the `TerminalConnection` object has changed to `TerminalConnection.PASSIVE`. This interface extends the `TermConnEv` interface and is reported via the `CallObserver` interface.

See Also:

[TerminalConnection](#), [CallObserver](#), [TermConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface [javax.telephony.events.Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface [javax.telephony.events.TermConnEv](#)

[getTerminalConnection](#)

Methods inherited from interface [javax.telephony.events.CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events***Interface TermConnRingEv***

public interface **TermConnRingEv**
 extends [TermConnEv](#)

The **TermConnRingEv** interface indicates that the state of the **TerminalConnection** object has changed to **TerminalConnection.RINGING**. This interface extends the **TermConnEv** interface and is reported via the **CalObserver** interface.

See Also:

[TerminalConnection](#), [CalObserver](#), [TermConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[TermConnEv](#)

[getTerminalConnection](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events***Interface TermConnUnknownEv***

public interface **TermConnUnknownEv**
 extends [TermConnEv](#)

The TermConnUnknownEv interface indicates that the state of the TerminalConnection object has changed to TerminalConnection.UNKNOWN. This interface extends the TermConnEv interface and is reported via the CallObserver interface.

See Also:

[TerminalConnection](#), [CallObserver](#), [TermConnEv](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[TermConnEv](#)

[getTerminalConnection](#)

Methods inherited from interface javax.telephony.events.[CallEv](#)

[getCall](#)

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Field Detail

ID

```
public static final int ID
```

Event id

javax.telephony.events

Interface TermEv

All Known Subinterfaces:

[ButtonInfoEv](#), [ButtonPressEv](#), [CallCtlTermDoNotDisturbEv](#), [CallCtlTermEv](#), [DisplayUpdateEv](#), [HookswitchStateEv](#), [LampModeEv](#), [MicrophoneGainEv](#), [PhoneTermEv](#), [RingerPatternEv](#), [RingerVolumeEv](#), [SpeakerVolumeEv](#), [TermObservationEndedEv](#)

public interface **TermEv**
extends [Ev](#)

The **TermEv** interface is the base interface for all Terminal-related events. All events which pertain to the Terminal object must extend this interface. Events which extend this interface are reported via the **TerminalObserver** interface.

The only event defined in the core package for the Terminal is the **TermObservationEndedEv**.

The **TermEv.getTerminal()** method on this interface returns the Terminal associated with the Terminal event.

See Also:

[TermObservationEndedEv](#), [Ev](#), [TerminalObserver](#), [Terminal](#)

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Method Summary

Terminal	getTerminal() Returns the Terminal associated with this Terminal event.
--------------------------	---

Methods inherited from interface <code>javax.telephony.events.Ev</code>
<code>getCause</code> , <code>getID</code> , <code>getMetaCode</code> , <code>getObserved</code> , <code>isNewMetaEvent</code>

Method Detail

getTerminal

public Terminal **getTerminal**()

Returns the Terminal associated with this Terminal event.

Returns:

The Terminal associated with this event.

javax.telephony.events***Interface TermObservationEndedEv***

public interface **TermObservationEndedEv**
 extends [TermEv](#)

The **TermObservationEndedEv** event indicates that the application will no longer receive Terminal events on the instance of the **TerminalObserver**. This interface extends the **TermEv** interface and is reported on the **TerminalObserver** interface.

See Also:

[TermEv](#), [TerminalObserver](#)

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE CALL CANCELLED](#), [CAUSE DEST NOT OBTAINABLE](#),
[CAUSE INCOMPATIBLE DESTINATION](#), [CAUSE LOCKOUT](#),
[CAUSE NETWORK CONGESTION](#), [CAUSE NETWORK NOT OBTAINABLE](#),
[CAUSE NEW CALL](#), [CAUSE NORMAL](#), [CAUSE RESOURCES NOT AVAILABLE](#),
[CAUSE SNAPSHOT](#), [CAUSE UNKNOWN](#), [META CALL ADDITIONAL PARTY](#),
[META CALL ENDING](#), [META CALL MERGING](#), [META CALL PROGRESS](#),
[META CALL REMOVING PARTY](#), [META CALL STARTING](#), [META CALL TRANSFERRING](#),
[META SNAPSHOT](#), [META UNKNOWN](#)

Methods inherited from interface javax.telephony.events.[TermEv](#)

[getTerminal](#)

Methods inherited from interface javax.telephony.events.[Ev](#)

[getCause](#), [getID](#), [getMetaCode](#), [getObserved](#), [isNewMetaEvent](#)

Field Detail

ID`public static final int ID`Event id

Package javax.telephony.mobile

The JTAPI Mobile package extends the core capabilities of a JTAPI implementation with capabilities specific to mobile networks.

See:

Description

Interface Summary	
<u>MobileAddress</u>	A <code>MobileAddress</code> interface represents what we commonly think of as a "telephone number." In the mobile world, the telephone number depends of the subscription and so often of the SIM card.
<u>MobileNetwork</u>	A <code>MobileNetwork</code> interface represents an actual cellular network.
<u>MobileProvider</u>	A <code>MobileProvider</code> represents the telephony software-entity that interfaces with a wireless subsystem.
<u>MobileProviderEvent</u>	The <code>MobileProviderEvent</code> interface is the interface for all mobileprovider events.
<u>MobileProviderListener</u>	The <code>MobileProviderListener</code> interface reports all changes which happen to the <code>MobileProvider</code> object.
<u>MobileRadio</u>	The <code>MobileRadio</code> interface is used for simple radio management capabilities (turning on and off), if the platform supports it.
<u>MobileRadioEvent</u>	The <code>MobileRadioEvent</code> class is the listener event class for all radio state change events.
<u>MobileRadioListener</u>	The <code>MobileRadioListener</code> interface reports all changes which happen to the <code>MobileRadio</code> object.
<u>MobileTerminal</u>	A <code>MobileTerminal</code> represents a physical hardware endpoint connected to a wireless telephony domain.
<u>NetworkSelection</u>	The <code>NetworkSelection</code> interface provides the specific list based services.

Package javax.telephony.mobile Description

The JTAPI Mobile package extends the core capabilities of a JTAPI implementation with capabilities specific to mobile networks.

Introduction

The overall architecture of the JTAPI packages was designed to flexibly represent a wide range of telephony devices. While the advanced call control and call center packages provide added features for third party world view at switches and server applications, other packages are designed to enhance the terminal side of first party only devices. e.g. the phone package for dealing with physical devices such as ringer, buttons and hookswitches.

The Mobile package adds to the first party view of the telephony network the features needed to support the unique characteristics visible to the wireless network device endpoint. This package was initially designed with GSM networks in mind, but has been crafted to work well in other wireless networks as well.

Mobile device characteristics

Several characteristics of the mobile device are worth highlighting here to understand some of the design decisions that were made in the Mobile package that may appear contrary to design patterns used in earlier releases of JTAPI :

- small footprint - mobile devices are typically very cost sensitive and require drastic design choices in minimizing the overall footprint of interface and implementation of the telephony applications.
 - No capabilities interfaces - optional static capabilities are supported via interfaces and checks for instance of interface.
 - Listener events - Observer interfaces are typically bulky in the use of the class name space to convey individual state transitions.
 - Return values - by defining meaningful return values for methods that were unused fewer methods are required, e.g. returning a null String for a get property operation for an not supported operation, or returning a boolean to indicate the success or failure of an operation
 - Exceptions - in cases where a partial implementation of a feature are provided, exceptions can be used to provide finer granularity of optional implementation features. e.g. NetworkSelection scanning for networks may be provided in an implementation that throws method not supported exceptions if the scanning operation can not be canceled once started.
- Emergency Provider State - in addition to basic in and out of service states of the telephony network, the cellular network attempts to provide emergency service even if the device has been compromised (SIM card removed), or the device is otherwise restricted (onboard a plane).
- Radio Management - mobile devices are very power conscious and the radio equipment very power intensive. Users typically have the ability to control whether the radio functions of the devices are automatically enabled when the device is powered on or whether these features are managed separately from other Personal Information Management applications on the device.
- Network Selection - In many locations a mobile phone user has a choice of cellular networks. The selection mechanism might be automatic or intelligently chosen based on costs or even manually with end user intervention.

Network Types

The term **available network** means a mobile cellular network that is in the general geographical area as the mobile device such that the device can detect and communicate with a cell of that network. And that the network can offer, at a minimum, restricted communication service to the mobile device.

Current network refers to the mobile network service provider that the mobile device is currently registered with. That is, the network the mobile device is receiving communications through.

Home network is the network the mobile device has a subscription of service with.

Hierarchy For Package javax.telephony.mobile

Package Hierarchies:

All Packages

Interface Hierarchy

- interface javax.telephony.**Address**
 - interface javax.telephony.mobile.**MobileAddress**
 - interface javax.telephony.**Event**
 - interface javax.telephony.mobile.**MobileRadioEvent**
 - interface javax.telephony.**ProviderEvent**
 - interface javax.telephony.mobile.**MobileProviderEvent**
 - interface java.util.EventListener
 - interface javax.telephony.**ProviderListener**
 - interface javax.telephony.mobile.**MobileProviderListener**
 - interface javax.telephony.mobile.**MobileNetwork**
 - interface javax.telephony.mobile.**MobileRadio**
 - interface javax.telephony.mobile.**MobileRadioListener**
 - interface javax.telephony.**Provider**
 - interface javax.telephony.mobile.**MobileProvider**
 - interface javax.telephony.mobile.**NetworkSelection**
 - interface javax.telephony.**Terminal**
 - interface javax.telephony.mobile.**MobileTerminal**
-
-

javax.telephony.mobile **Interface MobileAddress**

public interface **MobileAddress**
extends [Address](#)

A `MobileAddress` interface represents what we commonly think of as a "telephone number." In the mobile world, the telephone number depends of the subscription and so often of the SIM card.

Address and Call Objects

Address objects represent the *logical* endpoints of a telephone call. A logical view of a telephone call views the call as originating from one Address endpoint and terminates at another Address endpoint.

Address and Events

In the core package, the Address is described as static. In the mobile world, a subscription (and so the phone number) may appear or disappear with the insertion or extraction of the support (subscription included in the SIM card), but the MobileAddress object life is the same as the MobileProvider object life (as in the core). If the subscription changes, it will generate an event (through the [MobileProviderListener](#)).

See Also:

[Address](#), [MobileProvider](#)

Method Summary

boolean	<u>getCallWaiting()</u> Returns the state of call 'knocking' feature.
java.lang.String	<u>getSubscriptionId()</u> Returns the Subscription Identification String associated with this address.
void	<u>setCallWaiting(boolean state)</u> Sets the state of call 'knocking' feature.

Methods inherited from interface javax.telephony.Address

[addAddressListener](#), [addCallListener](#), [addCallObserver](#), [addObserver](#),
[getAddressCapabilities](#), [getAddressListeners](#), [getCallListeners](#),
[getCallObservers](#), [getCapabilities](#), [getConnections](#), [getName](#),
[getObservers](#), [getProvider](#), [getTerminals](#), [removeAddressListener](#),
[removeCallListener](#), [removeCallObserver](#), [removeObserver](#)

Method Detail*getCallWaiting*

```
public boolean getCallWaiting()
           throws MethodNotSupportedException
```

Returns the state of call 'knocking' feature.

In the mobile world the user can turn call waiting indication on and off, as a do not disturb. `getCallWaiting` does an inquiry to the network to get the setting on or off.

Returns:

True, if call waiting is enabled.

Throws:

[MethodNotSupportedException](#) - Indicates this method is not supported by the implementation.

setCallWaiting

```
public void setCallWaiting(boolean state)
           throws MethodNotSupportedException
```

Sets the state of call 'knocking' feature.

Parameters:

`state` - true, if call waiting is enabled.

Throws:

[MethodNotSupportedException](#) - Indicates this method is not supported by the implementation.

getSubscriptionId

```
public java.lang.String getSubscriptionId()
```

Returns the Subscription Identification String associated with this address. This methods returns the subscription ID string such as IMSI. (International Mobile Subscriber Identification)

Returns:

the subscriber ID String, NULL if no such ID is implemented.

javax.telephony.mobile **Interface MobileNetwork**

public interface **MobileNetwork**

A `MobileNetwork` interface represents an actual cellular network. The `MobileNetwork` object contains all available information about the wireless network.

See Also:

[MobileProvider](#), [MobileProviderEvent](#), [MobileProviderListener](#)

Method Summary

java.lang.String	<u>getCode()</u> Returns the network code in a String format.
java.lang.String[]	<u>getNames()</u> Returns the all available network name synonyms.
boolean	<u>isAvailable()</u> Returns true if the network is currently available.
boolean	<u>isCurrent()</u> Returns true if the network is current network in use.
boolean	<u>isRestricted()</u> Returns true if the network is currently restricted.

Method Detail

isAvailable

public boolean **isAvailable()**

Returns true if the network is currently available.

Returns:

true if mobile network is available

isCurrent

```
public boolean isCurrent()
```

Returns true if the network is current network in use. (convenience method equivalent to `MobileProvider.getCurrentNetwork()`)

Returns:

true if mobile network is current

isRestricted

```
public boolean isRestricted()
```

Returns true if the network is currently restricted.

Returns:

true if mobile network is restricted

getNames

```
public java.lang.String[] getNames()
```

Returns the all available network name synonyms. This usually means one short and long name (per network). The shortest name or the only name available must be the FIRST (=[0]) in the array. The other names must be sorted by character length, also. For example, [0] = "tl", [1] = "tele", [2] = "telecom"

Returns:

All available names of a network, null, if none available.

getCode

```
public java.lang.String getCode()
```

Returns the network code in a String format.

Example: MCC-MNC, SID-NID

where : MCC = Mobile Country Code, MNC = Mobile Network Code

SID = Subscriber ID, NID = Network ID

Returns:

The network code.

javax.telephony.mobile

Interface MobileProvider

All Known Subinterfaces:

NetworkSelection

public interface **MobileProvider**
 extends Provider

A **MobileProvider** represents the telephony software-entity that interfaces with a wireless subsystem. It is an extension of the core provider with various additions needed for wireless communication.

MobileProvider States

This interface defines states for the **Provider** which provide greater detail beyond the states defined in the **Provider** interface. The states defined by this interface are related to the states defined in the core package in certain, specific ways, as defined below. Applications may obtain the **MobileProvider** state via the **Provider.getState()** method defined on the core. This method returns one of the integer state constants defined in this interface.

Below is a description of each **MobileProvider** state in real-world terms. These real-world descriptions only serve to provide a more intuitive understanding of what is going on. Several methods in this specification state pre-conditions based upon the **MobileProvider** state. Most of the states are identical to those defined in the core package.

MobileProvider.RESTRICTED_SERVICE

This state indicates that the subscription is invalid (invalid subscription, forbidden network...); from the network point of view, only emergency calls are allowed. Local operations (such as Personal Information Management) are available.

Core vs. Mobile Package States

There is a strong relationship between the **MobileProvider** states and the **Provider** states. If an implementation supports the mobile package, it must ensure this relationship is properly maintained.

Since the states defined in the **MobileProvider** interface provide more detail to the states defined in the **Provider** interface, each state in the **Provider** interface corresponds to a state defined in the **MobileProvider** interface. Conversely, each **MobileProvider** state corresponds to exactly one **Provider** state. This arrangement permits applications to view either the core state or the mobile state and still see a consistent view.

The following table outlines the relationship between the core package Provider states and the mobile package Provider states.

<i>If the mobile package state is...</i>	<i>then the core package state must be...</i>
MobileProvider.RESTRICTED_SERVICE	Provider.IN_SERVICE

Listeners and Events

Each time a state changes occurs on a MobileProvider, the application is notified via an *event*. This event is reported via the MobileProviderListener interface. Applications instantiate objects which implement this interface and use the Provider.addProviderListener(MobileProviderListener) method to begin the delivery of events.

See Also:

MobileProviderEvent, MobileProviderListener

Field Summary

static int	<u>RESTRICTED_SERVICE</u> The MobileProvider.RESTRICTED_SERVICE state indicates a MobileProvider services are restricted because of missing user subscription or invalid/missing password such as four-digit PIN-code.
------------	--

Fields inherited from interface javax.telephony.Provider

IN_SERVICE, OUT_OF_SERVICE, SHUTDOWN

Method Summary

void	<u>cancelAvailableNetworkRequest()</u> Cancels an outstanding getAvailableNetworks-request, if possible.
<u>MobileNetwork</u> []	<u>getAvailableNetworks(boolean fullsearch)</u> Scans for available networks in the area or returns the previous search results, if available.
<u>MobileNetwork</u>	<u>getCurrentNetwork()</u> Returns the current network object.
<u>MobileNetwork</u>	<u>getHomeNetwork()</u> Get the home network object.
int	<u>getMobileState()</u> Returns the current state of the MobileProvider.
java.util.Date	<u>getNetworkTime()</u> Get the network time.
java.util.TimeZone	<u>getNetworkTimeZone()</u> Get the network time zone.
java.lang.String	<u>getType()</u> Returns the type of network/wireless service.
boolean	<u>isRoaming()</u> Return the current roaming status.
void	<u>setNetwork(MobileNetwork network)</u> Selects a network as the current servicing network.

Methods inherited from interface javax.telephony.Provider

addObserver, addProviderListener, createCall, getAddress,
getAddressCapabilities, getAddressCapabilities, getAddresses,
getCallCapabilities, getCallCapabilities, getCalls, getCapabilities,
getConnectionCapabilities, getConnectionCapabilities, getName,
getObservers, getProviderCapabilities, getProviderCapabilities,
getProviderListeners, getState, getTerminal, getTerminalCapabilities,
getTerminalCapabilities, getTerminalConnectionCapabilities,
getTerminalConnectionCapabilities, getTerminals, removeObserver,
removeProviderListener, shutdown

Field Detail

RESTRICTED_SERVICE

```
public static final int RESTRICTED_SERVICE
```

The `MobileProvider.RESTRICTED_SERVICE` state indicates a `MobileProvider` services are restricted because of missing user subscription or invalid/missing password such as four-digit PIN-code. In this state, only emergency calls are allowed.

Method Detail

getMobileState

```
public int getMobileState()
```

Returns the current state of the `MobileProvider`.

Returns:

The current state of the `mobileprovider`.

getAvailableNetworks

```
public MobileNetwork[] getAvailableNetworks(boolean fullsearch)  
                                     throws MethodNotSupportedException,  
                                     InvalidStateException,  
                                     ResourceUnavailableException
```

Scans for available networks in the area or returns the previous search results, if available. The state of particular network may or may not be known.

Parameters:

`fullsearch` - True, if full search is initiated. This may take several tens of seconds to complete. False, if previous results without a full scan are asked for.

Returns:

A list of networks in the area or previous search results.

Throws:

MethodNotSupportedException - Indicates that the scanning for available networks is not supported.

InvalidStateException - Indicates the implementation is in a such state which network search is not possible (call ongoing, for example.)

ResourceUnavailableException - indicates that a resource inside the system is not available to complete an operation. This may often be the case when the application wants the previous results.

See Also:MobileNetwork

cancelAvailableNetworkRequest

```
public void cancelAvailableNetworkRequest()  
           throws MethodNotSupportedException,  
                  InvalidStateException
```

Cancels an outstanding getAvailableNetworks-request, if possible.

Throws:

MethodNotSupportedException - Indicates the cancelling of the radio scan capability is not supported.

InvalidStateException - Indicates the implementation is in a such state in which the cancelling of an operation is not possible.

getCurrentNetwork

```
public MobileNetwork getCurrentNetwork()
```

Returns the current network object. If no connection to a network is available this must return null. The current network is the one the mobile has registered to.

Returns:

A network object of the current network.

getHomeNetwork

```
public MobileNetwork getHomeNetwork()  
           throws MethodNotSupportedException,  
                  ResourceUnavailableException
```

Get the home network object. The information capability is in some cases dynamic i.e., it may or may not be available. (Information may be stored in a changeable SIM-card, for example.)

Returns:

The home network.

Throws:

MethodNotSupportedException - Indicates the home network information capability is not supported.

ResourceUnavailableException - Indicates that a resource inside the system is not available to complete an operation.

setNetwork

```
public void setNetwork(MobileNetwork network)
    throws MethodNotSupportedException,
           InvalidStateException,
           ResourceUnavailableException
```

Selects a network as the current servicing network. If the selected network is available, it will become the current network. **Note: setting the network with this method forces the network selection mode to manual.**

Parameters:

network - Network to be selected.

Throws:

MethodNotSupportedException - Indicates network selection is not supported.

InvalidStateException - Indicates the implementation is in a state in which setting the network is not possible (call ongoing, for example.)

ResourceUnavailableException - Indicates that a resource inside the system is not available to complete the operation.

isRoaming

```
public boolean isRoaming()
    throws MethodNotSupportedException,
           InvalidStateException
```

Return the current roaming status. True, if roaming.

Returns:

The current roaming status.

Throws:

MethodNotSupportedException - Indicates this method is not supported by the implementation.

InvalidStateException - Indicates the `MobileProvider` or `MobileRadio` is in a state in which the roaming state is not available.

getType

```
public java.lang.String getType()
```

Returns the type of network/wireless service. Totally up the implementation. Example: "GSM", "PCS1900", "CDMA" ...

Returns:

A string representing the air interface type.

getNetworkTime

```
public java.util.Date getNetworkTime()  
                        throws MethodNotSupportedException
```

Get the network time.

Returns:

The current network time.

Throws:

MethodNotSupportedException - Indicates the home network information capability is not supported.

getNetworkTimeZone

```
public java.util.TimeZone getNetworkTimeZone()  
                        throws MethodNotSupportedException
```

Get the network time zone.

Returns:

The current network time zone.

Throws:

MethodNotSupportedException - Indicates the home network information capability is not supported.

javax.telephony.mobile ***Interface MobileProviderEvent***

public interface **MobileProviderEvent**
extends ProviderEvent

The `MobileProviderEvent` interface is the interface for all mobileprovider events. Each event is specified by unique identification number, specified in the `MobileProvider` interface.

State and State Cause Mapping

In each state, the state cause must be set as described below:

<i>MobileProvider State</i>	<i>MobileProviderEvent Cause</i>
IN_SERVICE	CAUSE_NORMAL
RESTRICTED_SERVICE	CAUSE_FORBIDDEN, CAUSE_FORBIDDEN_ZONE, CAUSE_NETWORK_NOT_SELECTED, CAUSE_SUBSCRIPTION_ERROR or CAUSE_SEARCHING
SHUTDOWN	CAUSE_NORMAL
OUT_OF_SERVICE	CAUSE_RADIO_OFF, CAUSE_NO_NETWORK or CAUSE_UNKNOWN

See Also:

MobileProvider, MobileProviderEvent, MobileProviderListener

Field Summary	
static int	<u>CAUSE_FORBIDDEN</u> The MobileProvider.CAUSE_FORBIDDEN cause indicates the provider is in the restricted state because the current network is forbidden i.e., emergency calls only.
static int	<u>CAUSE_FORBIDDEN_ZONE</u> The MobileProvider.CAUSE_FORBIDDEN_ZONE cause indicates the provider is in the restricted state because the mobile phone is currently in a forbidden regional zone i.e., only emergency calls are allowed.
static int	<u>CAUSE_ILLEGAL_MOBILE</u> The MobileProvider.CAUSE_ILLEGAL_MOBILE indicates there maybe a problem with mobile equipment ID information.
static int	<u>CAUSE_NETWORK_NOT_SELECTED</u> The MobileProvider.CAUSE_NETWORK_NOT_SELECTED value indicates that currently no network is selected but networks are available in the area.
static int	<u>CAUSE_NO_NETWORK</u> The MobileProvider.CAUSE_NO_NETWORK value indicates there is no cellular networks detected in the area.
static int	<u>CAUSE_NORMAL</u> The MobileProvider.CAUSE_NORMAL cause indicates there is no cause.
static int	<u>CAUSE_RADIO_OFF</u> The MobileProvider.CAUSE_RADIO_OFF cause indicates that the radio is turned off.
static int	<u>CAUSE_SEARCHING</u> The MobileProvider.CAUSE_SEARCHING indicates the implementation is searching for cellular networks.
static int	<u>CAUSE_SUSBCRIPTION_ERROR</u> The MobileProvider.CAUSE_SUSBCRIPTION_ERROR cause indicates the provider is in the restricted or in out_of_service state because of a subscription related error.
static int	<u>CAUSE_UNKNOWN</u> The MobileProvider.CAUSE_UNKNOWN value indicates the reason for state is not available or is none of the above causes.
static int	<u>MOBILEPROVIDER_NETWORK_CHANGE</u> The event id for MobileProvider network change.
static int	<u>MOBILEPROVIDER_NO_NETWORK</u> The event id for MobileProvider no network situation.
static int	<u>PROVIDER_RESTRICTED_SERVICE</u> The event id for MobileProvider state change for restricted network service.

Fields inherited from interface javax.telephony.ProviderEvent

PROVIDER EVENT TRANSMISSION ENDED, PROVIDER IN SERVICE,
PROVIDER OUT OF SERVICE, PROVIDER SHUTDOWN

Fields inherited from interface javax.telephony.Event

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN

Method Summary

<u>MobileProvider</u>	<u>getMobileProvider()</u> Returns the MobileProvider associated with this event.
int	<u>getMobileStateCause()</u> Returns the cause for a service state.

Methods inherited from interface javax.telephony.ProviderEvent

getProvider

Methods inherited from interface javax.telephony.Event

getCause, getID, getMetaEvent, getSource

Field Detail

CAUSE_NORMAL

```
public static final int CAUSE_NORMAL
```

The `MobileProvider.CAUSE_NORMAL` cause indicates there is no cause.

CAUSE_FORBIDDEN

```
public static final int CAUSE_FORBIDDEN
```

The `MobileProvider.CAUSE_FORBIDDEN` cause indicates the provider is in the restricted state because the current network is forbidden i.e., emergency calls only.

CAUSE_FORBIDDEN_ZONE

```
public static final int CAUSE_FORBIDDEN_ZONE
```

The `MobileProvider.CAUSE_FORBIDDEN_ZONE` cause indicates the provider is in the restricted state because the mobile phone is currently in a forbidden regional zone i.e., only emergency calls are allowed.

CAUSE_SUSBCRIPTION_ERROR

```
public static final int CAUSE_SUSBCRIPTION_ERROR
```

The `MobileProvider.CAUSE_SUSBCRIPTION_ERROR` cause indicates the provider is in the restricted or in out_of_service state because of a subscription related error. The SIM is either missing or there is some other problem such as invalid entry code.

CAUSE_RADIO_OFF

```
public static final int CAUSE_RADIO_OFF
```

The `MobileProvider.CAUSE_RADIO_OFF` cause indicates that the radio is turned off. In other words, the `MobileRadio` is off.

CAUSE_NO_NETWORK

```
public static final int CAUSE_NO_NETWORK
```

The `MobileProvider.CAUSE_NO_NETWORK` value indicates there is no cellular networks detected in the area.

CAUSE_NETWORK_NOT_SELECTED

public static final int **CAUSE_NETWORK_NOT_SELECTED**

The `MobileProvider.CAUSE_NETWORK_NOT_SELECTED` value indicates that currently no network is selected but networks are available in the area.

CAUSE_SEARCHING

public static final int **CAUSE_SEARCHING**

The `MobileProvider.CAUSE_SEARCHING` indicates the implementation is searching for cellular networks.

CAUSE_ILLEGAL_MOBILE

public static final int **CAUSE_ILLEGAL_MOBILE**

The `MobileProvider.CAUSE_ILLEGAL_MOBILE` indicates there maybe a problem with mobile equipment ID information. This cause is because sometimes the subscription ID is not available and the mobile equipment ID is used to authenticate the mobile. It is through this that invalid or stolen equipment can be identified and the network will refuse normal service, but will provide restricted (emergency) service. Some systems keep a list of stolen mobiles - called the black list. Gray list is mobiles that have some software or radio problem and need to be recalled.

CAUSE_UNKNOWN

public static final int **CAUSE_UNKNOWN**

The `MobileProvider.CAUSE_UNKNOWN` value indicates the reason for state is not available or is none of the above causes.

PROVIDER_RESTRICTED_SERVICE

public static final int **PROVIDER_RESTRICTED_SERVICE**

The event id for `MobileProvider` state change for restricted network service.

MOBILEPROVIDER_NETWORK_CHANGE

public static final int **MOBILEPROVIDER_NETWORK_CHANGE**

The event id for `MobileProvider` network change.

MOBILEPROVIDER_NO_NETWORK

public static final int **MOBILEPROVIDER_NO_NETWORK**

The event id for `MobileProvider` no network situation.

Method Detail

getMobileProvider

public `MobileProvider` **getMobileProvider()**

Returns the `MobileProvider` associated with this event.

Returns:

The `MobileProvider` associated with this event.

getMobileStateCause

public int **getMobileStateCause()**

Returns the cause for a service state.

Returns:

a cause value.

javax.telephony.mobile

Interface MobileProviderListener

public interface **MobileProviderListener**
extends [ProviderListener](#)

The `MobileProviderListener` interface reports all changes which happen to the `MobileProvider` object. These changes are reported as one method call with separate event ids for each event type. The applications must instantiate an object which implements this interface and then use the `Provider.addProviderListener` method to register the object to receive all future events associated with the `MobileProvider` object.

See Also:

[MobileProvider](#), [MobileProviderEvent](#)

Method Summary

void	networkChanged (MobileProviderEvent event) The network of <code>MobileProvider</code> has changed.
void	serviceRestricted (MobileProviderEvent event) The state of <code>MobileProvider</code> has changed to restricted use mode.

Methods inherited from interface `javax.telephony.ProviderListener`

[providerEventTransmissionEnded](#), [providerInService](#),
[providerOutOfService](#), [providerShutdown](#)

Method Detail

serviceRestricted

public void **[serviceRestricted](#)**([MobileProviderEvent](#) event)

The state of `MobileProvider` has changed to restricted use mode. The core `ProviderEvent` supplies `inService`, `outOfService` and `serviceShutdown` transitions. The `MobileProvider` adds the notification of `serviceRestricted` transitions.

Parameters:

event - service restricted event.

networkChanged

```
public void networkChanged(MobileProviderEvent event)
```

The network of MobileProvider has changed.

Parameters:

event - the network has changed due to roaming operation

javax.telephony.mobile ***Interface MobileRadio***

public interface **MobileRadio**

The **MobileRadio** interface is used for simple radio management capabilities (turning on and off), if the platform supports it, and getting information about the current signal strength.

Network States

The **MobileRadio** state can either be on (**setState(true)**) or off (**setState(false)**). If state management is not user controllable, then the **setState** method will throw a **MethodNotSupportedException**. If the device has been secured via password protection (e.g. PIN-code), the **setState** method may also throw the **InvalidStateException**, if an attempt is made to manipulate the radio while the device is secured.

Mobile Radio Listeners

All changes in a **MobileRadio** object are reported via the **MobileRadioListener** interface. Applications instantiate an object which implements this interface and begins this delivery of events to this object using the **MobileRadio.addListener()** method. Applications receive events on a listener until the listener is removed via the **MobileRadio.removeRadioListener()** method.

See Also:

MobileRadioEvent, **MobileRadioListener**

Method Summary

void	<u>addRadioListener</u> (<u>MobileRadioListener</u> listener) Add a listener for <code>MobileRadio</code> events.
int	<u>getMaxSignalLevel</u> () Get the maximum radio signal level.
boolean	<u>getRadioStartState</u> () Returns the <code>MobileRadio</code> startup state at boot.
boolean	<u>getRadioState</u> () Returns the current <code>MobileRadio</code> state.
int	<u>getSignalLevel</u> () Get the current radio signal level.
void	<u>removeRadioListener</u> (<u>MobileRadioListener</u> listener) Remove a listener from the <code>MobileRadio</code> .
void	<u>setRadioStartState</u> (boolean state) Sets the <code>MobileRadio</code> startup state at boot.
void	<u>setRadioState</u> (boolean state) Set the <code>MobileRadio</code> state.

Method Detail

getRadioState

```
public boolean getRadioState()
    throws MethodNotSupportedException
```

Returns the current `MobileRadio` state.

Returns:

true if the radio is on, otherwise false.

Throws:

MethodNotSupportedException - can not access the current radio state.

setRadioState

```
public void setRadioState(boolean state)
    throws MethodNotSupportedException,
           InvalidStateException
```

Set the `MobileRadio` state. The `MobileProvider` must be `OUT_OF_SERVICE` before the `MobileRadio` object may turn the physical radio off. After the radio is successfully turned off the `MobileRadio` state must be changed to reflect the radio off status. When the radio is turned off, no further network events will be generated and the radio signal level is set to zero, until the radio is reenabled.

When the radio is turned on, the `MobileProvider` can be in any valid state, depending on whether there are networks detected and subscription details are OK.

Parameters:

`state` - The desired radio state.

Throws:

[MethodNotSupportedException](#) - Indicates the radio state can not be manipulated on this implementation.

[InvalidStateException](#) - Indicates the network service state cannot be changed due to current `MobileProvider` state.

getRadioStartState

```
public boolean getRadioStartState()  
           throws MethodNotSupportedException
```

Returns the `MobileRadio` startup state at boot.

Returns:

The current radio startup state.

Throws:

[MethodNotSupportedException](#) - Indicates the radio state startup management capability is not supported.

setRadioStartState

```
public void setRadioStartState(boolean state)  
           throws MethodNotSupportedException,  
                  InvalidStateException
```

Sets the `MobileRadio` startup state at boot. A possible required password will not be given here for security reasons! The password required should be given some other way, determined by native implementation, or at next time when a `MobileProvider` object is requested from the `JtapiPeer`.

Parameters:

`state` - The new radio startup state. True, if radio is turned on. Otherwise, false.

Throws:

[MethodNotSupportedException](#) - Indicates the radio state startup management is not supported.

[InvalidStateException](#) - Indicates the network service startup state cannot be changed due to current platform state.

getSignalLevel

```
public int getSignalLevel()
```

Get the current radio signal level.

When the radio is turned off the signal level is set to zero. The radio signal is mapped to positive integer values from 1 to `getMaxSignalLevel`. In GSM networks the signal strength is mapped to the range 1-5. (e.g. Level 1 = 20%, Level 5 = 100%)

Returns:

The current radio signal level.

See Also:

[`getMaxSignalLevel\(\)`](#)

getMaxSignalLevel

```
public int getMaxSignalLevel()
```

Get the maximum radio signal level.

Returns:

The maximum radio signal level.

See Also:

[`getSignalLevel\(\)`](#)

addRadioListener

```
public void addRadioListener(MobileRadioListener listener)  
    throws MethodNotSupportedException
```

Add a listener for `MobileRadio` events.

Parameters:

`listener` - The listener object to add.

Throws:

[`MethodNotSupportedException`](#) - This implementation does not generate events.

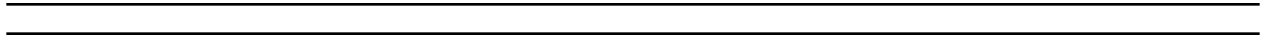
removeRadioListener

```
public void removeRadioListener(MobileRadioListener listener)
```

Remove a listener from the `MobileRadio`.

Parameters:

`listener` - The listener to remove.



javax.telephony.mobile **Interface MobileRadioEvent**

public interface **MobileRadioEvent**
extends Event

The `MobileRadioEvent` class is the listener event class for all radio state change events. Each event is specified by unique identification number.

Field Summary

static int	<u>MOBILERADIO_RADIO_OFF</u> The event id for radio turned off event.
static int	<u>MOBILERADIO_RADIO_ON</u> The event id for radio turned on event.
static int	<u>MOBILERADIO_SIGNAL_LEVEL_CHANGED</u> The event id for signal level change.
static int	<u>MOBILERADIO_STARTSTATE_OFF</u> The event id for radio start state change to off.
static int	<u>MOBILERADIO_STARTSTATE_ON</u> The event id for radio start state change to on.

Fields inherited from interface javax.telephony.Event

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN

Method Summary

<u>MobileRadio</u>	<u>getMobileRadio()</u> Returns the <code>MobileRadio</code> associated with this event.
--------------------	--

Methods inherited from interface `javax.telephony.Event``getCause`, `getID`, `getMetaEvent`, `getSource`**Field Detail*****MOBILERADIO_SIGNAL_LEVEL_CHANGED***

```
public static final int MOBILERADIO_SIGNAL_LEVEL_CHANGED
```

The event id for signal level change.

MOBILERADIO_STARTSTATE_ON

```
public static final int MOBILERADIO_STARTSTATE_ON
```

The event id for radio start state change to on.

MOBILERADIO_STARTSTATE_OFF

```
public static final int MOBILERADIO_STARTSTATE_OFF
```

The event id for radio start state change to off.

MOBILERADIO_RADIO_ON

```
public static final int MOBILERADIO_RADIO_ON
```

The event id for radio turned on event.

MOBILERADIO_RADIO_OFF

```
public static final int MOBILERADIO_RADIO_OFF
```

The event id for radio turned off event.

Method Detail

getMobileRadio

public MobileRadio **getMobileRadio()**

Returns the MobileRadio associated with this event.

Returns:

The MobileRadio associated with this event.

javax.telephony.mobile **Interface MobileRadioListener**

public interface **MobileRadioListener**

The `MobileRadioListener` interface reports all changes which happen to the `MobileRadio` object. These changes are reported as separate method calls. Applications must instantiate an object which implements this interface and then add the Listener to the network to receive all future events associated with the `MobileRadio`.

Method Summary

void	<u>radioOff</u> (<u>MobileRadioEvent</u> event) The radio was turned off.
void	<u>radioOn</u> (<u>MobileRadioEvent</u> event) The radio was turned on.
void	<u>radioStartStateOff</u> (<u>MobileRadioEvent</u> event) The radio startup state was changed to off.
void	<u>radioStartStateOn</u> (<u>MobileRadioEvent</u> event) The radio startup state was changed to on.
void	<u>signalLevelChanged</u> (<u>MobileRadioEvent</u> event) The radio signal level has been changed or the network connection has been established.

Method Detail

signalLevelChanged

public void **signalLevelChanged**(MobileRadioEvent event)

The radio signal level has been changed or the network connection has been established. The signal level must be always notified after the network contact is successfully set up. After that, only the level changes should be notified to the listeners.

Parameters:

event - the event object associated with this event.

radioStartStateOn

```
public void radioStartStateOn(MobileRadioEvent event)
```

The radio startup state was changed to on.

Parameters:

event - the event object associated with this event.

radioStartStateOff

```
public void radioStartStateOff(MobileRadioEvent event)
```

The radio startup state was changed to off.

Parameters:

event - the event object associated with this event.

radioOn

```
public void radioOn(MobileRadioEvent event)
```

The radio was turned on.

Parameters:

event - the event object associated with this event.

radioOff

```
public void radioOff(MobileRadioEvent event)
```

The radio was turned off.

Parameters:

event - the event object associated with this event.

javax.telephony.mobile ***Interface MobileTerminal***

public interface **MobileTerminal**
extends Terminal

A `MobileTerminal` represents a physical hardware endpoint connected to a wireless telephony domain. An example of a `MobileTerminal` is a mobile phone connected to a laptop via serial port. For example, computer workstations and handheld (PDA) devices are modeled as `MobileTerminals` if they act as physical endpoints in a cellular telephony network. This interface extends the core `Terminal` with capabilities to get the manufacturer name, the terminal identification and the type approval number.

DTMF tone generation is handled in this interface to allow for communication with legacy server applications which expect tone inputs to be sent from a dumb telephony terminal.

Note: in mobile networks the tones are not typically generated from the terminal node, but are handled by the network itself. An implementation of this interface is responsible for propagating the tones appropriately.

- GSM 04.08 section 9.3.24/25/26/28/29
- IS-136.2 section 2.7.3.1.3
- IS-95B section 6.7.2.3

See Also:

Terminal, MobileProvider

Method Summary

boolean	<u>generatedDTMF</u> (java.lang.String digits) Generates one or more DTMF tones.
java.lang.String	<u>getManufacturerName</u> () Returns the Mobile's Manufacturer Name.
java.lang.String	<u>getSoftwareVersion</u> () Returns the Software Version String.
java.lang.String	<u>getTerminalId</u> () Returns the Terminal Identification String.
java.lang.String	<u>getTypeApproval</u> () Returns the type approval String.
boolean	<u>startDTMF</u> (char digit) Generates one continuous DTMF tone.
void	<u>stopDTMF</u> () Stop the continuous DTMF tone being generated on the telephone line.

Methods inherited from interface javax.telephony.Terminal

addCallListener, addCallObserver, addObserver, addTerminalListener, getAddresses, getCallListeners, getCallObservers, getCapabilities, getName, getObservers, getProvider, getTerminalCapabilities, getTerminalConnections, getTerminalListeners, removeCallListener, removeCallObserver, removeObserver, removeTerminalListener

Method Detail

getTerminalId

```
public java.lang.String getTerminalId()
```

Returns the Terminal Identification String. This methods returns an equipment ID string such as IMEI or ESN. (International Mobile Equipment Identification) or (Electronic Serial Number)

Returns:

an equipment ID String, NULL if no such ID is implemented.

getManufacturerName

public java.lang.String **getManufacturerName()**

Returns the Mobile's Manufacturer Name.

Returns:

an manufacturer name String, NULL if not implemented.

getSoftwareVersion

public java.lang.String **getSoftwareVersion()**

Returns the Software Version String. This methods returns a software version string.

Returns:

a software version String, NULL if not implemented.

getTypeApproval

public java.lang.String **getTypeApproval()**

Returns the type approval String. This methods returns a type approval string.

Returns:

a type approval String, NULL if not implemented.

startDTMF

public boolean **startDTMF**(char digit)
throws MethodNotSupportedException

Generates one continuous DTMF tone. The character parameter must consist of one of the following characters: the numbers zero through nine (0 - 9), the letters A through D, asterisk (*), or pound(#).

Parameters:

digit - The DTMF-tone digit to generate on the telephone line.

Returns:

true if a tone was successfully delivered.

Throws:

MethodNotSupportedException - The implementation does not support generating DTMF tones.

stopDTMF

```
public void stopDTMF()
```

Stop the continuous DTMF tone being generated on the telephone line.

generateDTMF

```
public boolean generateDTMF(java.lang.String digits)  
    throws MethodNotSupportedException
```

Generates one or more DTMF tones. The string parameter must consists of one or more of the following characters: the numbers zero through nine (0 - 9), the letters A through D, asterisk (*), or pound(#). Any other characters in the string parameter are ignored.

Parameters:

`digits` - The string of DTMF-tone digits to generate on the telephone line.

Returns:

true if a tone was successfully delivered.

Throws:

MethodNotSupportedException - The implementation does not support generating DTMF tones.

javax.telephony.mobile **Interface NetworkSelection**

public interface **NetworkSelection**
extends MobileProvider

The NetworkSelection interface provides the specific list based services.

Users may choose between manual and automatic network selection and may interact with lists of preferred and forbidden network lists. (See GSM 03.22 Public Land Mobile Network)

See Also:

MobileProvider

Fields inherited from interface javax.telephony.mobile.<u>MobileProvider</u>

<u>RESTRICTED_SERVICE</u>

Fields inherited from interface javax.telephony.<u>Provider</u>
--

<u>IN_SERVICE</u> , <u>OUT_OF_SERVICE</u> , <u>SHUTDOWN</u>

Method Summary

java.lang.String	<u>getCurrentSelectionMode()</u> Returns the current network selection mode.
<u>MobileNetwork</u> []	<u>getForbiddenNetworks()</u> Returns the network list specified.
<u>MobileNetwork</u> []	<u>getPreferredNetworks()</u> Returns the network list specified.
java.lang.String[]	<u>getSelectionModes()</u> Returns the (automatic) network selection algorithm modes available.
void	<u>setSelectionMode(java.lang.String mode)</u> Sets the network selection mode.

Methods inherited from interface javax.telephony.mobile.MobileProvider

[cancelAvailableNetworkRequest](#), [getAvailableNetworks](#),
[getCurrentNetwork](#), [getHomeNetwork](#), [getMobileState](#), [getNetworkTime](#),
[getNetworkTimeZone](#), [getType](#), [isRoaming](#), [setNetwork](#)

Methods inherited from interface javax.telephony.Provider

[addObserver](#), [addProviderListener](#), [createCall](#), [getAddress](#),
[getAddressCapabilities](#), [getAddressCapabilities](#), [getAddresses](#),
[getCallCapabilities](#), [getCallCapabilities](#), [getCalls](#), [getCapabilities](#),
[getConnectionCapabilities](#), [getConnectionCapabilities](#), [getName](#),
[getObservers](#), [getProviderCapabilities](#), [getProviderCapabilities](#),
[getProviderListeners](#), [getState](#), [getTerminal](#), [getTerminalCapabilities](#),
[getTerminalCapabilities](#), [getTerminalConnectionCapabilities](#),
[getTerminalConnectionCapabilities](#), [getTerminals](#), [removeObserver](#),
[removeProviderListener](#), [shutdown](#)

Method Detail*getCurrentSelectionMode*

```
public java.lang.String getCurrentSelectionMode()
                        throws MethodNotSupportedException
```

Returns the current network selection mode. The mode must be null if the current mode is manual i.e., the network was selected by using the setNetwork-method.

Returns:

The current selection mode String object.

Throws:

[MethodNotSupportedException](#) - Indicates network selection is not supported.

getSelectionModes

```
public java.lang.String[] getSelectionModes()
                        throws MethodNotSupportedException
```

Returns the (automatic) network selection algorithm modes available. If no other selection modes than the manual selection is available, than this must return null. The modes are cellular network system and implementation specific. For example, in GSM-system this might return only one string: "AUTOMATIC" and in CDMA-systems, the modes could be "PREFERRED", "ALTERNATE" and

"HOME" or similar. In other words, this is totally up to the implementation.

Returns:

The selection modes available names as text strings.

Throws:

MethodNotSupportedException - Indicates network selection is not supported.

setSelectionMode

```
public void setSelectionMode(java.lang.String mode)
    throws InvalidArgumentException,
           MethodNotSupportedException,
           InvalidStateException
```

Sets the network selection mode. These modes are cellular system and implementation specific and are usually automatic selection algorithms. If the mode is null, then the action must be same as for setNetwork i.e., manual.

Parameters:

mode - Mode to be selected (or null for manual mode)

Throws:

MethodNotSupportedException - Indicates the setting of the selection mode capability is not supported.

InvalidArgumentException - Indicates that the input parameter is invalid.

InvalidStateException - Indicates the implementation is in a state in which setting the network selection mode is not possible (call ongoing, for example).

getPreferredNetworks

```
public MobileNetwork[] getPreferredNetworks()
    throws MethodNotSupportedException,
           ResourceUnavailableException
```

Returns the network list specified. The information in network lists might change at any time.

Preferred Networks are network service carriers the user has found to offer service, and is preferred over other networks.

Returns:

A list of networks.

Throws:

MethodNotSupportedException - Indicates the capability to get network lists is not supported.

ResourceUnavailableException - Indicates that a resource inside the system is not available to complete an operation.

getForbiddenNetworks

```
public MobileNetwork[] getForbiddenNetworks()  
                                throws MethodNotSupportedException,  
                                       ResourceUnavailableException
```

Returns the network list specified. The information in network lists might change at any time.

Forbidden Networks are network service carriers to which the user's home network carrier has no agreement of cross service or to which the user has reason (cost) not to use this network.

Returns:

A list of networks.

Throws:

MethodNotSupportedException - Indicates the capability to get network lists is not supported.

ResourceUnavailableException - Indicates that a resource inside the system is not available to complete an operation.

Package javax.telephony.phone

The JTAPI Phone package provides objects that model the physical characteristics of the terminal, such as, speakerphone, microphone, display, buttons, ringer, handset, and lamp.

See:

Description

Interface Summary	
<u>Component</u>	The Component interface is the base interface for all individual components used to model telephone hardware.
<u>ComponentGroup</u>	A ComponentGroup is a grouping of Component objects.
<u>PhoneButton</u>	Phone button interface.
<u>PhoneDisplay</u>	Phone display interface.
<u>PhoneGraphicDisplay</u>	A PhoneGraphicsDisplay represents a display device that is pixel-addressable, and which can be drawn into using AWT primitives.
<u>PhoneHookswitch</u>	Phone hook switch interface.
<u>PhoneLamp</u>	Phone lamp interface.
<u>PhoneMicrophone</u>	Phone microphone interface.
<u>PhoneRinger</u>	Phone ringer interface.
<u>PhoneSpeaker</u>	Phone speaker interface
<u>PhoneTerminal</u>	The PhoneTerminal interface extends the Terminal interface to provide functionality for the Phone package.
<u>PhoneTerminalObserver</u>	The PhoneTerminalObserver interface is used to report all Phone-related events.

Package javax.telephony.phone Description

The JTAPI Phone package provides objects that model the physical characteristics of the terminal, such as, speakerphone, microphone, display, buttons, ringer, handset, and lamp.

Introduction

This document provides an overview of the *Phone Extension Package* to the Java™ Telephony API. The Phone Extension Package permits implementations to describe physical Terminals in terms of standardized components. Applications may query each Terminal, using this package, for the components of which it is made and control certain attributes of these components.

Note: *The Phone Extension Package was previously named the **Set Extension Package** in Version 1.0 of this specification. Because of overwhelming feedback on this specification, the Java™ Telephony Specification working partners decided the term **Set** was ambiguous and has many meanings. Additionally, backwards compatability to the name **Set** will **NOT** be maintained. We apologize for any inconvenience this may have caused.*

The Phone Extension Package defines a number of components standard to traditional telephone-set hardware. These components include speakerphone, microphone, display, buttons, ringer, handset, and lamp. Each component exports a standard interface to control its attributes. For example, the ringer permits applications to control its volume, while the buttons permit applications to simulate pressing buttons.

Overview Document Organization

This overview document is organized into the following sections:

Phone Package Architecture

This section summarizes the architecture used by the Phone Extension Package to describe Terminals in term of the standard telephone-set components.

Extending the Terminal interface

The section summarizes the **java.telephony.PhoneTerminal** interface. This interface extends the core Terminal interfaces to permit applications to query for the number and kinds of components of which it is composed.

Phone Package Standard Components

This section describes the standard components in this package used to described traditional telephone-set hardware.

Phone Package Architecture.

This section explains how the Phone package describes Terminals in terms of their individual components.

Component Interface

The Phone package consists of a set of interfaces, each modelling some physical feature of a traditional telephone hardware set. There exists interfaces to model buttons, displays, lamps, ringers, hookswitches, speakers, and microphones. Each of these distinct interfaces provide methods to control the attributes inherent to each component type.

Each individual component extends the **Component** interface. These corresponding individual component interfaces are: **PhoneButton**, **PhoneDisplay**, **PhoneLamp**, **PhoneRinger**, **PhoneHookswitch**, **PhoneSpeaker**, and **PhoneMicrophone**. Each component is identified by its type and a string name.

ComponentGroup Interface

One or more individual components are organized into component groups. These component groups represent an association between components as they exist on the actual, physical hardware. Applications obtain an array of the components associated with a component group via the *getComponents()* method on this interface.

There may be more than one ComponentGroup associated with a single Terminal object. Applications obtain an array of the component groups associated with a Terminal via the *getComponentGroups()* method on the **PhoneTerminal** interface.

Terminals may have more than one ComponentGroup for the following reasons: at times an implementation may want to model a physical endpoint (i.e. Terminal) with two groups of components. One of these groups may model a telephone hardware set sitting on a person's desktop, while the other group may model a "virtual" telephone set drawn on the person's workstation's display.

Only a single ComponentGroup may be "activated" on a Terminal at a time. The act of activating a ComponentGroup enables its components to become actively part of the telephone call. For example, if a speaker component is part of a ComponentGroup which is activated, that speaker will receive all of the media sent to it. Applications activate a ComponentGroup with the *activate()* method and deactivate a group with the *deactivate()* method.

Extending the Terminal Interface

The Phone Extension Package extends the core Terminal interface with the **PhoneTerminal** interface. This interface permits applications to query for the components which make up the Terminal object.

As described in the previous section, each Terminal is composed of zero or more *ComponentGroups*. These groups are obtained via the **PhoneTerminal.getComponentGroups()** method. The following code segment queries for the Terminal named "5551212" and for the groups of components associated with this Terminal.

```

try {
    Terminal terminal = myprovider.getTerminal("5551212");

    ComponentGroup groups[];
    if (terminal instanceof PhoneTerminal) {
        groups = ((PhoneTerminal)terminal).getComponentGroups();
    }
} catch (Exception excp) {
    // Handle exceptions
}

```

Once an application obtains a list of groups, it may choose to activate one of these groups. The meaning of activating and deactivating a group of components was described in the previous section. The following code segment chooses the first group and activates it.

```

if (groups != null && groups.length >= 1) {
    try {
        groups[0].activate();
    } catch (Exception excp) {
        // Handle exceptions
    }
}

```

Next, applications may query for the array of individual components in the component group. An application may iterate through this array and control the attributes of each component-type. In the code sample below, the application looks for the microphone component and sets it on the maximum volume.

```

Component components[] = groups[0].getComponents();

for (int i = 0; i < components.length; i++) {

    if (components[i] instanceof MicrophoneComponent) {
        MicrophoneComponent mic = (MicrophoneComponent)components[i];

        mic.setVolume(MicrophoneComponent.FULL);
    }
}

```

Phone Package Standard Components

The Phone Extension Package defines a number of standard components found on traditional telephone hardware. Each component exports its own interfaces and controls over its attributes. Each component also has specific events indicating when its attributes change. This section briefly describes each component, the methods available on its interfaces, and the event associated with them.

Button Component

The **PhoneButton** component models a button on a telephone keypad. Applications may "press" this button via the *buttonPress()* on the PhoneButton interface. Each button has an indentifying piece of information associated with it (e.g. the text label on the telephone keypad button). Applications may obtain and set this information via the *getinfo()* and *setInfo()* methods, respectively. Additionally,

there may be a lamp-component associated with a button, obtained via the *getAssociatedPhoneLamp()* method.

Applications receive the *ButtonPressEv* event when the button component has been pressed and the *ButtonInfoEv* when the identifying information on the button has changed.

Display Component

The **PhoneDisplay** component models the display on the telephone hardware set. This display is composed of a certain number of rows and columns. Applications may set and obtain the text displayed at a particular coordinate via the *setDisplay()* and *getDisplay()* methods, respectively.

When any information on the display has changed, the application receives the *DisplayUpdateEv*.

Hookswitch Component

The **PhoneHookswitch** component models the physical hookswitch on a telephone hardware set. (A *hookswitch* is the level component on the telephone set on which the handset rests, which is depressed when the phone is on-hook.) The Hookswitch component can either be in an *ON_HOOK* state or an *OFF_HOOK* state. The application may set and obtain the hookswitch state via the *setHookswitchState()* and *getHookswitchState()* methods, respectively.

Applications receive the *HookswitchStateEv* event when the state of a hookswitch changes.

Lamp Component

The **PhoneLamp** component models a light-indicator on the telephone hardware set. Often, this lamp is a single green-color or red-color LED. Lamp components have a "mode" attribute which indicate the pattern of its flashing light. For example, the lamp mode can be *LAMPMODE_FLUTTER*, *LAMPMODE_STEADY*, or *LAMPMODE_WINK*, to name a few. Applications may set and obtain the lamp mode via the *setMode()* and *getMode()* methods, respectively. Additionally, lamp components may be associated with a button component. This button component is obtained via the *getAssociatedPhoneButton()* method.

Applications receive the *LampModeEv* when the mode of the lamp component has changed.

Microphone Component

The **PhoneMicrophone** component represents the audio input device on a telephone hardware set. The microphone has a single attribute: its gain. Applications may set or obtain the microphone's gain via the *setGain()* and *getGain()* methods, respectively.

Applications receive the *MicrophoneGainEv* event when the gain of a microphone component has changed.

Ringer Component

The **PhoneRinger** component represents the audible ringing device on a telephone hardware set. The ringer component has several attributes. First, applications may control the ringing pattern of the ringer component. In order to control the ringing pattern, applications obtain the number of ringing patterns via the *getNumberOfRingPatterns()* method. Applications then set the ringing pattern via the *setRingerPattern()*. It uses an integer index value between zero and the number of ringing patterns minus one. Applications may obtain the current ringing pattern via the *getRingingPattern()* method, and the number of rings which have already happened via the *getNumberOfRings()* method.

The ringer component also has a volume attribute that may be set or obtained via the *setRingerVolume()* and *getRingerVolume()* methods, respectively.

Applications receive the *RingerPatternEv* when the ringing pattern has changed on the ringer component and the *RingerVolumeEv* when the ringer volume has changed.

Speaker Component

The **PhoneSpeaker** component represents the audio output device on a telephone hardware set. The speaker has a single attribute: its volume. Applications may set or obtain the speaker's volume via the *setVolume()* and *getVolume()* methods, respectively.

Applications receive the *SpeakerVolumeEv* event when the volume of a speaker component has changed.

Hierarchy For Package javax.telephony.phone

Package Hierarchies:

All Packages

Interface Hierarchy

- interface javax.telephony.phone.**Component**
 - interface javax.telephony.phone.**PhoneButton**
 - interface javax.telephony.phone.**PhoneDisplay**
 - interface javax.telephony.phone.**PhoneGraphicDisplay**
 - interface javax.telephony.phone.**PhoneHookswitch**
 - interface javax.telephony.phone.**PhoneLamp**
 - interface javax.telephony.phone.**PhoneMicrophone**
 - interface javax.telephony.phone.**PhoneRinger**
 - interface javax.telephony.phone.**PhoneSpeaker**
 - interface javax.telephony.phone.**ComponentGroup**
 - interface javax.telephony.**Terminal**
 - interface javax.telephony.phone.**PhoneTerminal**
 - interface javax.telephony.**TerminalObserver**
 - interface javax.telephony.phone.**PhoneTerminalObserver**
-
-

javax.telephony.phone **Interface Component**

All Known Subinterfaces:

[PhoneButton](#), [PhoneDisplay](#), [PhoneGraphicDisplay](#), [PhoneHookswitch](#), [PhoneLamp](#),
[PhoneMicrophone](#), [PhoneRinger](#), [PhoneSpeaker](#)

public interface **Component**

The Component interface is the base interface for all individual components used to model telephone hardware. Each individual component extends this interface.

Each component is identified not only by its type, but also by an identifying name, which may be obtained via the *getName()* method on this interface.

Method Summary

<u>ComponentCapabilities</u>	<u>getCapabilities()</u> Returns the dynamic capabilities for this Component instance.
java.lang.String	<u>getName()</u> Returns the name of the Component.

Method Detail

getName

```
public java.lang.String getName()
```

Returns the name of the Component.

Returns:

The name of this component.

getCapabilities

```
public ComponentCapabilities getCapabilities()
```

Returns the dynamic capabilities for this `Component` instance. Static capabilities are not available for components.

Returns:

The dynamic component capabilities.

javax.telephony.phone **Interface ComponentGroup**

public interface **ComponentGroup**

A ComponentGroup is a grouping of Component objects. Terminals may be composed of zero or more ComponentGroups. Applications query the PhoneTerminal interface for the available ComponentGroups. Then they query this interface for the components which make up this component group.

Field Summary

static int	<u>HAND_SET</u> The component group is of type HAND_SET.
static int	<u>HEAD_SET</u> The component group is of type HEAD_SET.
static int	<u>OTHER</u> The component group is of type OTHER.
static int	<u>PHONE_SET</u> The componet group is of type PHONE_SET.
static int	<u>SPEAKER_PHONE</u> The component group is of type SPEAKER_PHONE.

Method Summary	
boolean	<u>activate()</u> Enables all routing of events or media stream between all Components of this group and calls on any of the Addresses asociated with the parent Terminal.
boolean	<u>activate(Address address)</u> Enables all routing of events or media stream between all Components of this group and calls to the specified Address.
boolean	<u>deactivate()</u> Disables all routing of events or media stream between all Components of this group and calls on any of the Addresses associated with the parent Terminal.
boolean	<u>deactivate(Address address)</u> Disables all routing of events or media stream between all Components of this group and the specified Address.
<u>ComponentGroupCapabilities</u>	<u>getCapabilities()</u> Returns the dynamic capabilities for this ComponentGroup instance.
<u>Component[]</u>	<u>getComponents()</u> Returns the groups components, null if the group contains zero components.
java.lang.String	<u>getDescription()</u> Returns a string describing the component group.
int	<u>getType()</u> Returns the type of group, either HEAD_SET, HAND_SET, SPEAKER_PHONE, PHONE_SET or OTHER.

Field Detail

HEAD_SET

```
public static final int HEAD_SET
```

The component group is of type HEAD_SET.

HAND_SET

```
public static final int HAND_SET
```

The component group is of type HAND_SET.

SPEAKER_PHONE

```
public static final int SPEAKER_PHONE
```

The component group is of type SPEAKER_PHONE.

PHONE_SET

```
public static final int PHONE_SET
```

The componet group is of type PHONE_SET.

OTHER

```
public static final int OTHER
```

The component group is of type OTHER.

Method Detail

getType

```
public int getType()
```

Returns the type of group, either HEAD_SET, HAND_SET, SPEAKER_PHONE, PHONE_SET or OTHER.

Returns:

The type of group.

getDescription

```
public java.lang.String getDescription()
```

Returns a string describing the component group.

Returns:

A string description of the component group.

getComponents

```
public Component[] getComponents()
```

Returns the groups components, null if the group contains zero components.

Returns:

An array of Component objects.

activate

```
public boolean activate()
```

Enables all routing of events or media stream between all Components of this group and calls on any of the Addresses asociated with the parent Terminal.

Returns:

true if successful and false if unsuccessful.

deactivate

```
public boolean deactivate()
```

Disables all routing of events or media stream between all Components of this group and calls on any of the Addresses associated with the parent Terminal.

Returns:

true if successful and false if unsuccessful.

activate

```
public boolean activate(Address address)  
    throws InvalidArgumentException
```

Enables all routing of events or media stream between all Components of this group and calls to the specified Address.

Parameters:

address - The Address that the group is to be activated on.

Returns:

true if successful and false if unsuccessful.

Throws:

InvalidArgumentException - The provided Address is not valid for the Terminal.

deactivate

```
public boolean deactivate(Address address)  
    throws InvalidArgumentException
```

Disables all routing of events or media stream between all Components of this group and the specified Address.

Parameters:

address - The Address that the group is to be deactivated on.

Returns:

true if successful and false if unsuccessful.

Throws:

InvalidArgumentException - The provided Address is not valid for the Terminal.

getCapabilities

```
public ComponentGroupCapabilities getCapabilities()
```

Returns the dynamic capabilities for this `ComponentGroup` instance. Static capabilities are not available for component groups.

Returns:

The dynamic component group capabilities.

javax.telephony.phone **Interface PhoneButton**

public interface **PhoneButton**
extends Component

Phone button interface.

Method Summary

void	<u>buttonPress()</u> Press the button.
<u>PhoneLamp</u>	<u>getAssociatedPhoneLamp()</u> Returns the associated lamp information.
java.lang.String	<u>getInfo()</u> Returns the button information.
void	<u>setInfo(java.lang.String buttonInfo)</u> Sets button information.

Methods inherited from interface javax.telephony.phone.Component

getCapabilities, getName

Method Detail

getInfo

public java.lang.String **getInfo()**

Returns the button information.

Returns:

The string button information.

setInfo

public void **setInfo**(java.lang.String buttonInfo)

Sets button information.

Parameters:

buttonInfo - The button information.

getAssociatedPhoneLamp

public PhoneLamp **getAssociatedPhoneLamp**()

Returns the associated lamp information.

Returns:

The associated lamp object.

buttonPress

public void **buttonPress**()

Press the button.

javax.telephony.phone **Interface PhoneDisplay**

public interface **PhoneDisplay**
extends [Component](#)

Phone display interface.

Method Summary

java.lang.String	getDisplay (int x, int y) Returns the displayed string starting at coordinates (x, y).
int	getDisplayColumns () Returns the number of display columns.
int	getDisplayRows () Returns the number of display rows.
void	setDisplay (java.lang.String string, int x, int y) Displays the given string starting at coordinates (x, y).

Methods inherited from interface javax.telephony.phone.[Component](#)

[getCapabilities](#), [getName](#)

Method Detail

getDisplayRows

public int **[getDisplayRows](#)**()

Returns the number of display rows.

Returns:

The number of display rows.

getDisplayColumns

```
public int getDisplayColumns()
```

Returns the number of display columns.

Returns:

The number of display columns.

getDisplay

```
public java.lang.String getDisplay(int x,  
                                     int y)  
    throws InvalidArgumentException
```

Returns the displayed string starting at coordinates (x, y).

Parameters:

x - The x-coordinate.

y - The y-coordinate.

Returns:

The string displayed starting at coordinates (x, y).

Throws:

InvalidArgumentException - Either the coordinates provided were invalid.

setDisplay

```
public void setDisplay(java.lang.String string,  
                       int x,  
                       int y)  
    throws InvalidArgumentException
```

Displays the given string starting at coordinates (x, y).

Parameters:

string - The string to display.

x - The x-coordinate.

y - The y-coordinate.

Throws:

InvalidArgumentException - Either the coordinates provided were invalid.

javax.telephony.phone ***Interface PhoneGraphicDisplay***

public interface **PhoneGraphicDisplay**
extends [Component](#)

A PhoneGraphicsDisplay represents a display device that is pixel-addressable, and which can be drawn into using AWT primitives.

Method Summary

java.awt.Graphics	getGraphics() Returns a Graphics object for drawing into the display.
java.awt.Dimension	size() Returns the size of the display.

Methods inherited from interface [javax.telephony.phone.Component](#)

[getCapabilities](#), [getName](#)

Method Detail

getGraphics

public java.awt.Graphics **getGraphics()**

Returns a Graphics object for drawing into the display.

Returns:

A Graphic object, as defined in the AWT.

size

public java.awt.Dimension **size()**

Returns the size of the display.

Returns:

The size of the display, packaged in an AWT Dimension object.

javax.telephony.phone **Interface PhoneHookswitch**

public interface **PhoneHookswitch**
extends [Component](#)

Phone hook switch interface.

Field Summary

static int	<u>OFF_HOOK</u> The Hookswitch is OFF_HOOK.
static int	<u>ON_HOOK</u> The Hookswitch is ON_HOOK.

Method Summary

int	<u>getHookSwitchState()</u> Returns the current state of the hookswitch.
void	<u>setHookSwitch(int hookSwitchState)</u> Sets the state of the hookswitch to either ON_HOOK or OFF_HOOK.

Methods inherited from interface javax.telephony.phone.[Component](#)

[getCapabilities](#), [getName](#)

Field Detail

ON_HOOK

public static final int **ON_HOOK**

The Hookswitch is ON_HOOK.

OFF_HOOK

```
public static final int OFF_HOOK
```

The Hookswitch is OFF_HOOK.

Method Detail

setHookSwitch

```
public void setHookSwitch(int hookSwitchState)  
    throws InvalidArgumentException
```

Sets the state of the hookswitch to either ON_HOOK or OFF_HOOK.

Parameters:

hookSwitchState - The desired state of the hook switch.

Throws:

InvalidArgumentException - The provided hookswitch state is not valid.

getHookSwitchState

```
public int getHookSwitchState()
```

Returns the current state of the hookswitch.

Returns:

The current state of the hookswitch.

javax.telephony.phone *Interface PhoneLamp*

public interface **PhoneLamp**
extends Component

Phone lamp interface.

Field Summary

static int	<u>LAMPMODE_BROKENFLUTTER</u> The lamp mode is BROKENFLUTTER, which is the superposition of flash and flutter.
static int	<u>LAMPMODE_FLASH</u> The lamp mode is FLASH, which means slow on and off.
static int	<u>LAMPMODE_FLUTTER</u> The lamp mode is FLUUTER, which means fast on and off.
static int	<u>LAMPMODE_OFF</u> The lamp mode is OFF.
static int	<u>LAMPMODE_STEADY</u> The lamp is STEADY, which means continuously lit.
static int	<u>LAMPMODE_WINK</u> The lamp mode is WINK.

Method Summary

<u>PhoneButton</u>	<u>getAssociatedPhoneButton()</u> Returns the button associated with the lamp.
int	<u>getMode()</u> Returns the current lamp mode.
int[]	<u>getSupportedModes()</u> Returns an array of supported lamp modes.
void	<u>setMode(int mode)</u> Sets the current lamp mode to a mode supported by the lamp and returns by getSupportedModes().

Methods inherited from interface <code>javax.telephony.phone.Component</code>
--

<code>getCapabilities</code> , <code>getName</code>

Field Detail

LAMPMODE_OFF

public static final int **LAMPMODE_OFF**

The lamp mode is OFF.

LAMPMODE_FLASH

public static final int **LAMPMODE_FLASH**

The lamp mode is FLASH, which means slow on and off.

LAMPMODE_STEADY

public static final int **LAMPMODE_STEADY**

The lamp is STEADY, which means continuously lit.

LAMPMODE_FLUTTER

public static final int **LAMPMODE_FLUTTER**

The lamp mode is FLUUTER, which means fast on and off.

LAMPMODE_BROKENFLUTTER

public static final int **LAMPMODE_BROKENFLUTTER**

The lamp mode is BROKENFLUTTER, which is the superposition of flash and flutter.

LAMPMODE_WINK

```
public static final int LAMPMODE_WINK
```

The lamp mode is WINK.

Method Detail

getSupportedModes

```
public int[] getSupportedModes()
```

Returns an array of supported lamp modes.

Returns:

An array of supported lamp modes.

setMode

```
public void setMode(int mode)
           throws InvalidArgumentException
```

Sets the current lamp mode to a mode supported by the lamp and returns by getSupportedModes().

Parameters:

mode - The desired lamp mode.

Throws:

[InvalidArgumentException](#) - The provided lamp mode is not valid.

getMode

```
public int getMode()
```

Returns the current lamp mode.

Returns:

The current lamp mode.

getAssociatedPhoneButton

```
public PhoneButton getAssociatedPhoneButton()
```

Returns the button associated with the lamp.

Returns:

The button associated with the lamp.

javax.telephony.phone **Interface PhoneMicrophone**

public interface **PhoneMicrophone**
extends [Component](#)

Phone microphone interface.

Field Summary

static int	<u>FULL</u> The full microhphone gain.
static int	<u>MID</u> The microphone gain is MID.
static int	<u>MUTE</u> The microphone gain is MUTE.

Method Summary

int	<u>getGain</u> () Returns the current microphone gain.
void	<u>setGain</u> (int gain) Sets the microphone gain to a value between MUTE and FULL, inclusive.

Methods inherited from interface javax.telephony.phone.[Component](#)

[getCapabilities](#), [getName](#)

Field Detail

MUTE

public static final int **MUTE**

The microphone gain is MUTE.

MID

public static final int **MID**

The microphone gain is MID.

FULL

public static final int **FULL**

The full microphone gain.

Method Detail

getGain

public int **getGain()**

Returns the current microphone gain.

Returns:

The current microphone gain.

setGain

public void **setGain**(int gain)
throws [InvalidArgumentException](#)

Sets the microphone gain to a value between MUTE and FULL, inclusive.

Parameters:

gain - A microphone gain between MUTE and FULL, inclusive.

Throws:

[InvalidArgumentException](#) - The microphone gain is not valid.

javax.telephony.phone **Interface PhoneRinger**

public interface **PhoneRinger**
extends Component

Phone ringer interface.

Field Summary

static int	<u>FULL</u> Ringer volume definition for the ringer at maximum volume.
static int	<u>MIDDLE</u> Ringer volume definition for the middle volume.
static int	<u>OFF</u> Ringer volume definition for the ringer off.

Method Summary

int	<u>getNumberOfRingPatterns()</u> Returns the number of available ringing patterns.
int	<u>getNumberOfRings()</u> Returns the number of complete ring cycles that the ringer has been ringing.
int	<u>getRingerPattern()</u> Returns the current ringer pattern.
int	<u>getRingerVolume()</u> Returns the current ringer volume.
int	<u>isRingerOn()</u> Returns true if the ringer is on, false otherwise.
void	<u>setRingerPattern(int ringerPattern)</u> Set the ringer pattern given an valid index number returned by getNumberOfRingPatterns().
void	<u>setRingerVolume(int volume)</u> Sets the ringer volume between ZERO or FULL, inclusive.

Methods inherited from interface `javax.telephony.phone.Component``getCapabilities`, `getName`**Field Detail***OFF*

```
public static final int OFF
```

Ringer volume definition for the ringer off.

MIDDLE

```
public static final int MIDDLE
```

Ringer volume definition for the middle volume.

FULL

```
public static final int FULL
```

Ringer volume definition for the ringer at maximum volume.

Method Detail*isRingerOn*

```
public int isRingerOn()
```

Returns true if the ringer is on, false otherwise.

Returns:

True if the ringer is on, false otherwise

getRingerVolume

```
public int getRingerVolume()
```

Returns the current ringer volume.

Returns:

The current ringer volume.

setRingerVolume

```
public void setRingerVolume(int volume)  
    throws InvalidArgumentException
```

Sets the ringer volume between ZERO or FULL, inclusive.

Parameters:

volume - The ringer volume, between ZERO and FULL, inclusive.

Throws:

InvalidArgumentException - The volume provided was not valid.

getRingerPattern

```
public int getRingerPattern()
```

Returns the current ringer pattern.

Returns:

The current ringer pattern.

getNumberOfRingPatterns

```
public int getNumberOfRingPatterns()
```

Returns the number of available ringing patterns. An index between zero and the returns value minus one may be used for the setRingerPattern() method.

Returns:

The number of available ringer patterns.

setRingerPattern

```
public void setRingerPattern(int ringerPattern)  
    throws InvalidArgumentException
```

Set the ringer pattern given an valid index number returned by `getNumberOfRingPatterns()`.

Parameters:

`ringerPattern` - The desired ringer pattern.

Throws:

InvalidArgumentException - The ring pattern provided was not valid.

getNumberOfRings

```
public int getNumberOfRings()
```

Returns the number of complete ring cycles that the ringer has been ringing. A value of 0 indicates that the ringer is not being rung.

Returns:

The current ringer count.

javax.telephony.phone **Interface PhoneSpeaker**

public interface **PhoneSpeaker**
extends [Component](#)

Phone speaker interface

Field Summary

static int	<u>FULL</u> Speaker volume definition for highest volume.
static int	<u>MID</u> Speaker volume definition for the middle volume.
static int	<u>MUTE</u> Speaker volume definition for muting.

Method Summary

int	<u>getVolume()</u> Returns the volume of the speaker.
void	<u>setVolume(int volume)</u> Sets the speaker or handset volume.

Methods inherited from interface javax.telephony.phone.[Component](#)

[getCapabilities](#), [getName](#)

Field Detail

MUTE

```
public static final int MUTE
```

Speaker volume definition for muting.

MID

```
public static final int MID
```

Speaker volume definition for the middle volume.

FULL

```
public static final int FULL
```

Speaker volume definition for highest volume.

Method Detail

getVolume

```
public int getVolume()
```

Returns the volume of the speaker.

Returns:

The volume of the speaker.

setVolume

```
public void setVolume(int volume)
```

Sets the speaker or handset volume. The volume value may be anything between MUTE or FULL, inclusive.

Parameters:

`volume` - The volume, between MUTE and FULL.

javax.telephony.phone ***Interface PhoneTerminal***

public interface **PhoneTerminal**
extends Terminal

The PhoneTerminal interface extends the Terminal interface to provide functionality for the Phone package. It allows applications to obtain arrays of telephony Components (each group is called a ComponentGroup) which represents the physical components of telephones.

Method Summary

<u>ComponentGroup</u> []	<u>getComponentGroups</u> () Returns an array of ComponentGroup objects available on the Terminal.
--------------------------	---

Methods inherited from interface javax.telephony.Terminal

addCallListener, addCallObserver, addObserver, addTerminalListener,
getAddresses, getCallListeners, getCallObservers, getCapabilities,
getName, getObservers, getProvider, getTerminalCapabilities,
getTerminalConnections, getTerminalListeners, removeCallListener,
removeCallObserver, removeObserver, removeTerminalListener

Method Detail

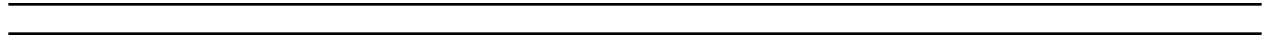
getComponentGroups

public ComponentGroup[] **getComponentGroups**()

Returns an array of ComponentGroup objects available on the Terminal. A ComponentGroup object is composed of a number of Components. Examples of Component objects include headsets, handsets, speakerphones, and buttons. ComponentGroup objects group Components together.

Returns:

An array of ComponetGroup objects on this Terminal.



javax.telephony.phone ***Interface PhoneTerminalObserver***

public interface **PhoneTerminalObserver**
extends TerminalObserver

The PhoneTerminalObserver interface is used to report all Phone-related events. Note that this observer does not have any method associated with it. Applications which implement a TerminalObserver class should also implement this interface to indicate to the implementation that it wants Phone-related events sent to it. If an application's observer does not implement this interface, phone-related events will not be sent to the application.

Methods inherited from interface javax.telephony.<u>TerminalObserver</u>

<u>terminalChangedEvent</u>

Package javax.telephony.phone.capabilities

The JTAPI Phone Capabilities package defines the optional features that may be present as part of the JTAPI implementation.

See:

Description

Interface Summary	
<u>ComponentCapabilities</u>	Component capabilities
<u>ComponentGroupCapabilities</u>	Component Group Capabilities interface

Package javax.telephony.phone.capabilities Description

The JTAPI Phone Capabilities package defines the optional features that may be present as part of the JTAPI implementation.

Hierarchy For Package javax.telephony.phone.capabilities

Package Hierarchies:

All Packages

Interface Hierarchy

- interface javax.telephony.phone.capabilities.**ComponentCapabilities**
 - interface javax.telephony.phone.capabilities.**ComponentGroupCapabilities**
-
-

javax.telephony.phone.capabilities **Interface ComponentCapabilities**

public interface **ComponentCapabilities**

Component capabilities

Method Summary

boolean	<u>canControl</u> () Returns true if the component can be controlled.
boolean	<u>canObserve</u> () Returns true if the component can be observed.

Method Detail

canObserve

public boolean **canObserve**()

Returns true if the component can be observed. For example, this method on a PhoneMicrophone component would return true, if events for changes in gain setting can be received through the TerminalObserver interface and also if the "get" methods on each of the component interfaces is expected to be successful.

Returns:

True if the component can be observed, false otherwise.

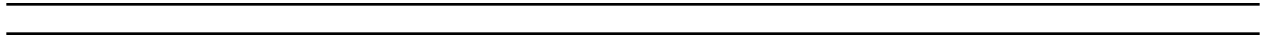
canControl

public boolean **canControl**()

Returns true if the component can be controlled. For example, this method on a PhoneMicrophone component would return true, if the gain setting can be adjusted programmatically.

Returns:

True if the componet can be controlled, false otherwise.



javax.telephony.phone.capabilities **Interface ComponentGroupCapabilities**

public interface **ComponentGroupCapabilities**

Component Group Capabilities interface

Method Summary

boolean	<u>canActivate()</u> Returns true if the ComponentGroup can be "activated" on the Terminal that the ComponentGroup is associated with.
boolean	<u>canActivate(Address address)</u> Returns true if the ComponentGroup can be "activated" on the specified Address at the Terminal that the ComponentGroup is associated with.

Method Detail

canActivate

public boolean **canActivate()**

Returns true if the ComponentGroup can be "activated" on the Terminal that the ComponentGroup is associated with. For example, activation of a headset on a certain Terminal allows media to flow between the headset and the telephone line associated with the terminal for all calls on the line . This method allows the application to determine if activation of the ComponentGroup on its Terminal is supported.

Returns:

True if the component group can be activated on its Terminal, false otherwise.

canActivate

public boolean **canActivate(Address address)**

Returns true if the ComponentGroup can be "activated" on the specified Address at the Terminal that the ComponentGroup is associated with. For example, activation of a headset on a certain Address at a Terminal allows media to flow between the headset and the telephone line associated with the Terminal for all calls on the specified Address. This method allows the application to determine if

activation of the ComponentGroup on a specific Address at a Terminal is supported.

Parameters:

address - test if feature available for this address

Returns:

True if the component group can be activated on its Terminal at the specified Address, false otherwise.

Package javax.telephony.phone.events

The JTAPI Phone Events package defines the specific event transitions associated with the physical terminal characteristics.

See:

Description

Interface Summary	
<u>ButtonInfoEv</u>	The ButtonInfoEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface.
<u>ButtonPressEv</u>	The ButtonPressEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface.
<u>DisplayUpdateEv</u>	The DisplayUpdateEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface.
<u>HookswitchStateEv</u>	The HookswitchStateEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface.
<u>LampModeEv</u>	The LampModeEv interface extends the PhoneTermEv and is reported via the PhoneTerminalObserver interface.
<u>MicrophoneGainEv</u>	The MicrophoneGainEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface.
<u>PhoneEv</u>	The PhoneEv is the base event for all events in the Phone package.
<u>PhoneTermEv</u>	The PhoneTermEv interface extends the TermEv interface and is the base event interface for all phone-components related events.
<u>RingerPatternEv</u>	The RingerPatternEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface.
<u>RingerVolumeEv</u>	The RingerVolumeEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface.
<u>SpeakerVolumeEv</u>	The SpeakerVolumeEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface.

Package javax.telephony.phone.events Description

The JTAPI Phone Events package defines the specific event transitions associated with the physical terminal characteristics.

Hierarchy For Package javax.telephony.phone.events

Package Hierarchies:

[All Packages](#)

Interface Hierarchy

- interface javax.telephony.events.**Ev**
 - interface javax.telephony.phone.events.**PhoneEv**
 - interface javax.telephony.phone.events.**PhoneTermEv**(also extends javax.telephony.events.**TermEv**)
 - interface javax.telephony.phone.events.**ButtonInfoEv**
 - interface javax.telephony.phone.events.**ButtonPressEv**
 - interface javax.telephony.phone.events.**DisplayUpdateEv**
 - interface javax.telephony.phone.events.**HookswitchStateEv**
 - interface javax.telephony.phone.events.**LampModeEv**
 - interface javax.telephony.phone.events.**MicrophoneGainEv**
 - interface javax.telephony.phone.events.**RingerPatternEv**
 - interface javax.telephony.phone.events.**RingerVolumeEv**
 - interface javax.telephony.phone.events.**SpeakerVolumeEv**
 - interface javax.telephony.events.**TermEv**
 - interface javax.telephony.phone.events.**PhoneTermEv**(also extends javax.telephony.phone.events.**PhoneEv**)
 - interface javax.telephony.phone.events.**ButtonInfoEv**
 - interface javax.telephony.phone.events.**ButtonPressEv**
 - interface javax.telephony.phone.events.**DisplayUpdateEv**
 - interface javax.telephony.phone.events.**HookswitchStateEv**
 - interface javax.telephony.phone.events.**LampModeEv**
 - interface javax.telephony.phone.events.**MicrophoneGainEv**
 - interface javax.telephony.phone.events.**RingerPatternEv**
 - interface javax.telephony.phone.events.**RingerVolumeEv**
 - interface javax.telephony.phone.events.**SpeakerVolumeEv**

javax.telephony.phone.events

Interface ButtonInfoEv

public interface **ButtonInfoEv**
 extends PhoneTermEv

The ButtonInfoEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface. This event interface indicates the information associated with a button component has changed.

Applications may obtain the new information associated with this button via the *getInfo()* method on this interface. The old information (before the change) may be obtained via the *getOldInfo()* method on this interface.

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

java.lang.String	<u>getInfo()</u> Returns the button information.
java.lang.String	<u>getOldInfo()</u> Returns the information previously associated with this button.

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail

ID

```
public static final int ID
```

Event id

Method Detail*getInfo*

```
public java.lang.String getInfo()
```

Returns the button information.

Returns:

The string button information.

getOldInfo

```
public java.lang.String getOldInfo()
```

Returns the information previously associated with this button.

Returns:

The old button information.

javax.telephony.phone.events **Interface ButtonPressEv**

public interface **ButtonPressEv**
extends PhoneTermEv

The ButtonPressEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface. This event interface indicates that a button component has been pressed.

Applications may obtain the identifying information associated with this button via the *getInfo()* method.

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

java.lang.String	<u>getInfo()</u> Returns the button information.
------------------	--

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail*ID*

```
public static final int ID
```

Event id

Method Detail

getInfo

```
public java.lang.String getInfo()
```

Returns the button information.

Returns:

The string button information.

javax.telephony.phone.events

Interface DisplayUpdateEv

public interface **DisplayUpdateEv**
 extends PhoneTermEv

The DisplayUpdateEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface. This event interface indicates that the contents of the display component has changed.

Applications may obtain the new contents of the display component via the *getDisplay(int x, int y)* method on this interface.

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
META SNAPSHOT, META UNKNOWN

Method Summary

java.lang.String	<u>getDisplay</u> (int x, int y) Returns the displayed string starting at coordinates (x, y).
------------------	---

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail*ID*

public static final int **ID**

Event id

Method Detail

getDisplay

```
public java.lang.String getDisplay(int x,  
                                     int y)
```

Returns the displayed string starting at coordinates (x, y).

Parameters:

x - The x-coordinate.

y - The y-coordinate.

Returns:

The string displayed starting at coordinates (x, y).

javax.telephony.phone.events **Interface HookswitchStateEv**

public interface **HookswitchStateEv**
extends PhoneTermEv

The HookswitchStateEv interface extends the PhoneTermEv interface and is reported via the PhoneTermObserver interface. This event interface indicates that the state of the hookswitch component has changed.

Applications may obtain the new state of the hookswitch (either on-hook or off-hook) via the *getHookSwitchState()* method on this interface.

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getHookSwitchState()</u> Returns the current state of the hookswitch.
-----	--

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail

ID

```
public static final int ID
```

Event id

Method Detail

getHookSwitchState

```
public int getHookSwitchState()
```

Returns the current state of the hookswitch.

Returns:

The current state of the hookswitch.

javax.telephony.phone.events

Interface LampModeEv

public interface **LampModeEv**
 extends PhoneTermEv

The LampModeEv interface extends the PhoneTermEv and is reported via the PhoneTerminalObserver interface. This event indicates that the mode of the lamp has changed.

Applications may use the *getMode()* method on this interface to obtain the new mode of the lamp.

Field Summary

static int	<u>ID</u>	Event id
------------	------------------	----------

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getMode()</u> Returns the current lamp mode.
-----	---

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail*ID*

```
public static final int ID
```


Event id

Method Detail

getMode

```
public int getMode()
```

Returns the current lamp mode.

Returns:

The current lamp mode.

javax.telephony.phone.events **Interface MicrophoneGainEv**

public interface **MicrophoneGainEv**
extends PhoneTermEv

The MicrophoneGainEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface. This event interface indicates that the gain of a microphone component has changed.

Applications may use the *getGain()* method on this interface to obtain the new gain of the microphone component.

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getGain()</u> Returns the gain of the microphone.
-----	--

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail

ID

```
public static final int ID
```

Event id

Method Detail

getGain

```
public int getGain()
```

Returns the gain of the microphone.

Returns:

The gain of the microphone.

javax.telephony.phone.events

Interface PhoneEv

All Known Subinterfaces:

[ButtonInfoEv](#), [ButtonPressEv](#), [DisplayUpdateEv](#), [HookswitchStateEv](#), [LampModeEv](#),
[MicrophoneGainEv](#), [PhoneTermEv](#), [RingerPatternEv](#), [RingerVolumeEv](#), [SpeakerVolumeEv](#)

public interface **PhoneEv**
 extends [Ev](#)

The PhoneEv is the base event for all events in the Phone package. Each event in this package must extend this interface. This interface is not meant to be a public interface, it is just a building block for other event interfaces.

The PhoneEv interface contains `getPhoneCause()`, which returns the reason for the event.

Field Summary

static int	<u>CAUSE_NORMAL</u> Cause code indicating normal operation
static int	<u>CAUSE_UNKNOWN</u> Cause code indicating the cause was unknown

Fields inherited from interface javax.telephony.events.[Ev](#)

[CAUSE_CALL_CANCELLED](#), [CAUSE_DEST_NOT_OBTAINABLE](#),
[CAUSE_INCOMPATIBLE_DESTINATION](#), [CAUSE_LOCKOUT](#),
[CAUSE_NETWORK_CONGESTION](#), [CAUSE_NETWORK_NOT_OBTAINABLE](#),
[CAUSE_NEW_CALL](#), [CAUSE_NORMAL](#), [CAUSE_RESOURCES_NOT_AVAILABLE](#),
[CAUSE_SNAPSHOT](#), [CAUSE_UNKNOWN](#), [META_CALL_ADDITIONAL_PARTY](#),
[META_CALL_ENDING](#), [META_CALL_MERGING](#), [META_CALL_PROGRESS](#),
[META_CALL_REMOVING_PARTY](#), [META_CALL_STARTING](#), [META_CALL_TRANSFERRING](#),
[META_SNAPSHOT](#), [META_UNKNOWN](#)

Method Summary

int	<u>getPhoneCause()</u> Returns the phone and core causes associated with this event.
-----	--

Methods inherited from interface `javax.telephony.events.Ev``getCause, getID, getMetaCode, getObserved, isNewMetaEvent`**Field Detail***CAUSE_NORMAL*

```
public static final int CAUSE_NORMAL
```

Cause code indicating normal operation

CAUSE_UNKNOWN

```
public static final int CAUSE_UNKNOWN
```

Cause code indicating the cause was unknown

Method Detail*getPhoneCause*

```
public int getPhoneCause()
```

Returns the phone and core causes associated with this event. Every event has a cause. The various cause values are defined as public static final variables in this interface, with the exception of CAUSE_NORMAL and CAUSE_UNKNOWN, which are defined in the core.

Returns:

The cause of the event.

javax.telephony.phone.events

Interface PhoneTermEv

All Known Subinterfaces:

ButtonInfoEv, ButtonPressEv, DisplayUpdateEv, HookswitchStateEv, LampModeEv,
MicrophoneGainEv, RingerPatternEv, RingerVolumeEv, SpeakerVolumeEv

public interface **PhoneTermEv**
 extends PhoneEv, TermEv

The PhoneTermEv interface extends the TermEv interface and is the base event interface for all phone-components related events. All component events must extends this interface. These events are reported through the TerminalObserver interface.

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Method Summary

<u>Component</u>	getComponent () Returns the Component object responsible for this event.
<u>ComponentGroup</u>	getComponentGroup () Returns the ComponentGroup object associated with this event.

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Method Detail

getComponentGroup

public ComponentGroup **getComponentGroup()**

Returns the ComponentGroup object associated with this event.

Returns:

The ComponentGroup object associated with this event.

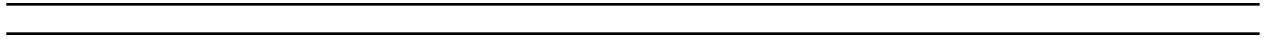
getComponent

public Component **getComponent()**

Returns the Component object responsible for this event.

Returns:

The Component object responsible for this event.



javax.telephony.phone.events **Interface RingerPatternEv**

public interface **RingerPatternEv**
extends PhoneTermEv

The RingerPatternEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface. This event interface indicates that the pattern of a ringer component has changed.

Applications may use the *getPattern()* method on this interface to obtain the new pattern of the ringer component.

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getRingerPattern()</u> Returns the pattern of the ringer.
-----	--

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail*ID*

```
public static final int ID
```

Event id

Method Detail

getRingerPattern

```
public int getRingerPattern()
```

Returns the pattern of the ringer.

Returns:

The pattern of the ringer.

javax.telephony.phone.events **Interface RingerVolumeEv**

public interface **RingerVolumeEv**
extends PhoneTermEv

The RingerVolumeEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface. This event interface indicates that the volume of a ringer component has changed.

Applications may use the *getVolume()* method on this interface to obtain the new volume of the ringer component.

Field Summary

static int	<u>ID</u>
	Event id

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getVolume()</u> Returns the volume of the ringer.
-----	--

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail*ID*

```
public static final int ID
```

Event id

Method Detail

getVolume

```
public int getVolume()
```

Returns the volume of the ringer.

Returns:

The volume of the ringer.

javax.telephony.phone.events **Interface SpeakerVolumeEv**

public interface **SpeakerVolumeEv**
extends PhoneTermEv

The SpeakerVolumeEv interface extends the PhoneTermEv interface and is reported via the PhoneTerminalObserver interface. This event interface indicates that the volume of a speaker component has changed.

Applications may use the *getVolume()* method on this interface to obtain the new volume of the speaker component.

Field Summary

static int	<u>ID</u> Event id
------------	------------------------------

Fields inherited from interface javax.telephony.phone.events.PhoneEv

CAUSE_NORMAL, CAUSE_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE_CALL_CANCELLED, CAUSE_DEST_NOT_OBTAINABLE,
CAUSE_INCOMPATIBLE_DESTINATION, CAUSE_LOCKOUT,
CAUSE_NETWORK_CONGESTION, CAUSE_NETWORK_NOT_OBTAINABLE,
CAUSE_NEW_CALL, CAUSE_NORMAL, CAUSE_RESOURCES_NOT_AVAILABLE,
CAUSE_SNAPSHOT, CAUSE_UNKNOWN, META_CALL_ADDITIONAL_PARTY,
META_CALL_ENDING, META_CALL_MERGING, META_CALL_PROGRESS,
META_CALL_REMOVING_PARTY, META_CALL_STARTING, META_CALL_TRANSFERRING,
META_SNAPSHOT, META_UNKNOWN

Fields inherited from interface javax.telephony.events.Ev

CAUSE CALL CANCELLED, CAUSE DEST NOT OBTAINABLE,
 CAUSE INCOMPATIBLE DESTINATION, CAUSE LOCKOUT,
 CAUSE NETWORK CONGESTION, CAUSE NETWORK NOT OBTAINABLE,
 CAUSE NEW CALL, CAUSE NORMAL, CAUSE RESOURCES NOT AVAILABLE,
 CAUSE SNAPSHOT, CAUSE UNKNOWN, META CALL ADDITIONAL PARTY,
 META CALL ENDING, META CALL MERGING, META CALL PROGRESS,
 META CALL REMOVING PARTY, META CALL STARTING, META CALL TRANSFERRING,
 META SNAPSHOT, META UNKNOWN

Method Summary

int	<u>getVolume()</u> Returns the volume of the speaker.
-----	---

Methods inherited from interface javax.telephony.phone.events.PhoneTermEv

getComponent, getComponentGroup

Methods inherited from interface javax.telephony.phone.events.PhoneEv

getPhoneCause

Methods inherited from interface javax.telephony.events.TermEv

getTerminal

Field Detail*ID*

```
public static final int ID
```

Event id

Method Detail

getVolume

```
public int getVolume()
```

Returns the volume of the speaker.

Returns:

The volume of the speaker.

Package javax.telephony

Table of Contents

Package javax.telephony	1
Package javax.telephony	1
Package javax.telephony Description	3
The Java™ Telephony API	3
Introduction	3
Overview Document Organization	4
History of the Java Telephony API	5
Java Telephony Features	6
Supported Configurations	6
Network Computer (NC) Configuration	6
Desktop Computer Configuration	7
Java Telephony Package Architecture	7
Java Telephony Standard Extension Packages	9
The Java Telephony Call Model	10
Provider Object	11
Call Object	11
Address Object	11
Connection Object	11
Terminal Object	11
TerminalConnection Object	11
Core Package Methods	12
Call.connect()	12
TerminalConnection.answer()	12
Connection.disconnect()	12
Connection Object States	12
IDLE state	13
INPROGRESS state	13
ALERTING state	13
CONNECTED state	13
DISCONNECTED state	14
FAILED state	14
UNKNOWN state	14
TerminalConnection Object States	14
IDLE state	15
ACTIVE state	15
RINGING state	15
DROPPED state	15
PASSIVE state	15
UNKNOWN state	15
Placing a Telephone Call	15
The Java Telephony Observer Model	17

Application Code Examples	18
Outgoing Telephone Call Example	18
Incoming Telephone Call Example	20
Locating and Obtaining Providers	22
JtapiPeerFactory	22
JtapiPeer	22
JtapiPeerFactory: Getting Started	23
JtapiPeer: Getting a Provider Object	23
Security in the Java Telephony API	23
javax.telephony Class Hierarchy	25
Hierarchy For Package javax.telephony	25
Class Hierarchy	25
Interface Hierarchy	25
Interface Address	27
javax.telephony Interface Address	27
Introduction	27
Address and Terminal Objects	27
Address and Call Objects	28
Existing Telephone Calls	28
Address Listeners and Events	28
Call Listeners	28
getName	30
getProvider	30
getTerminals	30
getConnections	31
addObserver	31
getObservers	32
removeObserver	32
addCallObserver	32
JTAPI v1.0 Semantics	33
JTAPI v1.1 Semantics	33
Call Observer Lifetime	33
Call Observer on Multiple Addresses and Terminals	34
getCallObservers	34
removeCallObserver	35
getCapabilities	35
getAddressCapabilities	35
addAddressListener	36
getAddressListeners	37
removeAddressListener	37
addCallListener	37
JTAPI v1.0 Semantics	38
JTAPI v1.1 Semantics	38
Call Listener Lifetime	38
Call Listener on Multiple Addresses and Terminals	39
getCallListeners	39

removeCallListener	40
Interface AddressEvent	41
javax.telephony Interface AddressEvent	41
ADDRESS_EVENT_TRANSMISSION_ENDED	42
getAddress	42
Interface AddressListener	43
javax.telephony Interface AddressListener	43
Introduction	43
Address Observation Ending	43
addressListenerEnded	44
Interface AddressObserver	45
javax.telephony Interface AddressObserver	45
Introduction	45
Address Observation Ending	45
addressChangedEvent	46
Interface Call	47
javax.telephony Interface Call	47
Introduction	47
Creating Call Objects	47
Call States	47
Call State Transitions	48
Calls and Connections	48
The <code>Call.connect()</code> method	48
Listeners and Observers	48
Listeners and Events	48
When Call Event Transmission Ends	49
Event Granularity	49
Registering Call Listeners via Address and Terminal	49
IDLE	51
ACTIVE	52
INVALID	52
getConnections	52
getProvider	52
getState	53
connect	53
Method Arguments	53
Method Post-conditions	54
Call Scenarios or Flows	54
Normal <code>Call.connect()</code> Scenario	54
Failure <code>Call.connect()</code> Scenario	56
addObserver	57
Observer Lifetime	57
Event Snapshots	57
CallObservers from Addresses and Terminals	57
Multiple Invocations	58
getObservers	58

removeObserver	59
getCapabilities	59
getCallCapabilities	60
addCallListener	60
Listener Lifetime	61
Event Snapshots	61
CallListeners from Addresses and Terminals	61
Multiple Invocations	61
getCallListeners	62
removeCallListener	62
Interface ConnectionListener	64
javax.telephony Interface ConnectionListener	64
Introduction	64
Adding a Listener to a Call	64
Call State Changes and Connection Events	64
Connection Events	64
connectionAlerting	66
connectionConnected	66
connectionCreated	66
connectionDisconnected	66
connectionFailed	67
connectionInProgress	67
connectionUnknown	67
Interface CallEvent	68
javax.telephony Interface CallEvent	68
CALL_ACTIVE	69
CALL_INVALID	69
CALL_EVENT_TRANSMISSION_ENDED	70
getCall	70
Interface CallListener	71
javax.telephony Interface CallListener	71
Introduction	71
Call Events and Call MetaEvents	71
Registering as a Listener	72
Connection Event and TerminalConnection Event listeners	72
Adding a Listener to a Call	72
Call State Changes	73
Meta Events	73
Call Listener Ending	73
callActive	75
callInvalid	75
callEventTransmissionEnded	75
singleCallMetaProgressStarted	75
singleCallMetaProgressEnded	76
singleCallMetaSnapshotStarted	76
singleCallMetaSnapshotEnded	76

multiCallMetaMergeStarted	76
multiCallMetaMergeEnded	77
multiCallMetaTransferStarted	77
multiCallMetaTransferEnded	77
Interface CallObserver	78
javax.telephony Interface CallObserver	78
Introduction	78
Adding an Observer to a Call	78
Call State Changes	78
Connection Events	78
TerminalConnection Events	79
Call Observation Ending	79
callChangedEvent	79
Interface TerminalConnectionListener	80
javax.telephony Interface TerminalConnectionListener	80
Introduction	80
Adding a Listener to a Call	80
Call State Changes and TerminalConnection Events	80
TerminalConnection Events	80
terminalConnectionActive	82
terminalConnectionCreated	82
terminalConnectionDropped	82
terminalConnectionPassive	82
terminalConnectionRinging	83
terminalConnectionUnknown	83
Interface Connection	84
javax.telephony Interface Connection	84
Introduction	84
Calls and Addresses	84
TerminalConnections	85
Connection States	85
Connection State Transitions	86
The Connection.disconnect() Method	86
Listeners and Events	86
IDLE	89
INPROGRESS	89
ALERTING	90
CONNECTED	90
DISCONNECTED	90
FAILED	90
UNKNOWN	90
getState	91
getCall	91
getAddress	91
getTerminalConnections	91
disconnect	92

Allowable Connection States	92
Method Return Conditions	92
Dropping Additional Connections	93
getCapabilities	94
getConnectionCapabilities	94
Interface ConnectionEvent	96
javax.telephony Interface ConnectionEvent	96
CONNECTION_ALERTING	98
CONNECTION_CONNECTED	98
CONNECTION_CREATED	99
CONNECTION_DISCONNECTED	99
CONNECTION_FAILED	99
CONNECTION_IN_PROGRESS	99
CONNECTION_UNKNOWN	99
getConnection	100
Interface Event	101
javax.telephony Interface Event	101
Introduction	101
Event Identification	101
Cause Codes	102
Core Package Event Hierarchy	102
JTAPI 1.2, Observer Model Meta Events	102
JTAPI 1.3, Listener Model Meta Events	103
CAUSE_NORMAL	105
CAUSE_UNKNOWN	105
CAUSE_CALL_CANCELLED	105
CAUSE_DEST_NOT_OBTAINABLE	105
CAUSE_INCOMPATIBLE_DESTINATION	106
CAUSE_LOCKOUT	106
CAUSE_NEW_CALL	106
CAUSE_RESOURCES_NOT_AVAILABLE	106
CAUSE_NETWORK_CONGESTION	106
CAUSE_NETWORK_NOT_OBTAINABLE	106
CAUSE_SNAPSHOT	106
getCause	107
getID	107
getSource	107
getMetaEvent	107
Class InvalidArgumentException	109
javax.telephony Class InvalidArgumentException	109
InvalidArgumentException	110
InvalidArgumentException	110
Class InvalidPartyException	111
javax.telephony Class InvalidPartyException	111
ORIGINATING_PARTY	112
DESTINATION_PARTY	112

UNKNOWN_PARTY	112
InvalidPartyException	113
InvalidPartyException	113
getType	113
Class InvalidStateException	114
javax.telephony Class InvalidStateException	114
PROVIDER_OBJECT	115
CALL_OBJECT	116
CONNECTION_OBJECT	116
TERMINAL_OBJECT	116
ADDRESS_OBJECT	116
TERMINAL_CONNECTION_OBJECT	116
InvalidStateException	116
InvalidStateException	117
getObjectType	117
getObject	117
getState	117
Interface JtapiPeer	118
javax.telephony Interface JtapiPeer	118
Introduction	118
Obtaining a Provider	118
getName	119
getServices	119
getProvider	119
Class JtapiPeerFactory	121
javax.telephony Class JtapiPeerFactory	121
Introduction	121
Obtaining a JtapiPeer Object	121
Default JtapiPeer	121
getJtapiPeer	122
Class JtapiPeerUnavailableException	124
javax.telephony Class JtapiPeerUnavailableException	124
JtapiPeerUnavailableException	125
JtapiPeerUnavailableException	125
Interface MetaEvent	126
javax.telephony Interface MetaEvent	126
Class MethodNotSupportedException	128
javax.telephony Class MethodNotSupportedException	128
MethodNotSupportedException	129
MethodNotSupportedException	129
Interface SingleCallMetaEvent	130
javax.telephony Interface SingleCallMetaEvent	130
SINGLECALL_META_PROGRESS_STARTED	132
SINGLECALL_META_PROGRESS_ENDED	132
SINGLECALL_META_SNAPSHOT_STARTED	132
SINGLECALL_META_SNAPSHOT_ENDED	132

getCall	133
Interface MultiCallMetaEvent	134
javax.telephony Interface MultiCallMetaEvent	134
MULTICALL_META_MERGE_STARTED	135
MULTICALL_META_MERGE_ENDED	136
MULTICALL_META_TRANSFER_STARTED	136
MULTICALL_META_TRANSFER_ENDED	136
getNewCall	137
getOldCalls	137
Class PlatformException	138
javax.telephony Class PlatformException	138
PlatformException	139
PlatformException	139
Class PrivilegeViolationException	140
javax.telephony Class PrivilegeViolationException	140
ORIGINATOR_VIOLATION	141
DESTINATION_VIOLATION	141
UNKNOWN_VIOLATION	141
PrivilegeViolationException	142
PrivilegeViolationException	142
getType	142
Interface Provider	143
javax.telephony Interface Provider	143
Introduction	143
Provider States	143
Obtaining a Provider	144
Listeners and Events	144
Call Objects and Providers	144
The Provider's domain	145
Address and Terminal Objects	145
Capabilities: Static and Dynamic	145
Multiple Providers and Multiple Applications	146
IN_SERVICE	148
OUT_OF_SERVICE	148
SHUTDOWN	149
getState	149
getName	149
getCalls	149
getAddress	150
getAddresses	150
getTerminals	151
getTerminal	151
shutdown	152
createCall	152
addObserver	153
getObservers	154

removeObserver	154
getProviderCapabilities	154
getCallCapabilities	155
getAddressCapabilities	155
getTerminalCapabilities	155
getConnectionCapabilities	155
getTerminalConnectionCapabilities	156
getCapabilities	156
getProviderCapabilities	156
getCallCapabilities	157
getConnectionCapabilities	157
getAddressCapabilities	158
getTerminalConnectionCapabilities	159
getTerminalCapabilities	159
addProviderListener	160
getProviderListeners	160
removeProviderListener	161
Interface ProviderEvent	162
javax.telephony Interface ProviderEvent	162
PROVIDER_IN_SERVICE	163
PROVIDER_EVENT_TRANSMISSION_ENDED	163
PROVIDER_OUT_OF_SERVICE	164
PROVIDER_SHUTDOWN	164
getProvider	164
Interface ProviderListener	165
javax.telephony Interface ProviderListener	165
Provider State Changes	165
Provider Observation Ending	165
providerInService	166
providerEventTransmissionEnded	166
providerOutOfService	167
providerShutdown	167
Interface ProviderObserver	168
javax.telephony Interface ProviderObserver	168
Introduction	168
Provider State Changes	168
Provider Observation Ending	168
providerChangedEvent	169
Class ProviderUnavailableException	170
javax.telephony Class ProviderUnavailableException	170
CAUSE_UNKNOWN	172
CAUSE_NOT_IN_SERVICE	172
CAUSE_INVALID_SERVICE	172
CAUSE_INVALID_ARGUMENT	172
ProviderUnavailableException	172
ProviderUnavailableException	173

ProviderUnavailableException	173
ProviderUnavailableException	173
getCause	173
Class ResourceUnavailableException	174
javax.telephony Class ResourceUnavailableException	174
UNKNOWN	175
ORIGINATOR_UNAVAILABLE	176
OBSERVER_LIMIT_EXCEEDED	176
TRUNK_LIMIT_EXCEEDED	176
OUTSTANDING_METHOD_EXCEEDED	176
UNSPECIFIED_LIMIT_EXCEEDED	176
NO_DIALTONE	176
USER_RESPONSE	176
ResourceUnavailableException	177
ResourceUnavailableException	177
getType	177
Interface Terminal	178
javax.telephony Interface Terminal	178
Introduction	178
Address and Terminal objects	178
Terminals and Call objects	179
Existing Telephone Calls	179
Terminal Listeners and Events	179
Call Listeners	179
getName	181
getProvider	181
getAddresses	182
getTerminalConnections	182
addObserver	182
getObservers	183
removeObserver	183
addCallObserver	184
JTAPI v1.0 Semantics	184
JTAPI v1.1 Semantics	184
Call Observer Lifetime	184
Call Observer on Multiple Addresses and Terminals	185
getCallObservers	185
removeCallObserver	186
getCapabilities	186
getTerminalCapabilities	186
addTerminalListener	187
getTerminalListeners	188
removeTerminalListener	188
addCallListener	188
JTAPI v1.0 Semantics	189
JTAPI v1.1 Semantics	189

Call Listener Lifetime	189
Call Listener on Multiple Addresses and Terminals	190
getCallListeners	190
removeCallListener	191
Interface TerminalConnection	192
javax.telephony Interface TerminalConnection	192
Introduction	192
Requirements for TerminalConnections	192
TerminalConnection States	193
TerminalConnection state transitions	194
Relationship between Connections and TerminalConnections	194
The TerminalConnection.answer() Method	195
Listeners and Events	195
IDLE	197
RINGING	197
PASSIVE	197
ACTIVE	198
DROPPED	198
UNKNOWN	198
getState	198
getTerminal	198
getConnection	199
answer	199
Allowable TerminalConnection States	199
Multiple TerminalConnections	199
Events	200
getCapabilities	200
getTerminalConnectionCapabilities	201
Interface TerminalConnectionEvent	202
javax.telephony Interface TerminalConnectionEvent	202
TERMINAL_CONNECTION_ACTIVE	204
TERMINAL_CONNECTION_CREATED	204
TERMINAL_CONNECTION_DROPPED	204
TERMINAL_CONNECTION_PASSIVE	205
TERMINAL_CONNECTION_RINGING	205
TERMINAL_CONNECTION_UNKNOWN	205
getTerminalConnection	205
Interface TerminalEvent	207
javax.telephony Interface TerminalEvent	207
TERMINAL_EVENT_TRANSMISSION_ENDED	208
getTerminal	208
Interface TerminalListener	209
javax.telephony Interface TerminalListener	209
Introduction	209
Terminal Listener Ending	209
terminalListenerEnded	209

Interface TerminalObserver	211
javax.telephony Interface TerminalObserver	211
Introduction	211
Terminal Observation Ending	211
terminalChangedEvent	212
Package javax.telephony.callcontrol	213
Package javax.telephony.callcontrol	213
Package javax.telephony.callcontrol Description	213
Java Telephony API - Call Control Extension Package	214
Introduction	214
Call Control Package Methods	214
The CallControlCall Interface	214
The CallControlAddress Interface	215
The CallControlConnection Interface	215
The CallControlTerminal Interface	215
The CallControlTerminalConnection Interface	215
Call Control Package Extended States	216
CallControlConnection States	216
State Transitions for the CallControlConnection Interface	217
Relationship between Connection and CallControlConnection States	219
CallControlTerminalConnection States	219
State Transitions for the CallControlTerminalConnection Interface	220
Relationship between TerminalConnection and CallControlTerminalConnection States	221
Timeline for Telephone Calls	222
OutCall Code Example	223
InCall Code Example	225
javax.telephony.callcontrol Class Hierarchy	229
Hierarchy For Package javax.telephony.callcontrol	229
Class Hierarchy	229
Interface Hierarchy	229
Interface CallControlAddress	230
javax.telephony.callcontrol Interface CallControlAddress	230
Introduction	230
Address Forwarding	230
Do Not Disturb and Message Waiting	230
Observers and Events	231
setForwarding	232
getForwarding	233
cancelForwarding	233
getDoNotDisturb	234
setDoNotDisturb	234
getMessageWaiting	235
setMessageWaiting	235
Interface CallControlAddressObserver	236
javax.telephony.callcontrol Interface CallControlAddressObserver	236

Interface CallControlCall	237
javax.telephony.callcontrol Interface CallControlCall	237
Introduction	237
Additional Call Information	237
Conferencing Telephone Calls	237
Transferring Telephone Calls	238
Consultation Calls	238
Additional CallControlCall Methods	239
Observers and Events	239
getCallingAddress	241
getCallingTerminal	241
getCalledAddress	241
getLastRedirectedAddress	242
addParty	242
States of the Existing Connections	242
The New Connection	243
drop	243
offHook	244
conference	246
The Conference Controller	246
The Telephone Call Argument	246
Other Shared Participants	247
transfer	248
The Transfer Controller	248
The Telephone Call Argument	248
Other Shared Participants	249
transfer	250
The Transfer Controller	250
The New Connection	251
setConferenceController	252
getConferenceController	253
setTransferController	253
getTransferController	254
setConferenceEnable	255
getConferenceEnable	255
setTransferEnable	256
getTransferEnable	256
consult	257
This Call Object	257
The TerminalConnection Argument	257
The Destination Address String	257
The Consultation Purpose	257
The Telephone Call	258
consult	259
Interface CallControlCallObserver	261
javax.telephony.callcontrol Interface CallControlCallObserver	261

Interface CallControlConnection	263
javax.telephony.callcontrol Interface CallControlConnection	263
Introduction	263
CallControlConnection State	263
State Transitions	265
Core vs. CallControl Package States	265
Observers and Events	266
IDLE	269
OFFERED	269
QUEUED	269
ALERTING	269
INITIATED	269
DIALING	270
NETWORK_REACHED	270
NETWORK_ALERTING	270
ESTABLISHED	270
DISCONNECTED	270
FAILED	271
UNKNOWN	271
OFFERING	271
getCallControlState	271
accept	272
reject	272
redirect	273
addToAddress	274
park	275
Class CallControlForwarding	278
javax.telephony.callcontrol Class CallControlForwarding	278
ALL_CALLS	280
INTERNAL_CALLS	280
EXTERNAL_CALLS	280
SPECIFIC_ADDRESS	280
FORWARD_UNCONDITIONALLY	280
FORWARD_ON_BUSY	280
FORWARD_ON_NOANSWER	281
CallControlForwarding	281
CallControlForwarding	281
CallControlForwarding	281
CallControlForwarding	282
getDestinationAddress	282
getType	282
getFilter	282
getSpecificCaller	283
Interface CallControlTerminal	284
javax.telephony.callcontrol Interface CallControlTerminal	284
Introduction	284

Do Not Disturb	284
Picking Up Telephone Calls	284
Observers and Events	285
getDoNotDisturb	286
setDoNotDisturb	286
pickup	287
The Address and Connection Arguments	287
pickup	289
pickup	289
pickupFromGroup	290
The Pickup Group Code	291
pickupFromGroup	291
The Pickup Group Code	292
Interface CallControlTerminalConnection	293
javax.telephony.callcontrol Interface CallControlTerminalConnection	293
Introduction	293
CallControlTerminalConnection State	293
State Transitions	294
Core vs. CallControl Package States	295
Observers and Events	296
IDLE	298
RINGING	298
TALKING	299
HELD	299
BRIDGED	299
INUSE	299
DROPPED	299
UNKNOWN	300
getCallControlState	300
hold	300
unhold	301
join	301
leave	302
Interface CallControlTerminalObserver	304
javax.telephony.callcontrol Interface CallControlTerminalObserver	304
Package javax.telephony.callcontrol.capabilities	305
Package javax.telephony.callcontrol.capabilities	305
Package javax.telephony.callcontrol.capabilities Description	305
javax.telephony.callcontrol.capabilities Class Hierarchy	306
Hierarchy For Package javax.telephony.callcontrol.capabilities	306
Interface Hierarchy	306
Interface CallControlAddressCapabilities	307
javax.telephony.callcontrol.capabilities Interface CallControlAddressCapabilities	307
canSetForwarding	308
canGetForwarding	308
canCancelForwarding	308

canGetDoNotDisturb	308
canSetDoNotDisturb	309
canGetMessageWaiting	309
canSetMessageWaiting	309
Interface CallControlCallCapabilities	310
javax.telephony.callcontrol.capabilities Interface CallControlCallCapabilities	310
canDrop	311
canOffHook	312
canSetConferenceController	312
canSetTransferController	312
canSetTransferEnable	312
canSetConferenceEnable	313
canTransfer	313
canTransfer	313
canTransfer	314
canConference	314
canAddParty	314
canConsult	315
canConsult	315
canConsult	315
Interface CallControlConnectionCapabilities	317
javax.telephony.callcontrol.capabilities Interface CallControlConnectionCapabilities	317
canRedirect	317
canAddToAddress	318
canAccept	318
canReject	318
canPark	318
Interface CallControlTerminalCapabilities	319
javax.telephony.callcontrol.capabilities Interface CallControlTerminalCapabilities	319
canGetDoNotDisturb	321
canSetDoNotDisturb	321
canPickup	321
canPickup	321
canPickup	322
canPickup	322
canPickupFromGroup	323
canPickupFromGroup	323
canPickupFromGroup	324
Interface CallControlTerminalConnectionCapabilities	325
javax.telephony.callcontrol.capabilities Interface CallControlTerminalConnectionCapabilities	325
canHold	325
canUnhold	326
canJoin	326
canLeave	326
Package javax.telephony.callcontrol.events	327
Package javax.telephony.callcontrol.events	327

Package javax.telephony.callcontrol.events Description	329
javax.telephony.callcontrol.events Class Hierarchy	330
Hierarchy For Package javax.telephony.callcontrol.events	330
Interface Hierarchy	330
Interface CallCtlAddrDoNotDisturbEv	333
javax.telephony.callcontrol.events Interface CallCtlAddrDoNotDisturbEv	333
ID	335
getDoNotDisturbState	335
Interface CallCtlAddrEv	336
javax.telephony.callcontrol.events Interface CallCtlAddrEv	336
Interface CallCtlAddrForwardEv	338
javax.telephony.callcontrol.events Interface CallCtlAddrForwardEv	338
ID	340
getForwarding	340
Interface CallCtlAddrMessageWaitingEv	341
javax.telephony.callcontrol.events Interface CallCtlAddrMessageWaitingEv	341
ID	343
getMessageWaitingState	343
Interface CallCtlCallEv	344
javax.telephony.callcontrol.events Interface CallCtlCallEv	344
Additional Call Information	344
getCallingAddress	346
getCallingTerminal	346
getCalledAddress	347
getLastRedirectedAddress	347
Interface CallCtlConnAlertingEv	348
javax.telephony.callcontrol.events Interface CallCtlConnAlertingEv	348
ID	350
Interface CallCtlConnDialingEv	351
javax.telephony.callcontrol.events Interface CallCtlConnDialingEv	351
ID	353
getDigits	353
Interface CallCtlConnDisconnectedEv	354
javax.telephony.callcontrol.events Interface CallCtlConnDisconnectedEv	354
ID	356
Interface CallCtlConnEstablishedEv	357
javax.telephony.callcontrol.events Interface CallCtlConnEstablishedEv	357
ID	359
Interface CallCtlConnEv	360
javax.telephony.callcontrol.events Interface CallCtlConnEv	360
Interface CallCtlConnFailedEv	363
javax.telephony.callcontrol.events Interface CallCtlConnFailedEv	363
ID	365
Interface CallCtlConnInitiatedEv	366
javax.telephony.callcontrol.events Interface CallCtlConnInitiatedEv	366
ID	368

Interface CallCtlConnNetworkAlertingEv	369
javax.telephony.callcontrol.events Interface CallCtlConnNetworkAlertingEv	369
ID	371
Interface CallCtlConnNetworkReachedEv	372
javax.telephony.callcontrol.events Interface CallCtlConnNetworkReachedEv	372
ID	374
Interface CallCtlConnOfferedEv	375
javax.telephony.callcontrol.events Interface CallCtlConnOfferedEv	375
ID	377
Interface CallCtlConnQueuedEv	378
javax.telephony.callcontrol.events Interface CallCtlConnQueuedEv	378
ID	380
getNumberInQueue	380
Interface CallCtlConnUnknownEv	381
javax.telephony.callcontrol.events Interface CallCtlConnUnknownEv	381
ID	383
Interface CallCtlEv	384
javax.telephony.callcontrol.events Interface CallCtlEv	384
CAUSE_ALTERNATE	386
CAUSE_BUSY	386
CAUSE_CALL_BACK	387
CAUSE_CALL_NOT_ANSWERED	387
CAUSE_CALL_PICKUP	387
CAUSE_CONFERENCE	387
CAUSE_DO_NOT_DISTURB	387
CAUSE_PARK	387
CAUSE_REDIRECTED	387
CAUSE_REORDER_TONE	388
CAUSE_TRANSFER	388
CAUSE_TRUNKS_BUSY	388
CAUSE_UNHOLD	388
getCallControlCause	388
Interface CallCtlTermConnBridgedEv	389
javax.telephony.callcontrol.events Interface CallCtlTermConnBridgedEv	389
ID	391
Interface CallCtlTermConnDroppedEv	392
javax.telephony.callcontrol.events Interface CallCtlTermConnDroppedEv	392
ID	394
Interface CallCtlTermConnEv	395
javax.telephony.callcontrol.events Interface CallCtlTermConnEv	395
Interface CallCtlTermConnHeldEv	398
javax.telephony.callcontrol.events Interface CallCtlTermConnHeldEv	398
ID	400
Interface CallCtlTermConnInUseEv	401
javax.telephony.callcontrol.events Interface CallCtlTermConnInUseEv	401
ID	403

Interface CallCtlTermConnRingingEv	404
javax.telephony.callcontrol.events Interface CallCtlTermConnRingingEv	404
ID	406
Interface CallCtlTermConnTalkingEv	407
javax.telephony.callcontrol.events Interface CallCtlTermConnTalkingEv	407
ID	409
Interface CallCtlTermConnUnknownEv	410
javax.telephony.callcontrol.events Interface CallCtlTermConnUnknownEv	410
ID	412
Interface CallCtlTermDoNotDisturbEv	413
javax.telephony.callcontrol.events Interface CallCtlTermDoNotDisturbEv	413
ID	415
getDoNotDisturbState	415
Interface CallCtlTermEv	416
javax.telephony.callcontrol.events Interface CallCtlTermEv	416
Package javax.telephony.capabilities	418
Package javax.telephony.capabilities	418
Package javax.telephony.capabilities Description	418
javax.telephony.capabilities Class Hierarchy	419
Hierarchy For Package javax.telephony.capabilities	419
Interface Hierarchy	419
Interface AddressCapabilities	420
javax.telephony.capabilities Interface AddressCapabilities	420
isObservable	420
Interface CallCapabilities	421
javax.telephony.capabilities Interface CallCapabilities	421
canConnect	421
isObservable	422
Interface ConnectionCapabilities	423
javax.telephony.capabilities Interface ConnectionCapabilities	423
canDisconnect	423
Interface ProviderCapabilities	425
javax.telephony.capabilities Interface ProviderCapabilities	425
isObservable	425
Interface TerminalCapabilities	426
javax.telephony.capabilities Interface TerminalCapabilities	426
isObservable	426
Interface TerminalConnectionCapabilities	427
javax.telephony.capabilities Interface TerminalConnectionCapabilities	427
canAnswer	427
Package javax.telephony.events	429
Package javax.telephony.events	429
Package javax.telephony.events Description	431
javax.telephony.events Class Hierarchy	432
Hierarchy For Package javax.telephony.events	432
Interface Hierarchy	432

Interface AddrEv	433
javax.telephony.events Interface AddrEv	433
getAddress	434
Interface AddrObservationEndedEv	435
javax.telephony.events Interface AddrObservationEndedEv	435
ID	436
Interface CallActiveEv	437
javax.telephony.events Interface CallActiveEv	437
ID	438
Interface CalEv	439
javax.telephony.events Interface CalEv	439
getCall	440
Interface CallInvalidEv	441
javax.telephony.events Interface CallInvalidEv	441
ID	442
Interface CallObservationEndedEv	443
javax.telephony.events Interface CallObservationEndedEv	443
ID	444
getEndedObject	444
Interface ConnAlertingEv	445
javax.telephony.events Interface ConnAlertingEv	445
ID	446
Interface ConnConnectedEv	447
javax.telephony.events Interface ConnConnectedEv	447
ID	448
Interface ConnCreatedEv	449
javax.telephony.events Interface ConnCreatedEv	449
ID	450
Interface ConnDisconnectedEv	451
javax.telephony.events Interface ConnDisconnectedEv	451
ID	452
Interface ConnEv	453
javax.telephony.events Interface ConnEv	453
getConnection	454
Interface ConnFailedEv	455
javax.telephony.events Interface ConnFailedEv	455
ID	456
Interface ConnInProgressEv	457
javax.telephony.events Interface ConnInProgressEv	457
ID	458
Interface ConnUnknownEv	459
javax.telephony.events Interface ConnUnknownEv	459
ID	460
Interface Ev	461
javax.telephony.events Interface Ev	461
Introduction	461

Event IDs	461
Cause Codes	462
Core Package Event Hierarchy	462
Meta Codes	462
CAUSE_NORMAL	465
CAUSE_UNKNOWN	466
CAUSE_CALL_CANCELLED	466
CAUSE_DEST_NOT_OBTAINABLE	466
CAUSE_INCOMPATIBLE_DESTINATION	466
CAUSE_LOCKOUT	466
CAUSE_NEW_CALL	466
CAUSE_RESOURCES_NOT_AVAILABLE	466
CAUSE_NETWORK_CONGESTION	467
CAUSE_NETWORK_NOT_OBTAINABLE	467
CAUSE_SNAPSHOT	467
META_CALL_STARTING	467
META_CALL_PROGRESS	467
META_CALL_ADDITIONAL_PARTY	467
META_CALL_REMOVING_PARTY	468
META_CALL_ENDING	468
META_CALL_MERGING	468
META_CALL_TRANSFERRING	468
META_SNAPSHOT	468
META_UNKNOWN	468
getCause	469
getMetaCode	469
isNewMetaEvent	469
getID	469
getObserved	470
Interface ProvEv	471
javax.telephony.events Interface ProvEv	471
getProvider	472
Interface ProvInServiceEv	473
javax.telephony.events Interface ProvInServiceEv	473
ID	474
Interface ProvObservationEndedEv	475
javax.telephony.events Interface ProvObservationEndedEv	475
ID	476
Interface ProvOutOfServiceEv	477
javax.telephony.events Interface ProvOutOfServiceEv	477
ID	478
Interface ProvShutdownEv	479
javax.telephony.events Interface ProvShutdownEv	479
ID	480
Interface TermConnActiveEv	481
javax.telephony.events Interface TermConnActiveEv	481

ID	482
Interface TermConnCreatedEv	483
javax.telephony.events Interface TermConnCreatedEv	483
ID	484
Interface TermConnDroppedEv	485
javax.telephony.events Interface TermConnDroppedEv	485
ID	486
Interface TermConnEv	487
javax.telephony.events Interface TermConnEv	487
getTerminalConnection	488
Interface TermConnPassiveEv	489
javax.telephony.events Interface TermConnPassiveEv	489
ID	490
Interface TermConnRingingEv	491
javax.telephony.events Interface TermConnRingingEv	491
ID	492
Interface TermConnUnknownEv	493
javax.telephony.events Interface TermConnUnknownEv	493
ID	494
Interface TermEv	495
javax.telephony.events Interface TermEv	495
getTerminal	496
Interface TermObservationEndedEv	497
javax.telephony.events Interface TermObservationEndedEv	497
ID	498
Package javax.telephony.mobile	499
Package javax.telephony.mobile	499
Package javax.telephony.mobile Description	499
Introduction	500
Mobile device characteristics	500
Network Types	501
javax.telephony.mobile Class Hierarchy	502
Hierarchy For Package javax.telephony.mobile	502
Interface Hierarchy	502
Interface MobileAddress	503
javax.telephony.mobile Interface MobileAddress	503
Address and Call Objects	503
Address and Events	503
getCallWaiting	504
setCallWaiting	504
getSubscriptionId	504
Interface MobileNetwork	506
javax.telephony.mobile Interface MobileNetwork	506
isAvailable	506
isCurrent	507
isRestricted	507

getNames	507
getCode	507
Interface MobileProvider	508
javax.telephony.mobile Interface MobileProvider	508
MobileProvider States	508
Core vs. Mobile Package States	508
Listeners and Events	509
RESTRICTED_SERVICE	511
getMobileState	511
getAvailableNetworks	511
cancelAvailableNetworkRequest	512
getCurrentNetwork	512
getHomeNetwork	512
setNetwork	513
isRoaming	513
getType	513
getNetworkTime	514
getNetworkTimeZone	514
Interface MobileProviderEvent	515
javax.telephony.mobile Interface MobileProviderEvent	515
State and State Cause Mapping	515
CAUSE_NORMAL	518
CAUSE_FORBIDDEN	518
CAUSE_FORBIDDEN_ZONE	518
CAUSE_SUSBCRIPTION_ERROR	518
CAUSE_RADIO_OFF	518
CAUSE_NO_NETWORK	518
CAUSE_NETWORK_NOT_SELECTED	519
CAUSE_SEARCHING	519
CAUSE_ILLEGAL_MOBILE	519
CAUSE_UNKNOWN	519
PROVIDER_RESTRICTED_SERVICE	519
MOBILEPROVIDER_NETWORK_CHANGE	520
MOBILEPROVIDER_NO_NETWORK	520
getMobileProvider	520
getMobileStateCause	520
Interface MobileProviderListener	521
javax.telephony.mobile Interface MobileProviderListener	521
serviceRestricted	521
networkChanged	522
Interface MobileRadio	523
javax.telephony.mobile Interface MobileRadio	523
Network States	523
Mobile Radio Listeners	523
getRadioState	524
setRadioState	524

getRadioStartState	525
setRadioStartState	525
getSignalLevel	526
getMaxSignalLevel	526
addRadioListener	526
removeRadioListener	526
Interface MobileRadioEvent	528
javax.telephony.mobile Interface MobileRadioEvent	528
MOBILERADIO_SIGNAL_LEVEL_CHANGED	529
MOBILERADIO_STARTSTATE_ON	529
MOBILERADIO_STARTSTATE_OFF	529
MOBILERADIO_RADIO_ON	529
MOBILERADIO_RADIO_OFF	529
getMobileRadio	530
Interface MobileRadioListener	531
javax.telephony.mobile Interface MobileRadioListener	531
signalLevelChanged	531
radioStartStateOn	532
radioStartStateOff	532
radioOn	532
radioOff	532
Interface MobileTerminal	533
javax.telephony.mobile Interface MobileTerminal	533
getTerminalId	534
getManufacturerName	535
getSoftwareVersion	535
getTypeApproval	535
startDTMF	535
stopDTMF	536
generateDTMF	536
Interface NetworkSelection	537
javax.telephony.mobile Interface NetworkSelection	537
getCurrentSelectionMode	538
getSelectionMode	538
setSelectionMode	539
getPreferredNetworks	539
getForbiddenNetworks	540
Package javax.telephony.phone	541
Package javax.telephony.phone	541
Package javax.telephony.phone Description	541
Introduction	542
Overview Document Organization	542
Phone Package Architecture.	542
Component Interface	543
ComponentGroup Interface	543
Extending the Terminal Interface	543

Phone Package Standard Components	544
Button Component	544
Display Component	545
Hookswitch Component	545
Lamp Component	545
Microphone Component	545
Ringer Component	546
Speaker Component	546
javax.telephony.phone Class Hierarchy	547
Hierarchy For Package javax.telephony.phone	547
Interface Hierarchy	547
Interface Component	548
javax.telephony.phone Interface Component	548
getName	548
getCapabilities	548
Interface ComponentGroup	550
javax.telephony.phone Interface ComponentGroup	550
HEAD_SET	551
HAND_SET	552
SPEAKER_PHONE	552
PHONE_SET	552
OTHER	552
getType	552
getDescription	552
getComponents	553
activate	553
deactivate	553
activate	553
deactivate	554
getCapabilities	554
Interface PhoneButton	555
javax.telephony.phone Interface PhoneButton	555
getInfo	555
setInfo	556
getAssociatedPhoneLamp	556
buttonPress	556
Interface PhoneDisplay	557
javax.telephony.phone Interface PhoneDisplay	557
getDisplayRows	557
getDisplayColumns	558
getDisplay	558
setDisplay	558
Interface PhoneGraphicDisplay	559
javax.telephony.phone Interface PhoneGraphicDisplay	559
getGraphics	559
size	559

Interface PhoneHookswitch	561
javax.telephony.phone Interface PhoneHookswitch	561
ON_HOOK	561
OFF_HOOK	562
setHookSwitch	562
getHookSwitchState	562
Interface PhoneLamp	563
javax.telephony.phone Interface PhoneLamp	563
LAMPMODE_OFF	564
LAMPMODE_FLASH	564
LAMPMODE_STEADY	564
LAMPMODE_FLUTTER	564
LAMPMODE_BROKENFLUTTER	564
LAMPMODE_WINK	565
getSupportedModes	565
setMode	565
getMode	565
getAssociatedPhoneButton	565
Interface PhoneMicrophone	567
javax.telephony.phone Interface PhoneMicrophone	567
MUTE	568
MID	568
FULL	568
getGain	568
setGain	568
Interface PhoneRinger	569
javax.telephony.phone Interface PhoneRinger	569
OFF	570
MIDDLE	570
FULL	570
isRingerOn	570
getRingerVolume	571
setRingerVolume	571
getRingerPattern	571
getNumberOfRingPatterns	571
setRingerPattern	572
getNumberOfRings	572
Interface PhoneSpeaker	573
javax.telephony.phone Interface PhoneSpeaker	573
MUTE	574
MID	574
FULL	574
getVolume	574
setVolume	574
Interface PhoneTerminal	575
javax.telephony.phone Interface PhoneTerminal	575

getComponentGroups	575
Interface PhoneTerminalObserver	577
javax.telephony.phone Interface PhoneTerminalObserver	577
Package javax.telephony.phone.capabilities	578
Package javax.telephony.phone.capabilities	578
Package javax.telephony.phone.capabilities Description	578
javax.telephony.phone.capabilities Class Hierarchy	579
Hierarchy For Package javax.telephony.phone.capabilities	579
Interface Hierarchy	579
Interface ComponentCapabilities	580
javax.telephony.phone.capabilities Interface ComponentCapabilities	580
canObserve	580
canControl	580
Interface ComponentGroupCapabilities	582
javax.telephony.phone.capabilities Interface ComponentGroupCapabilities	582
canActivate	582
canActivate	582
Package javax.telephony.phone.events	584
Package javax.telephony.phone.events	584
Package javax.telephony.phone.events Description	584
javax.telephony.phone.events Class Hierarchy	586
Hierarchy For Package javax.telephony.phone.events	586
Interface Hierarchy	586
Interface ButtonInfoEv	587
javax.telephony.phone.events Interface ButtonInfoEv	587
ID	589
getInfo	589
getOldInfo	589
Interface ButtonPressEv	590
javax.telephony.phone.events Interface ButtonPressEv	590
ID	591
getInfo	592
Interface DisplayUpdateEv	593
javax.telephony.phone.events Interface DisplayUpdateEv	593
ID	594
getDisplay	595
Interface HookswitchStateEv	596
javax.telephony.phone.events Interface HookswitchStateEv	596
ID	597
getHookSwitchState	598
Interface LampModeEv	599
javax.telephony.phone.events Interface LampModeEv	599
ID	600
getMode	601
Interface MicrophoneGainEv	602
javax.telephony.phone.events Interface MicrophoneGainEv	602

ID	603
getGain	604
Interface PhoneEv	605
javax.telephony.phone.events Interface PhoneEv	605
CAUSE_NORMAL	606
CAUSE_UNKNOWN	606
getPhoneCause	606
Interface PhoneTermEv	607
javax.telephony.phone.events Interface PhoneTermEv	607
getComponentGroup	608
getComponent	608
Interface RingerPatternEv	610
javax.telephony.phone.events Interface RingerPatternEv	610
ID	611
getRingerPattern	612
Interface RingerVolumeEv	613
javax.telephony.phone.events Interface RingerVolumeEv	613
ID	614
getVolume	615
Interface SpeakerVolumeEv	616
javax.telephony.phone.events Interface SpeakerVolumeEv	616
ID	617
getVolume	618